

FIREFLY SOFTWARE FOUNDATION



PyFly

by Example

Event-Driven Python Microservices
with the Firefly Framework



PyFly by Example

Event-Driven Python Microservices with the Firefly Framework

Firefly Software Foundation

Copyright © 2026 Firefly Software Foundation.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this material except in compliance with the License. You may obtain a copy of the License at <https://www.apache.org/licenses/LICENSE-2.0>.

Unless required by applicable law or agreed to in writing, software and documentation distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First edition, 2026.

All code listings in this book were written and verified against PyFly framework version 26.6.x.

Published by the Firefly Software Foundation.

Preface

Enterprise Python has long meant stitching together a dozen independent libraries — one for dependency injection, another for routing, yet another for async database access — with no shared idiom to bind them. **PyFly** changes that. It brings the cohesive, convention-over-configuration experience that Spring Boot gave the Java world, rebuilt from the ground up for Python 3.12+ and `async / await`.

This book teaches PyFly **by doing**. You build one real application from an empty folder to a secured, observable, event-driven service — making every concept concrete before moving to the next. Crucially, the code in these pages is not illustrative pseudocode: it is taken from a **real project that compiles, boots, and passes its tests** against PyFly v26.6.60. Every listing was verified against the running sample, so what you read is what actually works.

Who This Book Is For

This book is for intermediate Python developers comfortable with `async / await`, type hints, and the basics of HTTP services. You need no prior framework expertise — if you have built anything with FastAPI, Flask, or SQLAlchemy, you are well prepared.

Spring Boot developers will feel especially at home. Wherever PyFly mirrors a Spring concept — beans, stereotypes, declarative transactions, application events — a **Spring parity** callout draws the parallel explicitly, so you map what you already know rather than learning from zero.

What You Will Build

Every chapter advances **Lumen**, a digital-wallet and ledger service. The journey follows a deliberate arc, one part at a time:

- **Part I — Foundations (Chapters 1–4).** You scaffold the first Lumen service with `pyfly new`, run it under an ASGI server, wire PyFly's dependency-injection container, bind typed configuration and profiles, and expose your first validated REST endpoints.
- **Part II — Modeling & Persisting (Chapters 5–7).** You introduce the repository pattern over a port, persist wallets with async SQLAlchemy (SQLite, no infrastructure required), model the domain with a `Money` value object and a `Wallet` aggregate, and split reads from writes with CQRS command and query handlers dispatched through a bus.
- **Part III — Event-Driven (Chapters 8–10).** The aggregate raises domain events; a listener projects them; an **event-sourced ledger** rebuilds every balance by replaying its event stream; and the same events flow out to Kafka or RabbitMQ for other services.
- **Part IV — Into Microservices (Chapters 11–13).** Lumen reaches beyond its own process: a typed HTTP client calls an external Payments service, an orchestrated **transfer saga** moves money across wallets and *compensates* when a step fails, and caching plus resilience patterns keep the system fast and fault-tolerant.
- **Part V — Secure · Observe · Ship (Chapters 14–18).** You secure the endpoints with JWT and `@secure`, make the service observable with metrics, tracing, health checks, and the admin dashboard, test the whole stack, connect it to the outside world with scheduling, notifications, and webhooks, and finally extend and ship it to production.

By the last page you have a working, tested, observable, secured service — and the mental model to extend it.

How to Use This Book

Read sequentially. Each chapter builds on the one before, and the Lumen codebase grows incrementally; skipping ahead leaves gaps.

Type every listing yourself. Reading and typing code at the same time is how the patterns stick. Resist copy-pasting until you have written each listing at least once.

Run it. Lumen really runs — `uv run pyfly run` boots the service and `uv run --extra dev pytest` exercises it. Whenever a chapter adds a feature, start the app or the tests and watch it work. Seeing real JSON come back from a real endpoint is worth a hundred diagrams.

Each chapter closes with a **Recap** of what changed in the Lumen codebase and a set of **Exercises** that push one step further. The exercises are optional but recommended for anything you intend to apply immediately.

Conventions in Brief

Typographic and structural conventions — code-listing captions, callout types, and figure numbering — are demonstrated, with live examples, in the **Conventions** section that follows.

The Companion Code

The complete, runnable Lumen project lives in the framework's `samples/lumen` directory. It is a single, layered PyFly project — `interfaces`, `models`, `core`, `web` — that you grow chapter by chapter; the finished source there is the destination this book walks you to. Set it up once with `uv sync`, and use it to compare your work, catch up if you fall behind, or simply run the parts you are reading about.

Conventions

This page explains the typographic and structural conventions used throughout the book.

Code Listings

Every multi-line code example has a **file-name tab** in the top-left corner showing the file it belongs to, and a "**Listing N.N**" caption below the block identifying it by chapter and sequence number. For example:

wallet/domain/wallet.py

```
from pyfly.core import component
from dataclasses import dataclass, field
from decimal import Decimal

@Component
@dataclass
class Wallet:
    id: str
    balance: Decimal = field(default=Decimal("0.00"))

    def deposit(self, amount: Decimal) -> None:
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self.balance += amount
```

Listing 5.1 — Wallet aggregate root

Inline code references within prose use `monospace` font, as in "the `@component` decorator registers the class with PyFly's container."

Callouts

Four callout styles appear in the margins and body:

NOTE

Notes provide supplementary context or clarify a subtlety in the main text — worth reading, but not blocking.

TIP

Tips share a shortcut, idiom, or best practice that will save you time in real projects.

WARNING

Warnings flag a common mistake or a sharp edge that can cause hard-to-debug problems if ignored.

SPRING PARITY

Spring parity callouts map a PyFly concept directly to its Spring Boot equivalent — ideal for developers migrating from the JVM ecosystem.

Figures

Diagrams are numbered **Figure N.N** and captioned below the image. They are embedded as inline SVG, so they render crisply at any zoom level in both the screen and print editions. You will meet the first one on the opening page of Chapter 1.

Contents

PART I Foundations

1. Why PyFly?	12
2. Dependency Injection & the Application Context	29
3. Configuration, Profiles & Secrets	56
4. Your First HTTP API	77

PART II Modeling & Persisting the Domain

5. Persistence & the Repository Pattern	101
6. Domain-Driven Design	129
7. CQRS: Commands & Queries	158

PART III Event-Driven Architecture

8. Domain Events & Event-Driven Architecture.....	195
9. Event Sourcing the Ledger.....	221
10. Messaging with Kafka & RabbitMQ.....	257

PART IV Into Microservices

11. Splitting the Monolith: HTTP Clients & the BFF.....	293
12. Distributed Transactions: Sagas, Workflows & TCC	321
13. Caching & Resilience	356

PART V Secure, Observe & Ship

14. Security, Sessions & Identity	388
15. Observability & the Admin Dashboard	427
16. Testing PyFly Applications.....	467

CONTENTS

17. Scheduling, Notifications, Webhooks & Callbacks	502
18. Extending PyFly & Going to Production	530

Appendices

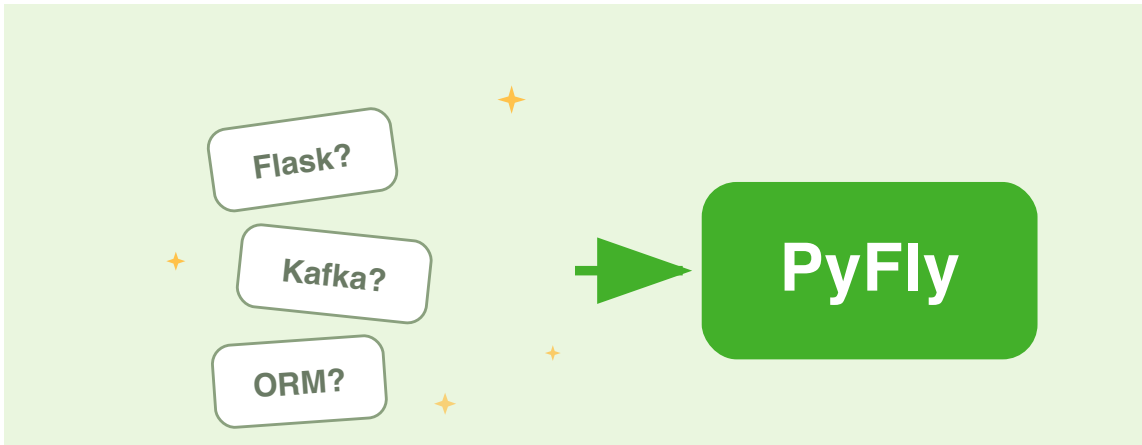
A. Spring Boot → PyFly Cheat-Sheet	558
B. MongoDB & Document Data	593
C. ECM: Content & E-Signature.....	611
D. CLI & Troubleshooting	623
Glossary	638

PART I

Foundations

CHAPTER 1

Why PyFly?



By the end of this chapter you will have installed PyFly, scaffolded the **Lumen** wallet service, and run it locally — with structured logging, a live health endpoint, and interactive API docs already working, all without a single line of boilerplate.

The cohesion problem

Picture your first day on a new Python microservice. Before you write one line of business logic, you face two weeks of architectural choices.

Which web framework do you reach for? FastAPI, Flask, Starlette, Django — each is reasonable, each introduces its own idioms. Which ORM? SQLAlchemy (sync or async?), Tortoise, Beanie? How do you wire dependencies — dependency-injector,

python-inject, or a hand-rolled factory module? How do you manage configuration — pydantic-settings, python-dotenv, dynaconf? And how should the project be laid out? Every team invents its own answer.

You eventually assemble a bespoke stack, glue it together with good intentions, and ship. Six months later a second team starts a new service — and makes entirely different choices. Now you have two codebases with incompatible conventions, different testing strategies, different deployment patterns, and no shared understanding of how anything works.

Python gives you infinite choice. What it does not give you is cohesion.

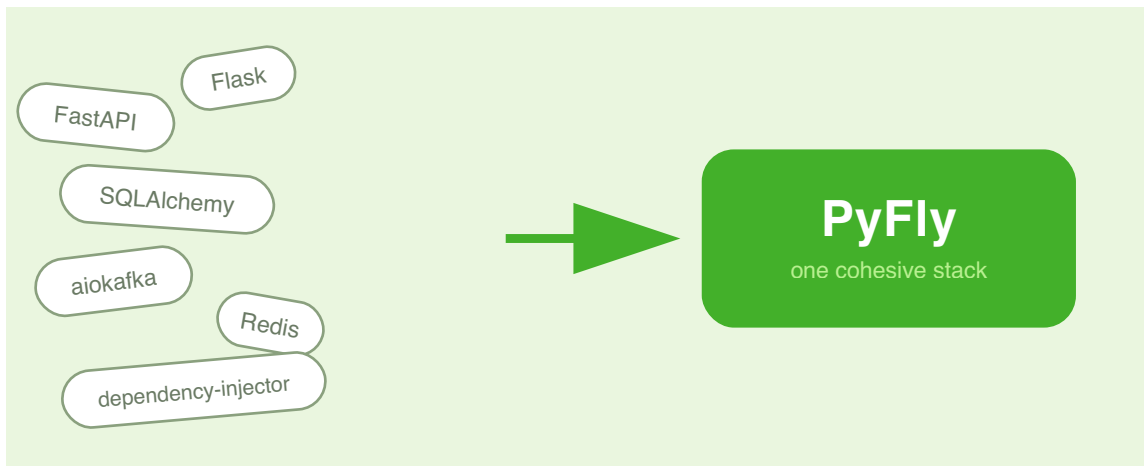


Figure 1.1 — Infinite choice, no cohesion.

The stack-assembly problem is not a skills failure — it is a tooling gap. Java developers solved it years ago with Spring Boot: one opinionated framework that makes sensible choices for you, lets you override what matters, and enforces a consistent idiom across every service. PyFly brings that same discipline to Python.

What is PyFly?

The solution to the cohesion problem is not to ban choice — it is to make one good set of choices and package them as a framework. That is precisely what PyFly does.

PyFly is a **cohesive, full-stack, async-native framework** for building production-grade Python applications — microservices, monoliths, and libraries alike. It makes the stack decisions for you: dependency injection, HTTP routing, database access, messaging, caching, security, observability — all integrated, all consistent, all with production-ready defaults from the very first `pyfly run`.

Under the hood, PyFly delegates to battle-tested async libraries in the Python ecosystem — Starlette for HTTP, SQLAlchemy (async) for relational data, structlog for logging, Pydantic for validation — but you never import them directly. You depend on **PyFly's ports** (Python `Protocol` classes), and the DI container wires the concrete adapters at startup. Swap PostgreSQL for MongoDB, or Kafka for RabbitMQ, without touching a single line of business logic.

PyFly is the **official Python implementation of the Firefly Framework**, a battle-tested enterprise platform originally built for Java (40+ modules in production). It brings the same programming model to Python 3.12+, not as a port but as a native reimplementation designed around `async/await` and type hints.

Its architecture rests on four layers — foundation, application, infrastructure, and integration — each composed of focused modules that interlock cleanly.

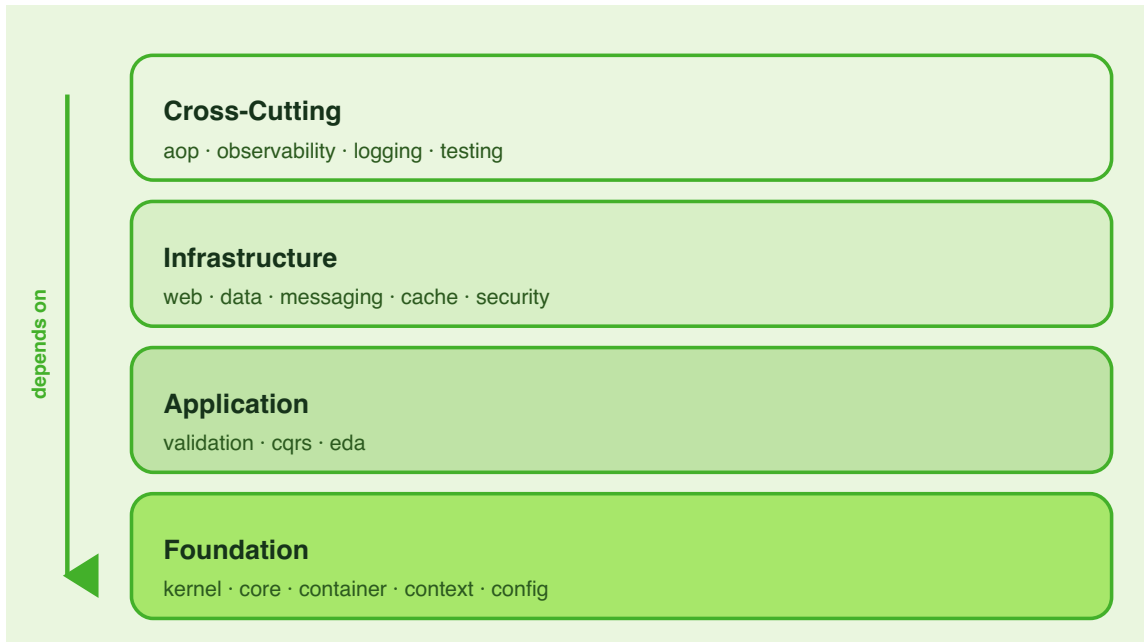


Figure 1.2 — PyFly's four module layers.

Every layer follows the same **hexagonal architecture** principle: your code lives in the centre and depends only on ports; adapters live at the edges and can be swapped without disturbing the core. This design lets the framework grow with you — start with a simple REST service and graduate to CQRS, event-driven messaging, or saga orchestration without rewriting the foundation you built on day one.

 SPRING PARITY

If you are coming from Spring Boot, PyFly will feel like home almost immediately. `@pyfly_application` is your `@SpringBootApplication`. `@service`, `@rest_controller`, and `@repository` are the exact stereotypes you know. Constructor-injection from type hints mirrors `@Autowired` with no XML or reflection magic. The `pyfly.yaml` configuration hierarchy (defaults → profile → env vars) maps directly to `application.yaml` + profiles. A **Spring parity** callout like this one appears throughout the book wherever the concepts align closely enough to save you the mental translation work.

Installing PyFly

To make these ideas concrete, you will build **Lumen** — a fintech wallet platform — throughout the book. Lumen starts simply: a REST API that records ledger entries and reports wallet balances. By the final chapter it will span multiple services communicating over events, with sagas coordinating cross-service transfers. Starting with a well-structured skeleton saves you the pain of retrofitting architecture later, so PyFly's scaffolder generates that structure for you up front.

First, verify you have Python 3.12 or later and [uv](#) (PyFly's recommended package manager):

terminal

```
python --version
# Python 3.12.0 or later

uv --version
# uv 0.5.0 or later

# Scaffold the Lumen wallet service (web-api archetype, web feature)
```



```

uv run pyfly new lumen --archetype web-api --features web

cd lumen

# Install dependencies (including the pyfly CLI and ASGI server)
uv sync

```

Listing 1.1 — Verify prerequisites and scaffold Lumen

`uv run pyfly new` calls the PyFly archetype registry and generates a production-shaped project directory in one shot. The `--archetype web-api` flag selects the REST service template; `--features web` adds the ASGI server dependency. `uv sync` installs the declared dependencies, including the `pyfly` command itself (available via the `cli` extra in `pyproject.toml`).

TIP

Run `pyfly new` **without arguments** to enter interactive mode. It walks you through archetype and feature selection with arrow-key navigation — handy when you want to pre-select extras like relational data or messaging support.

The scaffolder prints the generated layout as a tree:

```

┌─── Created web-api project ───┐
│ lumen-demo/                  │
│ ├── .env.example              │
│ ├── .gitignore                │
│ ├── Dockerfile                │
│ ├── README.md                 │
│ ├── pyfly.yaml                │
│ ├── pyproject.toml            │
│ ├── src/lumen_demo/__init__.py │
│ ├── src/lumen_demo/app.py      │
│ ├── src/lumen_demo/controllers/__init__.py │
│ └── src/lumen_demo/controllers/health_controller.py │
└────────────────────────────────┘

```

```
| └─ src/lumen_demo/controllers/todo_controller.py |  
| └─ src/lumen_demo/main.py |  
| └─ src/lumen_demo/models/__init__.py |  
| └─ src/lumen_demo/models/todo.py |  
| └─ src/lumen_demo/repositories/__init__.py |  
| └─ src/lumen_demo/repositories/todo_repository.py |  
| └─ src/lumen_demo/services/__init__.py |  
| └─ src/lumen_demo/services/todo_service.py |  
| └─ tests/__init__.py |  
| └─ tests/conftest.py |  
| └─ tests/test_todo_service.py |  
└──────────────────────────────────────────────────┘
```

Next steps:

```
cd lumen-demo
```

```
uv sync --group dev
```

```
pyfly run --reload
```

The apparent simplicity is deliberate. `pyproject.toml` gives you a standards-compliant project from day one — no `setup.py` legacy debt. `pyfly.yaml` is the single source of truth for every runtime setting, from log level to database URL, so configuration never scatters across a dozen files. The `src/` layout prevents the `lumen` package from being accidentally importable without an explicit install, catching import-path bugs early. The scaffolder also generates runnable sample controllers, services, and repositories so you have working code to study immediately rather than an empty shell.

Two files that matter

Understanding the scaffold's entry-point structure early will save you confusion later. The scaffold generates two files that work together: `app.py` declares the application and `main.py` exposes the ASGI `app` that the server imports. Each has a distinct responsibility.

Open `src/lumen/app.py`:

`lumen/app.py`

```
from pyfly.core import pyfly_application

@pyfly_application(
    name="lumen-demo",
    scan_packages=[
        "lumen_demo.controllers",
        "lumen_demo.services",
        "lumen_demo.repositories",
    ],
)
class Application:
    pass
```

Listing 1.2 — Application declaration (@pyfly_application)

Short as it is, every parameter pulls its weight:

- `name` — the service identity. It appears in every structured log event, in the OpenAPI title, and in health-check payloads — an unambiguous handle when you are reading aggregated logs from a dozen services.
- `scan_packages` — the instruction that drives the whole DI system. PyFly walks every listed module and registers any class decorated with `@service`,

`@rest_controller`, `@repository`, or `@configuration` in the `ApplicationContext`. Add a new service class anywhere in those packages and it is wired automatically; no explicit registration code required.

The `Application` class body is intentionally empty. You never instantiate it yourself — `PyFlyApplication` does that during startup: it loads configuration from `pyfly.yaml`, configures structured logging, prints the startup banner, scans packages, initialises the `ApplicationContext`, and logs startup timing.

Now open `src/lumen/main.py`:

lumen/main.py

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

from starlette.applications import Starlette

from pyfly.core import PyFlyApplication
from pyfly.web.adapters.starlette import create_app

from lumen_demo.app import Application

# Bootstrap: load config, scan packages, build DI context
_pyfly = PyFlyApplication(Application)

@asynccontextmanager
async def _lifespan(app: Starlette) -> AsyncIterator[None]:
    """Manage application startup and shutdown lifecycle."""
    _pyfly._route_metadata = getattr(
        app.state, "pyfly_route_metadata", []
    )
    _pyfly._docs_enabled = getattr(
        app.state, "pyfly_docs_enabled", False
    )
```

```

_pyfly._host = str(_pyfly.config.get("pyfly.web.host", "0.0.0.0"))
_pyfly._port = int(_pyfly.config.get("pyfly.web.port", 8080))
await _pyfly.startup()
yield
await _pyfly.shutdown()

app = create_app(
    title="lumen-demo",
    version="0.1.0",
    context=_pyfly.context,
    lifespan=_lifespan,
)

```

Listing 1.3 — ASGI entry point (main.py)

This is the file that `pyfly run` (and any ASGI server like Uvicorn) discovers. The module-level `app` object is a Starlette application: `create_app` mounts every `@rest_controller` found in the DI context and wires the lifespan hook so startup and shutdown happen cleanly. The `@pyfly_application` decorator alone does **not** create the ASGI `app` — `main.py` is the explicit bridge between the DI world (`app.py`) and the HTTP world.

Project files

The other two files you will edit most often are `pyproject.toml` and `pyfly.yaml`. Each plays a distinct role: `pyproject.toml` manages dependencies, while `pyfly.yaml` owns every runtime knob.

`pyproject.toml` records the project's dependencies. Notice the extras:

```
lumen/pyproject.toml
```

```

[project]
name = "lumen-demo"
version = "0.1.0"
description = "lumen-demo - built with PyFly"
requires-python = ">=3.12"
dependencies = [
    "pyfly[web]",
]

[dependency-groups]
dev = [
    "pytest>=8.0",
    "pytest-asyncio>=0.23",
    "pytest-cov>=5.0",
    "ruff>=0.3",
    "mypy>=1.8",
]

```

Listing 1.4 — `pyproject.toml` (key sections)

The `web` extra in `pyfly[web]` bundles Uvicorn together with the ASGI adapter. The `cli` extra (add it when you need the `pyfly` command outside of `uv run`) provides the command-line tooling. Other extras — `data-relational`, `granian` — are opt-in; you add them only when a feature needs them. The Lumen wallet sample, for example, declares `pyfly[cli,web,data-relational]` because it connects to SQLite.

`pyfly.yaml` is where all runtime settings live:

`lumen/pyfly.yaml`

```

pyfly:
  app:
    name: lumen-demo
    module: lumen_demo.main:app

  web:

```

```

port: 8080
adapter: auto

server:
  type: "auto"
  event-loop: "auto"
  workers: 0

actuator:
  endpoints:
    enabled: true

admin:
  enabled: true

logging:
  level:
    root: INFO

```

Listing 1.5 — `pyfly.yaml` (scaffold default)

The `module` key tells `pyfly run` exactly where the ASGI `app` lives — `lumen_demo.main:app`. Port, server type, actuator endpoints, and log level all have sensible defaults; you override only what you actually need to change.

DEFAULT SERVER: GRANIAN VS UVICORN

PyFly's default ASGI server is **Granian**, a high-performance Rust-based server. The scaffold's `pyfly[web]` dependency bundles **Uvicorn** instead (lighter install). The two are interchangeable: pass `--server uvicorn` on the command line, or add `pyfly[granian]` to your dependencies to use Granian. The Lumen wallet sample runs with `--server uvicorn` for this reason.

Your first run

With the project in place, start the server. From the project root:

terminal

```
uv run pyfly run --server uvicorn
```

Listing 1.6 — Run the server with uv

After a moment you will see the PyFly ASCII banner followed by a stream of structured startup events:

```

      _____
     |             |
     |  .  .  .  /  |  .  .  .
     |  <  |  |  \  |  <  |  | |
     |  |  >  >  |  |  |  |  \  |
     |  //  |  |  |  |  /  |
     |  \  \  |  |  |  \  \

:: PyFly Framework :: (v26.06.60) (Python 3.13.13)
Copyright 2026 Firefly Software Foundation. | Apache License 2.0
2026-06-07 20:34:32,442 [INFO] pyfly.core: starting_application |
    app=lumen version=1.0.0 python=3.13.13 pid=72300
2026-06-07 20:34:32,442 [INFO] pyfly.core: runtime_environment |
    os=Darwin os_version=25.5.0 arch=arm64 cpus=11
2026-06-07 20:34:32,442 [INFO] pyfly.core: loaded_config |
    source=pyfly-defaults.yaml (framework defaults)
2026-06-07 20:34:32,442 [INFO] pyfly.core: loaded_config |
    source=pyfly.yaml
2026-06-07 20:34:32,442 [INFO] pyfly.core: scanned_package |
    package=lumen.web.controllers beans_found=1
2026-06-07 20:34:32,580 [INFO] pyfly.core: bean_summary |
    total=127 services=6 repositories=2 controllers=4
2026-06-07 20:34:32,580 [INFO] pyfly.core: mapped_endpoints | count=5
2026-06-07 20:34:32,580 [INFO] pyfly.core: request_mapping |
    method=POST path=/api/v1/wallets handler=open_wallet

```



```
2026-06-07 20:34:32,580 [INFO] pyfly.core: api_documentation |
  swagger_ui=http://0.0.0.0:8080/docs
2026-06-07 20:34:32,581 [INFO] pyfly.core: server_started |
  server=uvicorn host=0.0.0.0 port=8080 workers=1
```

Read these log lines as a story of what the framework did on your behalf. Each event has a name rather than a raw message so log aggregators can filter by key:

1. **starting_application** — PyFly announces itself with the service name and version you declared, giving every log aggregator a clean filter from the very first event.
2. **runtime_environment** — OS, architecture, and CPU count are emitted once on boot, making it easy to correlate behaviour with the environment later.
3. **loaded_config** (x2) — configuration is layered: the framework ships `pyfly-defaults.yaml` with safe production defaults; your `pyfly.yaml` overrides only what you need. Activate a profile (e.g., `--profile prod`) and you will see a third entry.
4. **scanned_package** — the DI container found one `@rest_controller` in the web controllers package. `beans_found` grows as you add services and repositories.
5. **bean_summary** — the total DI context size, including framework-internal beans. Even with 127 beans registered, PyFly boots in well under a second.
6. **server_started** — Uvicorn is accepting connections.

Two endpoints are already live before you have written a single line of application logic. Try the health check first:

terminal

```
curl -s localhost:8080/actuator/health
```

Listing 1.7 — Health check

```
{
  "status": "UP",
  "components": {
    "cache_health_indicator": {
      "status": "UP",
      "details": {"adapter": "InMemoryCache", "latencyMs": 0.01}
    },
    "cqrs_health_indicator": {
      "status": "UP",
      "details": {"command_handlers": 3, "query_handlers": 2}
    },
    "eda_health": {
      "status": "UP",
      "details": {"adapter": "InMemoryEventBus"}
    },
    "db_health_indicator": {
      "status": "UP",
      "details": {"database": "sqlite"}
    }
  }
}
```

Then call the first business endpoint — opening a wallet:

terminal

```
curl -s -X POST localhost:8080/api/v1/wallets \
  -H 'content-type: application/json' \
  -d '{"owner_id":"u-1","currency":"EUR"}'
```

Listing 1.8 — Open a wallet

```
{"wallet_id": "wlt-c5bbb2a7-dd49-4321-932e-e4c6bfa5cc2c"}
```

Open <http://localhost:8080/docs> in your browser for the interactive Swagger UI; <http://localhost:8080/redoc> gives you ReDoc; the raw OpenAPI spec is at [/openapi.json](#).

NOTE

All three doc URLs are emitted in the boot log (`api_documentation` entries) so you never have to remember them. `http://localhost:8080/admin` opens the PyFly Admin Dashboard — a live view of beans, endpoints, health indicators, and recent log events.

✓ What you built

In under five minutes you installed PyFly, scaffolded a production-shaped service, and ran it locally. The table below summarises what Lumen already delivers — entirely from the framework — before you have written a single line of application logic.

Capability	How you got it
Structured JSON logging with runtime metadata	Framework default — emitted on every boot
Interactive API docs (Swagger UI + ReDoc)	Built-in; always on, opt out via <code>pyfly.yaml</code>
Health endpoint (<code>/actuator/health</code>)	<code>actuator.endpoints.enabled: true</code> in scaffold
Profile-aware configuration layering	<code>pyfly.yaml</code> + <code>--profile</code> flag
Auto-discovery DI container	<code>scan_packages</code> in <code>@pyfly_application</code>
Startup timing, bean summary, endpoint map	Emitted as structured log events on every boot
Admin Dashboard (<code>/admin</code>)	Enabled in the scaffold by default

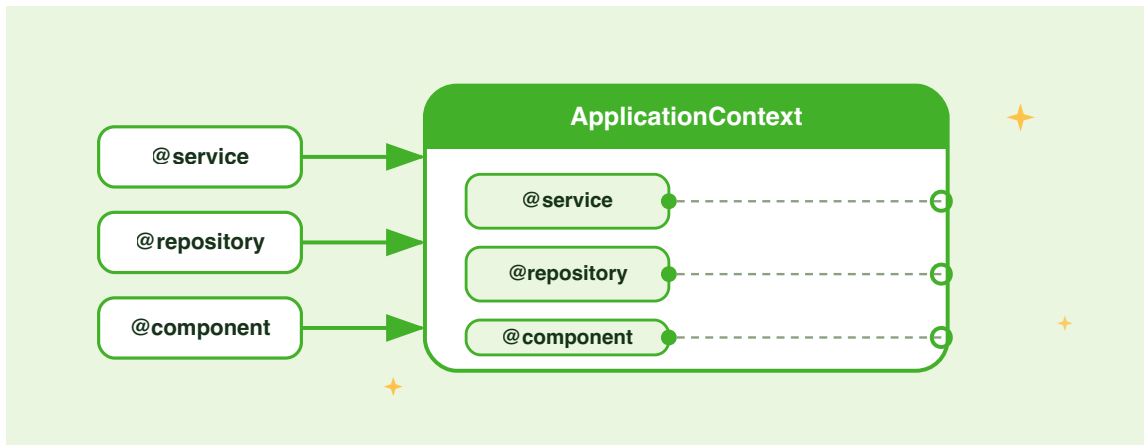
That is the PyFly promise: sensible decisions made for you, conventions consistent across every service, and production-ready defaults from the very first run.

Try it yourself

- 1. Rename the service.** Open `pyfly.yaml` and change `app.name` from `lumen-demo` to `lumen-wallet`. Also update the `name` parameter in `@pyfly_application`. Re-run with `uv run pyfly run --server uvicorn` and confirm the new name appears in the `starting_application` log line.
- 2. Explore the live docs.** With the server running, open `http://localhost:8080/docs`. Notice the service name and version at the top of the Swagger UI. Then navigate to `http://localhost:8080/redoc` and compare the two doc renderers. Check the health response at `http://localhost:8080/actuator/health` and note which health indicators are already reporting.
- 3. Switch to Granian.** Add `pyfly[granian]` to the `dependencies` list in `pyproject.toml`, run `uv sync`, then start the server without the `--server uvicorn` flag. Compare the `server_started` log line — the `server` field should now read `granian`.
- 4. Map files to responsibilities.** Look at the two entry-point files `app.py` and `main.py`. Write a one-sentence description of what each one is responsible for. Where does the DI container come from? Where does the ASGI `app` object live? Which one would you edit to change the scan packages?

CHAPTER 2

Dependency Injection & the Application Context



In the previous chapter you gave Lumen its application entry point and watched the container start. Now you will declare Lumen's first real components — a `WalletRepository` that subclasses the framework's Spring-Data-style `Repository`, a `WalletEntity` that maps the persistence row, and a CQRS handler that depends on both — and let PyFly wire them together from nothing but type hints. No factories, no manual `new`, no glue code.

Before a single line of Lumen code appears, it is worth pausing on *why* that matters. In a conventional Python project you would write something like:

```
handler = DepositFundsHandler(
    repository=InMemoryWalletRepository(),
    events=InMemoryEventBus(),
)
```

somewhere near the startup path. That one line seems harmless, but it locks every decision — which repository class, which event bus — at the point of construction. Swap the repository for a Postgres adapter and you must find every construction site. Add a test double and you need to restructure the wiring. **Dependency injection** inverts this relationship: classes *declare* what they need, and the container *decides* what to provide. The result is code that is open to extension but closed to modification — the `DepositFundsHandler` you write today will accept a production database adapter in Part II without a single change to its source.

Stereotypes: declaring your beans

Before the container can wire anything, it needs to know which classes to manage. A **bean** is any object the container creates, wires, and owns. You make a class visible to the container by applying a **stereotype decorator** — a thin annotation that registers the class and signals its architectural role.

PyFly ships five stereotypes:

Decorator	Meaning
<code>@service</code>	Business-logic layer: domain operations, use-case orchestration.
<code>@component</code>	Generic managed bean with no specific architectural role.
<code>@repository</code>	Data-access layer: databases, external storage, ports.
<code>@configuration</code>	Configuration class that can contain <code>@bean</code> factory methods.
<code>@rest_controller</code>	HTTP layer: handles requests and returns JSON responses.

All five stereotypes are **container-equivalent**: they share the same internal `_make_stereotype()` factory and accept the same optional keyword arguments (`name`, `scope`, `profile`, `condition`). The meaningful differences are the `__pyfly_stereotype__` label — used by the web layer to discover controllers and by the context to find `@configuration` classes — and the architectural clarity each name brings to readers of your code. Choosing `@repository` over `@component` costs nothing technically but tells every future reader exactly what the class is for.

Both bare and parenthesised forms work:

```
@service          # bare – all defaults
class SimpleService:
    pass

@service(name="wallet_svc") # with keyword args
class NamedService:
    pass
```

The scan_packages bootstrap

The container only discovers beans in packages it has been told to scan. In `lumen/app.py`, `@pyfly_application` lists every subpackage the container should introspect for stereotype declarations:

`lumen/app.py`

```
from pyfly.core import pyfly_application
from pyfly.starters.domain import enable_domain_stack

@enable_domain_stack
@pyfly_application(
    name="lumen",
    version="1.0.0",
    description=(
        "Lumen – a DDD digital-wallet service"
```

```

        " built on the PyFly framework."
    ),
    scan_packages=[
        "lumen.models.repositories",
        "lumen.core.services.wallets",
        "lumen.core.services.transfers",
        "lumen.core.services.listeners",
        "lumen.web.controllers",
    ],
)
class LumenApplication:
    pass

```

Listing 2.1 — Application entry point with `scan_packages`

How it works. `@pyfly_application` registers `LumenApplication` as the application root and seeds the container with framework auto-configurations. `scan_packages` is the exact list of Python package paths the container walks at startup, collecting every class decorated with a stereotype. Any package not listed here is invisible to the container — the most common source of "why is my bean not found?" confusion when adding new subpackages. `@enable_domain_stack` activates the CQRS, transactional engine, event sourcing, relational data, and rule-engine auto-configurations in a single line.

🌿 SPRING PARITY

`scan_packages` is the equivalent of Spring's `@ComponentScan(basePackages = {...})`. The semantics are identical: list every subpackage you want the framework to introspect, and it will register everything it finds.

The entity and the repository

Lumen stores wallets in a relational database. Two classes carry this responsibility:

`WalletEntity` (the persistence row) and `WalletRepository` (the data-access bean).

The entity. `WalletEntity` is a plain SQLAlchemy-mapped class that inherits the framework's `Base`:

`lumen/models/entities/v1/wallet_orm.py`

```
from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's own id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(
        String(64), primary_key=True
    )
    owner_id: Mapped[str] = mapped_column(
        String(255), nullable=False, index=True
    )
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(
        nullable=False, default=0
    )
    created_at: Mapped[datetime] = mapped_column(
        default=lambda: datetime.now(UTC)
    )
```

Listing 2.2a — `WalletEntity`: the persistence row

Inheriting `Base` (PyFly's declarative base) registers the `wallets` table in `Base.metadata`; the framework's engine lifecycle creates it on startup. No further wiring is needed.

The repository. `WalletRepository` subclasses the framework's generic `Repository[WalletEntity, str]`. The two type arguments tell the framework the *entity type* (`WalletEntity`) and the *primary-key type* (`str`); from that it generates and injects a full async CRUD surface — `find_by_id`, `save`, `find_all`, `find_all(pageable)`, `delete`, `delete_by_id`, `count`, and more — backed by the transactional `AsyncSession` from the relational auto-configuration:

`lumen/models/repositories/wallet_repository.py`

```
from __future__ import annotations

from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data import Page, Pageable
from pyfly.data.relational.sqlalchemy import (
    Repository,
    Specification,
)

def balance_at_least(min_minor: int) -> Specification[WalletEntity]:
    """Reusable predicate: wallets with balance >= min_minor."""
    return Specification(
        lambda root, q: q.where(root.balance_minor >= min_minor)
    )

@repository
class WalletRepository(Repository[WalletEntity, str]):
    """CRUD + derived + specification queries for WalletEntity."""

    # Derived query – compiled from the name by the post-processor
```

```

async def find_by_owner_id(
    self, owner_id: str
) -> list[WalletEntity]:
    """All wallets owned by owner_id (derived query stub)."""
    ...

# Specification query – composable predicate + pagination
async def find_rich(
    self, min_minor: int, pageable: Pageable
) -> Page[WalletEntity]:
    """Page of wallets with balance >= min_minor."""
    return await self.find_all_by_spec_paged(
        balance_at_least(min_minor), pageable
    )

# Upsert: one call for INSERT or UPDATE
async def upsert(self, entity: WalletEntity) -> WalletEntity:
    """Persist entity whether the row is new or already exists."""
    session = self._require_session()
    merged = await session.merge(entity)
    await session.flush()
    return merged

```

Listing 2.2b — WalletRepository: framework Repository subclass

How it works. `@repository` tells the container to manage `WalletRepository` as a DI bean. The framework reads `Repository[WalletEntity, str]` at startup, generates the CRUD implementation internally, and registers the class — you inject `WalletRepository` directly by type anywhere in the application. There is no hand-written port interface and no separate adapter to maintain: **the framework supplies and injects the implementation; you depend on the repository class itself by type.**

The three extra methods show the extension points the framework exposes on top of the inherited CRUD:

- `find_by_owner_id` is a **derived query** — the `RepositoryBeanPostProcessor` parses the method name and compiles a real `SELECT ... WHERE owner_id = :owner_id` at startup; you write the stub (`...`) and the framework fills it in.
- `find_rich` is a **Specification query** — it composes a reusable `Specification` predicate and runs it with pagination and sorting via the inherited `find_all_by_spec_paged`.
- `upsert` is a thin convenience over `session.merge` so a command handler can persist an entity whether it is new or already exists with a single call.

🍃 SPRING PARITY

`@service`, `@component`, `@repository`, and `@configuration` map directly to Spring's `@Service`, `@Component`, `@Repository`, and `@Configuration`. `@rest_controller` mirrors `@RestController`. `Repository[E, ID]` mirrors Spring Data's `JpaRepository<E, ID>`: declare the entity and key types; the framework generates and injects the full implementation. Derived query methods (names like `find_by_owner_id`) compile to SQL at startup — the same mechanism as Spring Data's query derivation from method names.

Constructor injection

With the repository declared, you need a handler that uses it. That is where the container's most important capability becomes visible: you never call constructors yourself. You declare what a class *needs* as `__init__` parameters with type annotations, and the container fills them in automatically. This is **constructor injection**, and it is the recommended approach for all mandatory dependencies.

The mental model is a simple wishlist: list the types you need; the container delivers the right instances. If a dependency does not exist at startup, you get a clear `NoSuchBeanError` immediately — not a cryptic `AttributeError` three call frames deep at runtime.

Stacking handler decorators on `@service`

In Lumen's CQRS design, every write-side handler carries two decorators:

`@command_handler` (or `@query_handler`) **stacked on** `@service`. The pattern is non-negotiable: `@service` registers the class as a bean; the CQRS decorator adds only routing metadata (`__pyfly_command_type__` or `__pyfly_query_type__`) so the command/query bus can dispatch to the right handler. Without `@service`, the container never sees the class and the bus raises `NoHandlerError` at dispatch time.

The `DepositFundsHandler` shows the pattern in full:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import (
    DepositFunds,
)
from lumen.core.services.wallets.event_publishing import (
    publish_domain_events,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
```

```

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle( # type: ignore[override]
        self, command: DepositFunds
    ) -> int:
        entity = await self._repository.find_by_id(
            command.wallet_id
        )
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.deposit(
            Money(amount=command.amount, currency=wallet.currency)
        )
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(
            self._events, wallet.clear_events()
        )
        return wallet.balance.amount

```

Listing 2.3 — `DepositFundsHandler`: `@command_handler` + `@service` stacked

How it works. Five decisions are visible in this listing:

- `@service` registers the class as a singleton bean. Without it, the container never sees the class.
- `@command_handler` (applied above `@service`, so it runs *after* registration) reads the first generic argument of `CommandHandler[DepositFunds, int]` and records that this bean handles `DepositFunds` commands.
- The `__init__` signature is the complete wiring specification: `repository: WalletRepository` — the framework-generated CRUD bean; `events: EventPublisher` — resolved by the CQRS auto-configuration; `session_factory: async_sessionmaker[AsyncSession]` — the shared connection factory provided by the relational auto-configuration. All three are resolved by type; `DepositFundsHandler` never imports a concrete class.
- `@transactional()` on `do_handle` wraps the entire body in a single committed unit of work. The decorator opens a session from `session_factory`, binds it to the repository for the duration of the call, and commits on success (or rolls back on error).
- The business logic follows the standard CQRS/DDD sequence: load the entity, rehydrate the aggregate via the mapper, mutate through domain methods that enforce invariants, persist via `upsert`, drain and publish the events. The wallet is saved *before* events are published, so any listener that queries the repository finds the updated record.

A read-side handler uses the same stacking pattern, only with `@query_handler` and `QueryHandler`:

```
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
```

```

from lumen.models.repositories.wallet_repository import WalletRepository

@query_handler
@service
class GetWalletHandler(QueryHandler[GetWallet, WalletDto | None]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle(
        self, query: GetWallet
    ) -> WalletDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_dto(entity) if entity is not None else None

```

The container resolves dependencies **recursively**. When it constructs `DepositFundsHandler` it also constructs `WalletRepository` (the framework-generated CRUD bean), the `EventPublisher`, and the `async_sessionmaker` — none of which the handler needs to know about.

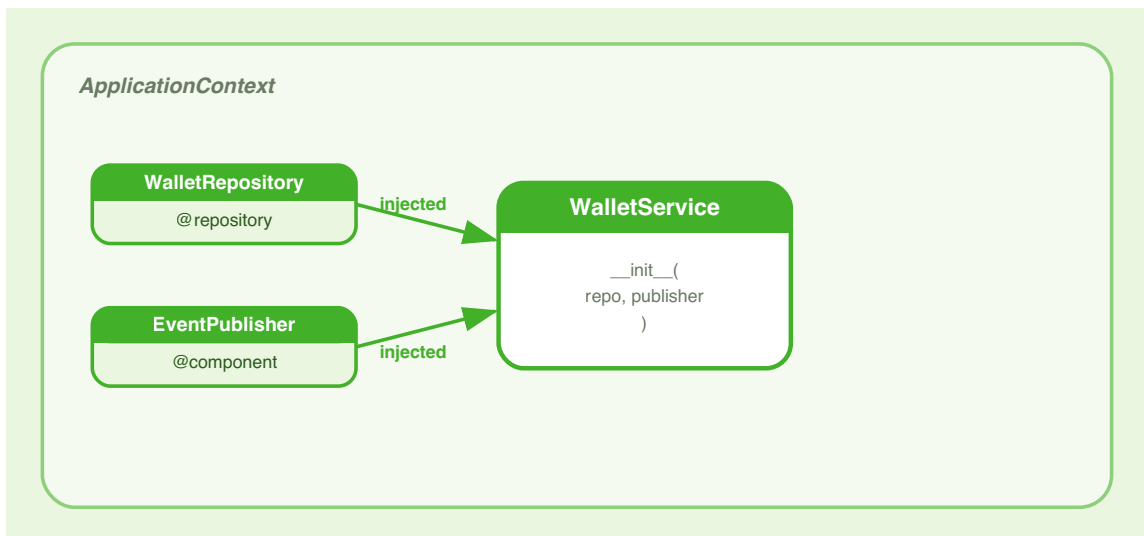


Figure 2.1 — The container injects dependencies from type hints.

🍃 SPRING PARITY

Constructor injection in PyFly is functionally identical to Spring's `@Autowired` constructor injection. In modern Spring you do not even write `@Autowired` — the framework infers injection from the single constructor, just as PyFly reads `__init__` type hints. The mental model is the same: declare what you need, let the container provide it.

💡 TIP

Prefer constructor injection for mandatory dependencies. It makes them visible in the class signature, lets you write plain-Python unit tests without a container (`handler = DepositFundsHandler(repo=MockRepo(), events=MockBus())`), and prevents accidental missing-dependency bugs at startup rather than at runtime.

The Container and the ApplicationContext

PyFly's DI system has two layers, and understanding the boundary between them will save you real debugging time. One layer handles object graphs; the other handles the full application lifecycle. Conflating them is a common source of confusion.

Container (from `pyfly.container`) is the low-level DI engine. It stores **Registration** objects, resolves types by constructor hints, manages scopes, applies **@primary** disambiguation, and handles **Qualifier**-based named lookups. It has no lifecycle awareness — it is a pure "give me a **T**" machine.

ApplicationContext (from `pyfly.context`) is the high-level orchestrator. It wraps **Container** and adds the full startup sequence: profile filtering, condition evaluation, **@configuration** / **@bean** processing, **BeanPostProcessor** weaving,

`@post_construct` / `@pre_destroy` hooks, event publishing, and auto-configuration. You interact with the `ApplicationContext` in application code; the raw `Container` is an implementation detail, accessible via `ctx.container` as an escape hatch.

Think of it this way: `Container` is the factory floor — it knows how to build things. `ApplicationContext` is the production manager — it decides what gets built, in what order, and what happens when the factory opens or closes.

Resolution rules

When the container needs to resolve a type `T`, it applies four rules in strict priority order:

1. **Direct registration** — if `T` is registered directly, resolve it.
2. **Interface binding** — if `T` is a `Protocol` or ABC with exactly one bound implementation, resolve that implementation.
3. **@primary disambiguation** — if multiple implementations are bound, the one decorated with `@primary` wins.
4. **Error** — `NoSuchBeanError` when nothing matches; `NoUniqueBeanError` when multiple candidates exist with no `@primary`.

Step 4 is deliberately loud. A missing or ambiguous dependency is a configuration error, and surfacing it at startup rather than burying it in a runtime traceback is one of the container's key guarantees.

@primary

`@primary` resolves ambiguity when several beans satisfy the same interface. Place it on the implementation you want as the default. This arises whenever you have a `@runtime_checkable Protocol` port with more than one registered adapter — a common pattern for swappable infrastructure (cache store, message bus, notification channel).

For example, suppose your application defines a `CacheStore` protocol and ships two adapters — an in-process one for local development and a Redis one for production:

```
from pyfly.container import repository, primary

@primary
@Repository
class InMemoryCacheStore(CacheStore):
    """Default: active in development and tests."""
    ...

@Repository
class RedisCacheStore(CacheStore):
    """Production cache – activated by profile or condition."""
    ...
```

Without `@primary`, resolving `CacheStore` with two registered implementations raises:

```
NoUniqueBeanError: Multiple beans of type 'CacheStore' found
but none is marked @primary
Candidates: ['InMemoryCacheStore', 'RedisCacheStore']
```

The message names every competing candidate so you can make a deliberate decision rather than guessing which one the container would have picked. Moving `@primary` from one adapter to the other is the only change needed to switch the application's backing store — nothing in the service code changes.

Note that Lumen's own `WalletRepository` is a framework `Repository` subclass, so only one bean is registered and no `@primary` is needed. `@primary` is relevant whenever you hand-roll a port/adaptor pair with multiple adapter implementations.

@order

The container initialises singleton beans eagerly during startup, but some beans genuinely must be ready before others — a security filter that must wrap every inbound request, or a schema migrator that must run before any repository is touched. `@order` gives you explicit control over initialisation sequence.

Lower values are resolved first during the eager startup pass. The constants `HIGHEST_PRECEDENCE` (`-(2**31)`) and `LOWEST_PRECEDENCE` (`2**31 - 1`) mark the extremes:

```
from pyfly.container import order, HIGHEST_PRECEDENCE, service

@order(HIGHEST_PRECEDENCE)
@service
class SecurityInitializer:
    """Must be ready before any other service."""
    ...
```

`@order` affects singleton resolution during startup, the sequence in which `BeanPostProcessor` instances run, and the ordering of `get_beans_of_type()` results.

Qualifier — named bean resolution

Type-based injection covers most scenarios. Occasionally, though, you genuinely need a particular *instance* rather than any satisfying implementation — the classic case being a `@configuration` class that produces two beans of the same type (say, a primary and a read-replica database connection) where a downstream service must receive a specific one.

Select a specific bean by name with `Annotated[T, Qualifier("name")]`:

```
from typing import Annotated
from pyfly.container import Qualifier, service
```

```

@service
class ReportService:
    def __init__(
        self,
        db: Annotated[object, Qualifier("analytics_db")],
    ) -> None:
        self.db = db # receives the bean named "analytics_db"

```

The container calls `resolve_by_name("analytics_db", expected_type=T)` and verifies assignability — a mistyped name pointing at the wrong type raises `NoSuchBeanError` with a clear message rather than silently injecting the wrong object.

Bean factories: @configuration and @bean

Stereotype decorators work beautifully for classes you own, but not every dependency is a class you control. Third-party clients need constructor arguments known only at runtime; related beans share configuration state; some families of beans are most clearly expressed as a single factory. For all of these situations, PyFly provides the `@configuration` / `@bean` pattern — explicit factory code that still participates fully in the container's resolution and lifecycle machinery.

A `@configuration` class acts as a factory. Its `@bean` methods are called during the startup sequence, and each method's return value is registered as a bean whose type comes from the method's return annotation:

`lumen/infra_config.py`

```

from pyfly.container import configuration, bean
from pyfly.eda import EventPublisher, InMemoryEventBus

```

```

@Configuration
class LumenInfraConfig:
    """Wires infrastructure beans that require explicit construction."""

    @Bean
    def event_publisher(self) -> EventPublisher:
        """In-memory event bus – replace with Kafka adapter in production."""
        return InMemoryEventBus()

```

Listing 2.4 — Producing an `EventPublisher` bean via `@configuration`

How it works. `@configuration` tells the context to scan `LumenInfraConfig` for `@bean` methods during startup, before any stereotype beans are constructed. The return annotation `EventPublisher` is the key: the context reads it and registers the produced `InMemoryEventBus` instance as an `EventPublisher`, not as an `InMemoryEventBus`. That distinction matters — when `DepositFundsHandler` later asks for an `EventPublisher`, it receives the `InMemoryEventBus` instance without knowing or caring about the concrete type.

Swapping to a Kafka adapter for production means replacing `InMemoryEventBus()` with `KafkaEventPublisher(settings.kafka_url)` in a single method. The rest of the codebase is untouched.

`@bean` methods can also declare parameters; the container resolves them automatically:

```

@Configuration
class MessagingConfig:

    @Bean
    def audited_publisher(self, base: EventPublisher) -> EventPublisher:
        """Wrap the base publisher with audit logging."""
        return AuditingEventPublisher(base)

```

@bean parameters

Parameter	Default	Description
<code>name</code>	method name	Bean name for named resolution.
<code>scope</code>	<code>Scope.SINGLETON</code>	Lifecycle scope of the produced bean.
<code>primary</code>	<code>False</code>	Mark this the primary candidate for its interface.
<code>profile</code>	<code>""</code>	Only create the bean when the profile expression matches.

NOTE

The return type annotation on a `@bean` method is **mandatory**. The context reads it to know which interface type to register the produced bean under. Omitting it will cause the bean to be unreachable by type.

Scopes

Every bean has a **scope** that controls how long its instance lives. Getting scope right is less about performance and more about correctness: sharing a stateful object designed for single-use produces race conditions; re-creating a singleton on every resolution wastes resources and defeats caching. The `Scope` enum defines three values that cover the vast majority of real-world needs.

`Scope.SINGLETON` (default) — one instance is created on first resolution and reused for the life of the application. Singletons are instantiated eagerly during `ApplicationContext.start()`, sorted by `@order`. Almost all application beans should be singletons.

Scope.TRANSIENT — a fresh instance is created on every resolution. Use this for stateful, non-shareable objects:

`lumen/contexts.py`

```
from pyfly.container import component, Scope

@component(scope=Scope.TRANSIENT)
class TransferContext:
    """Carries state for a single wallet transfer operation."""

    def __init__(self) -> None:
        self.steps: list[str] = []
        self.rolled_back: bool = False
```

Listing 2.5 — A transient bean for per-operation context

How it works. `TransferContext` accumulates the steps of a multi-hop transfer so that a saga can roll them back in reverse order if anything fails. Sharing a single instance across concurrent requests would blend their state; `Scope.TRANSIENT` ensures every resolution produces a fresh, empty `TransferContext`. The container still manages the class — injecting it, profiling it, post-processing it — but never caches the result.

Scope.REQUEST — scoped to a single HTTP request. A new instance is created when a request arrives and discarded when it completes. Use this for web-layer beans that carry request-specific state, such as the current authenticated user.

```
from pyfly.container import component, Scope

@component(scope=Scope.REQUEST)
class CurrentUser:
    user_id: str = ""
    roles: list[str] = []
```


A quick rule of thumb:

- **SINGLETON** — the bean is stateless, or its state is safe to share across all callers (connection pools, caches, service objects).
- **TRANSIENT** — the bean accumulates per-operation state that must not bleed between operations (sagas, builders, context carriers).
- **REQUEST** — the bean carries per-HTTP-request state that must be isolated between concurrent requests (authenticated user, request-scoped trace ID).

Lifecycle and conditions

Construction and wiring are only half the story. Real infrastructure beans need to *act* after they are built — reserving a thread pool, pre-loading a cache, subscribing to a message queue — and they need to *undo* those actions cleanly on shutdown. PyFly gives you two lifecycle hooks for this, plus a family of conditional decorators that control whether a bean participates in the container at all.

@post_construct and @pre_destroy

Once the container constructs a bean and injects all its dependencies, you often need one-time initialisation — opening a connection pool, warming a cache, registering a listener. Mark a method `@post_construct` and the context calls it after construction completes. Both synchronous and `async` methods are supported:

`lumen/wallet_audit_listener.py`

```
from pyfly.container import service
from pyfly.context import post_construct, pre_destroy
import logging

logger = logging.getLogger(__name__)
```

```

@service
class WalletAuditListenerWithLifecycle:
    def __init__(self) -> None:
        self._entries: list[dict] = []

    @post_construct
    async def on_start(self) -> None:
        logger.info("wallet_audit_listener_ready")

    @pre_destroy
    async def on_stop(self) -> None:
        logger.info("wallet_audit_listener_shutting_down")

```

Listing 2.6 — Lifecycle hooks on a `@service` bean

How it works. `on_start` fires *after* the constructor returns and all injected dependencies are set — making it safe to issue repository queries, open connections, or publish an application event. The `async` keyword works without any extra setup: the context calls `await on_start()` when it detects a coroutine, and falls back to a direct call for synchronous methods.

`@pre_destroy` is the counterpart, called during `ApplicationContext.stop()` before the bean is discarded. Beans are destroyed in **reverse** initialisation order, so a listener started after the repository is stopped before it.

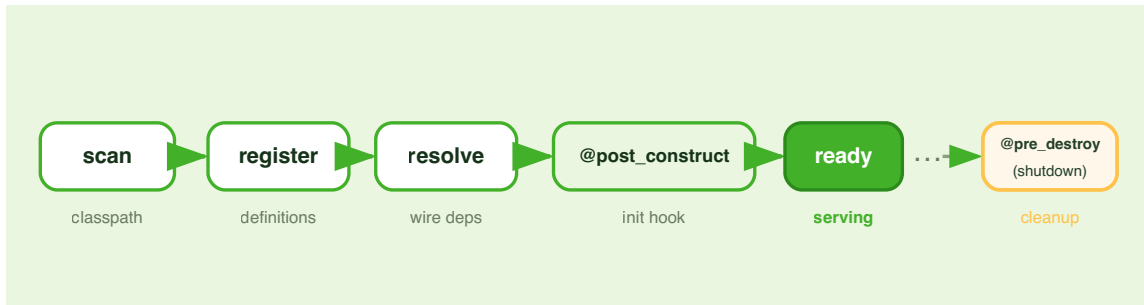


Figure 2.2 — A bean's lifecycle.

Conditional beans

Conditions answer a powerful question: *should this bean exist at all, given the current environment?* They are how the same codebase works in development (cheap in-memory adapters), in CI (Testcontainers), and in production (real infrastructure) — without a single `if` statement in your service code.

Conditional decorators are evaluated in a two-pass strategy during

`ApplicationContext.start()`:

Pass 1 (before user `@configuration` is processed) evaluates: -

- `@conditional_on_property(key, having_value="...")` — the config key must exist and optionally match a value.
- `@conditional_on_class("module.name")` — the Python module must be importable.
- The `condition` callable on a stereotype decorator.

Pass 2 (after user `@configuration` is processed) evaluates: -

- `@conditional_on_bean(SomeType)` — only register if another bean of that type already exists.
- `@conditional_on_missing_bean(SomeType)` — only register if no bean of that type exists yet.

The two-pass design is deliberate. Pass 1 conditions depend on external facts — configuration files and installed packages — that are knowable before any beans are constructed. Pass 2 conditions depend on *which beans got registered*, information available only after Pass 1 settles. Processing them in order ensures each condition evaluates against a stable, predictable view of the container.

The most powerful pattern is **"default with override"** — ship a fallback that automatically yields to any user-provided implementation:

`lumen/notifications.py`

```
from pyfly.container import service
from pyfly.context import conditional_on_missing_bean, conditional_on_property
import logging

logger = logging.getLogger(__name__)

class NotificationPort:
    async def send(self, owner_id: str, message: str) -> None:
        ...

@conditional_on_property("lumen.smtp.host")
@service
class SmtplibNotificationAdapter:
    """Real email sender — only active when SMTP is configured."""

    async def send(self, owner_id: str, message: str) -> None:
        logger.info("smtp_send owner=%s", owner_id)

@conditional_on_missing_bean(NotificationPort)
@service
class LoggingNotificationFallback:
    """Log-only fallback — active whenever no real sender is wired."""
```

```

async def send(self, owner_id: str, message: str) -> None:
    logger.info("notification_fallback owner=%s", owner_id)

```

Listing 2.7 — Default-with-override using `@conditional_on_missing_bean`

How it works. Read the two classes as a chain of intent. `SmtplibNotificationAdapter` activates only when `lumen.smtp.host` is present in configuration, keeping development environments free of half-configured mail clients.

`LoggingNotificationFallback` activates whenever no real `NotificationPort` is registered — in practice, any environment where SMTP is not configured. The fallback does not check *why* the real adapter is absent; it simply fills the gap.

Any handler that injects `NotificationPort` therefore always receives *something* — no `NoSuchBeanError`, no `None` guard. In development and CI you get structured log output; in production you get real email. The choice is made entirely in configuration, with no code change and no branch in service logic.

TIP

The `@conditional_on_missing_bean` / `@conditional_on_property` pair is how all of PyFly's own auto-configuration works. Every subsystem (cache, messaging, HTTP client) ships a default bean that backs off automatically the moment you register your own implementation.

✓ What you built

Lumen now has a `WalletEntity` mapped to the `wallets` table, a `WalletRepository` that subclasses the framework's `Repository[WalletEntity, str]` (giving a full async CRUD surface, a derived query, and a specification query), and a

`DepositFundsHandler` wired to the repository, the event publisher, and the session factory — all by type hints alone. You saw why `@command_handler` and `@query_handler` **must be stacked on `@service`** — the CQRS decorators add routing metadata, but `@service` is what registers the bean. You saw that the framework auto-generates and injects the `Repository` implementation so you depend on the repository class itself by type, with no hand-written port/adaptor pair required. You also saw how `@primary` resolves ambiguity when two adaptors compete for a hand-rolled port, how `@post_construct` / `@pre_destroy` bracket a bean's life, and how `@conditional_on_missing_bean` enables defaults that automatically yield to real implementations.

The through-line is consistent: you declare intent with decorators and type hints; PyFly provides the instances. That separation lets you test each class in isolation, swap adaptors without touching business logic, and drive the full application configuration from a YAML file — all of which become essential as Lumen grows through the rest of the book.

Try it yourself

1. **Practice `@primary` with a hand-rolled port.** Define a `CacheStore` protocol with a single `async def get(key)` method. Register two `@repository` adapters — an in-process one and a stub "remote" one — both inheriting `CacheStore`. Start the application and observe the `NoUniqueBeanError`. Then add `@primary` to the in-process adapter and watch startup succeed. Next, try injecting the stub by name: annotate a constructor parameter as `Annotated[CacheStore, Qualifier("remote_cache")]` after registering it with `@repository(name="remote_cache")`.

2. Add a `@post_construct` that logs startup metadata. Extend

`WalletAuditListener` with an `async def on_ready(self)` method decorated with `@post_construct`. Inside it, log the class name of any injected dependency. Run `pyfly run --reload`, start the server, and confirm the log line appears after the framework's own startup messages.

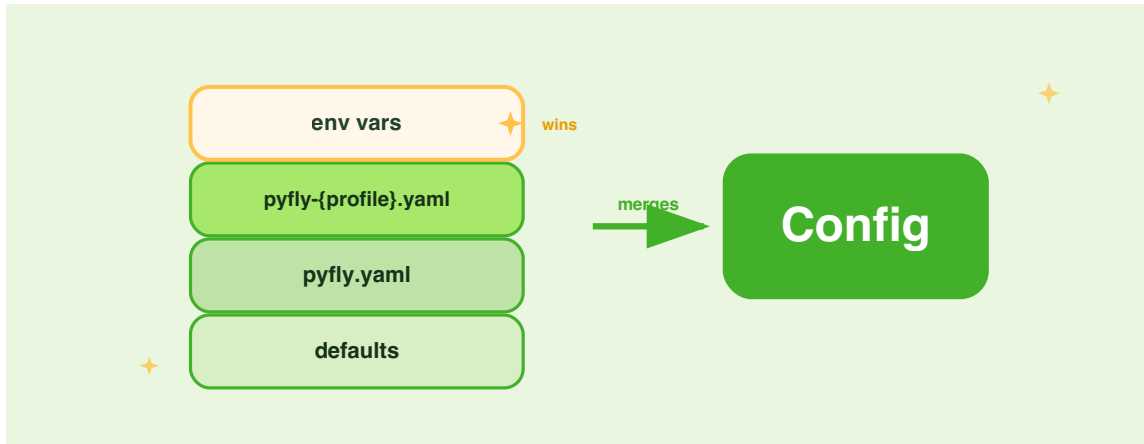
3. Make a bean conditional on a property. Add a `WalletAuditService` decorated with

`@conditional_on_property("lumen.audit.enabled", having_value="true")`.

Open `pyfly.yaml` and omit the key. Verify the service is absent from the bean list at startup. Then add `lumen.audit.enabled: "true"` to `pyfly.yaml` and re-run — confirm it appears. This is exactly how you gate optional subsystems without touching service code.

CHAPTER 3

Configuration, Profiles & Secrets



Lumen now has wired services — a `WalletService` backed by a repository, an event publisher, and a full DI lifecycle. The trouble is that every environment where Lumen runs — your laptop, a shared staging cluster, a hardened production deployment — needs different settings: different ports, different database URLs, different log verbosity, different secrets. Baking those differences into code is fragile; scattering them across a dozen `os.environ` calls is unreadable and impossible to audit.

This chapter shows how PyFly solves that with a single `pyfly.yaml`, a four-layer precedence system, environment-specific overlay files, and strongly-typed configuration classes. By the end, Lumen will have a clean configuration story that scales from `pyfly run` on your laptop to a containerised production deploy — without touching a line of business logic.

pyfly.yaml: your single source of settings

Every non-trivial application has at least two audiences for its configuration: a developer who wants verbose logs and a relaxed local database, and a production system that demands structured JSON logs, a real connection pool, and no debug mode. The naive solution — `if os.getenv("ENV") == "prod":` scattered through a dozen files — quickly becomes impossible to audit. PyFly's answer is one canonical YAML (or TOML) file that holds everything your application knows about itself, with separate mechanisms for what changes between environments.

PyFly auto-discovers this file in your project root. The framework checks candidates in order — `pyfly.yaml`, `pyfly.toml`, `config/pyfly.yaml`, `config/pyfly.toml` — and loads the first one it finds. Here is Lumen's base `pyfly.yaml`:

`pyfly.yaml`

```
pyfly:
  app:
    name: lumen
    version: 1.0.0
  banner:
    mode: console
  web:
    port: 8080
  observability:
    metrics:
      enabled: true
    tracing:
      enabled: false
  cqrs:
    enabled: true
  transactional:
    enabled: true
  persistence:
    provider: in-memory
```

```

eventsourcing:
  enabled: true
cache:
  provider: in-memory
# Event-Driven Architecture: in-memory bus (no broker needed).
# The EventPublisher bean that wallet command handlers publish
# through – and that @event_listener projections subscribe to –
# is activated by setting provider to "memory".
eda:
  provider: memory
# Relational data layer (SQLAlchemy + SQLite via aiosqlite).
# The framework creates the schema on startup (ddl-auto=create).
data:
  relational:
    enabled: true
    url: "sqlite+aiosqlite:///./lumen.db"
    ddl-auto: create

```

Listing 3.1 — Lumen's base configuration file

Three things are worth noting. First, the `pyfly:` top-level key is reserved exclusively for framework settings — web server, observability, CQRS, EDA, data access, and profiles all live there. Your own application keys go under a different top-level name (such as `lumen:`). Second, `pyfly.app.name` and `pyfly.app.version` identify the service throughout — startup banner, health endpoints, and trace metadata all read these values. Third, the `pyfly.data.relational.*` block configures the SQLAlchemy/aiosqlite layer; `url`, `ddl-auto`, and `enabled` are its three core keys.

Nested keys map directly to dot-notation access through `Config.get()`:

```

config.get("pyfly.app.name")      # "lumen"
config.get("pyfly.web.port")      # 8080
config.get("pyfly.data.relational.url") # "sqlite+aiosqlite:///./lumen.db"
config.get("pyfly.eda.provider")  # "memory"

```

`Config.get()` uses **relaxed segment matching**: `ddl-auto` and `ddl_auto` resolve to the same key. Your YAML can use kebab-case (the conventional YAML style) and your Python code can use snake_case — no need to remember which form you used in the file.

PyFly uses `PyYAML` (`yaml.safe_load`) for YAML parsing; native YAML types are preserved. The integer `8080` in YAML arrives as a Python `int` — no string parsing required.

 TIP

You can also use TOML if your team prefers INI-like syntax with strict typing. Rename the file to `pyfly.toml` and use TOML table syntax — `[pyfly.web]` , `[pyfly.data.relational]` — instead of YAML nesting. Every feature described in this chapter works identically with both formats.

How configuration is layered

A single file works well with one environment. Real projects have three or four — development, test, staging, production — and the differences between them are usually small: a database URL here, a log level there. Duplicating the entire file for each environment is a maintenance burden; the first time someone updates the port in one file and forgets the others, you have a configuration drift bug.

PyFly avoids this by layering four configuration sources, each deeply merged on top of the previous. Later layers always win:

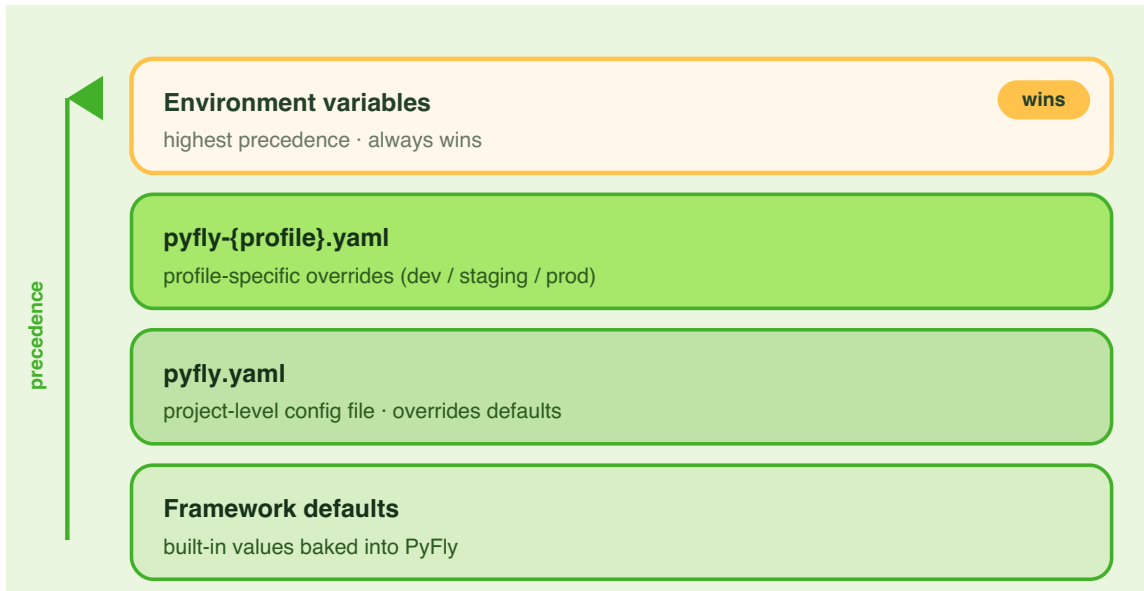


Figure 3.1 — Configuration precedence (later layers win).

Layer 1 — Framework defaults. The bundled `pyfly-defaults.yaml` inside the `pyfly.resources` package provides a sensible default for every key the framework reads. You never edit this file — it is loaded via `importlib.resources` and works correctly in packaged distributions. The framework always starts from a complete, working baseline.

Layer 2 — User configuration file. Your `pyfly.yaml` (or `pyfly.toml`). Include only the keys that differ from the framework defaults. In Listing 3.1, `pyfly.web.port: 8080` matches the default — it is included for clarity, not necessity.

Layer 3 — Profile overlay files. For each active profile, PyFly looks for a file named `pyfly-{profile}.yaml` alongside the base file and deep-merges it in. Profile overlays contain only the keys that change.

Layer 4 — Environment variables. Checked at **read time** on every `Config.get()` call, not baked in at startup. This means an env var set after the application starts still wins — the right behaviour for container deployments where secrets are injected at runtime. Environment variables always override everything else.

Deep merge, not replacement

Layers combine through a recursive deep merge (`Config._deep_merge()`). Nested dictionaries are merged key-by-key; scalar values are replaced. The distinction matters in practice: without deep merge, a production overlay that changes only the port would wipe out the `host` and `docs` keys sitting alongside it in the same `web:` section. With deep merge, you write only what you mean to change.

To make this concrete, consider a base file and a production overlay:

```
# pyfly.yaml (base)
```

```
pyfly:
```

```
  web:
```

```
    port: 8080
```

```
    host: "0.0.0.0"
```

```
    docs:
```

```
      enabled: true
```

```
# pyfly-prod.yaml (overlay)
```

```
pyfly:
```

```
  web:
```

```
    port: 443
```

After merge, the effective configuration is:

```
pyfly:
```

```
  web:
```

```
    port: 443           # overridden by prod overlay
```

```
    host: "0.0.0.0"    # preserved from base
```

```
    docs:
```

```
      enabled: true    # preserved from base
```

Only the keys that differ appear in the overlay. Everything else is preserved from the layer below.

SPRING PARITY

This four-layer model maps directly to Spring Boot's configuration hierarchy:

`application.yaml` (defaults embedded in the jar) → your `application.yaml` → `application-{profile}.yaml` → environment variables. The deep-merge behaviour, the env-var-always-wins rule, and the early profile resolution step are all deliberate parity decisions.

Profiles

The layering system provides the mechanism for varying configuration between environments. Profiles provide the vocabulary to name those environments and activate them cleanly, without any `if/else` logic in your application code.

A **profile** is a named environment variant — `dev`, `test`, `staging`, `prod`. Activating a profile loads an overlay file and can conditionally include or exclude beans.

Activating profiles

PyFly must know which profiles are active *before* loading the full configuration, because it needs to know which overlay files to merge. This **early profile resolution** follows a deliberate priority order:

1. **`PYFLY_PROFILES_ACTIVE` environment variable** — highest priority; comma-separated for multiple profiles.
2. **`pyfly.profiles.active` in the base config file** — fallback when the env var is not set.

3. **Passed programmatically** — via `Config.from_file("pyfly.yaml", active_profiles=["prod"])`.

In production, override with an env var — no code change, no file edit:

```
PYFLY_PROFILES_ACTIVE=prod python main.py
```

Profile overlay files

For each active profile `{name}`, PyFly looks for `pyfly-{name}.yaml` next to the base file. Lumen ships three overlays, each containing only the keys that differ from the base:

`pyfly-dev.yaml`

```
pyfly:
  web:
    debug: true
  data:
    relational:
      echo: true
  logging:
    level:
      root: "DEBUG"
```

Listing 3.2 — Development overlay: verbose logging, debug mode

Three keys cover everything the dev environment needs: debug mode for detailed tracebacks, SQL echo so every query appears in the terminal, and `DEBUG` log level so framework internals are visible. Everything else comes unchanged from the base file. Note that `echo` lives under `pyfly.data.relational.*`, consistent with the base file structure.

`pyfly-test.yaml`

```
pyfly:
  banner:
```

```

mode: "OFF"
data:
  relational:
    enabled: false
logging:
  level:
    root: "WARNING"

```

Listing 3.3 — Test overlay: in-memory SQLite, silent banner

The test overlay silences the startup banner so test output stays clean, disables data persistence (unit tests mock the repository layer), and raises the log threshold to **WARNING** so passing tests produce no noise.

pyfly-prod.yaml

```

pyfly:
  web:
    port: 443
    debug: false
    docs:
      enabled: false
  data:
    relational:
      enabled: true
      url: "postgresql+asyncpg://prod-db:5432/lumen"
  logging:
    level:
      root: "WARNING"
      format: "json"
  banner:
    mode: "OFF"

```

Listing 3.4 — Production overlay: real database, JSON logging, docs off

The production overlay makes several deliberate choices. It disables the interactive API docs — you do not want a live Swagger UI on a production endpoint. It switches logging to `json` format so aggregators like Datadog or CloudWatch can parse structured fields rather than scraping human-readable text. It points `pyfly.data.relational.url` at the real PostgreSQL instance. It sets `port: 443`, though in practice you will override this with `PYFLY_WEB_PORT` from your deployment pipeline so no topology details enter the repository.

TIP

Multiple profiles are comma-separated in the env var and are applied in order, so the last profile wins on conflicts: `PYFLY_PROFILES_ACTIVE=prod,metrics` first applies `pyfly-prod.yaml`, then `pyfly-metrics.yaml`. Use this to compose cross-cutting concerns — a `metrics` profile can enable Prometheus scraping without duplicating your entire prod config.

Profile-scoped beans

Sometimes the difference between environments is not a value but whether a component exists at all. A seed loader that populates test wallets must never run in production. A verbose audit logger that records every request field is useful in development but a compliance risk in prod.

The `profile` parameter on any stereotype controls when a bean participates in the container. The expression supports negation and comma-separated OR:

```
from pyfly.container import service

@service(profile="dev")
class DevSeedLoader:
    """Seeds the database with test wallets — only in dev."""
    ...
```

```
@service(profile="!prod")
class VerboseAuditLogger:
    """Detailed audit logging – active everywhere except prod."""
    ...
```

`Environment.accepts_profiles()` evaluates profile expressions during the first pass of `ApplicationContext.start()`. Beans whose expression does not match the active set are removed before any resolution takes place — never instantiated, never wired, never present in the container. The result is a container that is structurally different per environment, with no `if` statement in your application code.

Type-safe settings with `@config_properties`

String-key lookups like `config.get("pyfly.data.relational.url")` work for occasional reads, but they do not scale. Each call is an isolated read with no type information — you must remember to call `float()` on the result, and a typo in a key surfaces at the first request in production, not at startup. For anything beyond a handful of scattered values, the right approach is to group related settings into a typed Python class that is populated once at startup and injected wherever it is needed.

`@config_properties` solves exactly this by binding a config section to a typed Python dataclass.

Declaring a properties class

Decorate a `@dataclass` with `@config_properties(prefix="...")`. The `prefix` identifies the config section to bind; field names must match the keys under that section (kebab/snake interchangeable). Here is the framework's own `RelationalProperties`, which binds the `pyfly.data.relational.*` block:

pyfly/config/properties/data.py

```

from dataclasses import dataclass

from pyfly.core.config import config_properties

@config_properties(prefix="pyfly.data.relational")
@dataclass
class RelationalProperties:
    """Typed binding for pyfly.data.relational.*"""

    enabled: bool = False
    url: str = "sqlite+aiosqlite:///pyfly.db"
    echo: bool = False
    pool_size: int = 5

```

Listing 3.5 — RelationalProperties: typed settings for the data layer

The decorator sets `__pyfly_config_prefix__` on the class and marks it as an injectable bean. Field types must be `int`, `float`, `bool`, or `str` for automatic coercion; more complex types are left as-is.

Notice that each field carries a default value matching the framework's built-in `pyfly-defaults.yaml`. This is intentional: the class is self-documenting, and it can be constructed and used in unit tests without any YAML file on disk — just instantiate `RelationalProperties()` and you get the development defaults.

Apply the same pattern to your own application-level settings. Here is how a `WalletProperties` class would look for Lumen's business rules:

```

from dataclasses import dataclass
from pyfly.core import config_properties

@config_properties(prefix="lumen.wallet")

```

```
@dataclass
class WalletProperties:
    daily_transfer_limit: float = 10_000.0
    default_currency: str = "USD"
```

Add the matching block to `pyfly.yaml` under the `lumen:` top-level key (outside `pyfly:`) and the framework binds it automatically — no special registration required.

Binding and injecting

Call `config.bind(PropertiesClass)` to produce a populated, typed instance. `Config` is registered as a singleton bean, so you can inject it into any service and bind from there:

`lumen/wallet_service.py`

```
from pyfly.container import service
from pyfly.core import Config
from pyfly.config.properties import RelationalProperties
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.eda import EventPublisher

@service
class WalletService:
    def __init__(
        self,
        repo: WalletRepository,
        events: EventPublisher,
        config: Config,
    ) -> None:
        self.repo = repo
        self.events = events
        self.db: RelationalProperties = config.bind(
            RelationalProperties
        )

    async def transfer(
```

```

        self, from_id: str, to_id: str, amount: float
    ) -> dict:
        if not self.db.enabled:
            raise RuntimeError("Relational layer not enabled")
        # ... perform transfer using self.db.url for diagnostics ...
        return {"from": from_id, "to": to_id, "amount": amount}

```

Listing 3.6 — Injecting `RelationalProperties` via `config.bind()`

When the DI container starts Lumen, `WalletService.__init__` receives the shared `Config` singleton and immediately calls `config.bind(RelationalProperties)`. That call resolves the bound values once, at startup, and stores them in `self.db` — a plain Python dataclass with real types. From that point on, `transfer()` reads `self.db.enabled` as a `bool`, with full IDE autocompletion and no parsing code anywhere.

`config.bind()` works through five steps:

1. Reads `__pyfly_config_prefix__` from the class.
2. Calls `effective_section(prefix)` — a resolved copy of the subtree with `${...}` placeholders expanded, environment-variable overrides applied, and env-only keys injected.
3. Matches section keys to dataclass fields using relaxed (kebab/snake interchangeable) lookup.
4. Applies type coercion for fields whose values arrived as strings (for example, from environment variables).
5. Constructs the dataclass with the gathered kwargs; fields absent from config use their dataclass defaults.

The critical detail in step 2 is that `effective_section()` applies the full four-layer stack — defaults, file, profile overlay, env vars — before the dataclass is constructed. By the time `bind()` finishes, `RelationalProperties` reflects whatever the production overlay or a runtime env var says, not just the base YAML.

Injecting individual values with Value

For isolated settings that do not warrant a full properties class, PyFly provides a `Value` descriptor. Declare it as a class-level field and the DI container resolves it at bean creation time — exactly like Spring Boot's `@Value("${...}")`:

```
from pyfly.container import service
from pyfly.core import Value

@service
class WalletService:
    # Resolved from pyfly.app.name in the merged config.
    app_name: str = Value("${pyfly.app.name}")
    # Falls back to 10000 when the key is absent.
    transfer_limit: float = Value(
        "${lumen.wallet.daily-transfer-limit:10000}"
    )
```

`Value("${key}")` raises `KeyError` at startup when the key is missing and no default is provided — a fail-fast guarantee that keeps missing-config bugs out of production.

`Value("${key:default}")` uses the colon-delimited default when the key is absent.

Type coercion

Native YAML types arrive correctly typed — integers, booleans, and floats need no coercion. When a value arrives from an environment variable (always a string) and the target field has a non-string type, `bind()` coerces automatically:

Target type	Coercion rule
<code>int</code>	<code>int(value)</code>
<code>float</code>	<code>float(value)</code>
<code>bool</code>	<code>value.lower() in ("true", "1", "yes", "on")</code>
<code>str</code>	no coercion needed

Calling `bind()` on a class not decorated with `@config_properties` raises `ValueError` immediately — a clear fail-fast signal at startup rather than a silent wrong-value bug at request time.

🌱 SPRING PARITY

`@config_properties` is PyFly's answer to Spring Boot's `@ConfigurationProperties`. The mental model is identical: annotate a POJO (here, a dataclass) with a prefix, and the framework binds the matching config section to it with full type coercion. `Value("${...}")` maps to Spring's `@Value("${...}")` — same expression syntax, same fail-fast-on-missing guarantee. The combination of `pyfly.yaml` + profile overlays + `@config_properties` + `Value` maps to `application.yaml` + `application-{profile}.yaml` + `@ConfigurationProperties` + `@Value` — same concepts, Pythonic idioms.

Environment variables & secrets

Files are the right home for configuration that varies by environment but is safe to commit — ports, log levels, database hostnames. They are the wrong home for secrets: passwords, API keys, signing tokens, and database credentials must never enter

source control. The fourth layer of the configuration stack exists specifically to receive these values at deploy time, from a secrets manager or a CI/CD pipeline, without any of them touching the file system.

Environment variables are the fourth and highest-priority layer. PyFly checks them on every `Config.get()` call — at read time, not at startup — so they always win, even when set after the process begins.

Naming convention

Every dot-notation config key maps to a `PYFLY_`-prefixed environment variable through a mechanical three-step transformation:

1. Strip the `pyfly.` prefix (if present).
2. Replace dots (`.`) and hyphens (`-`) with underscores (`_`).
3. Uppercase the result and prefix with `PYFLY_`.

Config key	Environment variable
<code>pyfly.app.name</code>	<code>PYFLY_APP_NAME</code>
<code>pyfly.web.port</code>	<code>PYFLY_WEB_PORT</code>
<code>pyfly.web.debug</code>	<code>PYFLY_WEB_DEBUG</code>
<code>pyfly.data.relational.url</code>	<code>PYFLY_DATA_RELATIONAL_URL</code>
<code>pyfly.data.relational.pool-size</code>	<code>PYFLY_DATA_RELATIONAL_POOL_SIZE</code>
<code>pyfly.logging.level.root</code>	<code>PYFLY_LOGGING_LEVEL_ROOT</code>
<code>pyfly.eda.provider</code>	<code>PYFLY_EDA_PROVIDER</code>
<code>pyfly.profiles.active</code>	<code>PYFLY_PROFILES_ACTIVE</code>

For application-specific keys that do not start with `pyfly.`, the full dotted path is transformed the same way (no prefix stripping):

```
lumen.wallet.daily-transfer-limit
→ PYFLY_LUMEN_WALLET_DAILY_TRANSFER_LIMIT
```

The rule is consistent: when you need to tell a Kubernetes operator which env var controls a given setting, the answer is always "apply the three-step transformation" rather than hunting through framework source code.

Env vars always win

Activating the production profile and overriding the database URL for a specific container instance is a single command:

```
PYFLY_PROFILES_ACTIVE=prod \
PYFLY_DATA_RELATIONAL_URL="postgresql+asyncpg://rds-prod:5432/lumen" \
PYFLY_WEB_PORT=8080 \
python main.py
```

Here, `PYFLY_WEB_PORT=8080` overrides the prod overlay's `port: 443`. The precedence stack resolves like this:

1. Framework defaults → `port: 8080`
2. Base config → `port: 8080` (unchanged)
3. Prod overlay → `port: 443`
4. Env var → `port: 8080` (wins)

The effective port is `8080`. This is a useful pattern during a staged migration: keep `port: 443` in the overlay as the intended production default, then use a temporary env var to hold the service on `8080` for a traffic-splitting experiment. When the experiment ends, remove the env var and the overlay takes over — no file edits needed.

Keeping secrets out of files

Config files must never contain credentials, API keys, or signing secrets. The `pyfly-defaults.yaml` ships with a placeholder JWT secret (`"change-me-in-production"`) that exists only to keep the framework runnable out of the box. Replace it before going to production:

```
PYFLY_SECURITY_JWT_SECRET="$(vault kv get -field=jwt_secret secret/lumen)"
```

⚠ NEVER COMMIT SECRETS

Do not put passwords, API keys, database credentials, or JWT secrets in `pyfly.yaml` , `pyfly-prod.yaml` , or any file that enters source control. Use environment variables sourced from a secret manager (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets, or similar). The env-var layer exists precisely to receive these at deploy time, not at development time.

A note on env-only keys

`Config.bind()` also handles values that exist *only* as environment variables — no matching entry in any YAML file. `effective_section()` injects these env-only keys into the bound section so `bind()` sees the same value that `get()` would. Add a new field to a `@config_properties` class, set it exclusively via an env var in your deployment pipeline, and it is populated correctly even when the YAML files have not been updated yet:

```
# No YAML entry for pyfly.data.relational.pool-size?
# Set it exclusively via env var – bind() still picks it up.
PYFLY_DATA_RELATIONAL_POOL_SIZE=20 python main.py
```

This is a practical escape hatch during incremental rollouts: the deploying team can inject a new value before the YAML file is updated and reviewed, and the application picks it up without a code change.

✓ What you built

Lumen now has a clean configuration story across three environments. A `pyfly.yaml` holds the shared baseline — `pyfly.app.name`, `pyfly.eda.provider`, `pyfly.data.relational.*`, and every other framework knob Lumen uses. `pyfly-dev.yaml`, `pyfly-test.yaml`, and `pyfly-prod.yaml` hold only the per-environment deltas. Activating a profile takes a single env var (`PYFLY_PROFILES_ACTIVE=prod`). Typed settings live in `@config_properties` dataclasses, bound at startup with full type coercion — so services read typed fields rather than calling `float(os.environ.get(...))` in scattered service code. Individual values inject cleanly via `Value("${key}")`, which fails fast at startup when the key is missing. Secrets stay in environment variables, never in files.

The four-layer stack — defaults → file → profile overlay → env vars — gives you one mental model that works from `pyfly run` on your laptop to a locked-down container with secrets injected at deploy time, without touching a line of business logic.

✎ Try it yourself

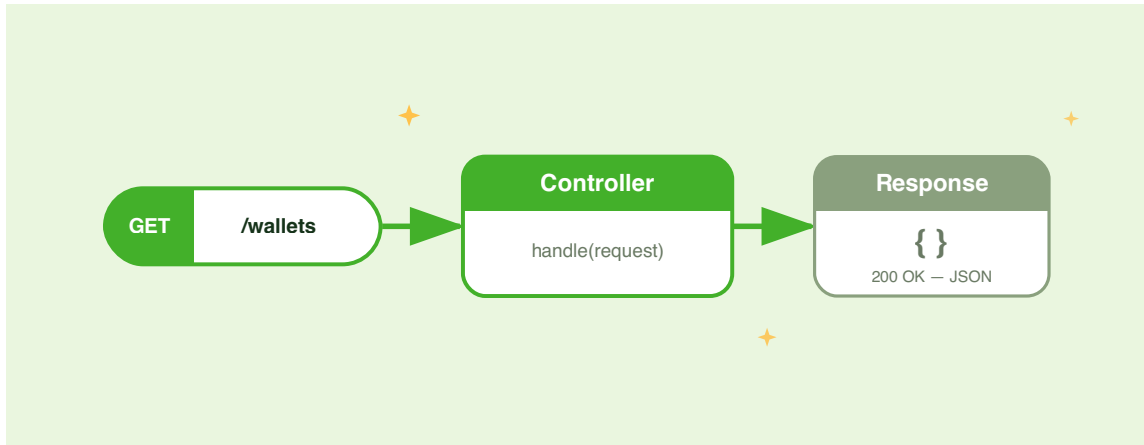
1. **Add a staging overlay.** Create `pyfly-staging.yaml` with a PostgreSQL URL for a shared test database under `pyfly.data.relational.url`, `pyfly.data.relational.enabled: true`, and logging at `INFO`. Activate it with `PYFLY_PROFILES_ACTIVE=staging python main.py` and verify from the startup log that the staging source was loaded. Compare the effective configuration to what the prod overlay would produce.

2. **Bind a new typed property and use it.** Add a `max_wallets_per_owner: int = 5` field to a new `WalletProperties` class decorated with `@config_properties(prefix="lumen.wallet")`, and a matching `lumen.wallet.max-wallets-per-owner: 5` key in `pyfly.yaml` (outside the `pyfly:` block). Inject `Config` into `WalletService`, call `config.bind(WalletProperties)`, and add a guard in `open_wallet` that raises `ValueError` when the owner already holds the maximum number of wallets. Write a quick test that overrides the limit to `1` by setting `PYFLY_LUMEN_WALLET_MAX_WALLETS_PER_OWNER=1` and verifying the error fires on the second wallet.

3. **Override a value via an env var and observe precedence.** Set `PYFLY_WEB_PORT=9090` before starting Lumen. Check the startup log and confirm the server binds to `9090`, not the `8080` in `pyfly.yaml`. Then unset the env var and restart — the port should revert to `8080`. This exercise makes the read-time nature of env-var resolution concrete: the env var always wins, and removing it immediately restores the file value without any code change.

CHAPTER 4

Your First HTTP API



Lumen has wired services, a clean configuration story, and a lifecycle that spans development through production. The only thing missing is a way for the outside world to talk to it. This chapter closes Part I by turning the wallet domain into a clean, validated REST API — one with automatic OpenAPI documentation, structured error responses that clients can trust, and the framework-managed conventions you have come to expect from the rest of PyFly.

Controllers and route mappings

Every web framework must answer two questions: how does a request find the right handler, and how does that handler get the dependencies it needs? Frameworks that answer these questions with separate mechanisms force you to maintain a router file, a DI glue file, and a documentation scaffold in three different places. PyFly collapses all three concerns into a single class.

A **controller** in PyFly is an ordinary Python class that the DI container manages and the web layer routes requests into. Two decorators mark it: `@rest_controller` from `pyfly.container` registers it as a bean and sets its stereotype; `@request_mapping` from `pyfly.web` sets the URL prefix inherited by every handler in the class.

Route handlers are plain `async def` methods, each decorated with `@get_mapping`, `@post_mapping`, `@put_mapping`, `@patch_mapping`, or `@delete_mapping`. Every mapping decorator accepts an optional relative path and an optional `status_code`. The full URL is the base path from `@request_mapping` concatenated with the relative path from the method decorator.

A pre-CQRS reading example

Chapter 7 introduces the full CQRS command/query bus that Lumen uses in production. Here in Part I the web-layer mechanics are taught on the same wallet domain, backed by a simple in-memory store instead of the bus. The controller structure, imports, and decorator shapes are *identical* to Chapter 7; only the dispatch target changes.

```
lumen/web/controllers/wallet_controller.py
```

```
from __future__ import annotations

from pyfly.container import rest_controller
from pyfly.kernel import ResourceNotFoundException
```

```

from pyfly.web import (
    Body,
    PathVar,
    QueryParam,
    Valid,
    get_mapping,
    post_mapping,
    request_mapping,
)

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto

# -----
# In-memory store (replaced by a database repository in Chapter 5)
# -----
_wallets: dict[str, WalletDto] = {}

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, inspect.

    In Part I the controller holds a minimal in-memory store so you can
    focus on the web-layer mechanics – decorators, binding, validation, and
    error handling – without persistence or CQRS machinery. Chapter 7
    replaces the store with DefaultCommandBus / DefaultQueryBus dispatching.
    """

    @post_mapping("", status_code=201)
    async def open_wallet(
        self, request: Valid[Body[OpenWalletRequest]]
    ) -> dict[str, str]:
        import uuid
        wallet_id = str(uuid.uuid4())

```

```

        wallet = WalletDto(
            id=wallet_id,
            owner_id=request.owner_id,
            currency=request.currency,
            balance_minor=0,
            balance=0.0,
            created_at=__import__("datetime").datetime.now(
                __import__("datetime").timezone.utc
            ),
        )
        _wallets[wallet_id] = wallet
        return {"wallet_id": wallet_id}

@get_mapping("/{wallet_id}")
async def get_wallet(self, wallet_id: PathVar[str]) -> WalletDto:
    result = _wallets.get(wallet_id)
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

@get_mapping("/{wallet_id}/balance")
async def get_balance(self, wallet_id: PathVar[str]) -> BalanceDto:
    wallet = _wallets.get(wallet_id)
    if wallet is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return BalanceDto(
        id=wallet.id,
        currency=wallet.currency,
        balance_minor=wallet.balance_minor,
        balance=wallet.balance,
    )

```



```

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) -> dict[str, int | str]:
    wallet = _wallets.get(wallet_id)
    if wallet is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    new_balance = wallet.balance_minor + request.amount
    _wallets[wallet_id] = wallet.model_copy(
        update={
            "balance_minor": new_balance,
            "balance": new_balance / 100,
        }
    )
    return {"wallet_id": wallet_id, "balance_minor": new_balance}

@get_mapping("")
async def list_wallets(
    self,
    owner_id: QueryParam[str] = None,
) -> list[WalletDto]:
    wallets = list(_wallets.values())
    if owner_id is not None:
        wallets = [w for w in wallets if w.owner_id == owner_id]
    return wallets

```

Listing 4.1 — WalletController using real PyFly web decorators

Four design choices in this listing are worth examining.

`@rest_controller` does two things at once: it registers `WalletController` as a singleton bean in the DI container and sets the `__pyfly_stereotype__` marker that `ControllerRegistrar` uses to discover and mount routes at startup. Pairing it with `@request_mapping("/api/v1/wallets")` means every method-level decorator inherits that prefix — you write the base path once.

This version has no `__init__` with injected collaborators. The in-memory `_wallets` dictionary is a module-level store that suffices for Part I. Chapter 5 introduces repositories; Chapter 7 shows the production pattern: a constructor that receives `DefaultCommandBus` and `DefaultQueryBus` from the DI container, dispatching commands and queries through the bus rather than reading `_wallets` directly.

Each handler returns a Pydantic model (`WalletDto` , `BalanceDto`) or a plain `dict` . The framework serialises the return value to JSON and sets the `Content-Type` header — the handler never builds a response object. The `status_code=201` argument to `@post_mapping` produces a 201 Created on success; all other handlers default to 200.

All five mapping decorators accept the same two parameters:

Parameter	Default	Description
<code>path</code>	<code>""</code>	Relative path appended to the base. Use <code>{name}</code> for path variables.
<code>status_code</code>	<code>200</code>	HTTP status code for a successful response.

`@post_mapping("", status_code=201)` maps `POST /api/v1/wallets` and returns 201 on success. `@get_mapping("/{wallet_id}")` maps `GET /api/v1/wallets/{wallet_id}` . Paths are concatenated at startup; duplicate or trailing slashes are normalised automatically.

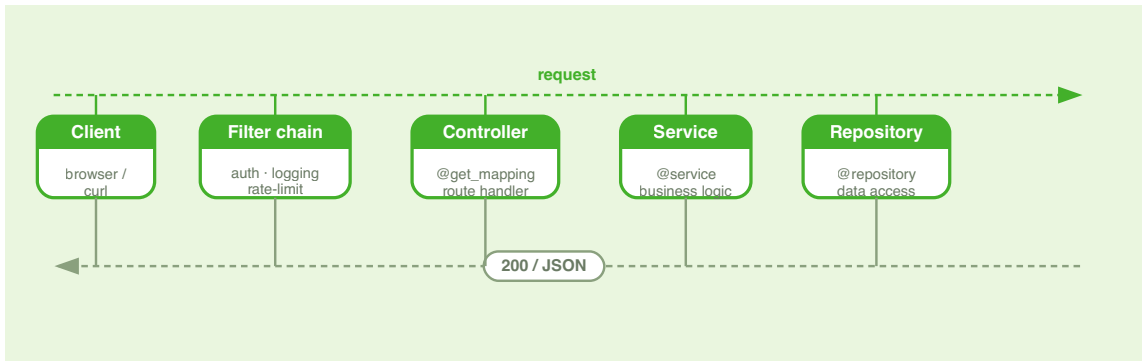


Figure 4.1 — How a request flows to your handler.

🌿 SPRING PARITY

`@rest_controller + @request_mapping + @get_mapping / @post_mapping` is a direct translation of Spring's `@RestController + @RequestMapping + @GetMapping / @PostMapping`. Handler methods return values directly (not `ResponseEntity`) and the framework converts them to JSON — exactly the pattern Spring encourages with `@ResponseBody` on `@RestController`.

Binding request data

A request carries data in several places at once: the URL path identifies the resource, the query string carries filters and pagination, the body carries the payload, and headers carry metadata. Most frameworks address these with separate mechanisms, each with its own conventions. PyFly unifies them under a single idea: **generic type annotations on handler parameters declare where data comes from**.

This approach makes handler signatures self-documenting. The parameter list of any handler tells you exactly which parts of the request it reads and what types it expects — without opening a router file or consulting the docs. The `ParameterResolver` inspects

each handler signature at startup and builds a resolution plan, so there is zero overhead per request for introspection. Five binding types cover every part of an HTTP request:

PathVar[T] — path variables

Extracts a named segment from the URL path. The parameter name must match a `{placeholder}` in the route.

```
@get_mapping("/{wallet_id}")
async def get_wallet(self, wallet_id: PathVar[str]) -> WalletDto:
    ...

@get_mapping("/{wallet_id}/transactions/{txn_id}")
async def get_transaction(
    self,
    wallet_id: PathVar[str],
    txn_id: PathVar[str],
) -> dict:
    ...
```

`PathVar` coerces the raw string segment to `T` automatically. `PathVar[int]`, `PathVar[float]`, and `PathVar[UUID]` all work — the coercion calls `int(value)`, `float(value)`, and `UUID(value)` respectively.

QueryParam[T] — query parameters

Extracts a value from the query string, with support for defaults and optional values.

```
@get_mapping("")
async def list_wallets(
    self,
    owner_id: QueryParam[str] = None,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
) -> list[WalletDto]:
    ...
```

A parameter is **required** when it has no Python default and its type does not admit `None`. A missing required `QueryParam` raises `InvalidRequestException` (HTTP 400). To make a parameter optional, give it a default value or annotate it `QueryParam[str | None]`.

Body[T] — request body

Deserialises the JSON (or XML) request body. When `T` is a Pydantic `BaseModel`, `model_validate_json()` is called automatically.

```
@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) -> dict[str, str]:
    ...
```

Header[T] and Cookie[T]

Extract values from request headers and cookies. For headers, the parameter name is converted from `snake_case` to `kebab-case` automatically:

```
@get_mapping("/me")
async def get_my_wallets(
    self,
    x_api_key: Header[str],
    session_id: Cookie[str | None],
) -> list[WalletDto]:
    ...
```

`x_api_key: Header[str]` reads the `x-api-key` header. A missing required header or cookie raises `InvalidRequestException` (HTTP 400), the same as a missing query parameter.

 TIP

All five binding types follow the same **required vs optional** rule: no default + non- `None` type = required (HTTP 400 when absent); any default or `T | None` = optional. The rule is uniform across `QueryParam`, `Header`, and `Cookie` — you learn it once, it applies everywhere.

Validation with `Valid[T]`

Binding tells the framework *where* data comes from. Validation tells it *what that data must look like* before your handler ever sees it. Without a layer that intercepts bad input early, validation logic scatters into service methods, manual `if` blocks litter business code, and different handlers produce inconsistent error responses depending on where they happen to catch the problem.

PyFly solves this at the type level. Pydantic `BaseModel` gives you field-level constraints for free. `Valid[T]` is PyFly's marker that converts a Pydantic `ValidationError` into a **structured 422 response** instead of letting it bubble up to a 500.

Pydantic DTOs for Lumen

The request and response DTOs used in Lumen's wallet API live under `lumen/interfaces/dtos/v1/` — one file per DTO. Here they are in full.

```
lumen/interfaces/dtos/v1/open_wallet_request.py
```

```
from __future__ import annotations

from pydantic import BaseModel, Field

from lumen.interfaces.enums.v1.currency import Currency
```

```

class OpenWalletRequest(BaseModel):
    """Wallet-opening request payload."""

    owner_id: str = Field(
        min_length=1,
        max_length=64,
        description="Identifier of the wallet owner",
    )
    currency: Currency = Field(
        default=Currency.EUR,
        description="ISO-4217 currency the wallet holds",
    )

```

Listing 4.2a — *OpenWalletRequest: wallet-opening payload*

`lumen/interfaces/dtos/v1/deposit_request.py`

```

from __future__ import annotations

from pydantic import BaseModel, Field

class DepositRequest(BaseModel):
    """Deposit/withdrawal request payload.

    Shared by POST /{id}/deposit and POST /{id}/withdraw – both move a
    positive amount of money in the wallet's own currency.
    """

    amount: int = Field(
        gt=0,
        description="Amount in minor units (cents); must be positive",
    )

```

Listing 4.2b — *DepositRequest: deposit/withdrawal payload*

`lumen/interfaces/dtos/v1/wallet_dto.py`

```

from __future__ import annotations

from datetime import datetime

from pydantic import BaseModel

from lumen.interfaces.enums.v1.currency import Currency


class WalletDto(BaseModel):
    """Full wallet representation returned to clients.

    ``balance_minor`` is in minor units (cents); ``balance`` is the same
    value rendered as a major-unit decimal for human-friendly display.
    """

    id: str
    owner_id: str
    currency: Currency
    balance_minor: int
    balance: float
    created_at: datetime

```

Listing 4.2c — *WalletDto: full wallet response*

`lumen/interfaces/dtos/v1/balance_dto.py`

```

from __future__ import annotations

from pydantic import BaseModel

from lumen.interfaces.enums.v1.currency import Currency


class BalanceDto(BaseModel):
    """Lightweight balance projection for the balance endpoint."""

    id: str

```



```

currency: Currency
balance_minor: int
balance: float

```

Listing 4.2d — BalanceDto: lightweight balance projection

These are pure Pydantic models — PyFly adds nothing to them. Four design decisions are worth unpacking.

`OpenWalletRequest.owner_id` uses `Field(min_length=1, max_length=64)`. The lower bound prevents phantom wallets from empty-string owner IDs that would silently pollute your data; the upper bound keeps identifiers within a sensible column width when the database layer arrives in Chapter 5.

`currency` is a `Currency` enum (a `StrEnum` with `EUR`, `USD`, `GBP`). Using an enum rather than a raw string means Pydantic rejects `"XYZ"` at deserialisation time — you never validate the currency code yourself. `Field(default=Currency.EUR)` provides a sensible default so callers can omit the field for EUR wallets.

`DepositRequest.amount` is an `int` with `Field(gt=0)`. Storing money in minor units avoids floating-point rounding: `1050` means €10.50 for an EUR wallet. The `gt=0` constraint makes a zero or negative deposit a 422 client error, not a business-logic decision — the constraint lives in the type, and Pydantic enforces it before your handler runs.

`WalletDto` and `BalanceDto` are response models. Returning a typed Pydantic model instead of a plain `dict` lets the framework generate accurate OpenAPI response schemas and gives clients a machine-readable contract.

Using `Valid[T]` in a handler

Wrap `Body[T]` in `Valid` to opt into structured 422 errors on validation failure:

```

@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) -> dict[str, str]:
    ...

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) -> dict[str, int | str]:
    ...

```

`Valid[Body[OpenWalletRequest]]` tells the resolver two things: bind from the request body (`Body`) and run Pydantic validation before the handler executes (`Valid`). When the body fails validation, the resolver catches the `ValidationError` and raises a `ValidationException` with `code="VALIDATION_ERROR"` and a `context.errors` array containing each field-level detail.

What the client sees on failure

To see validation in action, send a `POST /api/v1/wallets` with an empty `owner_id` :

```

POST /api/v1/wallets
Content-Type: application/json

{"owner_id": ""}

```

The response is HTTP 422:

```

{
  "error": {
    "message": "Validation failed: owner_id: String should have at least 1 character",
    "code": "VALIDATION_ERROR",
    "status": 422,
    "path": "/api/v1/wallets",
  }
}

```

```

"timestamp": "2026-06-07T10:30:00+00:00",
"transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"context": {
  "errors": [
    {
      "type": "string_too_short",
      "loc": ["owner_id"],
      "msg": "String should have at least 1 character",
      "input": "",
      "ctx": {"min_length": 1}
    }
  ]
}
}
}

```

Every field error carries a `type` (machine-readable), a `loc` (path to the failing field), a `msg` (human-readable), and an `input` (the rejected value). API consumers parse this array deterministically — no scraping of error strings.

The difference between bare `Body[T]` and `Valid[Body[T]]` is exactly this:

Annotation	On validation failure
<code>Body[T]</code>	Raw Pydantic <code>ValidationError</code> propagates — may become a 500 without extra handling
<code>Valid[Body[T]]</code>	Caught, converted to <code>ValidationException</code> , always produces a structured 422

Use `Valid[Body[T]]` for every endpoint that accepts user input.

🍃 SPRING PARITY

`Valid[Body[T]]` maps directly to Spring's `@Valid + @RequestBody` combination on a `@RestController` method. In Spring you write `@PostMapping public ResponseEntity create(@Valid @RequestBody OpenWalletRequest body)`; in PyFly you write `async def open_wallet(self, request: Valid[Body[OpenWalletRequest]])`. The 422 response shape (field-level errors with location paths) mirrors Spring Boot 3's `MethodArgumentNotValidException` payload.

Errors that clients can trust

A well-designed API fails loudly, consistently, and informatively. Clients must never parse exception stack traces or guess what went wrong from a generic 500. The challenge is achieving this without scattering HTTP-specific logic through your service code — the HTTP status code is an infrastructure concern, not a business one.

PyFly's exception hierarchy is the backbone of its error story. Every exception in the tree carries three things: a human-readable `message`, a machine-readable `code`, and an optional `context` dict for debugging detail. The web layer's global exception handler maps each subclass to the correct HTTP status code automatically — you `raise`, the framework responds.

The exception tree

```
PyFlyException
├── BusinessException      → 400 (catch-all)
│   ├── ValidationException → 422
│   ├── ResourceNotFoundException → 404
│   ├── ConflictException  → 409
│   ├── InvalidRequestException → 400
│   └── ...
└── SecurityException      → 403
```

```

|   |— UnauthorizedException → 401
|   |— ForbiddenException   → 403
|— InfrastructureException   → 502 (catch-all)
|   |— ServiceUnavailableException → 503
|   |— CircuitBreakerException → 503
|   |— ...

```

The hierarchy is intentionally shallow. `BusinessException` covers anything that is the caller's fault; `InfrastructureException` covers anything that is the system's fault. Subclasses pin the status code. When a new domain error does not fit an existing subclass, extend the nearest parent and the status code comes for free.

Import them from `pyfly.kernel`:

```

from pyfly.kernel import (
    ResourceNotFoundException,
    ConflictException,
    ValidationException,
    InvalidRequestException,
)

```

Raise them from handler code without worrying about HTTP:

```

raise ResourceNotFoundException(
    f"Wallet {wallet_id!r} not found",
    code="WALLET_NOT_FOUND",
    context={"wallet_id": wallet_id},
)

```

The global handler catches it, maps it to 404, and emits a structured JSON response:

```

{
  "error": {
    "message": "Wallet 'w-999' not found",
    "code": "WALLET_NOT_FOUND",
    "status": 404,
    "path": "/api/v1/wallets/w-999",
  }
}

```

```

"timestamp": "2026-06-07T10:30:00+00:00",
"transaction_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
"context": {
  "wallet_id": "w-999"
}
}
}

```

The `transaction_id` is free: the `TransactionIdFilter` assigns a UUID to every request and threads it through all error responses. Clients log it; support uses it to find the corresponding server log entry. A single ID is all that is needed to reconstruct what happened.

RFC 7807

The default error envelope — `{"error": {...}}` — is PyFly's own format. If your team prefers the IETF standard, set `pyfly.web.problem-details.enabled: true` in `pyfly.yaml`. With that flag on, the same `ResourceNotFoundException` produces an `application/problem+json` response with `type`, `title`, `status`, `detail`, and `instance` as the standard RFC 7807 members, plus `code` and `transactionId` as PyFly extension members. Both modes use the same exception hierarchy and status mapping.

Content negotiation & OpenAPI

JSON and XML

Returning a `dict` or a Pydantic model is not quite the end of the story. Somewhere between your handler's `return` statement and the bytes the client receives, the framework decides on a wire format. Rather than hardcoding JSON, PyFly runs the

return value through an ordered `HttpMessageConverter` chain — important for enterprise APIs that must serve XML partners or negotiate the lightest format for mobile clients.

JSON is the default. When no `Accept` header is sent, the response is `application/json`. When the client sends `Accept: application/xml`, the XML converter takes over and serialises the same return value as XML, with no change to your handler code:

```
GET /api/v1/wallets/w-001  Accept: application/json
→ {"id": "w-001", ...}

GET /api/v1/wallets/w-001  Accept: application/xml
→ <response><id>w-001</id>...</response>
```

The same negotiation applies to inbound data: a `Body[T]` or `Valid[Body[T]]` parameter accepts both `Content-Type: application/json` and `Content-Type: application/xml` request bodies. JSON is the fallback when no `Content-Type` header is present.

Auto-generated documentation

Manually maintained API specs drift. As routes change, parameters are renamed, and new models are added, hand-written specs fall behind the code. PyFly eliminates this entirely by generating documentation from the same metadata that drives routing — the spec is always in sync because it is the same source.

As soon as Lumen starts, three documentation endpoints are live at no cost:

Endpoint	Purpose
<code>/docs</code>	Swagger UI — interactive, try-it-now documentation
<code>/redoc</code>	ReDoc — clean, two-panel reference documentation
<code>/openapi.json</code>	Raw OpenAPI 3.0 specification

The `OpenAPIGenerator` introspects `ControllerRegistrar`'s route metadata — every path, method, path variable, query parameter, and request/response schema (from Pydantic model introspection) — and assembles the spec at startup. You never write the spec by hand. Disable it in production with `pyfly.web.docs.enabled: false` in `pyfly.yaml`.

TIP

Open `http://localhost:8080/docs` while Lumen is running. You will see `POST /api/v1/wallets`, `GET /api/v1/wallets/{wallet_id}`, `POST /api/v1/wallets/{wallet_id}/deposit`, and the others — each with the correct request and response schemas derived from your Pydantic models, and the `owner_id` query parameter on `list_wallets` already documented with its type and default.

The server underneath

Lumen now has routes, bindings, validation, and documentation. The last question is what actually listens on port 8080. The answer matters: different servers make different trade-offs in throughput, HTTP version support, OS compatibility, and ecosystem tooling. Locking an application to a single server at the framework level forces you to accept those trade-offs permanently.

PyFly does not hardcode an ASGI server. At startup, `ServerAutoConfiguration` runs a cascading selection based on what is installed:

Priority	Server	Characteristic
1st	Granian	Rust/tokio-powered; fastest single-worker throughput
2nd	Uvicorn	Ecosystem standard; best tooling support
3rd	Hypercorn	Native HTTP/2 and HTTP/3

All three start through the same `ApplicationServerPort` protocol, so your code is completely unaware of which one is running. Override with `pyfly.server.type: uvicorn` in `pyfly.yaml` or with the `--server` CLI flag:

```
pyfly run --server uvicorn --reload      # development: auto-reload
pyfly run --server granian --workers 4  # production: multi-worker
```

The event loop is pluggable too: `uvloop` (Linux/macOS) and `winloop` (Windows) are selected automatically when installed, delivering a 2–4× throughput improvement over the `asyncio` default. Install them with `uv add "pyfly[web-fast]"`.

💡 TIP

For development, `pyfly run --reload` is all you need — it picks the best available server and event loop automatically. For production, `pyfly run --server granian --workers 0` resolves `0` to the CPU count, maximising throughput. CLI flags always override `pyfly.yaml`.

✓ What you built

Part I is complete.

In four chapters you went from an empty scaffold to a production-shaped service. Lumen now **boots** (`@pyfly_application` , startup banner, structured logging), is **wired** (services and repositories connected through constructor injection with no glue code), **configured** (four-layer `pyfly.yaml` + profile overlays + env-var secrets, typed `WalletProperties`), and **serves** — a validated REST API at `/api/v1/wallets` with `PathVar` , `QueryParam` , and `Valid[Body[T]]` body binding; structured 422 errors from Pydantic constraints; domain-error-to- status mapping from the exception hierarchy; typed response models (`WalletDto` , `BalanceDto`) that drive OpenAPI schema generation; and a pluggable ASGI server running underneath.

Every part of this stack follows the same hexagonal principle you have seen throughout: your code depends on ports and decorators; the framework wires the adapters. Swap the in-memory store for a PostgreSQL adapter in Chapter 5, replace direct dispatch with a full CQRS bus in Chapter 7, or enable XML responses — none of it requires touching the controller's decorator structure or the DTO shapes.

Part II takes Lumen further: persistent data with SQLAlchemy, domain events, resilience with circuit breakers, and security with JWT. The foundations you built here carry forward intact.

Try it yourself

1. **Add a `DELETE /api/v1/wallets/{wallet_id}` endpoint.** Remove the wallet from `_wallets` and return 204 No Content. Raise `ResourceNotFoundException` if the wallet does not exist. Decorate with `@delete_mapping("/{wallet_id}", status_code=204)` — PyFly converts a `None` return with `status_code=204` into a 204 response with no body. Verify with `curl -X DELETE http://localhost:8080/api/v1/wallets/{id}`.

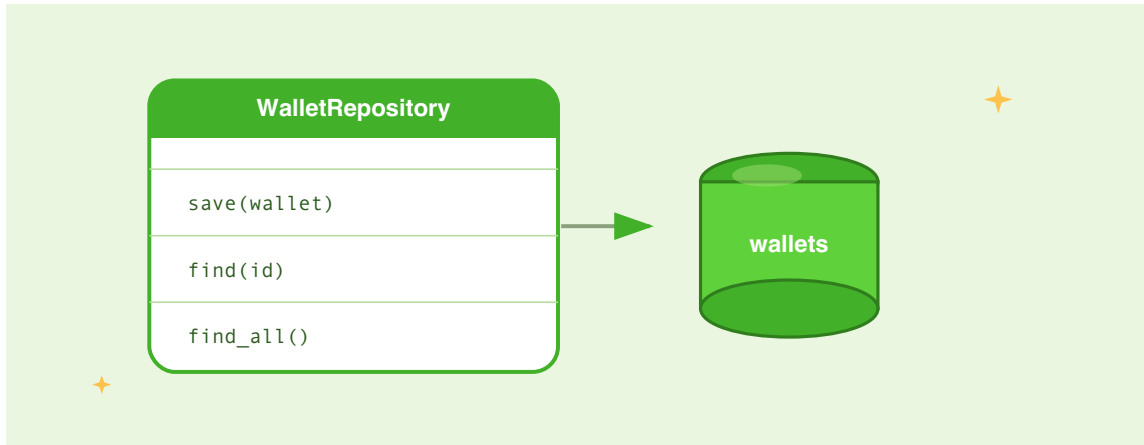
2. **Add currency filtering to `list_wallets`**. Add a `currency: QueryParam[str] = None` parameter and filter `_wallets.values()` when it is not `None`. Test with `GET /api/v1/wallets?currency=EUR` and confirm only EUR wallets are returned; confirm `GET /api/v1/wallets` without the parameter returns all wallets. Then make the parameter required by removing the default — observe the 400 response when you omit it from the request.
3. **Raise a domain-specific subclass of `ResourceNotFoundException`**. Create a `WalletNotFoundError` that subclasses `ResourceNotFoundException` and carry an extra `currency` field in `context`. Raise it from `get_wallet` and `get_balance` instead of bare `ResourceNotFoundException`. Verify the JSON error response includes the extra context field without any change to the global handler — the hierarchy maps it to 404 automatically.

PART II

Modeling & Persisting the Domain

CHAPTER 5

Persistence & the Repository Pattern



Lumen has a wallet API that works — but every wallet disappears the moment you restart the process. It is time to make wallets durable.

The naïve approach is to scatter SQLAlchemy `select()` and `session.commit()` calls through the command handlers. PyFly offers something far better: a **Spring-Data-style repository layer**. You declare an interface — `class WalletRepository(Repository[WalletEntity, str])` — and the framework *implements it for you*. Full async CRUD comes for free. Query methods are derived from their **names**. Pagination, sorting, composable filters, and read projections are first-class. There is no hand-written adapter and no SQL in the application code.

This chapter rebuilds Lumen's persistence on that layer, exactly as the running sample does it: the SQLAlchemy entity, the repository with its derived and specification queries, `Page / Pageable / Sort`, projections for read views, and the transaction seam that keeps the `Wallet` aggregate intact. Everything here runs against a real SQLite file with zero external infrastructure — the sample's 41 tests are green on it.

The repository, in one sentence

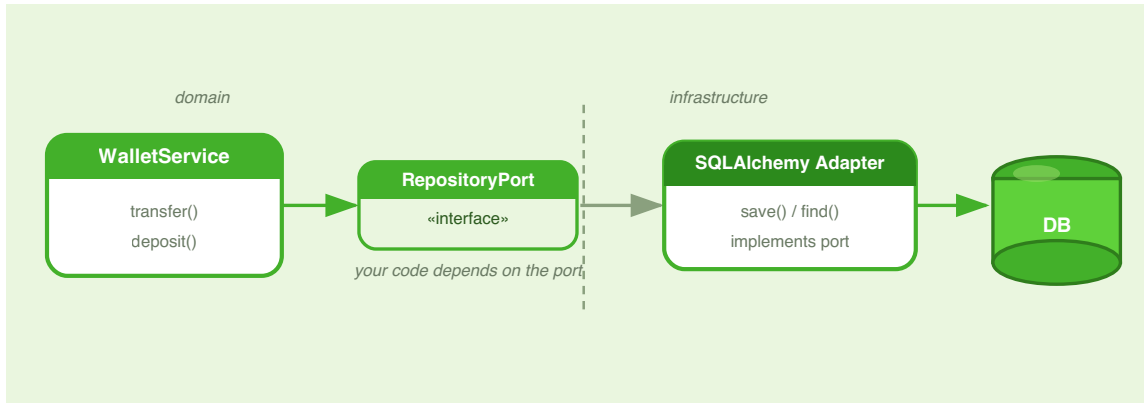


Figure 5.1 — Your code depends on the repository; the framework supplies the SQLAlchemy implementation behind it.

A PyFly repository is a class that subclasses the generic `Repository[Entity, ID]` and is marked with the `@repository` stereotype. That is the whole declaration. From the two type parameters the framework learns the **entity type** and the **primary-key type**, and from there it provides a complete async data-access surface — `save`, `find_by_id`, `find_all`, `delete / delete_by_id`, `count`, `exists_by_id`, plus pagination and specification queries — with the database `AsyncSession` injected for you.

This is the Repository pattern as Spring Data popularised it, translated to idiomatic async Python. You write *what* you want (the method) and the framework writes *how* (the SQL).

🌿 SPRING PARITY

`Repository[T, ID]` is PyFly's `JpaRepository<T, ID>`. Subclassing it to inherit CRUD, deriving queries from method names, `Pageable / Page`, `Specification`, and interface projections are all carried over almost name-for-name from Spring Data JPA. If you have written a Spring `interface OrderRepository` extends `JpaRepository<Order, UUID>`, you already know the shape of this chapter.

The entity: one row per wallet

Before a repository can store anything, it needs an **entity** — the on-disk shape of a wallet, one flat row per aggregate. PyFly entities are ordinary SQLAlchemy 2.0 models built on a declarative base the framework exports:

`lumen/models/entities/v1/wallet_orm.py`

```
from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's string id."""
```

```

__tablename__ = "wallets"

id: Mapped[str] = mapped_column(String(64), primary_key=True)
owner_id: Mapped[str] = mapped_column(
    String(255), nullable=False, index=True
)
currency: Mapped[str] = mapped_column(String(3), nullable=False)
balance_minor: Mapped[int] = mapped_column(nullable=False, default=0)
created_at: Mapped[datetime] = mapped_column(
    default=lambda: datetime.now(UTC)
)

```

Listing 5.1 — *WalletEntity*: the SQLAlchemy persistence row

The `Mapped[T]` / `mapped_column(...)` syntax is SQLAlchemy 2.0 style: each type annotation drives both the Python attribute type and the generated column DDL, so every column has a single source of truth. Because `WalletEntity` subclasses `Base`, importing this module registers the `wallets` table in `Base.metadata`; the framework's engine lifecycle then creates it on startup.

Two design choices are worth calling out.

Base, not BaseEntity. PyFly ships two declarative bases. `BaseEntity` gives you a surrogate **UUID** primary key plus four audit columns (`created_at`, `updated_at`, `created_by`, `updated_by`) populated automatically — the right default for most tables. Lumen deliberately uses plain `Base` instead, because the `Wallet` aggregate already owns its identity: a string id of the form `wlt-...`. Inheriting `Base` lets the row keep that **string** primary key, so the row and the aggregate share one identity rather than the row inventing a second, surrogate one.

Integer minor units. Amounts live in `balance_minor` as integer cents, never as a float. Floating-point columns accumulate rounding error over millions of transactions; integer arithmetic stays exact. A balance of `2500` means €25.00 — the major-unit decimal is computed only at the edges, for display.

💡 REACH FOR BASEENTITY BY DEFAULT

Unless your aggregate owns a natural key the way `Wallet` does, prefer `class Order(BaseEntity)`. You get a UUID PK and audit columns for free, and the `AuditingEntityListener` fills `created_by` / `updated_by` from the security context on every insert and update. Lumen is the exception, not the rule.

The repository: CRUD for free

Now the centrepiece. Lumen's `WalletRepository` subclasses `Repository[WalletEntity, str]` — entity type `WalletEntity`, primary-key type `str` — and is registered with `@repository`. That single declaration is enough for the framework to supply the entire CRUD surface:

`lumen/models/repositories/wallet_repository.py`

```
from __future__ import annotations

from lumen.models.entities.v1.wallet_orm import WalletEntity
from pyfly.container import repository
from pyfly.data import Page, Pageable
from pyfly.data.relational.sqlalchemy import Repository, Specification

@repository
class WalletRepository(Repository[WalletEntity, str]):
    """CRUD + derived + specification queries for WalletEntity.
```

The @repository stereotype registers this as a DI bean. The framework reads the entity/PK types from the Repository[WalletEntity, str] base and injects the shared AsyncSession.

"""

(query methods follow – see the next sections)

Listing 5.2 — WalletRepository: subclass the framework repository

There is no `__init__`, no SQL, and no adapter class. With just that declaration, any handler that injects a `WalletRepository` can already call:

Method	Returns	What it does
<code>save(entity)</code>	<code>T</code>	Insert or update; flushes + refreshes
<code>find_by_id(id)</code>	<code>T \ None</code>	Load by primary key
<code>find_all(**filters)</code>	<code>list[T]</code>	All rows, optional equality filters
<code>find_all(sort)</code>	<code>list[T]</code>	All rows in a given <code>Sort</code> order
<code>find_all(pageable)</code>	<code>Page[T]</code>	Paged + sorted query (see below)
<code>stream_all(sort)</code>	<code>AsyncIterator[T]</code>	Stream every row (the <code>Flux<T></code> analogue)
<code>delete(entity)</code>	<code>None</code>	Delete a given entity
<code>delete_by_id(id)</code>	<code>None</code>	Delete by primary key (no-op if absent)
<code>delete_all(entities=None)</code>	<code>None</code>	Delete the given entities (or all rows)
<code>delete_all_by_id(ids)</code>	<code>None</code>	Delete many rows by primary key
<code>count()</code>	<code>int</code>	Count every row in the table
<code>exists_by_id(id)</code>	<code>bool</code>	Whether a row with this id exists
<code>save_all(entities)</code>	<code>list[T]</code>	Bulk insert/update
<code>find_all_by_id(ids)</code>	<code>list[T]</code>	Load many rows by primary key
<code>find_all_by_spec(spec)</code>	<code>list[T]</code>	Rows matching a <code>Specification</code>

Method	Returns	What it does
<code>find_all_by_spec_paged(...)</code>	<code>Page[T]</code>	Paged + sorted <code>Specification</code> query

That is more than enough for most entities. Lumen adds three methods of its own on top — a derived query, a specification query, and an upsert — which the next sections build up.

How the framework knows the types

When you write `Repository[WalletEntity, str]`, the base class's `__init_subclass__` hook inspects `__orig_bases__` at class-definition time and pulls the entity type (`WalletEntity`) and id type (`str`) out of the generic parameters. The `AsyncSession` is then supplied as an injected dependency by the relational auto-configuration. Nothing is passed manually — the type parameters *are* the wiring.

Derived queries: the method name is the query

CRUD covers lookups by primary key. Real applications also need to query by other columns — "all wallets owned by this customer." In most frameworks you would write the SQL by hand. In PyFly you declare a **stub** and let the framework compile the query *from the method name*:

```
lumen/models/repositories/wallet_repository.py
```

```
@repository
class WalletRepository(Repository[WalletEntity, str]):

    # derived query: compiled from the method name by the post-processor
    async def find_by_owner_id(
        self, owner_id: str
```

```

) -> list[WalletEntity]:
  """All wallets owned by *owner_id* (derived query stub)."""
  ...

```

Listing 5.3 — A derived query: declared as a stub, compiled from its name

The body is literally `...`. At startup a `BeanPostProcessor` — the `RepositoryBeanPostProcessor` — scans the repository, spots that `find_by_owner_id` is a stub, parses the **name** into a parsed query, and replaces the stub with a real implementation that runs `SELECT ... FROM wallets WHERE owner_id = :owner_id`. Calling `await repo.find_by_owner_id("alice")` now returns exactly the rows for that owner.

The grammar is the Spring Data convention. A method name is a **prefix** followed by a **subject** built from field names, operators, connectors, and an optional ordering clause:

Part	Tokens
Prefix	<code>find_by</code> · <code>count_by</code> · <code>exists_by</code> · <code>delete_by</code>
Connectors	<code>_and_</code> · <code>_or_</code>
Operators	<code>_greater_than</code> · <code>_less_than</code> · <code>_between</code> · <code>_in</code> · <code>_like</code> · <code>_containing</code> · <code>_is_null</code> · <code>_is_not_null</code>
Ordering	<code>_order_by_<field>_<asc\ desc></code>

Each clause consumes the matching number of method arguments (equality and the comparisons take one; `_between` takes two; `_is_null` / `_is_not_null` take none). A few examples on a hypothetical orders repository:

```

@Repository
class OrderRepository(Repository[Order, UUID]):
  async def find_by_status(self, status: str) -> list[Order]: ...

```

```

async def find_by_customer_id_and_status(
    self, customer_id: str, status: str
) -> list[Order]: ...

async def find_by_total_greater_than(
    self, min_total: float
) -> list[Order]: ...

async def find_by_total_between(
    self, low: float, high: float
) -> list[Order]: ...

async def count_by_status(self, status: str) -> int: ...

async def exists_by_customer_id(self, customer_id: str) -> bool: ...

async def find_by_status_order_by_created_at_desc(
    self, status: str
) -> list[Order]: ...

```

The prefix decides the *shape* of the result: `find_by` returns a list, `count_by` returns an `int`, `exists_by` returns a `bool`, and `delete_by` issues a `DELETE` and returns the number of rows removed. You never write the SQL; you name the method and annotate the return type.

💡 WHEN A NAME WOULD GET SILLY, USE @QUERY

Derived names are perfect up to two or three predicates. Past that they become unreadable. For anything more complex, drop a `@query("SELECT w FROM WalletEntity w WHERE ...")` decorator (JPQL-like, or `native=True` for raw SQL) on the stub and write the query explicitly. Same stub-plus-decorator pattern; you just supply the query text instead of encoding it in the name.

Pagination: Page, Pageable, and Sort

A list endpoint should never return *every* wallet. PyFly's pagination types — `Pageable` (what page, what size, what sort), `Sort` (the ordering), and `Page[T]` (the slice plus metadata) — are inherited straight from the CRUD surface via `find_all(pageable)`.

Lumen's `ListWallets` query handler is the whole story in three lines:

`lumen/core/services/wallets/list_wallets_handler.py`

```
@query_handler
@service
class ListWalletsHandler(
    QueryHandler[ListWallets, Page[WalletDto]]
):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle( # type: ignore[override]
        self, query: ListWallets
    ) -> Page[WalletDto]:
        page = await self._repository.find_all(
            query.pageable
        )
        return page.map(entity_to_dto)
```

Listing 5.4 — Paginating with `find_all(pageable)`, then mapping the page

`find_all(pageable)` does three things in one call: it counts the total number of matching rows, applies the `Pageable`'s sort, and slices the result with `LIMIT / OFFSET`. It hands back a `Page[WalletEntity]`. The handler then calls `page.map(entity_to_dto)` to turn each row into a `WalletDto` **without losing the pagination metadata** — `.map` carries `total`, `page`, `size`, and the rest across to the new page.

A `Page[T]` exposes everything a client needs to render a pager:

Member	Meaning
<code>items</code>	The rows on this page (<code>list[T]</code>)
<code>total</code>	Total matching rows across all pages
<code>page</code>	Current page number (1-based)
<code>size</code>	Maximum items per page
<code>total_pages</code>	<code>ceil(total / size)</code>
<code>has_next</code>	Whether a next page exists
<code>has_previous</code>	Whether a previous page exists
<code>map(fn)</code>	Transform items, preserving metadata

The `Pageable` itself is built at the edge — the controller turns `?page=&size=` query params into a `Pageable` with a shared newest-first `Sort`:

`lumen/web/controllers/wallet_controller.py`

```
#: Newest-first ordering shared by the list endpoints.
_NEWEST_FIRST = Sort.by("created_at").descending()

@get_mapping("")
async def list_wallets(
    self, page: QueryParam[int] = 1, size: QueryParam[int] = 20
) -> PageDto[WalletDto]:
    """A page of wallets, newest first."""
    result = await self.queries.query(
        ListWallets(pageable=Pageable.of(page, size, _NEWEST_FIRST))
```



```
)
return PageDto.from_page(result)
```

Listing 5.5 — Building a Pageable from query params (controller)

`Sort.by("created_at").descending()` names the column and the direction; `Pageable.of(page, size, sort)` packages it with the page coordinates. The handler returns a framework `Page`, and the controller folds it into a serialisable `PageDto` — a plain Pydantic mirror of the page — so `GET /api/v1/wallets?page=1&size=20` returns JSON like `{"items": [...], "total": 42, "page": 1, "total_pages": 3, "has_next": true, ...}`.

Specifications: composable, reusable filters

Derived queries answer fixed questions. Sometimes you want a **reusable predicate** you can compose at the call site — "wallets with at least this balance," combined freely with other conditions. That is what a `Specification` is: a small object wrapping a `WHERE` fragment, composable with `&`, `|`, and `~`.

Lumen defines one as a module-level factory and uses it in a `find_rich` method:

```
lumen/models/repositories/wallet_repository.py
```

```
def balance_at_least(min_minor: int) -> Specification[WalletEntity]:
    """Wallets whose balance is at least *min_minor*.

    Returned as a Specification, so it composes via & / | / ~ and
    runs through find_all_by_spec / find_all_by_spec_paged.
    """
    return Specification(
        lambda root, q: q.where(root.balance_minor >= min_minor)
    )
```

```

@repository
class WalletRepository(Repository[WalletEntity, str]):

    async def find_rich(
        self, min_minor: int, pageable: Pageable
    ) -> Page[WalletEntity]:
        """A page of wallets with balance >= min_minor."""
        return await self.find_all_by_spec_paged(
            balance_at_least(min_minor), pageable
        )

```

Listing 5.6 — A Specification factory and a method that runs it paged

A `Specification` wraps a callable `(root, q) -> q` — given the entity class (`root`) and a SQLAlchemy `Select`, it returns the statement with a predicate added.

`balance_at_least(1000)` yields the predicate `balance_minor >= 1000`. Because specifications compose with Python operators, you can build arbitrarily complex filters from small pieces:

```

rich = balance_at_least(1000)
in_eur = Specification(
    lambda root, q: q.where(root.currency == "EUR")
)
rich_eur = rich & in_eur          # AND
rich_or_eur = rich | in_eur      # OR
not_rich = ~rich                 # NOT

```

You run a specification two ways. `find_all_by_spec(spec)` returns every matching row as a list; `find_all_by_spec_paged(spec, pageable)` applies the predicate, counts the matches, sorts, and slices — returning a `Page[T]`. `find_rich` uses the paged form, so the rich-wallets endpoint is itself paginated. The handler mirrors the list handler exactly, mapping rows to DTOs:

```
lumen/core/services/wallets/list_rich_wallets_handler.py
```

```

@query_handler
@service
class ListRichWalletsHandler(
    QueryHandler[ListRichWallets, Page[WalletDto]]
):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle( # type: ignore[override]
        self, query: ListRichWallets
    ) -> Page[WalletDto]:
        page = await self._repository.find_rich(
            query.min_minor, query.pageable
        )
        return page.map(entity_to_dto)

```

Listing 5.7 — The rich-wallets handler runs the *Specification* path

`GET /api/v1/wallets/rich?min_minor=1000&page=1&size=20` now returns a page of wallets at or above €10.00, newest first.

❗ FILTERS WITHOUT LAMBDA

For the common case — equality and a handful of comparisons — you don't even need to write a lambda. `FilterOperator.gte("balance_minor", 1000) & FilterOperator.eq("currency", "EUR")` produces the same composable *Specification* from static factory methods, and `FilterUtils.by(currency="EUR")` builds one from keyword arguments (Query-by-Example). Lumen uses an explicit lambda here because the intent reads clearly; both styles produce a *Specification* you can pass to the same repository methods.

Projections: read only the columns you need

The balance endpoint does not need the whole row — just the id, currency, and a computed balance. PyFly supports **interface projections**, Spring Data's idea of declaring the subset of fields a read-view wants and letting the framework copy exactly those.

A projection is a class marked `@projection`. In Lumen it is a concrete dataclass:

`lumen/interfaces/dtos/v1/balance_dto.py`

```
from dataclasses import dataclass

from pyfly.data import projection

@projection
@dataclass
class BalanceView:
    """Projection: just the fields the balance view needs.

    id, currency and balance_minor are copied straight from the
    WalletEntity; balance is a computed major-unit decimal supplied
    by a registered transform on the mapper.
    """

    id: str
    currency: str
    balance_minor: int
    balance: float
```

Listing 5.8 — *BalanceView: a @projection of just the balance fields*

The mapper reads those four fields off a `WalletEntity` and constructs the view. Three (`id`, `currency`, `balance_minor`) are copied straight across; the fourth (`balance`, the major-unit decimal) is supplied by a transform registered on the mapper:

lumen/core/mappers/wallet_mapper.py

```

from pyfly.data import Mapper

_mapper = Mapper()
_mapper.register_projection(
    WalletEntity,
    BalanceView,
    transforms={"balance": lambda e: round(e.balance_minor / 100, 2)},
)

def entity_to_balance_dto(entity: WalletEntity) -> BalanceDto:
    """Project a row onto the balance DTO via the projection."""
    view = _mapper.project(entity, BalanceView)
    return BalanceDto(
        id=view.id,
        currency=Currency(view.currency),
        balance_minor=view.balance_minor,
        balance=view.balance,
    )

```

Listing 5.9 — Registering and running the projection via Mapper

`Mapper.project(entity, BalanceView)` reads only the declared fields, applies the `balance` transform, and returns a `BalanceView`. The query handler then loads the row by id and projects it:

lumen/core/services/wallets/get_balance_handler.py

```

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

```

```

async def do_handle( # type: ignore[override]
    self, query: GetBalance
) -> BalanceDto | None:
    entity = await self._repository.find_by_id(query.wallet_id)
    return (
        entity_to_balance_dto(entity)
        if entity is not None
        else None
    )

```

Listing 5.10 — The balance read handler: find by id, then project

⚠ A PROJECTION MUST BE INSTANTIABLE

Spring lets a projection be a bare interface and returns a runtime proxy. Python has no such proxy, so a PyFly projection must be a **concrete** type the mapper can construct — here, a `@dataclass`. Marking a `Protocol` `@projection` will not work: a `Protocol` cannot be instantiated, and `Mapper.project` has nothing to build. Use a `dataclass` (or any plain class with matching fields) and you are safe.

Transactions and the aggregate seam

The repository surface is clean — but two honest subtleties decide whether your writes actually survive. Both come from how the framework manages the session, and Lumen handles both deliberately.

save() flushes; it does not commit

This is the single most important thing to understand about the data layer. The framework uses **one shared** `AsyncSession`, and `Repository.save()` calls `session.add()` followed by `session.flush()` and `session.refresh()` — it **flushes**,

making the write visible *within* the current session, but it never **commits**. If nothing commits, the write is rolled back when the session closes and the wallet does not survive a restart.

The commit happens at the **unit-of-work boundary**, and you declare that boundary with `@transactional()`. A handler that writes decorates its `do_handle` with `@transactional()`, injects the `async_sessionmaker` as `self._session_factory`, and the decorator opens a unit of work, swaps that transactional session onto the repository for the call, **commits on success**, and rolls back on failure:

`lumen/core/services/wallets/open_wallet_handler.py`

```
@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle( # type: ignore[override]
        self, command: OpenWallet
    ) -> str:
        wallet_id = f"wlt-{uuid4()}"
        wallet = Wallet.open(
            wallet_id=wallet_id,
```

```

        owner_id=command.owner_id,
        currency=command.currency,
    )
    await self._repository.upsert(to_entity(wallet))

    await publish_domain_events(
        self._events, wallet.clear_events()
    )
    return wallet_id

```

Listing 5.11 — A write handler: `@transactional()` commits the unit of work

`@transactional()` (imported from `pyfly.data.relational.sqlalchemy`) resolves the `async_sessionmaker` from `self._session_factory`, runs the body inside a `session.begin()` block, and commits at the end. Drop the decorator and the `upsert` would only flush — the wallet would never reach disk. The read handlers earlier in this chapter need no `@transactional`: a read makes no changes to commit.

upsert, not save, for an aggregate that owns its id

Notice the handler calls `self._repository.upsert(...)`, not `save(...)`. That is the second subtlety. The framework's `save()` issues `session.add()`, which SQLAlchemy treats as a **pending INSERT**. But the `Wallet` aggregate generates its *own* primary key up front (`wlt-...`), so by the time a deposit or withdrawal persists an already-loaded wallet, a row with that id already exists — and a second **INSERT** on the same primary key raises `IntegrityError`.

The fix is `session.merge`, which inserts when the id is new and updates when it already exists. Lumen wraps it in an `upsert` convenience method:

`lumen/models/repositories/wallet_repository.py`

```

@repository
class WalletRepository(Repository[WalletEntity, str]):

```



```

async def upsert(self, entity: WalletEntity) -> WalletEntity:
    """Insert *entity* or update the existing row with the same id.

    Uses session.merge so a freshly-mapped entity carrying the
    aggregate's id persists whether or not a row already exists –
    the aggregate owns its primary key, so identity is never
    ambiguous. Flushes so the write is visible in the current
    unit of work; the surrounding @transactional commits it.
    """

    session = self._require_session()
    merged = await session.merge(entity)
    await session.flush()
    return merged

```

Listing 5.12 — *upsert*: one call for both INSERT and UPDATE

`_require_session()` is the inherited accessor that returns the active session (the transactional one, once `@transactional` has swapped it in). `merge` keys on the primary key, so both the first write (open) and every later write (deposit, withdraw) take the same code path with no `IntegrityError`. For entities whose ids are database-generated, `save` is the natural choice; for an aggregate that owns its id, `upsert` is.

The aggregate ↔ entity mapper seam

There is one more boundary, and it is a feature, not an accident. Lumen keeps two distinct types:

- **Wallet** — the DDD *aggregate root* from Chapter 6. It owns the `balance >= 0` invariant, exposes intent-revealing methods (`open`, `deposit`, `withdraw`), and raises domain events. It knows nothing about SQLAlchemy.
- **WalletEntity** — the *persistence row*. It is a flat SQLAlchemy model with columns and no behaviour.

A small mapper bridges them — one pure function each way:

lumen/core/mappers/wallet_mapper.py

```

def to_entity(wallet: Wallet) -> WalletEntity:
    """Flatten a Wallet aggregate into a persistable row."""
    assert wallet.id is not None
    return WalletEntity(
        id=wallet.id,
        owner_id=wallet.owner_id,
        currency=wallet.currency.value,
        balance_minor=wallet.balance.amount,
        created_at=wallet.created_at,
    )

def to_aggregate(entity: WalletEntity) -> Wallet:
    """Rehydrate a Wallet aggregate from a persistence row."""
    currency = Currency(entity.currency)
    return Wallet(
        id=entity.id,
        owner_id=entity.owner_id,
        balance=Money(amount=entity.balance_minor, currency=currency),
        created_at=entity.created_at,
    )

```

Listing 5.13 — The aggregate ↔ row mapper

The write side calls `to_entity` before `upsert`; the read side either rehydrates with `to_aggregate` (when a command needs the rich aggregate) or projects straight to a DTO (when a query only needs data). Keeping the row separate from the aggregate means persistence concerns — column types, nullability, the merge dance — never leak into the domain model, and the domain's invariants never constrain the table schema. The repository stores rows; the mapper is the seam that keeps the aggregate pure.

i REHYDRATION SKIPS THE FACTORY

`to_aggregate` calls the `Wallet` **constructor** directly, never the `Wallet.open` factory. The factory is for *new* wallets: it validates inputs and raises a `WalletOpened` event. A row loaded from the database already represents a valid, committed wallet — re-running the factory would re-fire that event and re-check rules that passed long ago. The constructor sets fields quietly, producing a `Wallet` indistinguishable from a freshly opened one but with no spurious events.

Turning it on

Activating the relational layer is configuration, not code. Lumen's `pyfly.yaml` carries a `data.relational` block:

`pyfly.yaml`

```
pyfly:
  data:
    relational:
      enabled: true
      url: "sqlite+aiosqlite:///./lumen.db"
      ddl-auto: create
```

Listing 5.14 — Relational data layer configuration

`enabled: true` activates the relational auto-configuration, which builds the async SQLAlchemy engine and the `async_sessionmaker`, registers the `AsyncSession` and `session_factory` beans the repository and handlers inject, and installs the `RepositoryBeanPostProcessor` that compiles your derived-query stubs. `url` is a standard SQLAlchemy connection string — SQLite via `aiosqlite` here for zero-infrastructure development, `postgresql+asyncpg://...` in production. `ddl-auto:`

`create` runs `Base.metadata.create_all` on startup, so the `wallets` table (discovered because `WalletEntity` subclasses `Base`) is built automatically the first time the app boots.

The dependency footprint is tiny: `pyfly[data-relational]` pulls in `sqlalchemy[asyncio]` and `aiosqlite`, and nothing else. No database server, no driver install — which is exactly why the sample runs anywhere.

💡 SCHEMA LIFECYCLE IN PRODUCTION

`ddl-auto: create` is right for development and samples: it creates missing tables and leaves existing ones alone. In production set `ddl-auto: none` and manage the schema with a migration tool (Alembic), which generates versioned scripts from the diff between `Base.metadata` and the live database. The application code does not change — only the `ddl-auto` setting and the migration pipeline.

Proving it works

Because the repository is an ordinary class, you can test it directly against a real SQLite file — no application context, no HTTP. Lumen's repository test creates a temp database, runs `Base.metadata.create_all`, and exercises the surface end to end, including the `RepositoryBeanPostProcessor` that compiles the derived query (the same processor the live `ApplicationContext` runs):

`lumen/tests/test_sql_wallet_repository.py`

```
def _make_repo(session: AsyncSession) -> WalletRepository:
    repo = WalletRepository(WalletEntity, session)
    # Mirror the ApplicationContext: compile derived-query stubs.
    RepositoryBeanPostProcessor().after_init(repo, "walletRepository")
    return repo
```

```

@pytest.mark.asyncio
async def test_derived_find_by_owner_id(sqlite_factory) -> None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        await repo.upsert(_entity("wlt-1", "alice", 100))
        await repo.upsert(_entity("wlt-2", "alice", 200))
        await repo.upsert(_entity("wlt-3", "bob", 300))
        await session.commit()

    owned = await repo.find_by_owner_id("alice")
    assert sorted(w.id for w in owned) == ["wlt-1", "wlt-2"]
    assert await repo.find_by_owner_id("nobody") == []

@pytest.mark.asyncio
async def test_specification_find_rich_paged_and_sorted(
    sqlite_factory,
) -> None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        # age_days drives created_at for newest-first ordering.
        await repo.upsert(_entity("wlt-poor", "a", 50, age_days=3))
        await repo.upsert(_entity("wlt-mid", "b", 1000, age_days=2))
        await repo.upsert(_entity("wlt-rich", "c", 5000, age_days=1))
        await session.commit()

    # balance_minor >= 1000, newest first, page size 1.
    newest_first = Sort.by("created_at").descending()
    page = await repo.find_rich(1000, Pageable.of(1, 1, newest_first))
    assert page.total == 2 # mid + rich
    assert page.total_pages == 2
    assert page.has_next is True
    assert [w.id for w in page.items] == ["wlt-rich"]

    # The bare predicate also works through find_all_by_spec.

```

```
rich = await repo.find_all_by_spec(balance_at_least(5000))
assert [w.id for w in rich] == ["wlt-rich"]
```

Listing 5.15 — Testing CRUD, the derived query, and the Specification path

The first test drives the derived query: three wallets in, two owners out, and `find_by_owner_id("alice")` returns exactly the two — proof that the framework compiled `WHERE owner_id = :owner_id` from the method name. The second drives the `Specification` path: it asserts the threshold filter (`total == 2` , only mid and rich match `>= 1000`), the newest-first sort (`wlt-rich` is the newest of the two), the page metadata (`total_pages == 2` , `has_next`), and that the same `balance_at_least` predicate also runs unpaged through `find_all_by_spec` .

The fixture mirrors what the framework does at startup — build the engine, run `Base.metadata.create_all` inside a `begin()` block so the DDL commits, hand back a session factory — so the test exercises the exact table the application creates. Other tests in the same file prove `upsert` round-trips through a *fresh* engine (durability across reconnect) and that `find_all(pageable)` counts and slices a five-wallet table correctly.

🌿 SPRING PARITY

Constructing the repository directly against a real in-process database mirrors Spring's `@DataJpaTest` slice, which boots an H2 database and the JPA layer in isolation to test repositories without the full context. `Base.metadata.create_all` is the analogue of `spring.jpa.hibernate.ddl-auto=create` , and running `RepositoryBeanPostProcessor` by hand stands in for the Spring proxy that materialises derived queries on a `JpaRepository` at startup.

✓ What you built

Lumen now persists wallets through PyFly's Spring-Data-style repository layer:

- **Entity** — `WalletEntity(Base)`, a SQLAlchemy 2.0 row with a string primary key (the aggregate's own id) and integer minor-unit balances.
- **Repository** — `WalletRepository(Repository[WalletEntity, str])`, marked `@repository`. The framework supplies full async CRUD (`save`, `find_by_id`, `find_all`, `delete` / `delete_by_id`, `delete_all` / `delete_all_by_id`, `count`, `exists_by_id`, `save_all`, `find_all_by_id`, `stream_all`, pagination, specifications) with no hand-written adapter.
- **Derived query** — `find_by_owner_id`, declared as a `...` stub and compiled from its name by the `RepositoryBeanPostProcessor`.
- **Pagination** — `find_all(pageable)` returning a `Page[T]` with `total` / `total_pages` / `has_next`, mapped to DTOs with `Page.map`, exposed at `GET /api/v1/wallets`.
- **Specification** — `balance_at_least(n)` composed with `&` / `|` / `~` and run via `find_all_by_spec_paged`, exposed at `GET /api/v1/wallets/rich`.
- **Projection** — `@projection BalanceView`, a concrete dataclass the `Mapper` projects rows onto for the balance read view.
- **Transactions** — write handlers decorated `@transactional()` (because `save` / `upsert` only *flush*), using `upsert` / `session.merge` for an aggregate that owns its id, with the aggregate ↔ entity mapper keeping the domain model pure.

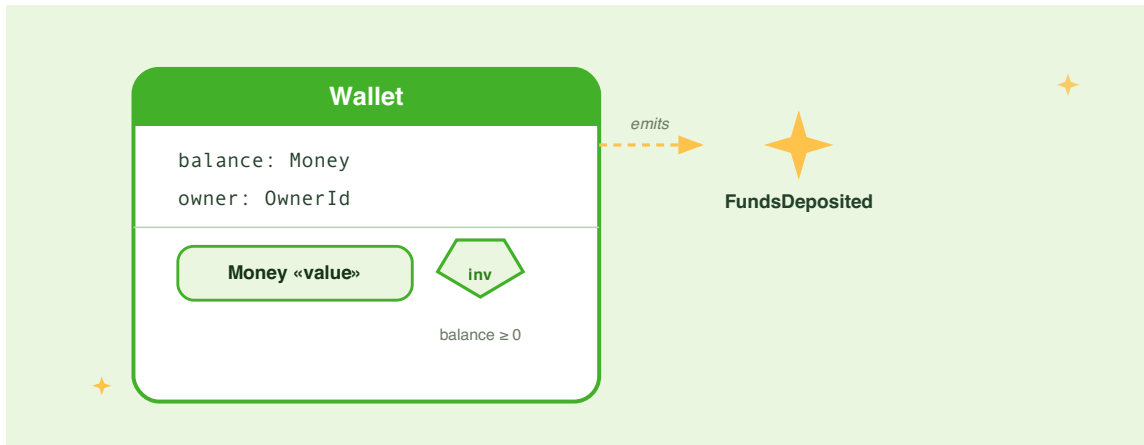
You wrote interfaces and stubs; the framework wrote the SQL. That is the payoff of the repository pattern.

Try it yourself

- 1. Add a derived counter.** Declare `async def count_by_currency(self, currency: str) -> int: ...` on `WalletRepository` (body `...`). Write a test that upserts wallets in two currencies and asserts the count for each — confirming the `count_by` prefix compiles to `SELECT COUNT(*) ... WHERE currency = :currency` with no SQL on your part.
- 2. Compose two specifications.** Define a second factory `in_currency(code: str) -> Specification[WalletEntity]` (predicate `currency == code`), then add a repository method that runs `balance_at_least(min_minor) & in_currency(code)` through `find_all_by_spec_paged`. Test that it returns only rich wallets in the chosen currency, newest first.
- 3. Trace the transaction boundary.** Temporarily change `OpenWalletHandler.do_handle` to call `self._repository.save(to_entity(wallet))` instead of `upsert`, open the same wallet twice in one test, and observe the `IntegrityError`. Restore `upsert`. Then remove the `@transactional()` decorator, open a wallet, and assert it does **not** survive a fresh-engine reconnect — proving that without the unit-of-work commit, `flush` alone is not durability.
- 4. Project a different view.** Add an `@projection OwnerView` dataclass with just `id` and `owner_id`, register it on a `Mapper`, and write a handler-free test that loads a `WalletEntity` and projects it — verifying that only the two declared columns are read and the rest of the row is ignored.

CHAPTER 6

Domain-Driven Design



Lumen's wallet feature works. Deposits land, balances update, and the database persists everything across restarts. But look closely at the service layer and you will notice something uncomfortable: the overdraft check lives in the service method, not in the wallet itself. The currency validation is a property comparison scattered across a handful of `if`-statements. Nothing stops a future developer — or a future you at 11 pm — from bypassing those guards by calling `repo.save(entity)` directly.

Domain-Driven Design addresses this by making the model responsible for its own rules. Data stops being a passive bag of values that any caller can mutate; it becomes an object with opinions — one that enforces its invariants, announces what happened, and cooperates with the persistence layer while remaining free of any database import.

This chapter promotes the wallet into a proper DDD aggregate: a `Money` value object, a `Wallet` aggregate root that guards the overdraft rule and the currency-match rule, and a set of `DomainEvent`s emitted every time the wallet changes state. A thin mapper converts between the rich domain model and the flat persistence record — neither side needs to know the shape of the other.

Entities and value objects

Before you can build a model that enforces its own rules, you need a vocabulary for the two fundamentally different kinds of objects that appear in every domain.

Think about what makes two wallets distinct. Even if two wallets happen to hold exactly one hundred euros, they are still separate wallets belonging to separate owners — you care *which one* you have. Now think about the amount itself. One hundred euros is one hundred euros; the exact Python object that holds the value is irrelevant. If a deposit adds fifty euros to a wallet's balance, you do not want to update the existing amount in place — you want to derive a brand-new amount that records the result. Mutating in place invites aliasing bugs where two parts of the code unknowingly share a reference to the same object and see each other's changes.

DDD names these two roles **entities** and **value objects**, and PyFly's `pyfly.domain` module makes them first-class concepts:

Concept	PyFly base	Equality	Mutation
<code>Entity[TID]</code>	<code>Entity</code>	Identity — equal only when <code>id</code> matches	Allowed through owned methods
<code>ValueObject</code>	<code>ValueObject</code>	Structural — all fields compared	Forbidden; <code>replace(**changes)</code> creates a new instance

Transient entities (those with `id=None`) compare equal only by Python's object identity, so you can safely put entities in sets and dicts without worrying about hash collisions from unsaved objects.

Money is the textbook value object. An amount of one hundred euros is not a specific object you track over time; it is a value. Two separate `Money(100, "EUR")` instances are equal. A deposit does not mutate the existing amount — it produces a new one, leaving the original untouched and the model free of hidden side-effects.

Here is the `Money` value object for Lumen:

`lumen/models/entities/v1/money.py`

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.domain import BusinessRuleViolation, ValueObject

@dataclass(frozen=True)
class Money(ValueObject):
    """An exact monetary amount in a single currency.

    ``amount`` is in minor units (e.g. cents): ``Money(1050,

```

```

Currency.EUR)` is €10.50. Arithmetic returns new ``Money``
instances and refuses to mix currencies.
"""

amount: int
currency: Currency

def __post_init__(self) -> None:
    if not isinstance(self.amount, int) or isinstance(self.amount, bool):
        raise BusinessRuleViolation(
            "money-amount-integer",
            "amount must be an integer number of minor units",
        )

    @classmethod
    def zero(cls, currency: Currency) -> Money:
        """The additive identity for *currency* (a zero balance)."""
        return cls(amount=0, currency=currency)

    def add(self, other: Money) -> Money:
        """Return ``self + other``; both must share a currency."""
        self._assert_same_currency(other)
        return Money(amount=self.amount + other.amount,
            currency=self.currency)

    def subtract(self, other: Money) -> Money:
        """Return ``self - other``; both must share a currency."""
        self._assert_same_currency(other)
        return Money(amount=self.amount - other.amount,
            currency=self.currency)

    @property
    def is_positive(self) -> bool:
        return self.amount > 0

    @property
    def is_negative(self) -> bool:
        return self.amount < 0

```

```

@property
def major_units(self) -> float:
    """The amount rendered as a major-unit decimal (cents / 100)."""
    return round(self.amount / 100, 2)

def _assert_same_currency(self, other: Money) -> None:
    if self.currency is not other.currency:
        raise BusinessRuleViolation(
            "money-currency-mismatch",
            f"cannot combine {self.currency.value} "
            f"with {other.currency.value}",
        )

def __str__(self) -> str:
    return f"{self.major_units:.2f} {self.currency.value}"

```

Listing 6.1 — Money: an immutable value object with currency-aware arithmetic

How it works. The amount is stored in **minor units** — integer cents, pence, or whatever the currency's smallest denomination is — to eliminate floating-point rounding entirely. Financial calculations that use `float` are a chronic source of off-by-one-cent bugs that only surface in production, usually during reconciliation. Storing €10.50 as `amount=1050` keeps all arithmetic exact. `__post_init__` rejects non-integer amounts immediately with `BusinessRuleViolation("money-amount-integer")`, so a stray float like `10.5` never silently enters the model.

The `currency` field uses the `Currency` enum (`Currency.EUR`, `Currency.USD`, `Currency.GBP`) rather than a bare string, ruling out typos at construction time. Currency comparisons inside `_assert_same_currency` use Python's identity check (`is`) — raising `BusinessRuleViolation("money-currency-mismatch")` if they differ — so the error surfaces exactly where the mistake was made.

Both `add` and `subtract` return a new `Money` instance rather than modifying `self`, a direct consequence of `frozen=True`. This immutability guarantee means the aggregate holding a `Money` value can never be partially updated: either the whole replacement succeeds or the old value remains in place. The `is_positive` and `is_negative` properties expose the sign without leaking the raw integer. `major_units` converts to a decimal for display, and `__str__` formats as `"10.50 EUR"` via `currency.value`.

MINOR UNITS VS DECIMAL

Storing money as integer cents is one convention; another is Python's `decimal.Decimal` with a fixed scale. Both are valid. What matters is picking one and sticking to it within the bounded context. For Lumen, integer minor units keep the model free of import-time precision configuration, and `__post_init__` enforces the constraint with `BusinessRuleViolation("money-amount-integer")` so a float never silently enters the model.

SPRING PARITY

`ValueObject` mirrors the `@ValueObject` / `@Embeddable` cluster in Spring's JPA ecosystem and the `ValueObject` marker interface from Spring Modulith. The `frozen=True` dataclass maps to Java's `record` type introduced in Java 16 — immutable, value-based equality, concise syntax. `jMolecules`'s `@ValueObject` annotation carries the same intent.

The aggregate root

`Money` solves the representation problem — amounts are now immutable and currency-aware. But Lumen still needs something to *own* the wallet's balance and decide when a deposit or withdrawal is permitted. That is the role of the **aggregate root**.

An entity becomes an aggregate root when it owns a cluster of related objects and acts as the single point of entry for all changes within that cluster. The aggregate root is the **consistency boundary**: no external code reaches inside and mutates an inner object directly. All changes flow through the root's methods, which enforce the rules. This is the design that prevents the 11 pm bypass described in the chapter introduction — once every change must flow through the root, there is no back-channel.

`AggregateRoot[TID]` extends `Entity[TID]` with one addition: an internal buffer of **pending domain events**. Every state-changing method calls `self.raise_event(event)` to record what happened. When the repository saves the aggregate, the application service drains that buffer with `clear_events()` and publishes the events to the event bus. You will see the full publish cycle in the Domain events section; for now, focus on the aggregate itself.

Here is the `Wallet` aggregate root:

`lumen/models/entities/v1/wallet_entity.py`

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import AggregateRoot, BusinessRuleViolation, DomainEvent

# — Domain events



---



@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
```

```

currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

# — Aggregate root


---



class Wallet(AggregateRoot[str]):
    """Wallet aggregate root – owns the ``balance >= 0`` invariant."""

    __slots__ = ("owner_id", "balance", "created_at")

    def __init__(
        self,
        id: str,
        owner_id: str,
        balance: Money,
        created_at: datetime | None = None,
    ) -> None:
        super().__init__(id)
        self.owner_id = owner_id
        self.balance = balance
        self.created_at = created_at or datetime.now(UTC)

```



```

@property
def currency(self) -> Currency:
    return self.balance.currency

# — Factory method —————

@classmethod
def open(cls, wallet_id: str, owner_id: str, currency: Currency) ->
Wallet:
    """Open a new, empty wallet; raises WalletOpened."""
    if not owner_id.strip():
        raise BusinessRuleViolation(
            "wallet-owner-required", "owner_id is required"
        )
    wallet = cls(
        id=wallet_id,
        owner_id=owner_id,
        balance=Money.zero(currency),
    )
    wallet.raise_event(
        WalletOpened(
            wallet_id=wallet_id,
            owner_id=owner_id,
            currency=currency.value,
        )
    )
    return wallet

# — Behaviour —————

def deposit(self, amount: Money) -> None:
    """Credit *amount* to the balance; raises FundsDeposited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "wallet-deposit-positive",
            "deposit amount must be > 0",
        )
    self.balance = self.balance.add(amount)

```

```

    assert self.id is not None
    self.raise_event(
        FundsDeposited(
            wallet_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=self.balance.amount,
        )
    )

def withdraw(self, amount: Money) -> None:
    """Debit *amount*; refuses to overdraw. Raises FundsWithdrawn."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "wallet-withdrawal-positive",
            "withdrawal amount must be > 0",
        )
    remaining = self.balance.subtract(amount)
    if remaining.is_negative:
        raise BusinessRuleViolation(
            "wallet-insufficient-funds",
            f"cannot withdraw {amount}; balance is {self.balance}",
        )
    self.balance = remaining
    assert self.id is not None
    self.raise_event(
        FundsWithdrawn(
            wallet_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=self.balance.amount,
        )
    )

# — Helpers —————

def _assert_currency(self, amount: Money) -> None:
    if amount.currency is not self.balance.currency:

```

```

raise BusinessRuleViolation(
    "wallet-currency-mismatch",
    f"wallet holds {self.balance.currency.value}, "
    f"got {amount.currency.value}",
)

```

Listing 6.2 — *Wallet: the aggregate root that owns balance and enforces its rules*

How it works. The aggregate boundary is enforced at three levels. First, `__slots__` locks the attribute set, and `balance` and `owner_id` are deliberately public-but-owned: only the aggregate's own methods (`deposit` , `withdraw`) mutate them, while the `currency` property is a read-only convenience that delegates to `balance.currency` . Second, the factory classmethod `open` is the sole legitimate way to create a new wallet: the caller supplies the `wallet_id` (so the application layer controls ID generation), `open` validates that `owner_id` is non-blank, initializes the balance with `Money.zero(currency)` , and immediately queues `WalletOpened` . Using a factory rather than calling `__init__` directly ensures the opening event is *never* forgotten, even in a test fixture. Third, the domain events — `WalletOpened` , `FundsDeposited` , `FundsWithdrawn` — are frozen dataclasses. `FundsDeposited` and `FundsWithdrawn` carry a `balance` field (the post-operation balance in minor units), so a subscriber never needs to call back into the aggregate to learn the current state.

The diagram below shows the complete picture: state, invariants, and the events the wallet emits.

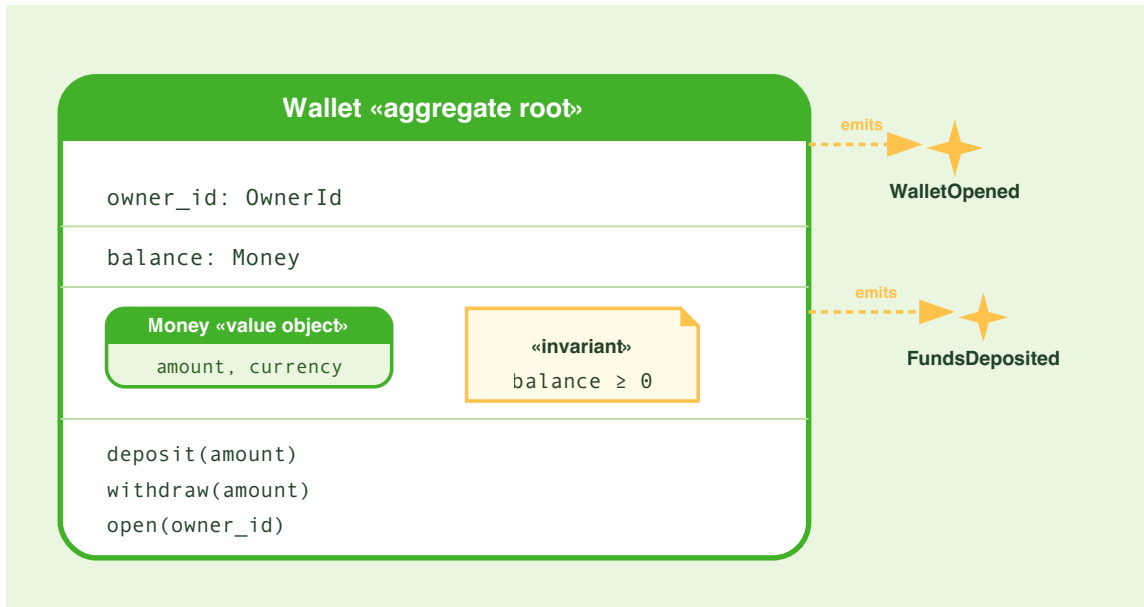


Figure 6.1 — The Wallet aggregate: state, invariants, and the events it emits.

🍃 SPRING PARITY

`AggregateRoot[str]` maps to `jMolecules`'s `org.jmolecules.ddd.types.AggregateRoot<A, ID>` and to Spring Data's `AbstractAggregateRoot<A>`, which offers the same `registerEvent()` / `@DomainEvents` / `@AfterDomainEventPublication` mechanism. The pattern is identical in spirit: the aggregate accumulates events in a buffer; the repository drains them after a successful save; a `DomainEventPublisher` dispatches them. PyFly's `raise_event` + `clear_events` is the Python equivalent of `registerEvent` + `@AfterDomainEventPublication`.

Protecting invariants

The aggregate root is only valuable if the rules it enforces are genuinely unreachable by any other path. That is what **invariant** means in DDD: a condition the model must uphold regardless of how it is called, who calls it, or how many services exist in the application. An invariant is not a suggestion — it is a constraint that cannot be violated because the model exposes no mechanism to do so.

Lumen's `Wallet` has three invariants:

1. The balance must never go below zero (no overdraft).
2. Funds can only be deposited or withdrawn in the wallet's native currency.
3. Deposit and withdrawal amounts must be strictly positive.

All three are enforced inside the aggregate methods. The framework exception for this is `BusinessRuleViolation` from `pyfly.domain`. It takes two required arguments: a stable machine-readable `rule` slug and a human-readable `message`. Lumen's slugs — `"wallet-insufficient-funds"`, `"wallet-currency-mismatch"`, `"wallet-deposit-positive"`, `"wallet-withdrawal-positive"` — are kebab-cased labels that travel in the RFC 7807 response body and in structured log fields.

`BusinessRuleViolation` extends `pyfly.kernel.BusinessException`, so the RFC 7807 problem-details mapper from Chapter 4 translates it automatically into an HTTP 422 response — no extra handler required.

⚠ KEEP INVARIANTS IN THE MODEL, NOT THE SERVICE

Moving the overdraft check back into `WalletService` creates two problems. First, any code that calls `repo.save(entity)` directly bypasses the check entirely. Second, you end up duplicating the rule across every path that modifies a wallet — the service, a background job, an admin command. When the rule changes — say, the product team introduces a configurable overdraft buffer — there is exactly one place to update: the aggregate method. That is the whole point.

The difference between a service-level guard and an aggregate invariant is enforceability. A service guard is a convention; an aggregate invariant is a physical constraint enforced by encapsulation. To make that concrete, here is what the service-level approach looks like and why it is fragile:

`lumen/wallet_service_before.py`

```
# DO NOT DO THIS — rules that belong in the model
from pyfly.container import service

@service
class WalletServiceBefore:

    async def withdraw(self, wallet_id: str, amount: float) -> None:
        # Rule lives here — but anyone calling repo.save directly skips it
        wallet = {"id": wallet_id, "balance": 50.0, "currency": "EUR"}
        if wallet["balance"] < amount:
            raise ValueError("Insufficient funds")
        wallet["balance"] -= amount
        # ... save
```

Listing 6.3 — Before: business rules scattered across the service (fragile)

And here is what the service looks like after the model takes ownership:

lumen/wallet_service_after.py

```

from pyfly.container import service
from pyfly.domain import AggregateNotFound

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money

@service
class WalletService:

    async def withdraw(
        self,
        wallet_id: str,
        amount_cents: int,
        currency: Currency,
    ) -> None:
        # The service orchestrates; the aggregate decides.
        wallet = await self._repo.find(wallet_id)
        if wallet is None:
            raise AggregateNotFound("Wallet", wallet_id)
        wallet.withdraw(Money(amount=amount_cents, currency=currency))
        await self._repo.save(wallet)
        # Events are drained and published by the repository/service boundary

```

Listing 6.4 — After: the service delegates to the aggregate

How it works. The after version reads as an instruction: `wallet.withdraw(...)` means "ask the wallet to withdraw". The service does not know — or care — what that entails. It trusts the aggregate to either succeed or raise a `BusinessRuleViolation`. That thin-orchestrator pattern has a practical benefit for team workflows: a new developer can implement a `transfer` endpoint without reading `WalletService` at all. The constraints live in `Wallet` — one place to look.

The `rule` slug on `BusinessRuleViolation` matters too. Strings like `"wallet-insufficient-funds"` and `"wallet-currency-mismatch"` travel in the RFC 7807 response body, where client code can match them without parsing free-text messages. They also appear in structured log fields, making production alerts straightforward to write.

AGGREGATENOTFOUND

`AggregateNotFound` is the second domain exception in `pyfly.domain`. Raise it when the requested aggregate does not exist — it maps to a 404 problem-details response via the same RFC 7807 handler. The constructor takes the aggregate type name and the ID: `AggregateNotFound("Wallet", wallet_id)`.

Domain events

Your aggregate now enforces its invariants and controls all state changes. But Lumen will eventually need to react to those changes: update an audit log, send a push notification, trigger fraud detection, publish a ledger entry. The tempting solution is to put those side-effects directly inside `deposit` and `withdraw`. That couples the domain model to infrastructure — suddenly your wallet needs to know about Kafka topics and email templates, and every unit test drags in a broker connection.

Domain events cut that coupling. A domain event records something that *happened* inside the aggregate — past tense, immutable fact. The aggregate does not know what will be done with the fact; it only records it. Downstream consumers — event listeners, projectors, notification services — subscribe and react in their own context, without the aggregate ever depending on them.

`DomainEvent` from `pyfly.domain` is a frozen-dataclass base that auto-populates three fields when an instance is created:

- `event_id` — a UUID v4 that uniquely identifies this occurrence.
- `occurred_at` — a UTC timestamp at the moment of construction.
- `event_type` — a property that defaults to the subclass's class name (`"WalletOpened"`, `"FundsDeposited"`, `"FundsWithdrawn"`).

You already saw the three wallet events defined in Listing 6.2. Here they are in isolation with an explicit look at what you get from the base:

`lumen/models/entities/v1/wallet_entity.py`

```
from dataclasses import dataclass
from pyfly.domain import DomainEvent

@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0  # balance after the deposit, in minor units

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
```

```

currency: str = ""
balance: int = 0 # balance after the withdrawal, in minor units

# Each event carries event_id (UUID), occurred_at (UTC datetime),
# and event_type (class name) – all set by DomainEvent.__post_init__.

def demonstrate_event_fields() -> None:
    evt = FundsDeposited(
        wallet_id="w-1",
        amount=5000,
        currency="EUR",
        balance=15000,
    )
    print(evt.event_id) # e.g. "3fa85f64-5717-4562-b3fc-2c963f66afa6"
    print(evt.occurred_at) # e.g. datetime(2026, 6, 7, 9, 30, 0,
tzinfo=UTC)
    print(evt.event_type) # "FundsDeposited"

```

Listing 6.5 — Domain events and the fields DomainEvent provides automatically

How it works. Each event class declares only the fields unique to that occurrence. Everything else comes from `DomainEvent`: a UUID `event_id` for idempotent processing, `occurred_at` for the audit trail, and `event_type` — the class name — for consumer routing without inspecting the Python class. All fields default to zero or empty string so that the `frozen=True` dataclass machinery can provide keyword-argument construction without requiring positional arguments.

Notice that `FundsDeposited` carries both `amount` (the transaction) and `balance` (the post-operation balance, in minor units). A subscriber updating a read-model balance needs no callback into the aggregate or the database — everything is in the event. That self-contained design keeps consumers simple and eliminates extra round-trips.

The event lifecycle spans two phases. Inside the aggregate: when

`wallet.deposit(amount)` succeeds, it calls

`self.raise_event(FundsDeposited(...))`, appending the event to a private buffer in

`AggregateRoot`. Nothing is published yet. At the service boundary: after the repository

saves the aggregate and the transaction commits, the application service drains the

buffer and publishes. This *save-first, publish-after* sequence guarantees that an event is

never dispatched for a change that failed to persist. Listing 6.6 shows that boundary in

full:

`lumen/wallet_application_service.py`

```
import uuid

from pyfly.container import service
from pyfly.eda import EventPublisher

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.entities.v1.wallet_entity import Wallet

@service
class WalletApplicationService:

    def __init__(
        self,
        repo: object,          # typed as WalletRepository in practice
        events: EventPublisher,
    ) -> None:
        self._repo = repo
        self._events = events

    async def open_wallet(self, owner_id: str, currency: Currency) -> str:
        wallet_id = str(uuid.uuid4())
        wallet = Wallet.open(
            wallet_id=wallet_id, owner_id=owner_id, currency=currency
```

```

    )
    await self._repo.save(wallet)
    for event in wallet.clear_events():
        await self._events.publish(event)
    return wallet_id

    async def deposit(
        self,
        wallet_id: str,
        amount_cents: int,
        currency: Currency,
    ) -> None:
        wallet = await self._repo.find(wallet_id)
        wallet.deposit(Money(amount=amount_cents, currency=currency))
        await self._repo.save(wallet)
        for event in wallet.clear_events():
            await self._events.publish(event)

```

Listing 6.6 — Draining domain events after a successful save

How it works. `open_wallet` calls `Wallet.open`, which queues a `WalletOpened` event internally; after the `save`, `wallet.clear_events()` returns that one event and `publish` dispatches it. `deposit` follows the same three-step pattern: load, mutate, save — then drain. The `for event in wallet.clear_events()` loop is intentionally explicit rather than hidden inside the repository, because the application service is the right place to decide *when* publishing happens — after the transaction boundary, not before.

💡 EVENT ORDERING

`raise_event` appends to the buffer in call order. `clear_events` drains and clears it, returning events in the same order. If a single aggregate method raises multiple events (a batch operation, for example), they arrive at the event bus in the order they were raised — oldest first.

Domain vs persistence

With the domain model and its events in place, one tension remains: how does the `Wallet` aggregate reach the database?

The tempting shortcut is to annotate `Wallet` directly with SQLAlchemy `Mapped[]` fields and a `__tablename__`. That merges two concerns that change at very different rates: business rules evolve with the product; column definitions evolve with the schema. Mixing them means a schema change forces you to touch the aggregate, and a rule change risks accidentally breaking a column mapping. It also drags SQLAlchemy into every unit test.

The alternative is two models that coexist without knowing about each other — and a thin mapper that converts between them:

Model	Contains	Knows about
<code>Wallet</code>	Business rules, domain events, invariants	Nothing outside <code>pyfly.domain</code>
<code>WalletEntity</code>	Five columns: <code>id</code> , <code>owner_id</code> , <code>currency</code> , <code>balance_minor</code> , <code>created_at</code>	Only SQLAlchemy + <code>pyfly.data</code>

`Wallet` is pure Python: no `Mapped[]` annotations, no `__tablename__`. You can instantiate it in a unit test with two lines and exercise every invariant without a database connection. `WalletEntity` is pure persistence: it subclasses `Base` from `pyfly.data.relational.sqlalchemy`, carries SQLAlchemy 2.0 typed columns, and knows nothing about domain rules or events. The framework's `Repository[WalletEntity, str]` (Chapter 5) stores and retrieves rows; a thin mapper converts between the row and the aggregate on every crossing.

Listing 6.7 shows `WalletEntity` — the ORM row — followed by the two mapper functions that cross the boundary:

`lumen/models/entities/v1/wallet_orm.py`

```
from __future__ import annotations

from datetime import UTC, datetime

from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

from pyfly.data.relational.sqlalchemy import Base

class WalletEntity(Base):
    """One persisted wallet row, keyed by the aggregate's own string id."""

    __tablename__ = "wallets"

    id: Mapped[str] = mapped_column(String(64), primary_key=True)
    owner_id: Mapped[str] = mapped_column(
        String(255), nullable=False, index=True
    )
    currency: Mapped[str] = mapped_column(String(3), nullable=False)
    balance_minor: Mapped[int] = mapped_column(nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(
        default=lambda: datetime.now(UTC)
    )
```

Listing 6.7 — `WalletEntity`: the SQLAlchemy persistence row

`lumen/core/mappers/wallet_mapper.py`

```
from __future__ import annotations

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
```

```

from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.entities.v1.wallet_orm import WalletEntity

def to_entity(wallet: Wallet) -> WalletEntity:
    """Flatten a Wallet aggregate into a persistable row."""
    assert wallet.id is not None
    return WalletEntity(
        id=wallet.id,
        owner_id=wallet.owner_id,
        currency=wallet.currency.value,
        balance_minor=wallet.balance.amount,
        created_at=wallet.created_at,
    )

def to_aggregate(entity: WalletEntity) -> Wallet:
    """Rehydrate a Wallet aggregate from a persistence row."""
    currency = Currency(entity.currency)
    return Wallet(
        id=entity.id,
        owner_id=entity.owner_id,
        balance=Money(amount=entity.balance_minor, currency=currency),
        created_at=entity.created_at,
    )

```

Listing 6.8 — *wallet_mapper*: pure functions that cross the domain/persistence boundary

How it works. `WalletEntity` subclasses `Base` rather than carrying any domain logic, so importing it registers the `wallets` table in `Base.metadata`; the framework's `EngineLifecycle` creates the table on startup when `ddl-auto=create` is set. The primary key is the aggregate's own string id (`wlt-...`) — not a surrogate — so the row and the `Wallet` share one identity and no translation is needed.

`to_entity` writes `wallet.balance.amount` (an integer) directly into `balance_minor` — no float conversion. `to_aggregate` reconstructs a `Currency` enum from the stored ISO-4217 string via `Currency(entity.currency)`, then builds a `Money` from the raw integer minor-unit value. The `created_at` field is preserved on the round-trip so rehydrated aggregates carry their original timestamp. There is no floating-point boundary crossing anywhere.

The mapper is intentionally narrow. It does not enforce rules — `Wallet.__init__` and the behaviour methods do that. It does not publish events — the application service does that. It only translates shape.

The repository. The application service never interacts with `WalletEntity` directly. Instead a command handler calls `wallet_mapper.to_entity(wallet)` before persisting and `wallet_mapper.to_aggregate(entity)` after loading, while the framework `WalletRepository(Repository[WalletEntity, str])` handles all SQL. Chapter 5 covers `Repository` in full; the key point here is that `Wallet` itself never imports SQLAlchemy — the aggregate stays free of persistence concerns across both sides of the mapper boundary.

🌿 SPRING PARITY

This two-model-plus-mapper structure is the Python equivalent of the pattern advocated in Vaughn Vernon's *Implementing Domain-Driven Design* for Spring: a `WalletJpaEntity` annotated with `@Entity` (the persistence row), a `Wallet` domain object (the aggregate), and a `WalletAssembler` or MapStruct-generated mapper that translates between them. Spring Data JPA's `JpaRepository<WalletJpaEntity, String>` corresponds to PyFly's `Repository[WalletEntity, str]`. The structure is identical; the boilerplate is less.

Specifications for business rules

The aggregate guards state-changing operations well — you cannot overdraw a wallet or deposit the wrong currency. But not every rule is about mutation. Some rules are eligibility checks: "before we show this user the withdrawal button, is the wallet in an operable state?" or "out of ten thousand wallets, which ones qualify for the loyalty bonus?" These are read-only predicates, and encoding them as aggregate methods would clutter `Wallet` with query logic unrelated to state transitions.

The **Specification pattern** solves this cleanly. A specification is a named, reusable predicate: a single `is_satisfied_by(obj) -> bool` method wrapped in an object that composes with others using Boolean operators. Because each rule is its own class, you can name rules clearly, reuse them across services, and combine them at runtime based on context — something an `if` chain cannot do.

`Specification[T]` from `pyfly.domain` is a composable in-memory predicate. Subclass it, implement `is_satisfied_by`, and combine instances with `&` (and), `|` (or), and `~` (not). A specification is also directly callable, so you can pass it to Python's built-in `filter` without any adapter code.

TWO KINDS OF SPECIFICATION

`pyfly.domain.Specification` is the in-memory predicate used inside domain services. `pyfly.data.relational.sqlalchemy.Specification` (Chapter 5) is the database-aware query predicate that pushes the rule down into SQL. The two coexist. Domain specifications are for business logic; data specifications are for queries.

Here is a specification that expresses the "eligible for withdrawal" rule:

```
lumen/domain/specs.py
```

```

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.wallet_entity import Wallet
from pyfly.domain import Specification

class HasPositiveBalance(Specification[Wallet]):
    """The wallet has at least one cent remaining."""

    def is_satisfied_by(self, wallet: Wallet) -> bool:
        return wallet.balance.is_positive

class IsInCurrency(Specification[Wallet]):
    """The wallet holds a specific currency."""

    def __init__(self, currency: Currency) -> None:
        self._currency = currency

    def is_satisfied_by(self, wallet: Wallet) -> bool:
        return wallet.balance.currency is self._currency

# Compose: a wallet is eligible for withdrawal if it has a positive
# balance in the requested currency.
def eligible_for_withdrawal(currency: Currency) -> Specification[Wallet]:
    return HasPositiveBalance() & IsInCurrency(currency)

# Use as a predicate:
def filter_eligible(
    wallets: list[Wallet],
    currency: Currency,
) -> list[Wallet]:
    spec = eligible_for_withdrawal(currency)
    return list(filter(spec, wallets))

```

Listing 6.9 — EligibleForWithdrawal: a composable domain Specification

How it works. `HasPositiveBalance` delegates to `wallet.balance.is_positive` (a property, no parentheses). `IsInCurrency` uses identity comparison (`is`) because `Currency` is a `StrEnum` with singleton members. The `eligible_for_withdrawal` factory combines them with `&`, producing a composite whose `is_satisfied_by` returns `True` only when both checks pass. Because `Specification` implements `__call__`, you pass the composite directly to `filter()` — no lambda wrapper needed.

The key design discipline: a specification is a *predicate*, not a *guard*. It returns `True` or `False` and never raises. Aggregate invariants (overdraft, currency mismatch) belong inside `deposit` and `withdraw` because they must *prevent* a state change. Specifications belong in services and query handlers because they *select* or *classify* — they never mutate.

Specifications are especially useful where rules combine dynamically: an admin search that adds filters based on the operator's role, or a batch job that partitions a list into eligible and ineligible wallets. Each class has exactly one method and no side-effects, making isolated unit tests trivial.

💡 SPECIFICATION.OF FOR QUICK LAMBDA'S

For one-off predicates that do not need a class, use the factory method: `spec = Specification.of(lambda w: w.balance.amount >= 1000, name="minimum-balance")`. It composes with `&`, `|`, and `~` the same way as a full subclass.

✓ What you built

Lumen's wallet is now a first-class domain model.

`Money` is a frozen `ValueObject` that stores amounts as integer minor units, enforces currency homogeneity via `_assert_same_currency`, and is replaced rather than mutated. `__post_init__` rejects floats immediately. `is_positive` and `is_negative` are properties; `major_units` and `__str__` handle display.

`Wallet(AggregateRoot[str])` is the consistency boundary. Its factory `open` and behaviour methods `deposit / withdraw` enforce all three invariants — no overdraft, no cross-currency operations, no non-positive amounts — by raising `BusinessRuleViolation` with a stable rule slug. Every state change queues a domain event (`WalletOpened`, `FundsDeposited`, `FundsWithdrawn`); the post-operation balance is recorded in each event so subscribers need no callback. After a successful save the application service drains events with `clear_events()` and hands them to `EventPublisher`.

The persistence layer sees only `WalletEntity` (five columns, no domain logic). `to_aggregate` in `wallet_mapper` rehydrates the row into a `Wallet`, and `to_entity` flattens it back; the framework `WalletRepository(Repository[WalletEntity, str])` handles all SQL without the aggregate ever importing SQLAlchemy. `Specification[Wallet]` gives you a composable, callable predicate for eligibility checks that live outside the aggregate boundary.

The controller is untouched. The service shrank. The rules are enforced by the object that owns them.

Try it yourself

1. Add a `transfer_to` method and a `DailyLimit` value object. Add a `DailyLimit(ValueObject)` with `max_amount: int` and `currency: Currency`, decorated with `@dataclass(frozen=True)`. Then add

`Wallet.transfer_to(target: Wallet, amount: Money) -> None`. The method should call `self.withdraw(amount)` and `target.deposit(amount)` in sequence, raising `BusinessRuleViolation("transfer-currency-mismatch", ...)` if the two wallets hold different currencies. Because `Wallet` uses `__slots__`, add `"_frozen"` to the tuple if you also do exercise 2. Verify that both wallets each accumulate a `FundsWithdrawn` / `FundsDeposited` event, respectively, and that a transfer between mismatched wallets raises before modifying either balance.

2. Add a `WalletFrozen` event and a `freeze()` behaviour. Define

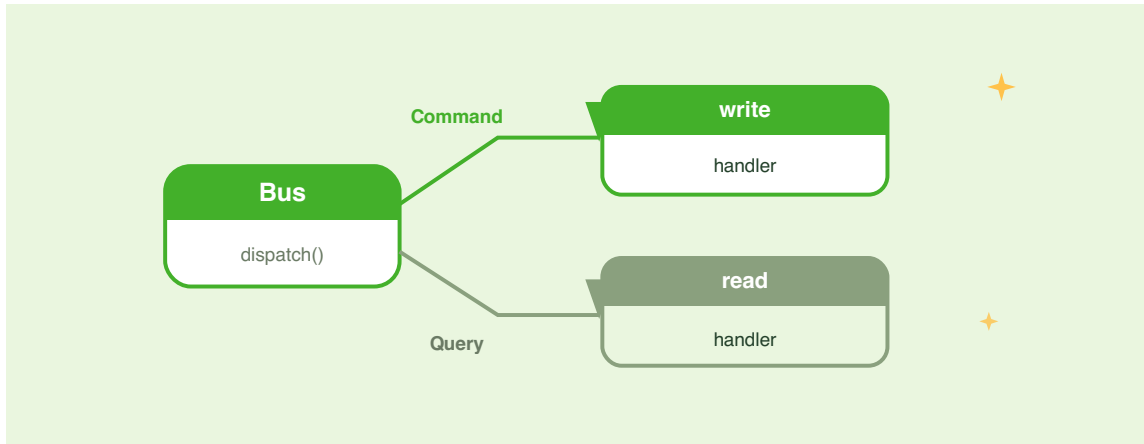
`WalletFrozen(DomainEvent)` with `wallet_id: str = ""` and `reason: str = ""`. Add `"frozen"` to `__slots__` and a `frozen: bool` attribute (default `False`) to `Wallet.__init__`. Add a `freeze(reason: str) -> None` method that sets `self.frozen = True` and calls `self.raise_event(WalletFrozen(...))`. Guard `deposit` and `withdraw` with `if self.frozen: raise BusinessRuleViolation("wallet-frozen", ...)` at the top of each method. Then write an event listener using `@event_listener` from `pyfly.eda` that logs a structured warning every time a `WalletFrozen` event is published.

3. Express a business rule as a Specification. Write a

`MinimumBalance(Specification[Wallet])` that checks whether a wallet's balance is at or above a threshold amount (in minor units) passed to its `__init__`. Combine it with `IsInCurrency` from Listing 6.9 using `&` to produce a `premium_eligible(currency: Currency, threshold: int)` factory function. Call `list(filter(premium_eligible(Currency.EUR, 50000), wallets))` over a list of test wallets and assert that only wallets with at least 500.00 EUR (50 000 cents) appear in the result.

CHAPTER 7

CQRS: Commands & Queries



Lumen's wallet is now a first-class citizen of the domain. The `Wallet` aggregate enforces its own invariants, emits domain events, and persists through a clean repository boundary. The controller, though, still calls `WalletApplicationService` directly — one method per operation, reads and writes sharing the same code path. That design is fine at small scale, but it shows friction as the system grows. The team wants to cache wallet balances, maintain a single audit trail for every write, add authorization rules to specific operations, and test each piece of logic in complete isolation from the others.

CQRS — Command Query Responsibility Segregation — addresses all of this by drawing a bright line between the two things a service can do: *change state* and *read state*. Writes become **commands**: strongly typed, named, immutable messages that flow through a `CommandBus`. Reads become **queries**: equally typed messages that flow

through a `QueryBus`. Each bus runs a fixed pipeline — validation, authorization, execution, then (for commands) domain event publishing. Your handler implements exactly one intent; the bus handles everything else.

By the end of this chapter Lumen's controller dispatches commands and queries instead of calling the service directly. `OpenWallet`, `DepositFunds`, and `WithdrawFunds` travel the command path; `GetWallet`, `GetBalance`, `ListWallets`, and `ListRichWallets` travel the query path. The `Wallet` aggregate you built in Chapter 6 remains untouched — CQRS does not replace the domain model; it is the delivery mechanism for instructions to it.

Why separate reads from writes

Picture Lumen at the end of Chapter 6. `WalletController` calls `WalletApplicationService.credit(wallet_id, amount)`. That call mutates state, but nothing in the method signature makes that obvious. The team wants to add a balance cache. Where does it go? Inside `credit`? In a decorator around the service? The question reveals the problem: a single service method is asked to serve two masters — the write path, which must always touch the database, and the read path, which should avoid it whenever possible. Bolting caching onto a write method is awkward at best and dangerous at worst.

Writes and reads have fundamentally different shapes. A write carries intent and data: "deposit 1 500 minor units into wallet wlt-001". A read carries a question: "what is the current balance of wallet wlt-001?" The first must reach the database every time. The second is repeatable — asking twice should return the same answer without doubling the database load. Funnelling both through the same method conflates concerns that scale differently, test differently, and need different cross-cutting behaviour.

The deeper benefit is **clarity of intent**. When a teammate reads `wallet_service.credit(wallet_id, amount)`, they must inspect the implementation to know whether it is safe to call twice, whether it publishes events, and whether it is idempotent. When they read `DepositFunds(wallet_id=..., amount=...)`, the intent is unambiguous — and if the intent turns out to be wrong, you rename the command, not the service signature.

Three concrete benefits matter for Lumen:

Independent scaling. Reads typically outnumber writes by an order of magnitude or more. Once the two paths are separate, the bus can cache query results without touching the write path. You can route queries to a read replica and commands to the primary database with a configuration change, not a code change.

Focused handlers. Each handler implements exactly one operation.

`DepositFundsHandler` loads a wallet, drives its domain behaviour, persists it, and drains events — nothing more. `GetBalanceHandler` loads one wallet and returns a lightweight projection — nothing more. Because handlers are plain Python classes with injected dependencies, you can unit-test each in complete isolation from the HTTP layer.

Centralized cross-cutting concerns. Validation, authorization, and distributed tracing are implemented once in the bus pipeline and apply uniformly to every handler — no boilerplate in the handler itself. Adding per-operation authorization later is a matter of overriding `authorize()` on the command; the bus ensures it runs before `do_handle` is ever reached.

Commands and command handlers

Before writing a single line of handler code, name your system's intentions. In Lumen's wallet domain three things can happen: a wallet can be opened, funds can be deposited, and funds can be withdrawn. Each is a **command** — a named, immutable message that expresses one intent. The bus delivers it; the handler acts on it; the domain aggregate enforces the rules. Commands are not method calls dressed up as objects: they are explicit contracts that live in your codebase as first-class citizens.

A command is a frozen dataclass that inherits from `Command[R]`, where `R` is the type the handler returns. The generic parameter is documentation and a type-checker hint; the bus does not enforce it at runtime.

Lumen's commands live in three separate files under `lumen/core/services/wallets/`, one per intent. Here is the first:

`lumen/core/services/wallets/open_wallet_command.py`

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class OpenWallet(Command[str]):
    """Open a new wallet. Returns the generated wallet id."""

    owner_id: str
    currency: Currency

    async def validate(self) -> ValidationResult: # type: ignore[override]
        if not self.owner_id.strip():
```

```

        return ValidationResult.failure(
            "owner_id", "Owner id is required"
        )
    return ValidationResult.success()

```

Listing 7.1 — *OpenWallet: a frozen command with built-in validation*

And the deposit and withdrawal commands:

`lumen/core/services/wallets/deposit_funds_command.py`

```

from __future__ import annotations

from dataclasses import dataclass

from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class DepositFunds(Command[int]):
    """Deposit ``amount`` minor units. Returns the new balance."""

    wallet_id: str
    amount: int

    async def validate(self) -> ValidationResult: # type: ignore[override]
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount <= 0:
            return ValidationResult.failure(
                "amount", "Deposit amount must be > 0"
            )
        return ValidationResult.success()

```

Listing 7.2 — *DepositFunds: amount in minor units, no currency field*

lumen/core/services/wallets/withdraw_funds_command.py

```

from __future__ import annotations

from dataclasses import dataclass

from pyfly.cqrs import Command, ValidationResult

@dataclass(frozen=True)
class WithdrawFunds(Command[int]):
    """Withdraw ``amount`` minor units. Returns the new balance."""

    wallet_id: str
    amount: int

    async def validate(self) -> ValidationResult: # type: ignore[override]
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount <= 0:
            return ValidationResult.failure(
                "amount", "Withdrawal amount must be > 0"
            )
        return ValidationResult.success()

```

Listing 7.3 — *WithdrawFunds*: same shape as *DepositFunds*

Four design choices are baked into every command:

- **frozen=True** makes the dataclass immutable the moment it is constructed. Fields cannot be accidentally mutated in one layer of the pipeline before reaching another, and immutable messages are hashable by default — useful when storing or comparing them in tests.

- `validate()` is an async hook the bus calls before dispatching the handler.
`OpenWallet.validate` checks that `owner_id` is not blank;
`DepositFunds.validate` and `WithdrawFunds.validate` check that the amount is positive. These pre-conditions belong on the command — they require no database lookup and do not belong in the domain aggregate. The aggregate enforces invariants that need loaded state (overdraft, currency match); commands enforce invariants knowable from the fields alone. Keeping these two layers separate means the aggregate is never called with structurally wrong data.
- **No currency field** on `DepositFunds` or `WithdrawFunds`. The wallet's own currency is the only valid currency for a deposit or withdrawal, and the repository resolves that once the aggregate is loaded. Carrying a currency on the command would invite mismatches; the aggregate enforces the invariant from its own state.
- **Imperative-mood naming:** `DepositFunds`, not `WalletDeposit` or `DepositFundsCommand`. This makes the command log read like a business audit trail — a sequence of things that *happened* — rather than a list of technical operations.

Implementing a command handler

A command handler inherits from `CommandHandler[C, R]` and implements exactly one method: `do_handle`. You write the *what*; the bus wraps it with the *how*.

Both decorators on every handler are required. `@command_handler` registers the class with the `HandlerRegistry` by introspecting the first generic type argument — no manual registration needed. `@service` wires the handler into PyFly's DI container so constructor arguments are resolved and injected automatically at startup. The order matters: `@command_handler` on top, `@service` directly below. Without `@service`, the DI container never instantiates the class and the bus cannot find the handler; without `@command_handler`, the registry never maps the command type to the class. Omitting either decorator is a silent failure — the bus raises "no handler found" at dispatch time.

`@transactional()` turns `do_handle` into a committed unit of work. Command handlers inject `session_factory: async_sessionmaker[AsyncSession]` and store it as `self._session_factory`. When `@transactional()` runs `do_handle` it opens a fresh session from that factory, swaps it onto the repository for the duration of the call, commits on success, and rolls back on any exception. Without `@transactional()` the framework's shared session only flushes — the write survives within the request but is never committed to the database.

Here is `OpenWalletHandler`:

`lumen/core/services/wallets/open_wallet_handler.py`

```
from __future__ import annotations

from uuid import uuid4

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
```

```

        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: OpenWallet) -> str: # type:
        ignore[override]
        wallet_id = f"wlt-{uuid4()}"
        wallet = Wallet.open(
            wallet_id=wallet_id,
            owner_id=command.owner_id,
            currency=command.currency,
        )
        await self._repository.upsert(to_entity(wallet))
        await publish_domain_events(self._events, wallet.clear_events())
        return wallet_id

```

Listing 7.4 — OpenWalletHandler: @transactional() unit of work + upsert

Walk through `do_handle` step by step. `f"wlt-{uuid4()}"` generates a stable prefixed identifier. `Wallet.open(...)` calls the factory, which enforces the non-empty owner pre-condition and buffers a `WalletOpened` event. `to_entity(wallet)` maps the aggregate to a flat `WalletEntity` row. `repository.upsert(...)` calls `session.merge` — a single call that inserts if no row exists or updates if one does — then flushes. Using `upsert` instead of `save` avoids an `IntegrityError` on the primary key: the aggregate owns its id, so both INSERT and UPDATE key on the same stable string. `wallet.clear_events()` drains the buffer and `publish_domain_events`

forwards each event to the EDA bus. The `@transactional()` decorator commits the session on the way out. The handler returns the wallet ID, which flows back to the controller as the `send` return value.

Note the constructor requirement: `super().__init__()` is mandatory on `CommandHandler`. Skip it and the base-class bookkeeping — correlation context, lifecycle hooks — is never initialized. The repository, `EventPublisher`, and `session_factory` are all injected by the DI container from type hints; no factory configuration is needed.

Here are the deposit and withdrawal handlers:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cQRS import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
```

```

        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: DepositFunds) -> int: # type:
        ignore[override]
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(self._events, wallet.clear_events())
        return wallet.balance.amount

```

Listing 7.5 — *DepositFundsHandler: find_by_id → to_aggregate → act → upsert*

`lumen/core/services/wallets/withdraw_funds_handler.py`

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler

```



```

from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

@command_handler
@service
class WithdrawFundsHandler(CommandHandler[WithdrawFunds, int]):
    """Debit funds from an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: WithdrawFunds) -> int: # type:
        ignore[override]
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.withdraw(Money(amount=command.amount,
            currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(self._events, wallet.clear_events())
        return wallet.balance.amount

```

Listing 7.6 — *WithdrawFundsHandler*: identical pattern, overdraft refused by the aggregate

`DepositFundsHandler` and `WithdrawFundsHandler` follow the classic pattern: **find** → **to_aggregate** → **act** → **to_entity** → **upsert** → **drain**. `repository.find_by_id` returns the flat `WalletEntity` row; `to_aggregate(entity)` rehydrates the rich domain object so the aggregate's invariants are in scope. `Money` is constructed from the command's `amount` and the *wallet*'s currency — never a currency from the command itself — because the wallet owns that invariant. If `wallet.withdraw` refuses (balance would go negative), it raises `BusinessRuleViolation`, which propagates as HTTP 422 without a single line of error-handling code in the handler.

Notice what is absent: no try/except blocks, no logging calls, no tracing setup. All of that belongs to the bus pipeline. The handler is a pure expression of business intent.

The entity↔aggregate mapper

Command handlers do not interact with the repository through the domain aggregate. They interact through a flat `WalletEntity` row — the persistence shape the framework `Repository[WalletEntity, str]` understands — and use `wallet_mapper` to translate between the two worlds:

```
# Aggregate → row (before upsert)
to_entity(wallet)      # Wallet → WalletEntity

# Row → aggregate (after find_by_id)
to_aggregate(entity)   # WalletEntity → Wallet
```

This separation keeps the aggregate free of SQLAlchemy annotations and the repository free of domain logic. The mapper is a single module; changing the storage schema touches one file, not every handler.

Sending a command

The `CommandBus` is the single entry point for all writes. PyFly's auto-configuration registers a `DefaultCommandBus` as a singleton in the DI container; declare it as a constructor argument and the framework injects it. Sending a command is a single awaited call:

```
from pyfly.cqrs import DefaultCommandBus
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.interfaces.enums.v1.currency import Currency

wallet_id: str = await command_bus.send(
    OpenWallet(owner_id="u-1", currency=Currency.EUR)
)
balance: int = await command_bus.send(
    DepositFunds(wallet_id=wallet_id, amount=1500)
)
```

`send` is a coroutine — always `await` it. The return value is whatever `do_handle` returned: a `str` wallet ID for `OpenWallet`, and the new balance as an `int` (minor units) for `DepositFunds` and `WithdrawFunds`. If anything in the pipeline fails — validation, authorization, or the handler itself — the exception wraps in `CommandProcessingException` and propagates out of `send`, where the global error handler maps it to the appropriate HTTP status code.

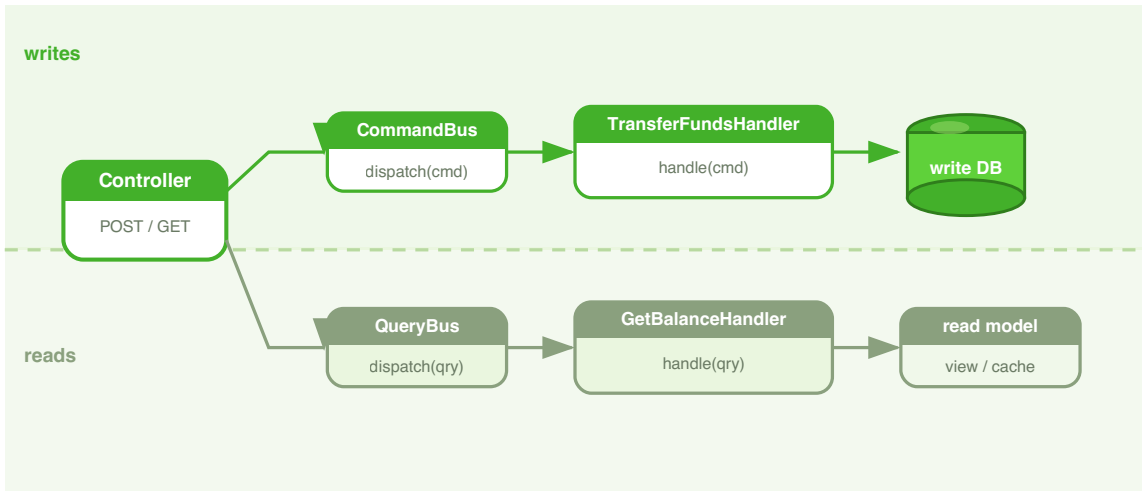


Figure 7.1 — Commands flow to the write model; queries to the read model.

🌿 SPRING PARITY

`CommandBus.send(command)` is the Python equivalent of Axon Framework's `CommandGateway.send(command)` or `CommandGateway.sendAndWait(command)`. Each command handler class corresponds to a method annotated with `@CommandHandler` in Axon, or a `@MessageHandler` in Spring Modulith's `ApplicationEventPublisher` model. The `@command_handler` decorator is PyFly's counterpart of `@CommandHandler`: it registers the handler with the registry by introspecting the generic type parameter, exactly as Axon resolves handler methods by parameter type. The `@service` stacking mirrors the fact that in Spring every `@CommandHandler` bean is also a Spring `@Component` — registration and injection are inseparable. The `@transactional()` decorator maps directly to Spring's `@Transactional`: both open a unit-of-work session, commit on success, and roll back on any exception — so `upsert` (backed by `session.merge`) is the Python analogue of `repository.save()` inside a `@Transactional` method.

Queries and query handlers

Commands travel one direction: into the write model. Queries are the return journey — they ask the system for a projection of current state and expect an answer, not a side effect.

A **query** is a frozen dataclass that inherits from `Query[R]`, where `R` is the result type. Like commands, queries are immutable messages, but they carry no intent to change state. `query_bus.query(GetBalance(...))` loads fresh data from the repository and returns a typed DTO. Queries do not need `@transactional()` — reads do not mutate state, so there is nothing to commit or roll back.

Queries return **read DTOs** rather than domain aggregates. The separation is deliberate. If `GetWalletHandler` returned a `Wallet` aggregate, the API layer would be coupled to every field on the aggregate — a change to the domain model could silently break the API contract. A dedicated `WalletDto` Pydantic model projects exactly the fields the HTTP response needs. Add a field to `Wallet`? The projection changes only if you explicitly include it in the DTO. Remove a field from `Wallet`? The projection compiles until you clean it up.

`lumen/core/services/wallets/get_wallet_query.py`

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.cqrs import Query

@dataclass(frozen=True)
class GetWallet(Query[WalletDto | None]):
    """Look up a wallet by its identifier."""
```

```
wallet_id: str
```

Listing 7.7 — *GetWallet*: a frozen query returning *WalletDto* or *None*

```
lumen/core/services/wallets/get_balance_query.py
```

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from pyfly.cqrs import Query

@dataclass(frozen=True)
class GetBalance(Query[BalanceDto | None]):
    """Look up just the balance of a wallet by its identifier."""

    wallet_id: str
```

Listing 7.8 — *GetBalance*: a lighter query returning only the balance projection

Both queries carry only `wallet_id`. `GetWallet` returns a `WalletDto` — the full representation including `id`, `owner_id`, `currency`, `balance_minor`, `balance`, and `created_at`. `GetBalance` returns a `BalanceDto` — a lighter projection that omits `owner_id` and `created_at`. A balance poll does not need the owner; leaving those fields out saves bandwidth and avoids accidentally exposing account ownership in a response that callers may log. Keeping the two queries separate means you can tune each independently — caching, authorization, or a dedicated read store — without touching the other.

The query handlers live under the same `wallets/` package as the commands. **The same `@query_handler` + `@service` stacking applies:** `@query_handler` registers the class with the handler registry; `@service` wires it into the DI container. Both decorators are required for the same reasons as on command handlers.

`lumen/core/services/wallets/get_wallet_handler.py`

```
from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cQRS import QueryHandler, query_handler

@query_handler
@service
class GetWalletHandler(QueryHandler[GetWallet, WalletDto | None]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: GetWallet) -> WalletDto | None: # type: ignore[override]
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_dto(entity) if entity is not None else None
```

Listing 7.9 — `GetWalletHandler`: `find_by_id` → `entity_to_dto` → `return`

`lumen/core/services/wallets/get_balance_handler.py`

```
from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
```

```

from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: GetBalance) -> BalanceDto | None: #
type: ignore[override]
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listing 7.10 — *GetBalanceHandler: @projection view via Mapper.project*

Both handlers delegate projection to `wallet_mapper` — the single module that owns the DTO shape. `entity_to_dto` fills in all six fields of `WalletDto` directly from the row. `entity_to_balance_dto` takes a different path: it calls `Mapper.project(entity, BalanceView)` against a `@projection`-marked interface that declares exactly the four fields the balance endpoint needs, with a registered transform that computes `balance` (major units) from `balance_minor`. The mapper copies only those declared fields — a read-side equivalent of Spring Data's interface projections. Neither handler touches the Pydantic model directly; a field rename touches one file.

Paged and specification queries

The read side does not stop at single-resource lookups. Production systems need lists with pagination metadata and the ability to filter by runtime predicates. The framework handles both through the `Repository` base class.

`ListWallets` wraps a `Pageable` (page number, size, sort) and asks the repository for a counted, sorted, limited slice. `ListRichWallets` adds a `min_minor` threshold and runs it through a composable `Specification` — a predicate object that can be combined with `&`, `|`, and `~` before execution.

`lumen/core/services/wallets/list_wallets_query.py`

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.data import Page, Pageable
from pyfly.cqrs import Query

@dataclass(frozen=True)
class ListWallets(Query[Page[WalletDto]]):
    """List wallets, one page at a time."""

    pageable: Pageable
```

Listing 7.11 — *ListWallets: a Pageable-carrying query*

`lumen/core/services/wallets/list_rich_wallets_query.py`

```
from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.data import Page, Pageable
from pyfly.cqrs import Query

@dataclass(frozen=True)
class ListRichWallets(Query[Page[WalletDto]]):
```

```
"""List wallets whose balance is at least ``min_minor``, paged."""
```

```
min_minor: int
pageable: Pageable
```

Listing 7.12 — *ListRichWallets*: adds a balance threshold for Specification filtering

```
lumen/core/services/wallets/list_wallets_handler.py
```

```
from __future__ import annotations

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.data import Page

@query_handler
@service
class ListWalletsHandler(QueryHandler[ListWallets, Page[WalletDto]]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: ListWallets) -> Page[WalletDto]:
        # type: ignore[override]
        page = await self._repository.find_all(query.pageable)
        return page.map(entity_to_dto)
```

Listing 7.13 — *ListWalletsHandler*: *find_all(pageable) + Page.map*

```
lumen/core/services/wallets/list_rich_wallets_handler.py
```

```
from __future__ import annotations
```

```

from lumen.core.mappers.wallet_mapper import entity_to_dto
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.data import Page

@query_handler
@service
class ListRichWalletsHandler(QueryHandler[ListRichWallets, Page[WalletDto]]):
    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    async def do_handle(self, query: ListRichWallets) -> Page[WalletDto]: #
type: ignore[override]
        page = await self._repository.find_rich(query.min_minor,
query.pageable)
        return page.map(entity_to_dto)

```

Listing 7.14 — ListRichWalletsHandler: Specification + find_all_by_spec_paged

`find_all(pageable)` is inherited from the framework `Repository` base. It counts the total rows, applies the `Pageable`'s sort, and slices with `LIMIT / OFFSET` — returning a `Page[WalletEntity]` that carries `items`, `total`, `page`, `size`, `total_pages`, `has_next`, and `has_previous`. `Page.map(entity_to_dto)` transforms the items without touching the metadata. The controller wraps the result in a `PageDto` for the wire.

`find_rich` is defined on `WalletRepository` itself and delegates to the inherited `find_all_by_spec_paged`. It constructs a `Specification` — a composable `WHERE` predicate — and passes it alongside the `Pageable`. The framework appends the

`WHERE` clause, the sort, and the `LIMIT / OFFSET`, then executes a count query for the total. The handler calls `repo.find_rich(query.min_minor, query.pageable)` and maps the page exactly as before.

Executing a query goes through `QueryBus.query`:

```
from pyfly.cqrs import DefaultQueryBus
from lumen.core.services.wallets.get_balance_query import GetBalance

balance_dto = await query_bus.query(GetBalance(wallet_id="wlt-001"))
```

The return value is whatever `do_handle` returned — a `BalanceDto` or `None`. `None` means the wallet was not found. The controller is responsible for translating that into HTTP 404, keeping HTTP concerns out of the handler.

QUERIES RETURN NONE, NOT EXCEPTIONS

Query handlers return `None` when the resource is not found rather than raising `AggregateNotFound`. This is a deliberate convention: a query that finds nothing is not an error — it is an answer. The controller turns a `None` result into a 404 response, keeping the HTTP concern out of the handler.

Wiring the bus into the controller

The controller is the system's HTTP boundary. Its only job is to translate an HTTP request into a domain message and map the result back to an HTTP response. Everything in between belongs to the bus and the handlers — and that boundary becomes much cleaner once the controller dispatches commands and queries instead of calling service methods directly.

Before CQRS, `WalletController` held a reference to `WalletApplicationService` and called methods on it directly. Every time the service interface changed — a new parameter, a renamed method, a different return type — the controller had to change too. With CQRS, the controller knows one thing: what message to send.

The controller injects `DefaultCommandBus` and `DefaultQueryBus` by type — the **concrete bus classes**, not abstract protocols. This is the correct import:

```
from pyfly.cQRS import DefaultCommandBus, DefaultQueryBus
```

Why concrete classes? PyFly's CQRS auto-configuration registers exactly one instance of each bus in the DI container. Injecting by the concrete type is unambiguous — no protocol dispatch, and the type checker sees the full `send` / `query` surface. Using a protocol alias would require an explicit container binding; the concrete type works out of the box.

Route ordering: why the single-resource handlers are named `wallet_*`

The framework registers a controller's routes in **alphabetical method-name order**; Starlette's router then applies first-registered-wins matching. This means a literal segment like `/rich` must be registered *before* the path variable `/ {wallet_id}` — otherwise every `GET /api/v1/wallets/rich` request would match the variable route and look for a wallet whose id is the string `"rich"`.

The collection handlers are named `list_wallets` and `list_rich_wallets`; the single-resource handlers are named `wallet_detail` and `wallet_balance`.

Alphabetically, `l` sorts before `w`, so the collection routes (`GET /`, `GET /rich`) are always registered ahead of the parameterised routes (`GET / {wallet_id}`, `GET / {wallet_id}/balance`). If you rename `wallet_detail` to something that sorts before `list_*`, the `/rich` route will silently break.

Here is the complete controller:

lumen/web/controllers/wallet_controller.py

```

from __future__ import annotations

from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest
from lumen.interfaces.dtos.v1.page_dto import PageDto
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.container import rest_controller
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
from pyfly.data import Pageable, Sort
from pyfly.kernel import ResourceNotFoundException
from pyfly.web import (
    Body, PathVar, QueryParam, Valid,
    get_mapping, post_mapping, request_mapping,
)

#: Newest-first ordering shared by the list endpoints.
_NEWEST_FIRST = Sort.by("created_at").descending()

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, withdraw, list, inspect."""

    def __init__(
        self, commands: DefaultCommandBus, queries: DefaultQueryBus
    ) -> None:
        self._commands = commands

```

```

        self._queries = queries

# --- commands -----

@post_mapping("", status_code=201)
async def open_wallet(
    self, request: Valid[Body[OpenWalletRequest]]
) -> dict[str, str]:
    wallet_id = await self._commands.send(
        OpenWallet(owner_id=request.owner_id, currency=request.currency)
    )
    return {"wallet_id": wallet_id}

@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) -> dict[str, int | str]:
    balance = await self._commands.send(
        DepositFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

@post_mapping("/{wallet_id}/withdraw")
async def withdraw(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
) -> dict[str, int | str]:
    balance = await self._commands.send(
        WithdrawFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# --- paged / specification queries (registered before /{wallet_id}) ---

@get_mapping("")
async def list_wallets(

```

```

        self, page: QueryParam[int] = 1, size: QueryParam[int] = 20
    ) -> PageDto[WalletDto]:
        result = await self._queries.query(
            ListWallets(pageable=Pageable.of(page, size, _NEWEST_FIRST))
        )
        return PageDto.from_page(result)

@get_mapping("/rich")
async def list_rich_wallets(
    self,
    min_minor: QueryParam[int] = 0,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
) -> PageDto[WalletDto]:
    result = await self._queries.query(
        ListRichWallets(
            min_minor=min_minor,
            pageable=Pageable.of(page, size, _NEWEST_FIRST),
        )
    )
    return PageDto.from_page(result)

# --- single-wallet queries (named wallet_* so they sort after list_*) ---

@get_mapping("/{wallet_id}")
async def wallet_detail(self, wallet_id: PathVar[str]) -> WalletDto:
    result = await self._queries.query(GetWallet(wallet_id=wallet_id))
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

@get_mapping("/{wallet_id}/balance")
async def wallet_balance(self, wallet_id: PathVar[str]) -> BalanceDto:
    result = await self._queries.query(
        GetBalance(wallet_id=wallet_id)
    )

```



```

    )
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

```

Listing 7.15 — *WalletController: DefaultCommandBus + DefaultQueryBus + paged list endpoints*

Compare the constructor to its pre-CQRS form. Before, the controller took `WalletApplicationService` — a concrete class whose method signatures leaked business logic into the HTTP layer. Now it takes `DefaultCommandBus` and `DefaultQueryBus` — two opaque channels. The controller knows *what* to send; it knows nothing about *how* the message is processed.

Look at `open_wallet`. Before, it called `self._service.open_wallet(owner_id=..., currency=...)` — a positional contract that breaks whenever the service grows a new parameter. Now it constructs `OpenWallet(owner_id=request.owner_id, currency=request.currency)` — a named, immutable object whose fields are its own API. Add a field to the command? The controller stays the same until you choose to populate it.

The request DTOs (`OpenWalletRequest`, `DepositRequest`) are Pydantic models in `lumen/interfaces/dtos/v1/`. `OpenWalletRequest` validates `owner_id` length and constrains `currency` to the `Currency` enum. `DepositRequest` is shared by both the deposit and withdraw endpoints — both move a positive `amount` in the wallet's own currency. Field-level constraints in those DTOs are enforced by `Valid[Body[...]]` before the handler is ever called.

The paged list endpoints (`list_wallets` , `list_rich_wallets`) build a `Pageable` from the query-string parameters, dispatch the query through the bus, and wrap the resulting `Page[WalletDto]` in a `PageDto` for the wire. `PageDto` is a Pydantic model that mirrors all the `Page` metadata fields — `total` , `total_pages` , `has_next` , `has_previous` — so clients get consistent pagination envelopes without a custom serializer.

The `wallet_detail` and `wallet_balance` methods show the only remaining HTTP concern in the controller: translating a `None` query result into a 404 via `ResourceNotFoundException` . That mapping belongs here because 404 is an HTTP status code and the handler deliberately has no HTTP knowledge. Return types are declared as `WalletDto` and `BalanceDto` — Pydantic models the framework serializes to JSON automatically.

💡 LET THE BUS RAISE

You do not need to catch `CommandProcessingException` or `QueryProcessingException` in the controller unless you want to customize the error shape. The global exception handler maps `AggregateNotFound` to 404 and `BusinessRuleViolation` to 422 — the same as before. The bus exceptions propagate those originals transparently.

The handler pipeline

A single `send` or `query` call triggers more than just the handler. Understanding the pipeline tells you where to put each cross-cutting concern — and, just as importantly, where *not* to put it.

The pipeline is defined once, in the bus, and applies uniformly to every handler. You never write pipeline logic inside a handler. The order is strict:

Step	Where it is defined	Applies to	Failure result
Business pre-condition validation	<code>validate()</code> hook on the message	Commands + Queries	<code>CqrsValidationException</code> (HTTP 422)
Authorization	<code>authorize()</code> hook on the message	Commands + Queries	<code>AuthorizationException</code> (HTTP 403)
Handler execution	<code>do_handle()</code>	Commands + Queries	Domain exceptions (4xx/5xx)
Domain event publishing	Bus pipeline (post-handler)	Commands only	—
Correlation ID cleanup	Bus pipeline (finally block)	Commands + Queries	—

Validation

Without a structured validation step, every handler would open with its own guard clauses: check this field is not blank, check that amount is positive. That logic would be duplicated across handlers and tested only through integration paths. Centralizing validation in the message itself solves both problems.

The bus invokes `validate()` before looking up the handler. If validation fails, the bus raises `CqrsValidationException` without ever reaching the handler.

The validation hook is also the right place for cross-field pre-conditions knowable from the fields alone — too simple for the domain aggregate, too application-specific for the request model:

```
@dataclass(frozen=True)
class DepositFunds(Command[int]):
```

```

wallet_id: str
amount: int

    async def validate(self) -> ValidationResult:
        if not self.wallet_id.strip():
            return ValidationResult.failure(
                "wallet_id", "Wallet id is required"
            )
        if self.amount <= 0:
            return ValidationResult.failure(
                "amount", "Deposit amount must be > 0"
            )
        return ValidationResult.success()

```

Authorization

Once a message is structurally valid, the bus asks: is the caller *allowed* to perform this operation? Authorization answers before any database access happens — more efficient and safer, since you never load sensitive data only to discard it because the caller lacked permission.

Both commands and queries expose an `authorize()` hook. Return `AuthorizationResult.success()` to allow execution, or `AuthorizationResult.failure(resource, message)` to deny it. The bus raises `AuthorizationException` on denial, mapping to HTTP 403 via the global error handler.

A clean rule of thumb: use `authorize()` on the command for **operation-level** checks — who is allowed to call this command at all — and leave **resource-level** decisions (can this caller access *this specific* wallet?) to the handler, which has the loaded aggregate in scope:

```
lumen/cqrs/commands_auth.py
```

```

from __future__ import annotations
from dataclasses import dataclass

from pyfly.cqrs.authorization.types import AuthorizationResult
from pyfly.cqrs import Command

@dataclass(frozen=True)
class CloseWallet(Command[None]):
    """Close a wallet. Only internal service accounts may do this."""
    wallet_id: str
    requested_by: str

    async def authorize(self) -> AuthorizationResult:
        internal_accounts = {"ops-service", "compliance-bot"}
        if self.requested_by not in internal_accounts:
            return AuthorizationResult.failure(
                "wallet",
                "Only internal service accounts may close wallets",
            )
        return AuthorizationResult.success()

```

Listing 7.16 — Authorization hook on a command

`CloseWallet.authorize` checks a known set of internal service accounts. If `requested_by` is not in the set, authorization fails before the handler is called. The set would normally come from a configuration value or a token claim injected at the controller boundary — it is hardcoded here for readability. The key point is that the check lives inside the command, not scattered across handler code.

Distributed tracing

When one HTTP request triggers multiple commands — and each command may call downstream services — you need a way to stitch all the logs and spans together. That is what `CorrelationContext` provides.

Both buses set a correlation ID at the start of every pipeline execution. If the message carries an ID already (set via `command.set_correlation_id(id)`), that ID is used; otherwise a new UUID is generated. The prior ID is always restored in a `finally` block, so nested command dispatches within the same request do not clobber the outer trace.

`CorrelationContext` propagates across `await` chains via Python's `contextvars` — no need to thread the ID manually through every function argument. For cross-service propagation, serialize the context to outgoing headers and restore it on the receiving side:

```
from pyfly.cqrs.tracing.correlation import CorrelationContext

# On the sending side
headers = CorrelationContext.create_context_headers()
# {"X-Correlation-ID": "...", "X-Trace-ID": "...", "X-Span-ID": "..."}

# On the receiving side
CorrelationContext.extract_context_from_headers(headers)
```

The three headers — `X-Correlation-ID`, `X-Trace-ID`, and `X-Span-ID` — follow W3C Trace Context naming, so they are compatible with OpenTelemetry-instrumented infrastructure out of the box.

💡 WHERE TO PUT CROSS-CUTTING LOGIC

The bus pipeline is the right home for concerns that apply to *all* operations: validation, authorization, tracing, and metrics. The handler is the right home for concerns specific to *one* operation: loading the aggregate, driving behaviour, saving, draining events. If you find yourself adding a try/except to every handler, or copying the same pre-condition check into multiple handlers, it belongs in the pipeline — either as a `validate()` hook on the command or as a bus-level service. The pipeline scales uniformly; handler boilerplate does not.

✓ What you built

Part II is complete. Lumen now has a full vertical slice from HTTP to domain and back — one built on architectural decisions that will scale without rewriting.

In Chapter 5 you gave the system persistence: a `WalletRepository` subclassing `Repository[WalletEntity, str]` — the framework's Spring-Data-style generic repository that provides `find_by_id`, `find_all(pageable)`, `find_all_by_spec_paged`, and more out of the box, with the `AsyncSession` injected by relational auto-configuration. In Chapter 6 you promoted the wallet to a proper DDD aggregate: `Money` as an immutable value object, `Wallet(AggregateRoot[str])` as the consistency boundary enforcing the overdraft, currency-match, and positive-amount invariants, with `WalletOpened`, `FundsDeposited`, and `FundsWithdrawn` domain events buffered in the aggregate and drained to the event bus after a successful save.

In this chapter you separated the write model from the read model. `OpenWallet`, `DepositFunds`, and `WithdrawFunds` are frozen, validated command messages that flow through `DefaultCommandBus` — a pipeline that runs validation, authorization, handler execution, domain event publishing, and distributed tracing automatically for every command. Each command handler carries `@transactional()` on `do_handle`: the decorator opens a committed unit of work from `self._session_factory`, swaps the session onto the repository, commits on success, and rolls back on failure. Persistence goes through `repository.upsert` — backed by `session.merge` — so INSERT and UPDATE share a single code path keyed on the aggregate's own id.

`GetWallet` and `GetBalance` are query messages that flow through `DefaultQueryBus` — the same pipeline without the event-publishing step, and without `@transactional()` because reads do not commit. `GetBalanceHandler` projects through a `@projection`-marked `BalanceView` interface and `Mapper.project`, copying only the

declared fields and applying a registered major-unit transform. `ListWallets` and `ListRichWallets` round out the query side: `find_all(pageable)` returns a counted, sorted, offset-limited `Page[WalletEntity]`; `find_all_by_spec_paged` runs a composable `Specification` predicate on top of the same pagination machinery. Both use `Page.map(entity_to_dto)` to project items without touching the metadata.

Each handler carries the `@command_handler` + `@service` (or `@query_handler` + `@service`) stack: the first decorator registers the class by introspecting its generic type argument; the second wires it into the DI container so constructor dependencies are injected automatically.

`WalletController` no longer knows about the service layer. It injects `DefaultCommandBus` and `DefaultQueryBus`, builds a command or query from the HTTP request, dispatches it, and either returns the result or raises a domain exception. Single-resource handler methods are named `wallet_detail` and `wallet_balance` — a deliberate choice so they sort alphabetically *after* the collection methods `list_wallets` and `list_rich_wallets`, ensuring the literal `/rich` segment is registered before the `/ {wallet_id}` variable route.

Adding a new command now means three things: define a frozen dataclass, implement one `do_handle` decorated with `@command_handler` + `@service` and annotated with `@transactional()`, and add one endpoint that calls `self._commands.send`. The pipeline applies automatically.

Try it yourself

1. **Trace the full lifecycle in the test suite.** Open `samples/lumen/tests/test_cqrs_flow.py` and run it against a real database using Testcontainers (Chapter 11). The test `test_full_wallet_lifecycle` opens a wallet, deposits 1

500 minor units, withdraws 500, then queries both `GetWallet` and `GetBalance`. Step through it with a debugger: confirm that `wallet.clear_events()` drains the `FundsDeposited` and `FundsWithdrawn` events after each `upsert` call, and that `GetWallet` returns a `WalletDto` with `balance_minor == 1000` and `balance == 10.0`.

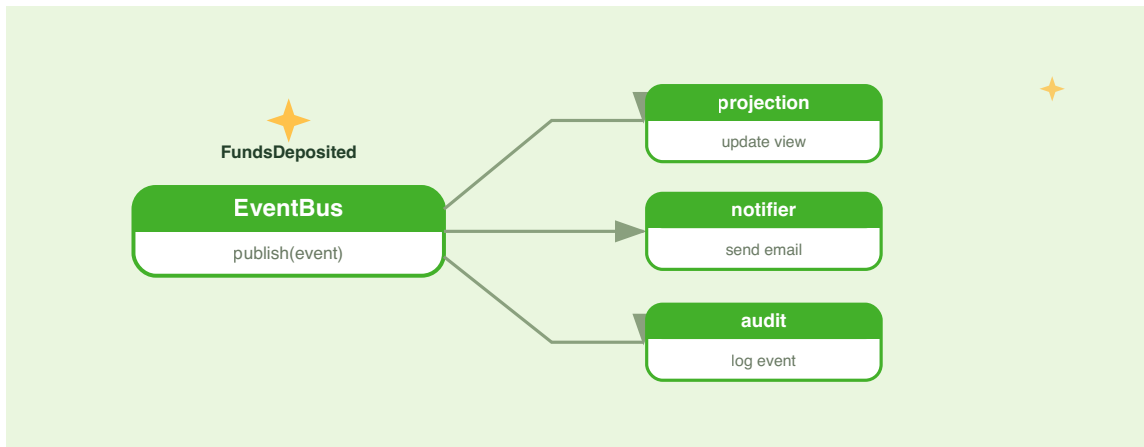
2. **Observe `upsert` vs `save`**. In a test, call `DepositFunds` twice on the same wallet without `@transactional()` and observe the `IntegrityError`. Then restore `@transactional()` and verify both deposits commit. Open `WalletRepository.upsert` and trace how `session.merge` resolves the primary-key conflict that a plain `INSERT` would raise.
3. **Add a `ListByOwner` query**. Define `ListByOwner(Query[list[WalletDto]])` with an `owner_id: str` field. Implement `ListByOwnerHandler` — decorated with `@query_handler` + `@service` — that calls `WalletRepository.find_by_owner_id(query.owner_id)` (the derived query stub already exists) and maps the result list with `entity_to_dto`. Add a `GET /api/v1/wallets/by-owner/{owner_id}` endpoint to `WalletController`. Ensure the new endpoint method name sorts before `wallet_detail` so Starlette matches the literal `/by-owner/...` segment first.
4. **Add authorization to `WithdrawFunds`**. Extend `WithdrawFunds` with an `initiated_by: str` field. Override `authorize()` to return `AuthorizationResult.failure("withdraw", "Initiator is required")` when `initiated_by` is blank, and `AuthorizationResult.success()` otherwise. Update `WithdrawFundsHandler.do_handle` to record `command.initiated_by` in the `FundsWithdrawn` event payload. Write a test that calls `await WithdrawFunds(wallet_id="wlt-1", amount=100, initiated_by="").authorize()` and asserts that the result denies authorization.

PART III

Event-Driven Architecture

CHAPTER 8

Domain Events & Event-Driven Architecture



Lumen's wallet saves correctly, validates rigorously, and dispatches every write through a typed command. But look at what the command handlers do after `repo.add`: nothing. The domain events that the **Wallet** aggregate buffers — **WalletOpened**, **FundsDeposited**, **FundsWithdrawn** — are drained and discarded. The bus pipeline that Chapter 7 promised would "publish domain events" has nowhere to send them yet.

The gap matters in practice. Lumen needs a balance read model that stays in sync without reloading the aggregate on every query, a welcome notification when a new wallet opens, and an immutable audit trail of every financial movement for compliance. All three features depend on knowing *that something happened* — not who requested it or how it was handled.

Domain events are the answer. Instead of having a command handler call the notification service directly — or coupling the audit log to the repository — you publish the event: a concise, timestamped, immutable record of a fact. Every interested party subscribes independently. The handler that saved the wallet does not need to know what the auditor does, or even that an auditor exists. You can add a new subscriber months later without touching a single line of existing handler code.

This chapter builds the reaction side of Lumen's architecture. You will wire `EventPublisher` into the command handlers, introduce the `publish_domain_events` bridge that drains the aggregate's buffer and forwards each event to the bus, and write a `WalletAuditListener` that subscribes using `@event_listener` and maintains two in-memory projections: an immutable audit trail and a running deposit total. By the end of the chapter the write path and the read infrastructure are fully decoupled — each side evolves without the other noticing.

Two kinds of events

Before touching any code, it is worth being precise about what "event" means in PyFly. The framework uses the word for two distinct things, and confusing them leads to the wrong bus, the wrong subscription API, and subtle runtime surprises.

Application events (`pyfly.context.events`) are framework lifecycle notifications. `ContextRefreshedEvent` fires when the DI container finishes wiring; `ApplicationReadyEvent` fires when the HTTP server begins accepting connections; `ContextClosedEvent` fires during shutdown. The `ApplicationEventBus` dispatches them to subscribers matched by Python class type — they are infrastructure plumbing for bootstrapping, deliberately separate from any business concept.

Domain events (`pyfly.eda`) are business-level facts: *a wallet was opened, funds were deposited, a transfer was completed*. The `EventPublisher` port wraps each payload in an `EventEnvelope` and routes it by the domain event class name — `"WalletOpened"`, `"FundsDeposited"`, `"FundsWithdrawn"` — so listeners subscribe to named business facts rather than implementation details. Domain events are the subject of this chapter.

The distinction shapes what you can do with each kind. The `ApplicationEventBus` dispatches to callables keyed by class; `InMemoryEventBus` routes by class name and can be swapped for a Kafka-backed adapter without touching subscriber code. The rule is simple: use lifecycle events for infrastructure bootstrapping, domain events for everything with business meaning.

APPLICATION EVENTS ARE STILL USEFUL

If you need to warm a cache as soon as the application is ready, `@app_event_listener` on `ApplicationReadyEvent` is the right tool. The two systems coexist; you can use both in the same service.

Publishing events

The EventPublisher port

The first question a new team member usually asks is: "which class do I import to fire an event?" The answer is deliberately not a class — it is a protocol. `EventPublisher` is a **port** in the hexagonal-architecture sense: any code that needs to publish an event depends on this interface, and the bus implementation is injected from outside. That design decision is what lets you run `InMemoryEventBus` locally today and swap in a Kafka adapter in production without touching a single handler.

The protocol exposes two methods:

```
from pyfly.eda import EventPublisher

class EventPublisher(Protocol):
    def subscribe(self, event_type_pattern: str, handler: EventHandler) -> None: ...

    async def publish(
        self,
        destination: str,
        event_type: str,
        payload: dict,
        headers: dict[str, str] | None = None,
    ) -> None: ...
```

`publish` wraps your data in an `EventEnvelope` before delivery — you never construct the envelope yourself. `subscribe` registers handlers programmatically, though in practice you will use the `@event_listener` decorator instead, because it lets the `ApplicationContext` wire subscriptions automatically at startup.

The bus bean exists only when `pyfly.eda.provider` is configured. For Lumen, `pyfly.yaml` sets it to `memory`:

`pyfly.yaml`

```
pyfly:
  eda:
    provider: memory
    # ... other keys omitted for brevity
```

Listing 8.0 — Enabling the in-memory EDA bus in `pyfly.yaml`

Without this line the `EventPublisher` bean is not registered and any handler that declares `events: EventPublisher` in its constructor will fail to start.

The EventEnvelope

Every domain event reaches its listeners wrapped in an `EventEnvelope`. Think of it as the metadata layer that transforms a bare Python dictionary into a traceable, auditable, first-class fact. It is a frozen dataclass — immutable once created — that pairs the payload with the context every listener needs.

Field	Type	Default	Description
<code>event_type</code>	<code>str</code>	required	The domain event class name, e.g. <code>"FundsDeposited"</code> . Used for routing.
<code>payload</code>	<code>dict[str, Any]</code>	required	The event data.
<code>destination</code>	<code>str</code>	required	Logical channel or topic, e.g. <code>"wallet.events"</code> .
<code>event_id</code>	<code>str</code>	auto UUID	Unique ID for this event instance.
<code>timestamp</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	UTC creation time.
<code>headers</code>	<code>dict[str, str]</code>	<code>{}</code>	Arbitrary metadata: correlation IDs, trace context, and so on.

Three fields deserve particular attention. `event_id` is a stable UUID generated by the bus at publish time — your **idempotency key** for exactly-once semantics, available in every listener with no extra work. `timestamp` records when the fact was observed, not when the listener processes it, so it stays accurate even if a listener runs with a delay. `headers` carries cross-cutting concerns such as distributed trace IDs — metadata that has nothing to do with the business payload but matters enormously for observability. Because the envelope is frozen, handlers can safely pass it across async boundaries without defensive copying.

`event_type` holds the **class name** of the domain event — `"WalletOpened"`, `"FundsDeposited"`, or `"FundsWithdrawn"` — not a dot-separated path. Listeners subscribe by those same class names, so the subscription contract is defined by the domain model, not by string conventions invented outside it.

The domain events in the Wallet aggregate

The `Wallet` aggregate raises typed, frozen-dataclass domain events. Each event's class name becomes its routing `event_type` on the bus:

`lumen/models/entities/v1/wallet_entity.py`

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import AggregateRoot, BusinessRuleViolation, DomainEvent

@dataclass(frozen=True)
class WalletOpened(DomainEvent):
    wallet_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0
```



```

@dataclass(frozen=True)
class FundsWithdrawn(DomainEvent):
    wallet_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

class Wallet(AggregateRoot[str]):
    """Wallet aggregate root – owns the ``balance >= 0`` invariant."""

    def deposit(self, amount: Money) -> None:
        """Credit *amount*; raises FundsDeposited."""
        self._assert_currency(amount)
        self.balance = self.balance.add(amount)
        self.raise_event(
            FundsDeposited(
                wallet_id=self.id,
                amount=amount.amount,
                currency=amount.currency.value,
                balance=self.balance.amount,
            )
        )
        # ... open() and withdraw() follow the same pattern

```

Listing 8.1 — Domain events raised by the Wallet aggregate

`DomainEvent` is a base frozen dataclass. Its `event_type` property returns `type(self).__name__` — the class name — which is exactly what `EventPublisher` uses as the routing key. `raise_event` buffers the event in the aggregate; the command handler drains that buffer by calling `wallet.clear_events()` after a successful persist.

The publish bridge

Rather than repeating the drain loop in every command handler, Lumen extracts it into a single `publish_domain_events` coroutine. The bridge serialises each drained event with `dataclasses.asdict`, then calls `publisher.publish` with the class name as `event_type` and `"wallet.events"` as the logical channel:

`lumen/core/services/wallets/event_publishing.py`

```
from __future__ import annotations

import dataclasses
from collections.abc import Iterable
from typing import Any

from lumen.core.services.listeners.wallet_audit_listener import (
    WALLET_EVENTS_DESTINATION,
)
from pyfly.domain import DomainEvent
from pyfly.eda import EventPublisher

def _to_payload(event: DomainEvent) -> dict[str, Any]:
    """Flatten a frozen-dataclass domain event into a dict."""
    payload: dict[str, Any] = dataclasses.asdict(event)
    payload.setdefault("event_type", event.event_type)
    return payload

async def publish_domain_events(
    publisher: EventPublisher, events: Iterable[DomainEvent]
) -> None:
    """Publish each drained domain event on the wallet events channel.

    The envelope's ``event_type`` is the domain event class name
    (``WalletOpened`` / ``FundsDeposited`` / ``FundsWithdrawn``).
    """
    for event in events:
```

```

await publisher.publish(
    destination=WALLET_EVENTS_DESTINATION,
    event_type=event.event_type,
    payload=_to_payload(event),
)

```

Listing 8.2 — `publish_domain_events` bridges drained events to the EDA bus

`WALLET_EVENTS_DESTINATION` is the constant `"wallet.events"` defined in `wallet_audit_listener.py` and shared by publisher and listener so the channel name cannot drift. `event.event_type` is the class-name property on `DomainEvent`: `"WalletOpened"`, `"FundsDeposited"`, or `"FundsWithdrawn"`.

Wiring the publisher into the command handlers

In Chapter 7 the command handlers loaded aggregates, drove domain behaviour, and saved — leaving buffered events on the floor. Now you close that gap. Inject an `EventPublisher` alongside the `WalletRepository` and an `async_sessionmaker`, decorate `do_handle` with `@transactional()`, and after `repo.upsert(...)` drain the aggregate's buffer and publish each event through the bridge.

The `@transactional()` decorator (from `pyfly.data.relational.sqlalchemy`) opens a dedicated `AsyncSession` from the injected `async_sessionmaker`, binds it to the repository for the call, commits on success, and rolls back on failure. That means the load → mutate → save sequence is one committed unit of work, and no event is published unless the row actually lands in the database.

Here is the updated `DepositFundsHandler`:

`lumen/core/services/wallets/deposit_funds_handler.py`

```

from __future__ import annotations

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

```

```

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.eda import EventPublisher

```

```
@command_handler
```

```
@service
```

```

class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

```

```

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

```

```
@transactional()
```

```

    async def do_handle(self, command: DepositFunds) -> int:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))

```

```

await publish_domain_events(self._events, wallet.clear_events())
return wallet.balance.amount

```

Listing 8.3 — *DepositFundsHandler*: @transactional unit-of-work, then publish

Three design decisions are worth noting. First, `events: EventPublisher` is typed as the protocol, not as `InMemoryEventBus` — the DI container injects whichever implementation is registered, so the handler never knows or cares which bus is active. Second, the publish call sits *after* `self._repository.upsert(...)` inside the `@transactional()` unit of work: if the save fails the decorator rolls back before `publish_domain_events` is reached, so listeners never see a fact that never persisted. Third, the handler works with the ORM entity directly via `to_aggregate` / `to_entity` mappers — the aggregate is rehydrated from the row, mutated, and mapped back to a row before the upsert. If the publish fails after a successful save you have an at-least-once delivery challenge — Chapter 10 addresses that with transactional outbox patterns. For now, the in-memory bus never fails.

The `OpenWalletHandler` follows the same pattern:

```
lumen/core/services/wallets/open_wallet_handler.py
```

```

from __future__ import annotations

from uuid import uuid4

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.models.entities.v1.wallet_entity import Wallet
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional

```

```

from pyfly.eda import EventPublisher

@command_handler
@service
class OpenWalletHandler(CommandHandler[OpenWallet, str]):
    """Open a new, empty wallet."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        # @transactional resolves the unit-of-work session from here.
        self._session_factory = session_factory

    @transactional()
    async def do_handle(self, command: OpenWallet) -> str:
        wallet_id = f"wlt-{uuid4()}"
        wallet = Wallet.open(
            wallet_id=wallet_id,
            owner_id=command.owner_id,
            currency=command.currency,
        )
        await self._repository.upsert(to_entity(wallet))

        await publish_domain_events(self._events, wallet.clear_events())
        return wallet_id

```

Listing 8.4 — OpenWalletHandler: @transactional, upsert, then publish WalletOpened

`return wallet_id` comes *after* the publish call — the handler fulfils its contract only once every fact produced by the operation has been dispatched. The wallet id is generated locally with `uuid4()` rather than delegated to a repository method; `to_entity` maps the aggregate to a row before the upsert so the framework `Repository` receives the ORM model it expects.

The `@publish_result` shortcut

In simpler services where a method's return value *is* the event payload — common in code that has not adopted the full aggregate pattern — `@publish_result` removes the manual publish call entirely:

`lumen/eda/publish_result_example.py`

```
from pyfly.eda import publish_result
from pyfly.eda.adapters.memory import InMemoryEventBus

bus = InMemoryEventBus()

@publish_result(bus, destination="wallet.events",
event_type="FundsTransferred")
async def transfer_funds(source_id: str, target_id: str, amount: int) -> dict:
    # Business logic omitted – the returned dict IS the event payload.
    return {
        "source_id": source_id,
        "target_id": target_id,
        "amount": amount,
    }
```

Listing 8.5 — `@publish_result` auto-publishes the method's return value

When `transfer_funds` returns, the decorator intercepts the result and calls `bus.publish` with it as the payload — no boilerplate loop needed. `destination` and `event_type` are fixed at decoration time, keeping the business function clean.

`@publish_result` also accepts an optional `condition` predicate: the event is published only when the result satisfies the test, which is useful for conditional workflows where not every successful execution should broadcast.

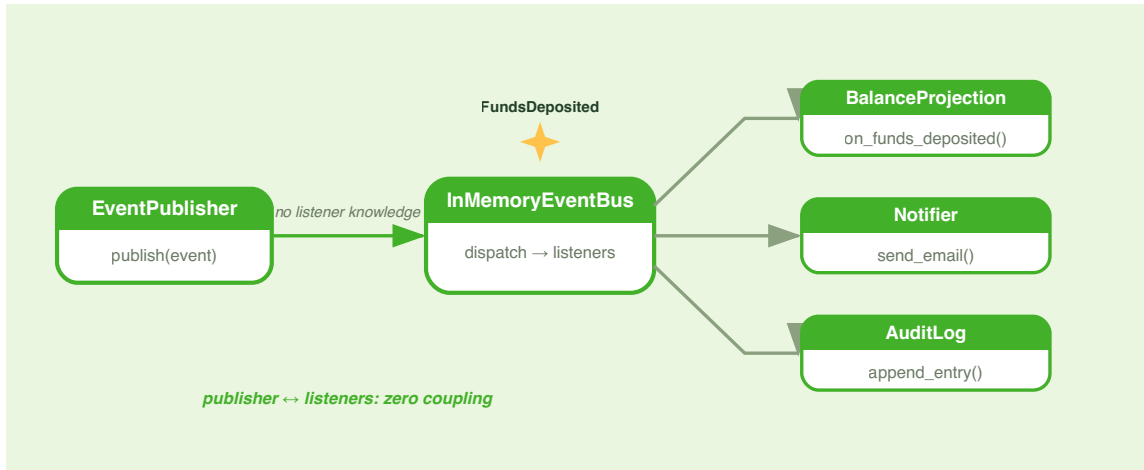


Figure 8.1 — One publisher, many independent listeners.

🌿 SPRING PARITY

`EventPublisher` is PyFly's counterpart of Spring's `ApplicationEventPublisher`. Calling `publisher.publish(...)` is equivalent to `applicationEventPublisher.publishEvent(event)`. The `@event_listener` decorator (next section) mirrors Spring's `@EventListener` for synchronous, same-transaction reactions. `@publish_result` achieves what Spring developers often wire manually with AOP `@AfterReturning` advice.

Reacting with `@event_listener`

Publishing an event is only half the picture. An event that nobody reacts to is just a log entry. The value of the event-driven model lies in the *reactions* it enables — independent behaviours that activate in response to the same published fact, each unaware of the others.

PyFly's `@event_listener` decorator is the simplest way to register a reaction.

Decorate any async method with the class names it cares about, and

`ApplicationContext` wires the subscription during startup — no bus reference needed at decoration time.

```
from pyfly.eda import event_listener, EventEnvelope

@event_listener(event_types=["FundsDeposited"])
async def on_funds_deposited(envelope: EventEnvelope) -> None:
    ...
```

`event_types` accepts exact class names. Listeners inside a `@service` class receive an `EventEnvelope` as their sole argument. Because matching happens at the bus level — not inside your function — a single listener method can subscribe to multiple event types in one declaration.

WalletAuditListener

Lumen's production listener is `WalletAuditListener`. It subscribes to all three wallet domain events and maintains two in-memory projections: an ordered **audit trail** and a **running net-deposit total** per wallet.

```
lumen/core/services/listeners/wallet_audit_listener.py
```

```
from __future__ import annotations

import logging
```

```

from dataclasses import dataclass
from datetime import datetime

from pyfly.container import service
from pyfly.eda import EventEnvelope, event_listener

logger = logging.getLogger(__name__)

WALLET_EVENTS_DESTINATION = "wallet.events"

@dataclass(frozen=True)
class AuditEntry:
    """One observed domain event, captured for the audit trail."""

    event_type: str
    wallet_id: str
    event_id: str
    occurred_at: datetime
    payload: dict[str, object]

@service
class WalletAuditListener:
    """In-memory audit log + running-total projection over wallet events."""

    def __init__(self) -> None:
        self._entries: list[AuditEntry] = []
        self._running_totals: dict[str, int] = {}

    @event_listener(
        event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"]
    )
    async def on_wallet_event(self, envelope: EventEnvelope) -> None:
        """Project every wallet domain event into the read models."""
        payload = dict(envelope.payload)
        wallet_id = str(payload.get("wallet_id", ""))

        self._entries.append(

```

```

    AuditEntry(
        event_type=envelope.event_type,
        wallet_id=wallet_id,
        event_id=str(payload.get("event_id", envelope.event_id)),
        occurred_at=envelope.timestamp,
        payload=payload,
    )
)

if envelope.event_type == "WalletOpened":
    self._running_totals.setdefault(wallet_id, 0)
elif envelope.event_type == "FundsDeposited":
    amount = int(payload.get("amount", 0))
    self._running_totals[wallet_id] = (
        self._running_totals.get(wallet_id, 0) + amount
    )
elif envelope.event_type == "FundsWithdrawn":
    amount = int(payload.get("amount", 0))
    self._running_totals[wallet_id] = (
        self._running_totals.get(wallet_id, 0) - amount
    )

logger.info(
    "wallet_audit_observed",
    extra={"event_type": envelope.event_type, "wallet_id": wallet_id},
)

@property
def entries(self) -> list[AuditEntry]:
    """A snapshot of the audit log, in observation order."""
    return list(self._entries)

def entries_for(self, wallet_id: str) -> list[AuditEntry]:
    """The audit entries recorded for one wallet."""
    return [e for e in self._entries if e.wallet_id == wallet_id]

def running_total(self, wallet_id: str) -> int:
    """Net funds (deposited minus withdrawn) for wallet_id, minor

```

```

units.""
    return self._running_totals.get(wallet_id, 0)

```

Listing 8.6 — *WalletAuditListener: audit trail + running-total projection*

Here is what the listener does, step by step.

`@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])` tells `ApplicationContext` to subscribe `on_wallet_event` to those three class names. Because the class is a `@service` bean, PyFly discovers it at startup and wires the subscriptions automatically — you never call `bus.subscribe` by hand.

`on_wallet_event` receives an `EventEnvelope`. `envelope.event_type` is the class name of the raised domain event. `envelope.payload` is the dict produced by `dataclasses.asdict` in the publish bridge, so its keys match the dataclass field names exactly — `wallet_id`, `amount`, `currency`, `balance`.

The method appends an `AuditEntry` for every event, then branches on `event_type` to update the running total. Notice what is absent: no import of the `Wallet` aggregate, no repository call, no knowledge of how the deposit was processed. The projection reacts purely to the published fact.

💡 ENVELOPE METADATA IN PROJECTIONS

`envelope.timestamp` gives you the authoritative event time — when the fact was recorded, not when the listener ran. Store it in your read model and you get a cheap `occurred_at` column for free, with no clock skew between writer and reader.

Testing the listener end-to-end

The test suite exercises the full publish-and-receive path with no mocks. The `conftest` wires a shared `InMemoryEventBus`, mirrors the `@event_listener` discovery step by subscribing `on_wallet_event` to each declared class name, and registers real command handlers that share the same bus reference:

```
# tests/conftest.py (abbreviated)
from pyfly.eda.adapters.memory import InMemoryEventBus

@pytest_asyncio.fixture
async def event_bus() -> InMemoryEventBus:
    yield InMemoryEventBus()

@pytest_asyncio.fixture
async def audit_listener(event_bus: InMemoryEventBus) -> WalletAuditListener:
    listener = WalletAuditListener()
    method = listener.on_wallet_event
    for pattern in method.__pyfly_event_patterns__:
        event_bus.subscribe(pattern, method)
    yield listener
```

With that wiring in place, the test sends real commands and asserts on the listener's read models:

`lumen/tests/test_event_listener.py`

```
from __future__ import annotations

import pytest
from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus
```

```

@pytest.mark.asyncio
async def test_listener_observes_wallet_events(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) -> None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=1500))
    await command_bus.send(WithdrawFunds(wallet_id=wallet_id, amount=400))

    entries = audit_listener.entries_for(wallet_id)
    assert [e.event_type for e in entries] == [
        "WalletOpened",
        "FundsDeposited",
        "FundsWithdrawn",
    ]

    deposited = entries[1]
    assert deposited.payload["amount"] == 1500
    assert deposited.payload["currency"] == "EUR"
    assert deposited.payload["balance"] == 1500

    # running_total = deposited - withdrawn
    assert audit_listener.running_total(wallet_id) == 1100

```

Listing 8.7 — End-to-end test: commands publish, listener projects

The test proves the full chain: `OpenWalletHandler` → `publish_domain_events` → `InMemoryEventBus` → `WalletAuditListener.on_wallet_event` → `audit_listener.entries_for(...)`. No mocks, no fakes — the production code path runs as written.

What makes this design compelling is that adding the listener required zero changes to the command handlers, the `Wallet` aggregate, or any repository.

`DepositFundsHandler` has no idea a projection exists. Both sides are entirely independent — each is a consequence of the same published fact, connected only by the bus.

When listeners fail: error strategies

A misbehaving listener raises a pointed question: should the failure stop the entire delivery chain, or should the bus continue notifying the remaining listeners? The right answer depends on the listener's role. PyFly gives you explicit control rather than imposing a single policy.

By default, `InMemoryEventBus` invokes listeners sequentially and propagates any exception — the right behaviour for development, where a failing listener should surface loudly. In production you usually need finer control.

`ErrorStrategy` is an enum that governs how the bus behaves when a listener raises:

```
from pyfly.eda import ErrorStrategy
```

Member	Value	Behaviour
<code>IGNORE</code>	<code>"IGNORE"</code>	Silently swallow the exception. Processing continues with the next handler.
<code>LOG_AND_CONTINUE</code>	<code>"LOG_AND_CONTINUE"</code>	Log the error then continue. The safest default for non-critical listeners.
<code>RETRY</code>	<code>"RETRY"</code>	Re-attempt delivery. Retry count and back-off are configured separately.
<code>DEAD_LETTER</code>	<code>"DEAD_LETTER"</code>	Move the failed event to a dead-letter destination for later inspection.
<code>FAIL_FAST</code>	<code>"FAIL_FAST"</code>	Propagate the exception immediately. No further handlers are invoked.

💡 MATCH STRATEGY TO LISTENER CRITICALITY

An audit listener should use `LOG_AND_CONTINUE` — a broken audit logger must not halt a financial transaction. A projection that drives query responses might warrant `RETRY` to ensure the read model stays consistent. A notifier can tolerate `IGNORE` since a missed welcome email is not a data integrity issue.

⚠️ SIDE EFFECTS AND IDEMPOTENCY

If a listener performs a side effect — writing a database row, sending an email — and the bus retries delivery after a transient failure, the effect can run more than once. Design listeners to be idempotent: write a row only if the `event_id` has not already been recorded, send an email only if the welcome flag is not already set. The `envelope.event_id` (a stable UUID generated by the bus) is your idempotency key.

In-memory today, a broker tomorrow

`InMemoryEventBus` is the out-of-the-box implementation — the default `EventPublisher` the `ApplicationContext` provides. It runs entirely in-process: `publish` is a direct async call, there is no serialisation, and undelivered events vanish if the process dies. For local development, integration tests, and monoliths that need no cross-process delivery that is perfectly acceptable.

Understanding how the in-memory bus works internally makes it easier to reason about behaviour at the edges — and to appreciate exactly what changes when you swap in a broker.

```
from pyfly.eda.adapters.memory import InMemoryEventBus

bus = InMemoryEventBus()

bus.subscribe("FundsDeposited", my_handler)

await bus.publish(
    destination="wallet.events",
    event_type="FundsDeposited",
    payload={"wallet_id": "w-001", "amount": 5000, "currency": "EUR", "balance": 5000}
)
```

A `publish` call executes four steps in sequence:

1. Wraps the arguments in an `EventEnvelope` with a generated `event_id` and a UTC `timestamp`.
2. Iterates every registered `(pattern, handler)` pair.
3. For each pair where `fnmatch.fnmatch(event_type, pattern)` is `True`, calls the handler with the envelope.
4. Handlers run sequentially in subscription order.

Subscriptions use Python's `fnmatch`, so `"Funds*"` matches both `"FundsDeposited"` and `"FundsWithdrawn"`, and `"*"` matches everything. The sequential invocation in step 4 makes listener order deterministic — useful in tests — but it also means a slow listener delays all subsequent ones. Broker-backed adapters typically dispatch in parallel; keep that difference in mind when reasoning about throughput.

Because every listener in Lumen depends on the `EventPublisher` *protocol*, not on `InMemoryEventBus` directly, the implementation swaps without touching a single listener. Chapter 10 introduces Kafka and RabbitMQ adapters; switching either in is a configuration change — `WalletAuditListener` keeps working without modification.

INMEMORYEVENTBUS AND TESTING

`InMemoryEventBus` is also the right tool for tests. Inject a fresh `InMemoryEventBus` as a fixture, subscribe a capturing handler, exercise your command handler, and assert on the `EventEnvelope` objects the handler received — including `event_type`, `payload`, `event_id`, and `timestamp`. No mocking, no fakes, just the real bus with controlled inputs.

✓ What you built

Part III is open.

This chapter closed the loop Chapter 7 began. `Wallet` raised domain events in Chapter 6; the command handlers published them here; `WalletAuditListener` reacts to those facts without knowing anything about the command path that triggered them.

The architecture is genuinely event-driven within a single process. Here is a quick reference to each piece:

Piece	Role
<code>EventPublisher</code>	Port — a protocol any bus implementation fulfils
<code>InMemoryEventBus</code>	Default adapter — in-process, zero config; activated by <code>pyfly.eda.provider: memory</code>
<code>EventEnvelope</code>	Carries payload + <code>event_id</code> , <code>timestamp</code> , <code>destination</code> , <code>headers</code>
<code>@event_listener(event_types=[...])</code>	Subscription decorator — class names; context wires it at startup
<code>publish_domain_events</code>	Bridge — drains <code>wallet.clear_events()</code> , serialises with <code>dataclasses.asdict</code> , calls <code>publisher.publish</code>
<code>ErrorStrategy</code>	Controls failure handling: <code>IGNORE</code> , <code>LOG_AND_CONTINUE</code> , <code>RETRY</code> , <code>DEAD_LETTER</code> , <code>FAIL_FAST</code>

Three principles carry forward into the rest of Part III: **save before you publish** — listeners must never see uncommitted facts; **design listeners for idempotency** — retries must be safe; **depend on the port, not the adapter** — the bus can be swapped without touching listener code.

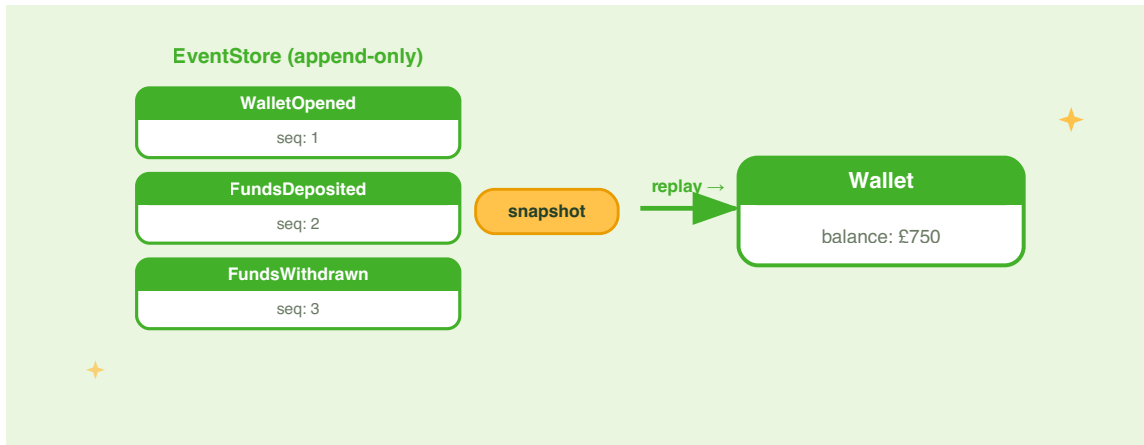
Chapter 9 pushes the event idea further. Instead of maintaining a separate read model alongside a mutable aggregate, you store the events themselves as the system of record — event sourcing the ledger so that every historical balance is computable from first principles.

Try it yourself

1. **Add a `FraudDetector` listener.** Create a `FraudDetector` service that subscribes to `"FundsDeposited"` using `@event_listener(event_types=["FundsDeposited"])`. If the `amount` in the payload exceeds `1_000_000` (ten thousand euros in minor units), log a warning that includes the `envelope.event_id`, `envelope.timestamp`, and the `wallet_id` from the payload. Verify it fires by publishing a `FundsDeposited` event directly to an `InMemoryEventBus` in a unit test, and assert that the warning was triggered.
2. **Extend `WalletAuditListener` with per-event filtering.** Add a method `entries_by_type(self, event_type: str) -> list[AuditEntry]` that returns only the entries with a matching `event_type`. Write a test that opens a wallet, makes two deposits and one withdrawal, and asserts that `entries_by_type("FundsDeposited")` returns exactly two entries.
3. **Observe error strategy behaviour.** Create a listener whose handler raises `RuntimeError("failure")` unconditionally. Register it alongside a list-appending capturing handler on an `InMemoryEventBus`. Configure `ErrorStrategy.LOG_AND_CONTINUE` and confirm the capturing handler still receives the event despite the failure. Then switch to `ErrorStrategy.FAIL_FAST` and confirm the capturing handler does *not* receive the event.

CHAPTER 9

Event Sourcing the Ledger



Chapter 8 left one structural gap unresolved. The `WalletAuditListener` keeps a fast read model by reacting to domain events — but the wallet's canonical state is still a row in the `wallets` table, holding a single `balance` column. Every time the balance changes, the previous value is gone forever. If a compliance auditor asks "what was the balance of wallet `w-001` at 14:32 on 3 March?", the honest answer is: you cannot know. The database remembers only the present.

Event sourcing turns this design inside out. Instead of storing the *current* state and discarding each change, you store the *sequence of changes* and derive the current state by replaying them. The `wallets` table disappears. In its place is an **event stream**: an append-only log of every `LedgerOpened`, `Credited`, and `Debited` event the ledger has ever produced. The balance at any point in time is a pure function of the events up to that moment. You can rewind to 14:32 on any date because you have a complete record of everything that happened between then and now.

A financial ledger is the ideal domain for event sourcing. Accountants have understood for centuries that a ledger's authority comes from its entries, not from a running total at the foot of the column. The running total is a *derived fact*; the entries are the *source of truth*. PyFly's `pyfly.eventsourcing` module brings that accounting intuition into code: aggregates emit domain events, an `EventStore` records them immutably, a repository replays the stream to reconstruct state, and a `ProjectionRunner` builds read models on top.

This chapter builds the `LedgerAccount` aggregate — a purpose-built event-sourced domain object that sits alongside the Chapter 6 state-stored `Wallet`. You will see every component of the `pyfly.eventsourcing` module, and how the event store, snapshotting, projections, and the transactional outbox work together to give Lumen a ledger that is both auditable and performant.

From state to events

The clearest way to grasp the shift is to compare what the database looks like in each model.

In the **state-storage model**, the database holds only the current state of the wallet:

wallet_id	owner_id	balance_cents	currency	updated_at
w-001	u-42	8500	EUR	2026-03-03 17:11

Every `deposit` and `withdraw` overwrites `balance_cents`. The history is gone. You know the wallet holds 85.00 EUR right now; you cannot know how it got there.

In the **event-storage model**, the database holds the event stream:

stream_id	seq	event_type	payload	occurred_at
led-001	1	LedgerOpened	<code>{"currency": "EUR", "owner_id": "u-42"}</code>	2026-03-01 09:00
led-001	2	Credited	<code>{"amount": 10000, "balance": 10000}</code>	2026-03-01 09:01
led-001	3	Debited	<code>{"amount": 1500, "balance": 8500}</code>	2026-03-03 17:11

The current balance is still 85.00 EUR — but now you can read every decision that led to it. An auditor, a regulator, or a fraud investigator can replay the stream from any offset and see exactly what happened and when.

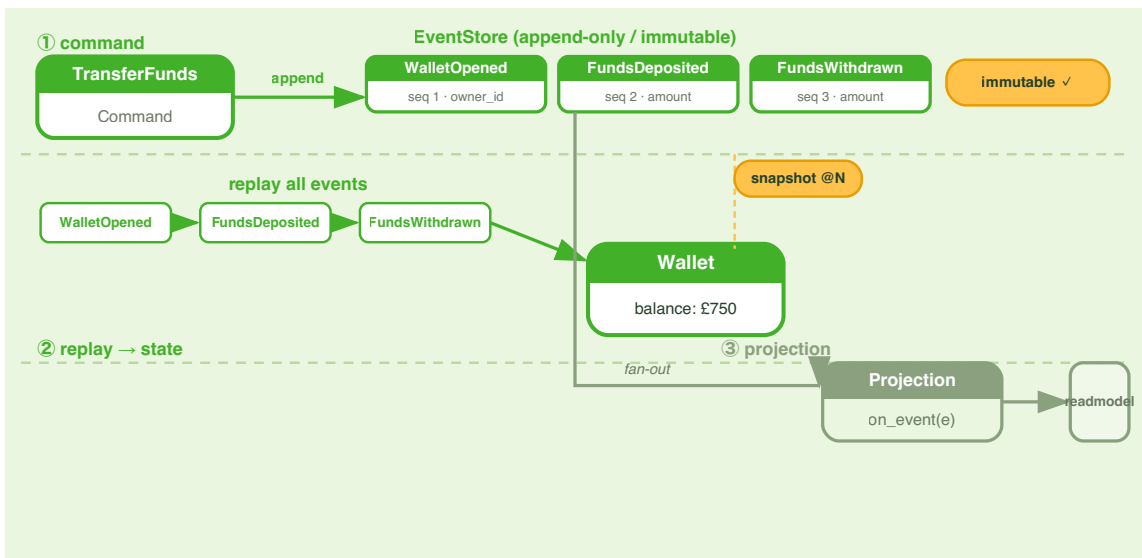


Figure 9.1 — State storage vs event storage: one model keeps a snapshot of the present; the other keeps every fact that led to it.

The trade-off is real. Event storage makes reads more expensive by default — you must replay the stream to compute the current balance — and it demands discipline around schema evolution (events are immutable; you cannot rename a field after the fact). Both concerns have solutions in PyFly: **snapshots** accelerate replay of long streams, and **upcasters** translate old event shapes to new ones during load. You will see both before the end of this chapter.

EVENTS AS THE SYSTEM OF RECORD

Event sourcing is not the same as event-driven architecture. Chapter 8 used EDA: the aggregate stored its state normally and published domain events as a side effect. Event sourcing goes further: the events *are* the state. There is no separate `balance` column to keep in sync — the balance is computed by the repository every time it loads the aggregate.

A separate base for event-sourced aggregates

Before writing a line of ledger code, one naming distinction matters. Chapter 6 built

`Wallet` on `pyfly.domain.AggregateRoot` — the state-stored base class.

`LedgerAccount` uses a **different** base class: `pyfly.eventsourcing.AggregateRoot`.

The two live in separate packages and are deliberately unrelated.

Concern	Chapter 6 <code>Wallet</code>	Chapter 9 <code>LedgerAccount</code>
Base class	<code>pyfly.domain.AggregateRoot</code>	<code>pyfly.eventsourcing.AggregateRoot</code>
Domain event	<code>pyfly.domain.DomainEvent</code>	<code>pyfly.eventsourcing.DomainEvent</code>
State lives in	a database row	the event stream
Repository	<code>WalletRepository</code> (R2DBC)	<code>LedgerAccountRepository</code> (<code>EventSourcedRepository</code>)

`pyfly.eventsourcing.AggregateRoot` supplies the event-sourcing machinery through four members:

- `when(EventType, handler)` — registers a handler for a given event class. The handler receives `(aggregate, event)` as two arguments and performs the mutation — a lambda, a free function, or a one-liner delegating to a private method.
- `apply(event)` — routes a brand-new event through its registered handler and queues it for the event store. Both happen atomically: in-memory state updates immediately, with no round-trip to the store.
- `replay(event_type, event)` — re-runs a persisted event through the same handler *without* re-queuing it. The repository calls this during load to rebuild state from the stored stream.
- `version` — an integer counter incremented after each dispatched event; the store uses it as the optimistic-concurrency token.

Dispatch order is: registered `when()` handler first; then a method named `on_{event_type}` if one exists on the aggregate; if neither is found, `EventHandlerException` is raised. A missing handler would silently corrupt reconstructed state, so the aggregate fails loudly rather than proceeding.

⚠ TWO-ARG HANDLER — THE MOST COMMON TRAP

Every handler registered with `when()` is called as `handler(aggregate, event)` — two arguments. A bound method like `self._on_opened` has signature `(self, event)`, which makes it a one-arg callable from the outside. Passing a bound method directly causes a `TypeError` at runtime. The pattern used throughout this chapter is a one-liner lambda — `lambda agg, evt: agg._on_opened(evt)` — which is correctly two-arg and keeps the real logic in a private method where it can be unit-tested and type-checked independently.

The event-sourced aggregate

In Chapter 6, `Wallet` held `_balance: Money` as direct Python state — `deposit` added to it, `withdraw` subtracted from it. In the event-sourced version the aggregate never mutates its own fields directly. Every state change is mediated by a domain event: the behaviour method *applies* the event, the event handler *updates the fields*, and the `EventStore` persists the event. On load, the repository replays all stored events through the same handlers, rebuilding in-memory state event by event.

This two-step indirection — apply, then handle — is the core mechanic of event sourcing. It enforces a strict discipline: every state transition is recorded exactly once as an event, and the aggregate's current state is always provable from its history.

Zero-arg constructor. `LedgerAccount.__init__` takes no arguments. This is required because `EventSourcedRepository` calls the factory as `LedgerAccount()` and then assigns `.id` before replaying the stream. Never construct a new ledger by passing arguments to `__init__` — call the `open` classmethod instead.

Here are the domain events and the aggregate:

lumen/models/entities/v1/ledger_account.py

```

from __future__ import annotations

from dataclasses import dataclass

from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from pyfly.domain import BusinessRuleViolation
from pyfly.eventsourcing import AggregateRoot, DomainEvent

# --- Domain events – the durable facts of the ledger -----

@dataclass
class LedgerOpened(DomainEvent):
    """The ledger was opened for an owner in a single currency."""
    account_id: str = ""
    owner_id: str = ""
    currency: str = ""

@dataclass
class Credited(DomainEvent):
    """Money moved *into* the ledger (a deposit / inbound transfer leg)."""
    account_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

@dataclass
class Debited(DomainEvent):
    """Money moved *out* of the ledger (withdrawal / outbound transfer leg)."""
    account_id: str = ""
    amount: int = 0
    currency: str = ""
    balance: int = 0

```

```

# --- Event-sourced aggregate root -----

class LedgerAccount(AggregateRoot):
    """An event-sourced money-movement ledger.

    Zero-arg constructible so it can serve as the repository's factory –
    EventSourcedRepository calls LedgerAccount() then assigns .id before
    replaying the stream. Use the open() classmethod to create new ledgers.
    """

    def __init__(self) -> None:
        super().__init__()
        self.owner_id: str = ""
        self.currency: Currency = Currency.EUR
        self.balance: Money = Money.zero(Currency.EUR)
        # Register apply-handlers. _dispatch calls handler(aggregate, event).
        # Use a lambda so the callable is two-arg; delegate to a private
        # method to keep the real logic type-checked and unit-testable.
        self.when(LedgerOpened, lambda agg, evt: agg._on_opened(evt))
        self.when(Credited, lambda agg, evt: agg._on_credited(evt))
        self.when(Debited, lambda agg, evt: agg._on_debited(evt))

# --- factory -----

@classmethod
def open(
    cls, account_id: str, owner_id: str, currency: Currency
) -> "LedgerAccount":
    """Open a new empty ledger; appends LedgerOpened."""
    if not owner_id.strip():
        raise BusinessRuleViolation(
            "ledger-owner-required", "owner_id is required"
        )
    account = cls()
    account.id = account_id
    account.apply(
        LedgerOpened(

```

```

        account_id=account_id,
        owner_id=owner_id,
        currency=currency.value,
    )
)
return account

# --- commands (validate invariants, then apply) -----

def credit(self, amount: Money) -> None:
    """Record money entering the ledger; appends Credited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "ledger-credit-positive", "credit amount must be > 0"
        )
    new_balance = self.balance.add(amount)
    self.apply(
        Credited(
            account_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=new_balance.amount,
        )
    )

def debit(self, amount: Money) -> None:
    """Record money leaving; refuses to overdraw. Appends Debited."""
    self._assert_currency(amount)
    if not amount.is_positive:
        raise BusinessRuleViolation(
            "ledger-debit-positive", "debit amount must be > 0"
        )
    remaining = self.balance.subtract(amount)
    if remaining.is_negative:
        raise BusinessRuleViolation(
            "ledger-insufficient-funds",
            f"cannot debit {amount}; balance is {self.balance}",
        )

```

```

    self.apply(
        Debited(
            account_id=self.id,
            amount=amount.amount,
            currency=amount.currency.value,
            balance=remaining.amount,
        )
    )

# --- apply-handlers (pure folds, shared by apply + replay) -----

def _on_opened(self, event: object) -> None:
    self.owner_id = event.owner_id          # type: ignore[attr-defined]
    self.currency = Currency(event.currency) # type: ignore[attr-defined]
    self.balance = Money.zero(self.currency)

def _on_credited(self, event: object) -> None:
    self.balance = Money(
        event.balance, Currency(event.currency) # type: ignore[attr-
defined]
    )

def _on_debited(self, event: object) -> None:
    self.balance = Money(
        event.balance, Currency(event.currency) # type: ignore[attr-
defined]
    )

# --- helpers -----

def _assert_currency(self, amount: Money) -> None:
    if amount.currency is not self.currency:
        raise BusinessRuleViolation(
            "ledger-currency-mismatch",
            f"ledger holds {self.currency.value}, "
            f"got {amount.currency.value}",
        )

```

Listing 9.1 — *LedgerAccount*: an event-sourced aggregate that derives its balance from replay

How it works. `__init__` registers three `when()` handlers — one per event class — each as a two-arg lambda delegating to a private method. No arithmetic happens inside the handlers; the behaviour methods (`credit`, `debit`) own all validation and compute the new state before constructing the event. The handler simply *applies* the already-computed result.

`account.apply(Credited(...))` does two things atomically: appends the event to the pending-events buffer (so the repository can persist it) and immediately dispatches it to the `Credited` handler (so `balance` updates in memory). The aggregate's in-memory state is therefore always consistent with its pending events, even before a save.

The factory method `open` calls `apply(LedgerOpened(...))` rather than setting fields directly. That is intentional: if you loaded this aggregate from its event stream, `LedgerOpened` would pass through exactly the same `_on_opened` handler and produce identical results. Write path and replay path are the same code — that symmetry is the correctness guarantee of event sourcing.

The `version` counter starts at zero and increments after each dispatched event. After `open`, `account.version == 1`; after one credit, `account.version == 2`. You will see this number again when the `EventStore` enforces optimistic concurrency.

`pending_events()` returns the events queued since the last save. The unit tests drive the aggregate in isolation — no repository needed — which makes invariant verification straightforward:

`tests/test_ledger_event_sourcing.py`

```
def test_open_emits_ledger_opened_and_starts_empty() -> None:
    account = LedgerAccount.open(
        "led-1", owner_id="owner-1", currency=Currency.EUR
    )
    assert account.id == "led-1"
    assert account.owner_id == "owner-1"
```

```

assert account.currency is Currency.EUR
assert account.balance == Money.zero(Currency.EUR)
# apply() queued the event for the store and bumped the version.
[event] = account.pending_events()
assert isinstance(event, LedgerOpened)
assert event.account_id == "led-1"
assert event.currency == "EUR"
assert account.version == 1

def test_credit_then_debit_track_the_balance() -> None:
    account = LedgerAccount.open("led-2", owner_id="o", currency=Currency.EUR)
    account.credit(Money(1000, Currency.EUR))
    account.debit(Money(400, Currency.EUR))
    assert account.balance == Money(600, Currency.EUR)
    kinds = [type(e).__name__ for e in account.pending_events()]
    assert kinds == ["LedgerOpened", "Credited", "Debited"]
    assert account.version == 3

def test_debit_cannot_overdraw() -> None:
    account = LedgerAccount.open("led-3", owner_id="o", currency=Currency.EUR)
    account.credit(Money(500, Currency.EUR))
    with pytest.raises(BusinessRuleViolation) as exc:
        account.debit(Money(501, Currency.EUR))
    assert exc.value.rule == "ledger-insufficient-funds"
    # Invariant held: balance unchanged, no Debited event queued.
    assert account.balance == Money(500, Currency.EUR)
    assert [type(e).__name__ for e in account.pending_events()] == [
        "LedgerOpened",
        "Credited",
    ]

```

Listing 9.2 — Unit tests: aggregate in isolation, commands and invariants

💡 ON_{EVENT_TYPE} AS AN ALTERNATIVE

Instead of `when()` lambdas, you can define a method on the aggregate named after the event in snake_case — `on_ledgeropened(self, evt)` is discovered automatically. Use `when()` for concise one-liners and named methods for handlers that need multiple statements or local variables. The dispatch order is: `when()` handler first; then `on_{event_type}` method; then `EventHandlerException` if neither exists.

🍃 SPRING PARITY

`AggregateRoot + apply() + when()` is PyFly's equivalent of Axon Framework's `@Aggregate + AggregateLifecycle.apply(event) + @EventSourcingHandler`. Axon uses annotation-driven handler discovery (`@EventSourcingHandler`); PyFly uses `when()` registration or `on_*` method convention. The replaying mechanic — load events from the store, call the same handlers, rebuild state — is identical in both frameworks.

The EventStore

The aggregate knows how to produce and replay events. The `EventStore` knows how to persist and retrieve them. These are deliberately separate concerns: the aggregate is pure business logic with no I/O; the event store is pure I/O with no business logic.

The `EventStore` protocol exposes two core operations:

- `append(aggregate_id, aggregate_type, events, *, expected_version)` — persists a batch of events for a stream. Raises `ConcurrencyError` if the stream's actual version does not match `expected_version`.
- `load(aggregate_id, *, after_sequence=0)` — returns the ordered sequence of `StoredEventEnvelope` objects from the first event (or from `after_sequence`) to the most recent.

`InMemoryEventStore` is the out-of-the-box implementation. Like `InMemoryEventBus` in Chapter 8, it runs entirely in-process with no I/O — ideal for development and tests. A production deployment swaps in a PostgreSQL- or EventStoreDB-backed adapter.

`EventSourcedRepository` wraps the `EventStore` and handles the full save/load cycle. Application code never calls the store directly; it calls `repo.save(aggregate)` and `repo.load(aggregate_id)`, and the repository handles the rest.

All imports come from two locations:

```
# Core event-sourcing types – all in the base package
from pyfly.eventsourcing import (
    AggregateRoot,
    DomainEvent,
    EventStore,
    InMemoryEventStore,
    SnapshotStore,
    InMemorySnapshotStore,
    StoredEventEnvelope,
)
# The generic repository lives in the .repository submodule
from pyfly.eventsourcing.repository import EventSourcedRepository
```

The LedgerAccountRepository

You subclass `EventSourcedRepository` for two reasons: to pass the concrete factory and snapshot store through a single well-named constructor, and — optionally — to override `_envelope_to_event` so that replayed events are real typed dataclasses rather than the generic attribute-bag the base class produces.

```
lumen/models/repositories/ledger_repository.py
```

```

from __future__ import annotations

from typing import ClassVar

from lumen.models.entities.v1.ledger_account import (
    Credited,
    Debited,
    LedgerAccount,
    LedgerOpened,
)
from pyfly.eventsourcing import (
    DomainEvent,
    EventStore,
    SnapshotStore,
    StoredEventEnvelope,
)
from pyfly.eventsourcing.repository import EventSourcedRepository

# Map a stored event_type (the event class name) back to its dataclass.
_EVENT_TYPES: dict[str, type[DomainEvent]] = {
    LedgerOpened.__name__: LedgerOpened,
    Credited.__name__: Credited,
    Debited.__name__: Debited,
}

class LedgerAccountRepository(EventSourcedRepository[LedgerAccount]):
    """Loads/saves LedgerAccount aggregates via the event store."""

    SNAPSHOT_INTERVAL: ClassVar[int] = 100

    def __init__(
        self,
        store: EventStore,
        *,
        snapshots: SnapshotStore | None = None,
    ) -> None:
        super().__init__(

```

```

        store,
        factory=LedgerAccount,
        snapshots=snapshots,
        snapshot_interval=self.SNAPSHOT_INTERVAL,
    )

    @staticmethod
    def _envelope_to_event(envelope: StoredEventEnvelope) -> object:
        """Rebuild the concrete event dataclass from a stored payload.

        Overrides the base-class generic hydration so that replayed events
        are the same dataclasses the aggregate applied on the write side.
        Unknown fields are ignored for forward-compatibility.
        """
        event_cls = _EVENT_TYPES.get(envelope.event_type)
        if event_cls is None:
            # Fall back to generic hydration for unrecognised event types.
            return EventSourcedRepository._envelope_to_event(envelope)
        field_names = {
            f.name for f in event_cls.__dataclass_fields__.values()
        }
        kwargs = {
            k: v for k, v in envelope.payload.items() if k in field_names
        }
        return event_cls(**kwargs)

```

Listing 9.3 — LedgerAccountRepository: typed replay via `_envelope_to_event`

How it works. `super().__init__(store, factory=LedgerAccount, ...)` tells the base repository to call `LedgerAccount()` — no arguments — to create a blank aggregate, assign `.id`, then replay the stream. `factory` accepts any zero-arg callable; passing the class itself (`factory=LedgerAccount`) is equivalent to `factory=lambda: LedgerAccount()`.

The `_envelope_to_event` override looks up the stored `event_type` string in `_EVENT_TYPES`, uses `__dataclass_fields__` to filter the payload to known fields, and reconstructs the real dataclass. Unknown fields are silently dropped — forward-compatibility in practice: if a future event version adds a field the old handler does not recognise, the ledger keeps replaying instead of crashing. The base-class fallback at the bottom handles any event type the repository does not know about.

Save, load, and the replay proof

The following test demonstrates the complete save-and-load cycle:

`tests/test_ledger_event_sourcing.py`

```
@pytest.mark.asyncio
async def test_balance_survives_reload_by_replay(
    event_store: InMemoryEventStore,
) -> None:
    # 1. Open + credit + debit, then persist the pending events.
    account = LedgerAccount.open(
        "acct-1", owner_id="owner-7", currency=Currency.EUR
    )
    account.credit(Money(2500, Currency.EUR)) # +25.00
    account.debit(Money(1000, Currency.EUR)) # -10.00 -> 15.00
    assert account.balance == Money(1500, Currency.EUR)

    repo = LedgerAccountRepository(event_store)
    await repo.save(account)
    # After committing, the aggregate has no pending events left.
    assert account.pending_events() == []

    # 2. Reconstruct from a *fresh* repository + aggregate. Nothing here
    # carries the in-memory object's state – load() rebuilds the
    # LedgerAccount purely by replaying the stored event stream.
    fresh_repo = LedgerAccountRepository(event_store)
```

```

recovered = await fresh_repo.load("acct-1")

assert recovered is not None
assert recovered is not account
assert recovered.owner_id == "owner-7"
assert recovered.currency is Currency.EUR
# The load-by-replay proof: balance recomputed from events, not stored.
assert recovered.balance == Money(1500, Currency.EUR)
# Three events were folded in: LedgerOpened, Credited, Debited.
assert recovered.version == 3
# A reconstructed aggregate has nothing pending – it was not "changed".
assert recovered.pending_events() == []

```

Listing 9.4 — Headline test: balance survives a reload by replay

How it works. `repo.save(account)` calls `store.append("acct-1", "LedgerAccount", pending_events, expected_version=0)`. The three events — `LedgerOpened`, `Credited`, `Debited` — are serialised into `StoredEventEnvelope` objects and written to the stream in order; the pending-events buffer on the aggregate is then cleared.

`fresh_repo.load("acct-1")` calls `store.load("acct-1")`, receives the three envelopes, constructs a blank `LedgerAccount()` via the factory, assigns `.id = "acct-1"`, and passes each envelope through `_envelope_to_event` before replaying it. The `_on_opened`, `_on_credited`, and `_on_debited` handlers run in sequence; after all three, `recovered.balance.amount` is `1500` — the same value the live aggregate held, computed without any shared state between the two objects.

The test uses *two independent repository instances* sharing the same in-memory store — `repo` for the write, `fresh_repo` for the read. This proves that replay, not in-process object identity, is the source of truth.

The event store also exposes raw envelopes for inspection — useful for audits and tests:

tests/test_ledger_event_sourcing.py

```

@pytest.mark.asyncio
async def test_store_holds_the_immutable_event_stream(
    event_store: InMemoryEventStore,
) -> None:
    account = LedgerAccount.open(
        "acct-2", owner_id="o", currency=Currency.GBP
    )
    account.credit(Money(800, Currency.GBP))
    account.debit(Money(300, Currency.GBP))
    await LedgerAccountRepository(event_store).save(account)

    envelopes = await event_store.load("acct-2")
    assert [e.event_type for e in envelopes] == [
        "LedgerOpened", "Credited", "Debited"
    ]
    assert [e.sequence for e in envelopes] == [1, 2, 3]
    assert all(e.aggregate_type == "LedgerAccount" for e in envelopes)
    # The Debited payload carries the post-debit running balance.
    assert envelopes[2].payload["balance"] == 500
    assert await event_store.latest_version("acct-2") == 3

```

Listing 9.5 — The store holds the immutable event stream

❗ FACTORY ARGUMENT

`EventSourcedRepository` accepts a `factory` callable that produces a blank aggregate instance. The factory must return an aggregate in its initial `__init__` state — no events applied — because the repository will apply the full history itself. Passing a factory that constructs an already-mutated aggregate (for example, one that calls `open()` internally) will corrupt the replay: `_on_opened` would run twice, the second time overwriting the fields that the real events already set.

Optimistic concurrency

Two concurrent requests — a credit from the mobile app and an automated fee debit from a background job — can load the same ledger at the same version, each apply their own change, and then attempt to save. Without a concurrency guard, one save silently wins, the other's events are lost, sequence numbers collide, and the resulting balance is wrong.

Optimistic concurrency prevents this. Before appending new events, the `EventStore` compares the stream's *current* version against the *expected* version the repository recorded at load time. If they match, the append proceeds and the version advances. If they do not match — because another writer already appended — `ConcurrencyError` is raised and the losing request must retry from a fresh load.

The `expected_version` is passed implicitly by the repository: it records the version at which it loaded the aggregate and supplies it to the store on save. You never manage version numbers in application code.

The version progression is deterministic: after a save-and-reload, further writes advance the version without conflict:

`tests/test_ledger_event_sourcing.py`

```
@pytest.mark.asyncio
async def test_continues_appending_after_a_reload(
    event_store: InMemoryEventStore,
) -> None:
    repo = LedgerAccountRepository(event_store)
    account = LedgerAccount.open(
        "acct-3", owner_id="o", currency=Currency.EUR
    )
    account.credit(Money(1000, Currency.EUR))
    await repo.save(account)
```



```

# Reload, mutate again, save again – version must advance, no conflict.
reloaded = await repo.load("acct-3")
assert reloaded is not None
assert reloaded.version == 2
reloaded.debit(Money(250, Currency.EUR))
await repo.save(reloaded)

final = await repo.load("acct-3")
assert final is not None
assert final.balance == Money(750, Currency.EUR)
assert final.version == 3
assert await event_store.latest_version("acct-3") == 3

```

Listing 9.6 — Continuing to append after a reload: optimistic lock advances correctly

How it works. After the first save the stream is at version 2. When loaded, `reloaded.version == 2`. `repo.save(reloaded)` appends with `expected_version=2`; the store advances to 3 and succeeds. The `final` load replays all three events and confirms the correct balance.

⚠ ALWAYS HANDLE CONCURRENCYERROR

When two writers race, the losing save raises `ConcurrencyError`. Your application service must catch it and decide what to do: retry the full load-mutate-save cycle (appropriate for low-contention writes), or surface a 409 Conflict to the caller (appropriate when the caller should re-submit with fresh data). Never silently swallow the error — a swallowed concurrency error leaves the stream in an inconsistent state.

Snapshots

Event sourcing trades write simplicity for read cost. Loading a ledger that has recorded 10 000 money movements means replaying 10 000 events every time the aggregate is needed. For most ledgers the stream stays short; for high-frequency accounts it can grow prohibitively long.

Snapshots address this. A snapshot is a serialised checkpoint of the aggregate's state at a specific version. On load, the repository looks for the most recent snapshot first, deserialises the state directly to that version, then replays only the events that arrived after it. A snapshot at version 9 000 reduces a 10 000-event replay to 1 000 events.

`InMemorySnapshotStore` stores snapshots in memory. Pass it to `EventSourcedRepository` alongside the event store via the `snapshots` keyword — exactly as `LedgerAccountRepository.__init__` accepts it:

```
store = InMemoryEventStore()
snapshots = InMemorySnapshotStore()
repo = LedgerAccountRepository(store, snapshots=snapshots)
```

The repository decides when to snapshot automatically using `snapshot_interval` (default `100`, set by `LedgerAccountRepository.SNAPSHOT_INTERVAL`). After every `save`, it checks whether the aggregate's new version **crosses** a multiple of `snapshot_interval`:

```
# crosses_interval is True when this batch pushes the stream past a
# 100-event boundary – e.g., version 95 → 105 crosses the 100 mark.
crossed = (
    (aggregate.version // snapshot_interval)
    > (previous_version // snapshot_interval)
)
```

This interval-crossing logic (rather than exact divisibility) handles the common case where a single save batch straddles the threshold. A bulk import that adds 10 events takes the version from 95 to 105 and correctly triggers a snapshot, even though neither 95 nor 105 is exactly divisible by 100.

The snapshot seam is harmless when the stream is shorter than the interval — the following test proves it:

tests/test_ledger_event_sourcing.py

```
@pytest.mark.asyncio
async def test_snapshot_store_round_trips_the_ledger() -> None:
    """With a snapshot store wired, reload still yields the right state.

    The ledger's stream is far shorter than the snapshot interval, so this
    proves the snapshot seam is harmless and the repository falls back
    to a full replay.
    """
    store = InMemoryEventStore()
    snapshots = InMemorySnapshotStore()
    repo = LedgerAccountRepository(store, snapshots=snapshots)

    account = LedgerAccount.open(
        "acct-4", owner_id="o", currency=Currency.EUR
    )
    account.credit(Money(5000, Currency.EUR))
    await repo.save(account)

    recovered = await repo.load("acct-4")
    assert recovered is not None
    assert recovered.balance == Money(5000, Currency.EUR)
```

Listing 9.7 — Snapshot store wired: reload still yields correct state

How it works. After the save, the repository checks: does `version 2 // 100 > 0 // 100`? No — the snapshot threshold has not been crossed, so no snapshot is taken. The next `load` performs a full replay of the two events and returns the correct balance. Once a ledger does cross a 100-event boundary, the repository serialises the aggregate's state into a snapshot envelope. The next load finds the snapshot, deserialises directly to that version, and then asks the event store for events with a sequence number greater than the snapshot version — reducing replay cost to the delta only.

💡 SNAPSHOT INTERVAL IN PRODUCTION

A `snapshot_interval` of 100 is the default and a sensible starting point. For high-frequency ledgers you might lower it; for accounts that only change a few times a day, a higher interval reduces snapshot-storage cost. Snapshots are an optimization, not a correctness requirement — removing them leaves the system correct but slower.

Boot wiring and auto-configuration

The `pyfly.eventsourcing` module ships in the **base** `pyfly` package — no extra dependency. Enabling it takes two steps: set `pyfly.eventsourcing.enabled: true` in `pyfly.yaml` and annotate the application with `@enable_domain_stack`. PyFly's auto-configuration then registers `event_store` and `snapshot_store` beans automatically.

The test suite confirms this directly:

`tests/test_ledger_event_sourcing.py`

```
def test_auto_configuration_registers_event_store_beans() -> None:
    """enable_domain_stack activates this auto-config when
    pyfly.eventsourcing.enabled=true, registering the in-memory
    event/snapshot stores the ledger repository depends on."""
```

```

from pyfly.eventsourcing.auto_configuration import (
    EventSourcingAutoConfiguration,
)

config = EventSourcingAutoConfiguration()
assert isinstance(config.event_store(), InMemoryEventStore)
assert isinstance(config.snapshot_store(), InMemorySnapshotStore)

```

Listing 9.8 — Auto-configuration registers the event store beans

In application code, the repository is wired through dependency injection:

```

# pyfly.yaml
pyfly:
  eventsourcing:
    enabled: true

# In a service or handler – the beans are injected automatically
@Component
class LedgerService:
    def __init__(
        self,
        event_store: EventStore,
        snapshot_store: SnapshotStore,
    ) -> None:
        self._repo = LedgerAccountRepository(
            event_store, snapshots=snapshot_store
        )

```

Projections

The event store is the system of record, but most application queries — "what is the current balance?", "show all ledgers for owner u-42", "which accounts are above 1 000 EUR?" — must not replay event streams on every read. They should hit a pre-computed read model: a table optimised for queries, kept in sync by a background process that consumes the event stream.

That background process is a **projection**. A projection subscribes to the event stream and updates a read model each time a relevant event arrives. PyFly provides

`FunctionProjection` and `ProjectionRunner` in `pyfly.eventsourcing.projection`:

- `FunctionProjection(name, handler_fn)` — wraps an async function that receives one `StoredEventEnvelope` and updates the read model.
- `ProjectionRunner(projection, store)` — drives the projection by iterating the `EventStore` in sequence-number order and calling the handler for each envelope.

Here is a `BalanceLedgerProjection` that builds a balance read model from the event stream:

`lumen/eventsourcing/balance_projection.py`

```
from __future__ import annotations

from pyfly.eventsourcing import InMemoryEventStore
from pyfly.eventsourcing.projection import FunctionProjection,
ProjectionRunner

# The in-process read model – in production, replace with a DB table.
_balance_store: dict[str, dict] = {}

async def _handle_envelope(envelope: object) -> None:
```

```

"""Update the balance read model for each ledger event."""
event_type: str = getattr(envelope, "event_type", "")
payload: dict = getattr(envelope, "payload", {})

if event_type == "LedgerOpened":
    _balance_store[payload["account_id"]] = {
        "account_id": payload["account_id"],
        "owner_id": payload.get("owner_id", ""),
        "balance_cents": 0,
        "currency": payload.get("currency", ""),
    }
elif event_type in ("Credited", "Debited"):
    account_id = payload["account_id"]
    if account_id in _balance_store:
        _balance_store[account_id]["balance_cents"] = (
            payload["balance"]
        )

def build_projection(store: InMemoryEventStore) -> ProjectionRunner:
    projection = FunctionProjection("balance_ledger", _handle_envelope)
    return ProjectionRunner(projection, store)

async def demo_projection(store: InMemoryEventStore) -> None:
    runner = build_projection(store)
    await runner.start()

    balance = _balance_store.get("led-001", {})
    print(f"Balance read model: {balance}")

```

Listing 9.9 — *BalanceLedgerProjection*: a read model built from the event stream

How it works. `FunctionProjection("balance_ledger", _handle_envelope)` wraps the async handler. `ProjectionRunner(projection, store)` links it to the `InMemoryEventStore`. `await runner.start()` iterates every envelope in the store in sequence-number order and calls `_handle_envelope` for each one. After `start()` returns, `_balance_store` reflects the current state of every ledger in the store.

The projection is intentionally stateless — it reads only `envelope.event_type` and `envelope.payload`. No aggregate is loaded; no repository is called. The read model is cheap to rebuild: stop the runner, clear `_balance_store`, call `start()` again. This rebuild-from-history property is unique to event sourcing — state-storage models have already discarded the history.

In production, `_handle_envelope` would write to a real database (PostgreSQL, Redis, Elasticsearch). The `ProjectionRunner` would persist a cursor in a checkpointing table so restarts continue from the last processed event instead of replaying everything from the beginning. The projection pattern is identical regardless of the underlying storage.

❗ PROJECTIONS VS CHAPTER 8 LISTENERS

Chapter 8's `BalanceProjection` (Listing 8.4) was an `@event_listener` subscriber on the `InMemoryEventBus` — it reacted to events as they were published. This chapter's `BalanceLedgerProjection` reads directly from the `EventStore` — it can replay history from the beginning, catch up to the present, and continue consuming future events. Both keep a balance read model; the event-store projection is rebuildable from history; the bus listener is not.

The transactional outbox

Consider the `repo.save(account)` call in Listing 9.4: three events are appended to the event store. Now suppose those events also need to reach an external broker — Kafka, RabbitMQ, another microservice. The naive approach is to call `broker.publish(envelope)` immediately after `store.append(...)`. But what if the process crashes between the append and the publish? The events are in the store, but the broker never received them. The downstream service never learned about the credit.

The **transactional outbox** pattern solves this. Instead of publishing directly, you enqueue the event into an *outbox* — a durable intermediary. The outbox persists the event alongside the aggregate's events in the same store operation. A separate background worker (the *relay*) drains the outbox and forwards each event to the broker with at-least-once semantics. If the relay crashes, it restarts and retries from the last unacknowledged event.

PyFly's `TransactionalOutbox` lives in `pyfly.eventsourcing`. It accepts a `publish` coroutine and a `max_attempts` limit, and exposes two methods:

- `enqueue(envelope)` — adds an event envelope to the outbox for delivery.
- `start()` — starts the background relay loop that calls `publish(envelope)` for each queued item, retrying up to `max_attempts` times on failure.

`lumen/eventsourcing/outbox_demo.py`

```
from __future__ import annotations

from pyfly.eventsourcing import (
    InMemoryEventStore,
    InMemorySnapshotStore,
    TransactionalOutbox,
)
```

```

from pyfly.eventsourcing.repository import EventSourcedRepository

from lumen.models.entities.v1.ledger_account import LedgerAccount
from lumen.models.entities.v1.money import Money
from lumen.interfaces.enums.v1.currency import Currency

# Simulated broker: collect published envelopes for inspection.
_published: list = []

async def _broker_publish(envelope: object) -> None:
    _published.append(envelope)

async def demo_outbox() -> None:
    store = InMemoryEventStore()
    repo = LedgerAccountRepository(store)
    outbox = TransactionalOutbox(publish=_broker_publish, max_attempts=5)
    await outbox.start()

    account = LedgerAccount.open("led-004", "u-11", Currency.EUR)
    account.credit(Money(5000, Currency.EUR))
    await repo.save(account)

    # Enqueue the stored envelopes into the outbox.
    for envelope in await store.load("led-004"):
        await outbox.enqueue(envelope)

    # The relay has delivered all envelopes to the broker.
    assert len(_published) == 2  # LedgerOpened + Credited

```

Listing 9.10 — *TransactionalOutbox: reliable at-least-once delivery to a broker*

How it works. The outbox holds envelopes in a durable queue. `_broker_publish` is the delivery function — replace it with your Kafka or RabbitMQ producer.

`max_attempts=5` means the relay retries a failing delivery up to five times before dead-lettering the envelope.

The critical guarantee: the outbox is drained independently of the request that created the events. If the process crashes after `repo.save(account)` but before the outbox finishes flushing, the next restart picks up from where it left off and completes the delivery. The aggregate state in the event store is already correct; only the broker-side delivery was interrupted.

⚠ AT-LEAST-ONCE, NOT EXACTLY-ONCE

The outbox guarantees that every event reaches the broker *at least once*. If the relay delivers an event and then crashes before marking it as acknowledged, the event is delivered again on restart. Your broker consumers — and downstream services — must be idempotent: use the `envelope.event_id` as a deduplication key. Chapter 10 shows how Kafka and RabbitMQ consumer adapters handle deduplication automatically.

The transactional outbox is the bridge between event sourcing and event-driven messaging. Chapter 10 picks up exactly here, introducing Kafka producers and RabbitMQ exchanges and showing how to configure the relay for reliable delivery to each.

🍃 SPRING PARITY

The transactional outbox pattern is well known in the Spring ecosystem under the same name. Spring Modulith's `EventPublicationRegistry` and Spring's `@TransactionalEventListener(phase = AFTER_COMMIT)` approximate the same guarantee — the event is recorded durably before being dispatched. Axon Server's event store fulfils a similar role for Axon-based applications: events are written to the store first, and projection groups / event processors consume them with at-least-once guarantees from the stored log. PyFly's `TransactionalOutbox` is the portable equivalent of that pattern, without requiring a dedicated event server.

Advanced: upcasting and multi-tenancy

Two concerns appear in every long-lived event-sourced system. This section introduces them briefly; full treatment is beyond the scope of this book.

Upcasting

Events are immutable. Once a `Credited` event is written to the stream you cannot go back and add a `reference_code` field. But product requirements change: three months from now the finance team will need a reference code on every credit for reconciliation. New events include it; old events do not.

An **upcaster** is a function that transforms an old event shape into the current shape during replay. The `EventStore` calls it transparently — the aggregate never sees the old shape. You register upcasters per event type and per version:

```
# Conceptual – upcaster API varies by adapter
def upcast_credited_v1(payload: dict) -> dict:
    payload.setdefault("reference_code", "LEGACY")
    return payload
```

The upcaster runs when the `EventStore` loads an event whose schema version is lower than the current one. Old data becomes readable without a migration; new data is written in the current schema.

Multi-tenancy

When multiple tenants share the same event store, stream IDs must be scoped by tenant. The canonical approach is to prefix every `aggregate_id` with the tenant identifier — `"tenant-A::led-001"` rather than `"led-001"`. `EventSourcedRepository` accepts a `tenant_id` parameter on construction and prepends the prefix to every stream operation transparently:

```
repo_tenant_a = EventSourcedRepository(  
    store,  
    factory=LedgerAccount,  
    tenant_id="tenant-A",  
)
```

Projections must scope their read models similarly — typically by including `tenant_id` as a column in the read-model table and filtering on it at query time.

CHOOSING EVENT SOURCING

Event sourcing adds operational complexity — upcasters, snapshot management, projection rebuild procedures, outbox relay monitoring. Choose it deliberately for domains where auditability and time-travel queries are first-class requirements: financial ledgers, medical records, supply-chain logs. For CRUD-heavy domains where the current state is all that matters, state storage is simpler and sufficient.

✓ What you built

Lumen's `LedgerAccount` is now a fully event-sourced ledger that coexists with the Chapter 6 state-stored `Wallet`.

The two aggregate bases are separate by design: `Wallet` extends `pyfly.domain.AggregateRoot` and stores its balance in a database row; `LedgerAccount` extends `pyfly.eventsourcing.AggregateRoot` and derives its balance from an immutable event stream. The distinction is not cosmetic — the event-sourcing machinery (`apply` , `replay` , `when`) lives only on the eventsourcing base class.

The three domain events — `LedgerOpened` , `Credited` , `Debited` — are `dataclass` es extending `pyfly.eventsourcing.DomainEvent` , each with default field values so the repository can reconstruct them from a stored payload. Handlers are registered with `when()` as two-arg lambdas delegating to private methods; the bound-method trap to avoid is that a bound method is already one-arg from the outside, which causes a `TypeError` at dispatch time.

`LedgerAccountRepository` is a thin subclass of `EventSourcedRepository[LedgerAccount]` that passes the zero-arg factory and overrides `_envelope_to_event` to hydrate concrete dataclasses on replay. The headline test confirmed it: after opening, crediting, and debiting a ledger, a brand-new repository and aggregate reconstructed the correct balance purely by replaying the stored stream — no stored balance column, no shared state.

The `version` counter drives the optimistic-concurrency guard: the store rejects any `append` whose `expected_version` does not match the stream's actual version, forcing the losing writer to retry from a fresh load. `InMemorySnapshotStore` proved harmless when the stream is shorter than the snapshot interval and will accelerate replay once a ledger crosses the threshold.

`BalanceLedgerProjection` used `FunctionProjection` and `ProjectionRunner` to maintain a fast balance read model from the raw event stream — one that can be rebuilt from history at any time, unlike the bus-listener approach from Chapter 8.

`TransactionalOutbox` then connected the event store to the broker world, enqueueing events for at-least-once delivery and retrying on failure so no fact is silently lost between the store and downstream consumers. Chapter 10 picks up exactly there, introducing the Kafka and RabbitMQ adapters that the outbox relay targets.

Try it yourself

1. Replay to a point in time. Apply ten credits of 100 cents each to a `LedgerAccount`.

Then load the aggregate manually by calling `await store.load("led-X")`, filter envelopes to those with `sequence <= 5`, and replay only those through a fresh `LedgerAccount()`. Assert that the resulting balance equals 400 cents (four credits after the open) rather than 1 000 cents (ten credits). This is the "time-travel query" that state-storage models cannot provide.

2. Implement a `Transferred` event and dual-aggregate save. Add a

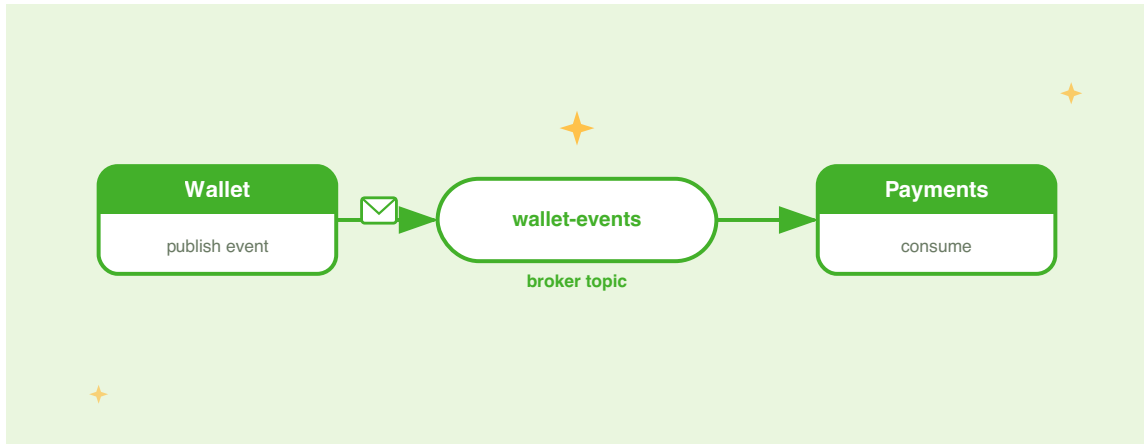
`Transferred(DomainEvent)` with fields `source_id: str`, `target_id: str`, `amount: int`, `currency: str`. Add a `transfer_to(target: LedgerAccount, amount: Money) -> None` method to `LedgerAccount` that calls `self.debit(amount)` and `target.credit(amount)` in sequence, then applies a

`Transferred` event on `self`. Wire a `demo_transfer` coroutine that opens two ledgers, credits 10 000 cents into the first, transfers 3 000 cents to the second, saves both aggregates independently, reloads both from the store, and asserts that the balances are 7 000 and 3 000 cents respectively.

3. **Add an `OwnerLedgerProjection`.** Write a second `FunctionProjection` named `owner_ledger` whose handler maintains a `dict[str, list[dict]]` mapping each `owner_id` to a chronological list of transaction records. Each record should include `event_type`, `amount` (from the payload for `Credited` / `Debited` events), and the envelope's sequence number. Feed it the same `InMemoryEventStore`, open three ledgers for the same owner, perform a mix of credits and debits, start the projection runner, and assert that the owner's transaction list has the correct number of entries in the correct order.

CHAPTER 10

Messaging with Kafka & RabbitMQ



Lumen's wallet service is genuinely event-driven. Commands flow through typed handlers; domain events publish to an in-process bus; listeners react independently without coupling to the write path. In Chapter 8 you built `WalletAuditListener` — a service that reacts to `WalletOpened`, `FundsDeposited`, and `FundsWithdrawn` events without knowing the command handlers. In Chapter 9 you went further, storing those events *as the source of truth* so every historical balance is computable from first principles.

There is one boundary neither chapter crossed: the network. `InMemoryEventBus` lives inside the Python process. The moment another service — a future `PaymentsService` that settles transfers, or a `NotificationsService` that sends push alerts — needs to react to Lumen's facts, you need a **message broker**: an independent infrastructure component that stores events durably, routes them to subscribers in other processes, and replays them when a consumer restarts after a crash.

This chapter takes Lumen's event-driven foundation across that boundary. You will see how PyFly wraps the complexity of Apache Kafka and RabbitMQ behind a single clean abstraction — `MessageBrokerPort` — so that application code never knows which broker is running beneath it. You will publish Lumen's wallet events to real topics, consume them with the `@message_listener` decorator, choose the right serialisation format for your schema-evolution requirements, handle poisoned messages with dead-letter queues built into the decorator, and protect your service against broker outages with circuit breakers and retries.

By the end of the chapter Lumen's integration events flow across process boundaries, ready for the Part IV services that will consume them.

One abstraction, many brokers

Why an abstraction matters

Before writing a line of Kafka or RabbitMQ code, it is worth asking: why does PyFly introduce an abstraction layer at all? `aiokafka` and `aio-pika` both expose perfectly usable async APIs. The answer is the same reason you depend on `EventPublisher` rather than `InMemoryEventBus` — the abstraction is what lets you swap infrastructure without touching business logic.

Without an abstraction, every service that produces or consumes a message imports Kafka-specific or RabbitMQ-specific types. Switching brokers — or running Kafka in production and an in-memory broker in CI — means changing import paths, constructor signatures, and consumer-loop boilerplate across every affected file. With `MessageBrokerPort`, the swap is a YAML change. The listeners and publishers that make up your business logic never change.

The abstraction pays dividends in testing too. `InMemoryMessageBroker` satisfies the port protocol. Inject it wherever `MessageBrokerPort` is expected and write fast, deterministic tests with no Docker dependency. Chapter 16 makes this concrete.

The MessageBrokerPort protocol

`MessageBrokerPort` is a `@runtime_checkable Protocol`. Use it as a type hint throughout your code; call `isinstance(obj, MessageBrokerPort)` at runtime if you need to verify that an injected bean satisfies the contract.

The protocol defines four methods:

```
from pyfly.messaging import MessageBrokerPort

class MessageBrokerPort(Protocol):
    async def publish(
        self,
        topic: str,
        value: bytes,
        *,
        key: bytes | None = None,
        headers: dict[str, str] | None = None,
    ) -> None: ...

    async def subscribe(
        self,
        topic: str,
        handler: MessageHandler,
        group: str | None = None,
    ) -> None: ...

    async def start(self) -> None: ...

    async def stop(self) -> None: ...
```

The four methods:

`publish` sends a single message to the named topic. `value` is raw bytes — the protocol deliberately leaves serialisation to you; encode the payload before calling `publish` and decode it inside your handler. `key` and `headers` are keyword-only so callers cannot accidentally transpose them. `key` drives Kafka partition assignment for ordering guarantees; RabbitMQ ignores it. `headers` carry cross-cutting metadata such as `event-type` and correlation IDs.

`subscribe` registers an async `MessageHandler` for a topic. The optional `group` parameter maps to Kafka consumer groups and RabbitMQ competing-consumer queues. Deploy three instances of a service, all subscribing with the same `group`, and only one instance processes each message. Omit `group` for broadcast semantics — every subscriber receives every message — which is useful for analytics that need a copy of every event.

`start` creates connections and begins consuming. Register all subscriptions *before* calling `start`, then call it once during application startup.

`stop` drains in-flight messages and closes connections cleanly. PyFly's application lifecycle calls `stop` automatically during shutdown, so you rarely need to invoke it manually.

The Message dataclass

Every handler receives a `Message` — a frozen dataclass carrying the full envelope of a received message:

```
from pyfly.messaging import Message

msg = Message(
    topic="wallet.events",
    value=b'{"wallet_id": "w-001", "amount": 5000}',
    key=b"w-001",
```

```
headers={"event-type": "FundsDeposited"},
)
```

Field	Type	Default	Description
<code>topic</code>	<code>str</code>	required	The topic or queue the message arrived on.
<code>value</code>	<code>bytes</code>	required	The raw payload. You decode it inside your handler.
<code>key</code>	<code>bytes \ None</code>	<code>None</code>	Partition or routing key. Kafka uses it for partition assignment.
<code>headers</code>	<code>dict[str, str]</code>	<code>{}</code>	String metadata attached by the publisher.

The dataclass is frozen: once the broker hands you a `Message` its fields are immutable — safe to pass across async boundaries without defensive copying, and immune to accidental mutation inside handlers.

Kafka vs RabbitMQ — choosing the right broker

Before diving into configuration, it helps to understand where each broker fits. The table below summarises the key trade-offs; neither choice is universally correct.

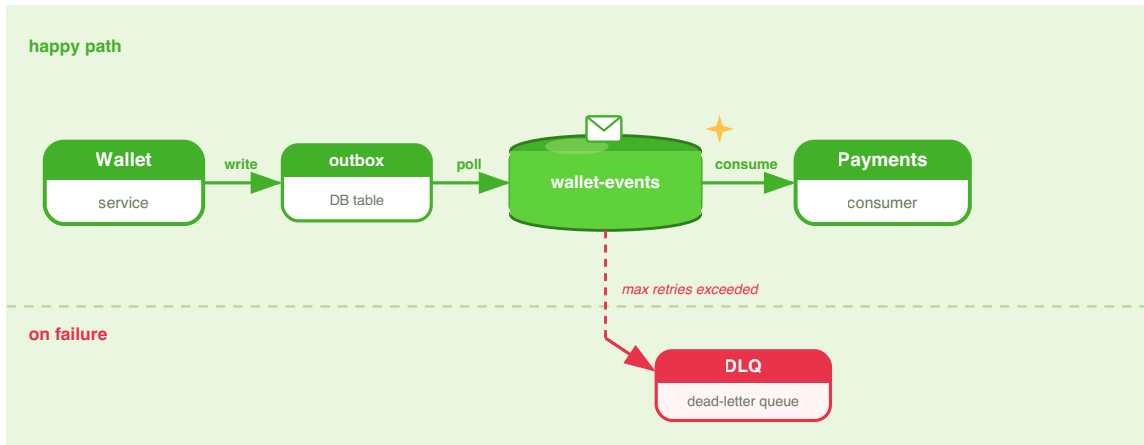


Figure 10.1 — MessageBrokerPort sits between application code and the broker adapters.

Dimension	Apache Kafka	RabbitMQ
Model	Distributed commit log; consumers maintain their own offset	Message broker; messages are removed from the queue after acknowledgement
Retention	Configurable (days to indefinite); consumers can replay from any offset	Messages are removed on delivery; dead-letter queues for failed messages
Throughput	Millions of messages/second; optimised for streaming	Tens of thousands/second; optimised for task routing
Ordering	Guaranteed within a partition (keyed producers)	Guaranteed within a single queue (FIFO)
Consumer groups	Native partition-level load balancing	Competing-consumer queues; one message per consumer
Schema evolution	Works well with Avro/Protobuf + Schema Registry	Works well with JSON; schema coupling is a user concern
When to choose	Event streaming, audit logs, replay, high throughput	Task queues, RPC patterns, per-message routing with complex bindings
PyFly extra	<code>uv add "pyfly[kafka]"</code>	<code>uv add "pyfly[rabbitmq]"</code>

For Lumen, Kafka is the natural fit: wallet events form an ordered stream per wallet, are worth replaying when a new consumer comes online, and will eventually feed high-throughput analytics. The examples in this chapter show both adapters interchangeably — from your code's perspective, the choice is a configuration detail.

i INSTALLING BOTH ADAPTERS

If you want to support either broker in one install, `uv add "pyfly[eda]"` pulls in both `aiokafka` and `aio-pika`. The auto-configuration then selects Kafka if `aiokafka` is importable, RabbitMQ if `aio_pika` is importable, and falls back to the in-memory broker if neither is present.

Configuring the adapters

Kafka

Add `pyfly[kafka]` to your project and declare the broker in `pyfly.yaml`:

```
pyfly:
  messaging:
    provider: "kafka"
  kafka:
    bootstrap-servers: "kafka-1:9092,kafka-2:9092"
```

That is all PyFly needs to auto-configure a `KafkaAdapter` and register it as the `MessageBrokerPort` bean. For most services the YAML is sufficient; if you need advanced producer options, construct the adapter manually as a `@bean` inside a `@configuration` class.

RabbitMQ

```
pyfly:
  messaging:
    provider: "rabbitmq"
  rabbitmq:
    url: "amqp://user:password@rabbitmq-host:5672/"
```


`RabbitMQAdapter` uses a durable direct exchange named `"pyfly"` by default. To customise the exchange name, construct the adapter manually:

`lumen/messaging/config.py`

```
from pyfly.container import configuration, bean
from pyfly.messaging import MessageBrokerPort
from pyfly.messaging.adapters.rabbitmq import RabbitMQAdapter

@configuration
class BrokerConfig:
    """Wire up the message broker bean."""

    @bean
    def broker(self) -> MessageBrokerPort:
        return RabbitMQAdapter(
            url="amqp://user:password@rabbitmq-host:5672/",
            exchange_name="lumen-events",
        )
```

Listing 10.1 — Custom RabbitMQ exchange name via `@bean`

How it works. `@configuration` marks the class as a factory that the DI container calls during startup. `@bean` on `broker` tells the container to call `broker()` once, cache the result, and inject it wherever `MessageBrokerPort` is requested. Any `@service` that declares `MessageBrokerPort` in its constructor receives this instance automatically — no import of `RabbitMQAdapter` required in the consumer class.

Auto-detection

When `provider` is `"memory"` (the default) or `"auto"`, PyFly probes installed packages in order:

Priority	Library checked	Adapter selected
1	<code>aiokafka</code>	<code>KafkaAdapter</code>
2	<code>aio_pika</code>	<code>RabbitMQAdapter</code>
3	<i>(fallback)</i>	<code>InMemoryMessageBroker</code>

Set `provider: "memory"` in `pyfly-test.yaml` and `provider: "kafka"` in `pyfly-prod.yaml`, and every test and production run uses the appropriate adapter without code changes.

Publishing integration events

From in-process events to integration events

In Chapter 8, Lumen's command handlers drained the `Wallet` aggregate's event buffer with `wallet.clear_events()` and published each domain event through `EventPublisher`. `WalletAuditListener` subscribed using `@event_listener` and reacted within the same process.

The **integration event** pattern crosses the process boundary. Where a *domain event* describes what happened inside an aggregate — a private fact, available to same-process listeners — an integration event is a sanitised, public representation of the same fact: designed for external consumers, stable across versions, and serialised to bytes for transport over a broker.

For Lumen, the integration event for a deposit carries only what an external consumer needs: the wallet identifier, the amount in minor units, the currency code, and the resulting balance. It does not expose the aggregate's internal implementation details.

How Lumen drains events to the broker

Lumen separates the publish bridge from the command handlers so every handler publishes events identically. `publish_domain_events` (in `lumen/core/services/wallets/event_publishing.py`) iterates the drained events, converts each frozen dataclass to a dict, and calls `EventPublisher.publish`:

```
# lumen/core/services/wallets/event_publishing.py (real Lumen code)
from pyfly.eda import EventPublisher
from pyfly.domain import DomainEvent

async def publish_domain_events(
    publisher: EventPublisher,
    events: Iterable[DomainEvent],
) -> None:
    for event in events:
        payload = dataclasses.asdict(event)
        payload.setdefault("event_type", event.event_type)
        await publisher.publish(
            destination="wallet.events",
            event_type=event.event_type, # "WalletOpened" / "FundsDeposited" / ...
            payload=payload,
        )
```

The `EventPublisher.publish` signature is:

```
async def publish(
    self,
    destination: str,
    event_type: str,
    payload: dict[str, Any],
    headers: dict[str, str] | None = None,
) -> None: ...
```

`destination` is the logical channel name (`"wallet.events"`). `event_type` is the domain event class name — `"WalletOpened"`, `"FundsDeposited"`, or `"FundsWithdrawn"` — which is exactly what `@event_listener` subscribers filter on.

Every command handler wires in `EventPublisher` via the constructor and calls `publish_domain_events` after persisting:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from __future__ import annotations

from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events

    async def do_handle(self, command: DepositFunds) -> int:
        wallet = await self._repository.find(command.wallet_id)
        if wallet is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet.deposit(Money(
            amount=command.amount,      # integer minor units (e.g. 5000 =
```

```

€50.00)
        currency=wallet.currency,
    ))
    await self._repository.add(wallet)

    # Drain pending events and forward them to the EDA bus.
    await publish_domain_events(
        self._events, wallet.clear_events()
    )
    return wallet.balance.amount

```

Listing 10.2 — *DepositFundsHandler drains events via EventPublisher*

Key design decisions:

`events: EventPublisher` is the port, not the adapter. The DI container injects whichever bus is configured — in-memory in tests, a broker-backed bus in production. This handler never mentions Kafka or RabbitMQ.

`command.amount` is the deposit in integer minor units (e.g. `5000` means €50.00 for a EUR wallet). The `FundsDeposited` domain event records the same `amount` field, plus `currency` (a string like `"EUR"`) and `balance` (the new balance in minor units).

`wallet.clear_events()` drains the aggregate's pending events list and returns them. Calling it *after* `repository.add` ensures the events describe a fact that persisted. Publishing before saving would create phantom events — facts about things that never happened.

The domain events raised during a deposit are instances of:

```

@dataclass(frozen=True)
class FundsDeposited(DomainEvent):
    wallet_id: str = ""
    amount: int = 0      # integer minor units

```

```

currency: str = ""    # e.g. "EUR"
balance: int = 0      # new balance after deposit, minor units

```

When `publish_domain_events` publishes this event, `event_type` is the class name `"FundsDeposited"` — *not* a dotted string like `"wallet.fundsdeposited"`.

Publishing an integration event directly to the broker

When a separate service running in a different process needs to receive Lumen's wallet events, the EDA bus must be backed by a real broker adapter. The payload flowing over the wire is the same dict the in-process listeners see. A dedicated `OutboxRelay` (covered in the resilience section) or a broker-backed `EventPublisher` handles the transport:

`lumen/messaging/deposit_publisher.py`

```

from __future__ import annotations

import json

from pyfly.messaging import MessageBrokerPort

async def publish_deposit_event(
    broker: MessageBrokerPort,
    wallet_id: str,
    amount: int,
    currency: str,
    balance: int,
) -> None:
    """Encode a FundsDeposited integration event and publish to the topic."""
    payload = json.dumps({
        "wallet_id": wallet_id,
        "amount": amount,          # integer minor units
        "currency": currency,      # e.g. "EUR"
        "balance": balance,        # new balance, minor units
        "event_type": "FundsDeposited",
    })

```

```

    }).encode()

    await broker.publish(
        "wallet.events",
        payload,
        key=wallet_id.encode(),
        headers={"event-type": "FundsDeposited"},
    )

```

Listing 10.3 — Publishing a wallet integration event to a Kafka topic

Key design decisions:

`broker: MessageBrokerPort` is the port, not the adapter. The DI container injects whichever adapter is configured — Kafka in production, the in-memory broker in tests.

`key=wallet_id.encode()` is the routing key. On Kafka, all messages sharing the same key land on the same partition, delivering them to consumers in publication order — critical for a ledger where deposit before withdraw must be preserved. On RabbitMQ the key is ignored (routing uses the exchange binding), so this field is safe to include regardless of which broker is running.

`headers={"event-type": "FundsDeposited"}` uses the domain event class name — not a dotted path like `"wallet.fundsdeposited"`. Consumers can inspect the event type without decoding the payload, which is useful for routing and filtering without full deserialisation.

⚠ PUBLISH AFTER SAVE, NOT BEFORE

Always drain and publish events *after* `repository.add(wallet)` . If the save fails, no message reaches the broker and external consumers never see a fact that never persisted. The transactional outbox pattern (where the outbox row and the aggregate row are written in the same database transaction) provides the stronger atomic guarantee for production; direct publishing as shown here is a reasonable starting point for simpler services.

Consuming events with `@message_listener`

The problem with polling

Before brokers, services reacted to another service's state changes by polling a shared database or a REST endpoint. Polling adds latency (the reaction waits until the next poll interval), wastes resources (most polls find nothing new), and couples consumer to producer at the API level. A message listener eliminates all three problems: the broker pushes the event as soon as it is available, idle connections consume negligible CPU, and the consumer depends only on the message schema — not on the producer's internal API.

Declarative listeners with `@message_listener`

`@message_listener` is the declarative subscription decorator. Decorate any async function or method with the topic it should consume, and PyFly wires the subscription during application startup — no bus reference, no `subscribe()` call, no lifecycle management in your code.

The decorator signature is:

```
def message_listener(
    topic: str,
```



```

group: str | None = None,
*,
retries: int = 0,
retry_delay: float = 0.0,
dead_letter_topic: str | None = None,
) -> ...: ...

```

Parameter	Type	Default	Description
<code>topic</code>	<code>str</code>	required	The topic to listen on.
<code>group</code>	<code>str \ None</code>	<code>None</code>	Consumer group name.
<code>retries</code>	<code>int</code>	<code>0</code>	Times to re-invoke the handler on failure.
<code>retry_delay</code>	<code>float</code>	<code>0.0</code>	Base delay (seconds) between retries — attempt N waits <code>retry_delay * N</code> .
<code>dead_letter_topic</code>	<code>str \ None</code>	<code>None</code>	When set, a message still failing after <code>retries</code> is re-published here.

`lumen/messaging/payments_consumer.py`

```

from __future__ import annotations

import json

from pyfly.messaging import Message, message_listener

@message_listener(topic="wallet.events", group="payments-service")
async def on_wallet_event(msg: Message) -> None:
    """React to every wallet event published to the topic."""
    event_type = msg.headers.get("event-type", "unknown")
    payload = json.loads(msg.value)

    if event_type == "FundsDeposited":

```

```

wallet_id: str = payload["wallet_id"]
amount: int = payload["amount"]          # minor units
currency: str = payload["currency"]
print(
    f"[Payments] Deposit received: "
    f"wallet={wallet_id} "
    f"amount={amount} {currency}"
)

```

Listing 10.4 — `@message_listener` on a standalone function

How it works. The decorator stores six metadata attributes on the wrapped function — `__pyfly_message_listener__ = True`, plus `__pyfly_listener_topic__`, `__pyfly_listener_group__`, `__pyfly_listener_retries__`, `__pyfly_listener_retry_delay__`, and `__pyfly_listener_dlq__`. During startup the framework scans all registered beans, finds functions carrying `__pyfly_message_listener__ = True`, and calls `broker.subscribe(topic, handler, group)` automatically. You never call `subscribe()` manually.

`group="payments-service"` places the consumer in a consumer group. Scale to multiple instances of the payments service and only one processes each message — the broker distributes load across the group. Omit `group` for broadcast semantics where every instance receives every message.

Inside the handler, `msg.headers.get("event-type", "unknown")` inspects the envelope metadata before touching the payload. The header value is the domain event class name — `"FundsDeposited"`, `"WalletOpened"`, or `"FundsWithdrawn"` — matching what Lumen sets on the publisher side.

Listeners on service classes

When a listener needs collaborators — a repository, another service — declare it as a method on a `@service` class. PyFly injects the dependencies through the constructor and wires the listener subscription after the bean is initialised:

`lumen/messaging/notifications_consumer.py`

```
from __future__ import annotations

import json

from pyfly.container import service
from pyfly.messaging import Message, message_listener

@service
class WalletNotificationConsumer:
    """Sends push notifications when wallet events arrive via the broker."""

    def __init__(self, smtp_client: object) -> None:
        # smtp_client would be an injected email/push service.
        self._smtp = smtp_client

    @message_listener(topic="wallet.events", group="notifications-service")
    async def on_wallet_event(self, msg: Message) -> None:
        event_type = msg.headers.get("event-type", "unknown")

        if event_type != "WalletOpened":
            return

        payload = json.loads(msg.value)
        owner_id: str = payload.get("owner_id", "")
        wallet_id: str = payload.get("wallet_id", "")
        currency: str = payload.get("currency", "")

        print(
            f"[Notification] Welcome {owner_id}! "
```

```
f"Your {currency} wallet {wallet_id} is ready."
)
```

Listing 10.5 — `@message_listener` on a `@service` method with dependencies

How it works. `@service` registers `WalletNotificationConsumer` in the DI container. The constructor receives `smtp_client` through injection. After the bean is created, the framework detects `on_wallet_event` carrying `__pyfly_message_listener__ = True` and registers it as a bound-method listener — `self` is already captured, so every invocation has full access to `self._smtp`.

The early return on `event_type != "WalletOpened"` is a filtering guard. A single topic (`wallet.events`) carries multiple event types, so each listener filters for the ones it cares about. This is simpler than maintaining a separate topic per event type, though for very high-volume streams, topic-per-type is a legitimate design trade-off.

💡 CONSUMER GROUP SEMANTICS AT A GLANCE

Two services with *different* group names each receive every message — the broker delivers a copy to each group. Two *instances* of the same service sharing the *same* group name share the load — each message goes to exactly one instance. Use different groups for fanout (payments and notifications both need the event); use the same group for horizontal scaling (three instances of the payments service share the work).

Serialisation and schema evolution

Why bytes, and why this matters

`MessageBrokerPort.publish` accepts raw `bytes`. That is a deliberate choice. A broker adaptor that forced a single serialisation format would be convenient for simple cases and painful for everything else — schema evolution, multi-language consumers, compliance requirements, and throughput constraints all push in different directions. By leaving serialisation to you, PyFly stays out of the way.

Three formats are worth knowing: JSON for simplicity, Avro for schema-registry-backed evolution, and Protobuf for performance-critical or multi-language environments:

Format	Human-readable	Schema enforcement	Schema evolution	Multi-language	PyFly encoding
JSON	Yes	Optional	Manual (consumer discipline)	Universal	<code>json.dumps(...).encode()</code>
Avro	No	Yes (via registry)	First-class (<code>BACKWARD</code> / <code>FORWARD</code> / <code>FULL</code>)	Good	<code>fastavro</code> library
Protobuf	No	Yes (<code>.proto</code> files)	First-class (field numbering)	Excellent	<code>protobuf</code> library

JSON — start here

JSON is the right default. It requires no tooling beyond the standard library, every language can parse it, and the payload is readable in broker monitoring UIs. The encoding pattern is two lines:

```
import json

payload: bytes = json.dumps({
    "wallet_id": "w-001",
    "amount": 5000,          # integer minor units (€50.00)
    "currency": "EUR",
    "balance": 10000,        # new balance, minor units
    "event_type": "FundsDeposited",
}).encode()

await broker.publish("wallet.events", payload)
```

Decoding in the consumer:

```
data: dict = json.loads(msg.value)
```

JSON's weakness is that the schema is unenforced. If a publisher adds a required field and the consumer has not been updated, the consumer breaks silently. For Lumen's internal events where producer and consumer are deployed together, this is manageable. For events shared with external teams or long-lived topics, you need stronger guarantees.

Avro — schema-registry-backed evolution

Avro schemas are JSON documents describing the shape of a message. A Schema Registry (Confluent's is the most common, but open-source alternatives exist) stores those schemas and enforces compatibility rules when producers register new versions. The `fastavro` library encodes and decodes the binary payload:

```
lumen/messaging/avro_publisher.py
```

```
from __future__ import annotations

import io

import fastavro # type: ignore[import]
```

```

from pyfly.messaging import MessageBrokerPort

WALLET_DEPOSITED_SCHEMA = {
    "type": "record",
    "name": "FundsDeposited",
    "namespace": "lumen.wallet",
    "fields": [
        {"name": "wallet_id", "type": "string"},
        {"name": "amount", "type": "long"},      # integer minor units
        {"name": "currency", "type": "string"},
        {"name": "balance", "type": "long"},    # new balance, minor units
    ],
}

_PARSED = fastavro.parse_schema(WALLET_DEPOSITED_SCHEMA)

async def publish_deposit_avro(
    broker: MessageBrokerPort,
    wallet_id: str,
    amount: int,
    currency: str,
    balance: int,
) -> None:
    """Encode a FundsDeposited event with Avro and publish to the topic."""
    record = {
        "wallet_id": wallet_id,
        "amount": amount,      # integer minor units
        "currency": currency,
        "balance": balance,
    }
    buf = io.BytesIO()
    fastavro.schemaless_writer(buf, _PARSED, record)

    await broker.publish(
        "wallet.events",
        buf.getvalue(),
        headers={"content-type": "avro/binary",

```

```
        "event-type": "FundsDeposited"},
    )
```

Listing 10.6 — Publishing a wallet event with Avro encoding

How it works. `fastavro.parse_schema` compiles the JSON schema document once at module load time — never parse it inside the publish function or you pay the compilation cost on every call. `fastavro.schemaless_writer` serialises the record into the `BytesIO` buffer without embedding the schema in each message (the registry provides the schema on the consumer side). `buf.getvalue()` extracts the bytes for `broker.publish`.

The

```
headers={"content-type": "avro/binary", "event-type": "FundsDeposited"}
```

headers signal to consumers that Avro decoding is required and carry the event type for routing — consistent with the JSON convention.

Protobuf — performance and polyglot

Protocol Buffers compile a `.proto` file into a generated class. They produce smaller messages than JSON or Avro, and the generated code is available in every major language — making Protobuf the right choice when the consumer is a Go or Java service.

```
# Assumes a generated class lumen_pb2.FundsDeposited
from lumen_pb2 import FundsDeposited # type: ignore[import]

event = FundsDeposited(
    wallet_id="w-001",
    amount=5000,      # integer minor units
    currency="EUR",
    balance=10000,
)
payload: bytes = event.SerializeToString()
```



```
await broker.publish(
    "wallet.events",
    payload,
    headers={"content-type": "application/protobuf",
            "event-type": "FundsDeposited"},
)
```

Decoding in the consumer follows the mirror pattern:

```
from lumen_pb2 import FundsDeposited # type: ignore[import]

event = FundsDeposited()
event.ParseFromString(msg.value)
```

💡 START WITH JSON, MIGRATE WHEN YOU FEEL THE PAIN

The correct progression for most teams is: JSON first (fast to ship, easy to debug); add Avro when multiple teams own different sides of a topic and schema drift becomes a real coordination cost; switch to Protobuf when binary size or multi-language interop is a hard requirement. Because PyFly's `publish` and `@message_listener` accept raw bytes, you can change serialization format without changing the broker API calls — just swap the encoding and decoding steps.

When delivery fails: dead-letter queues

The inevitable bad message

Even a well-designed consumer will eventually encounter a message it cannot process. A downstream database may be unavailable. The payload may violate an assumption the consumer relied on. A transient network error may interrupt a third-party API call mid-handler. The question is not whether a consumer will fail — it is what happens when it does.

Without a dead-letter strategy, a failed consumer either drops the message (data loss) or re-queues it indefinitely, creating an infinite retry loop that blocks all subsequent messages — a *poison pill*. A **dead-letter queue** (DLQ) is the structured answer: a separate topic or queue where messages that cannot be processed after a configurable number of attempts are parked for inspection and manual reprocessing.

Decorator-native retry and DLQ

In PyFly, retry and dead-letter routing are built into `@message_listener` — no try/except scaffolding, no manual publish to the DLQ. Declare `retries` and `dead_letter_topic` directly on the decorator:

`lumen/messaging/resilient_consumer.py`

```
from __future__ import annotations

import json
import logging

from pyfly.container import service
from pyfly.messaging import Message, message_listener

logger = logging.getLogger(__name__)

@service
class ResilientWalletConsumer:
    """Processes wallet events with built-in retry and DLQ fallback."""

    @message_listener(
        topic="wallet.events",
        group="payments-dlq",
        retries=3,
        retry_delay=0.5,          # waits 0.5s, 1.0s, 1.5s (linear)
        dead_letter_topic="wallet.events.DLQ",
    )
```

```

async def on_wallet_event(self, msg: Message) -> None:
    payload = json.loads(msg.value)
    event_type = msg.headers.get("event-type", "unknown")
    logger.info(
        "Processing event: type=%s wallet=%s",
        event_type,
        payload.get("wallet_id"),
    )
    # Any unhandled exception triggers a retry; after 3 retries
    # the original message is forwarded to wallet.events.DLQ.
    await self._charge(payload)

async def _charge(self, payload: dict) -> None:
    raise NotImplementedError("replace with real payment logic")

```

Listing 10.7 — Retry and DLQ wired through `@message_listener`

The three parameters:

`retries=3` re-invokes `on_wallet_event` up to three more times after the first failure. Retries are appropriate for *transient* failures (a single database node restarting); keep the count low and let the DLQ handle sustained failures.

`retry_delay=0.5` applies linear back-off: attempt 1 waits 0.5 s, attempt 2 waits 1.0 s, attempt 3 waits 1.5 s. With `retry_delay=0.0` (the default), retries are immediate.

`dead_letter_topic="wallet.events.DLQ"` is the safety net. When all retries are exhausted, the framework re-publishes the original message to the DLQ topic, preserving the original `value` and `key`, and adds two diagnostic headers:

Header	Value
<code>x-original-topic</code>	The topic the message was originally consumed from.
<code>x-exception</code>	The exception class name (e.g. <code>RuntimeError</code>).

The exception is then swallowed so the consumer keeps running — the message is parked, not lost, and the next message on the topic is processed normally.

Monitoring the DLQ

Subscribe to the DLQ topic like any other listener to observe and alert on dead-lettered messages:

`lumen/messaging/dlq_monitor.py`

```
from __future__ import annotations

import json
import logging

from pyfly.messaging import Message, message_listener

logger = logging.getLogger(__name__)

@message_listener(topic="wallet.events.DLQ", group="dlq-monitor")
async def on_dead_letter(msg: Message) -> None:
    """Log every message that failed all retries."""
    original = msg.headers.get("x-original-topic", "unknown")
    exc = msg.headers.get("x-exception", "unknown")
    payload = json.loads(msg.value) if msg.value else {}
    logger.warning(
        "DLQ message: original_topic=%s exception=%s wallet=%s",
        original,
        exc,
        payload.get("wallet_id"),
    )
```

Listing 10.8 — Subscribing to the DLQ topic

⚠️ DESIGN CONSUMERS FOR IDEMPOTENCY

A consumer that reaches the DLQ retry limit has consumed the message. If an operator later replays the DLQ message, the consumer will process it again. Without idempotency, that double-processing can corrupt data — crediting a wallet twice, sending a duplicate notification. Use the message's stable identifier as an idempotency key: before processing, check whether that ID has already been recorded in a `processed_events` table, and skip the work if it has. The check-and-record step should be in the same database transaction as the business write.

Resilience: circuit breakers and retries

Protecting Lumen from a broker outage

A healthy broker is not guaranteed. Network partitions, rolling upgrades, and resource exhaustion can all make the broker temporarily unavailable. If the command handler calls `broker.publish(...)` and the broker is down, you face two bad choices without a resilience layer: fail the entire command (refusing to deposit funds because the broker is unreachable) or silently drop the event (the deposit succeeds but the integration event is lost).

Neither is acceptable. The transactional outbox (Chapter 9) is the atomic solution — the event is captured in the database and a relay publishes it asynchronously, so a broker outage adds only latency, not data loss. Alongside the outbox, **circuit breakers** and **retries** protect the relay and any broker-calling code from cascading failures.

PyFly's resilience module (`pyfly.resilience`) provides both primitives. The circuit breaker opens after a configurable failure threshold and blocks calls to the broker during a cool-down period, preventing a thundering-herd reconnection storm. The retry decorator handles transient errors with configurable back-off:

```
lumen/messaging/resilient_publisher.py
```

```
from __future__ import annotations

import json
import logging

from pyfly.container import service
from pyfly.messaging import MessageBrokerPort
from pyfly.resilience import CircuitBreaker, circuit_breaker, retry

logger = logging.getLogger(__name__)

@service
class OutboxRelay:
    """
    Drains pending outbox records and forwards them to the broker.
    Applies retry and circuit-breaker protection on every publish call.
    """

    def __init__(self, broker: MessageBrokerPort) -> None:
        self._broker = broker

    @retry(max_attempts=3, delay=1.0, backoff=2.0)
    @circuit_breaker(CircuitBreaker(failure_threshold=5, recovery_timeout=30))
    async def forward(
        self,
        topic: str,
        payload: dict,
        event_type: str,
    ) -> None:
        """Forward a single outbox record to the broker."""
        await self._broker.publish(
            topic,
            json.dumps(payload).encode(),
            headers={"event-type": event_type},
        )
        logger.info(
```

```

        "Event forwarded: topic=%s event-type=%s",
        topic,
        event_type,
    )

```

Listing 10.9 — Resilient broker publishing with retry and circuit breaker

The two decorators:

`@retry(max_attempts=3, delay=1.0, backoff=2.0)` wraps `forward` in a retry loop of up to three attempts. After the first failure it waits `delay` seconds (1 s); after the second it waits `delay * backoff` (2 s) — the wait grows as `delay * backoff ** attempt`. If the third attempt still fails, the exception propagates. Retries suit *transient* failures (a single broker node restarting); they are counterproductive for *permanent* failures (a misconfigured topic). Keep `max_attempts` low and let the circuit breaker handle sustained outages.

`@circuit_breaker(CircuitBreaker(failure_threshold=5, recovery_timeout=30))` guards `forward` with a shared `CircuitBreaker` instance that tracks consecutive failures across all calls. When the count reaches `failure_threshold`, the circuit *opens* and subsequent calls fail immediately with `CircuitBreakerException` rather than attempting to reach an unreachable broker — preventing a reconnection storm. After `recovery_timeout` seconds the circuit enters a *half-open* state: the next call is allowed through as a probe. If it succeeds, the circuit closes; if it fails, it re-opens. Decorator order matters: `@retry` is the outer decorator, so all three attempts of one logical call happen before the circuit breaker registers a single failure.

🍃 SPRING PARITY

`MessageBrokerPort` with `KafkaAdapter` is PyFly's counterpart of Spring Kafka's `KafkaTemplate` (publishing) and `@KafkaListener` (consuming). `RabbitMQAdapter` mirrors Spring AMQP's `RabbitTemplate` and `@RabbitListener`. The lifecycle model is the same: register listeners before starting the container, and the framework manages the consumer threads. Dead-letter queues in Spring Kafka are configured via `DeadLetterPublishingRecoverer` on the `DefaultErrorHandler`; in Spring AMQP via `RabbitListenerContainerFactory` with a `MessageRecoverer`. PyFly implements the same pattern declaratively through `@message_listener(retries=..., dead_letter_topic=...)` rather than requiring broker-specific container configuration. The `@retry` and `@circuit_breaker` decorators mirror Resilience4j's `@Retryable` and `@CircuitBreaker` annotations used with Spring's messaging infrastructure.

✓ What you built

Part III is complete.

Lumen is now fully event-driven, event-sourced, and broker-connected. Here is where each chapter left things.

Chapter 8 introduced the two-bus model: `ApplicationEventBus` for framework lifecycle events, `InMemoryEventBus` for domain events. `EventPublisher` was wired into the command handlers so every aggregate mutation produced a fact that independent listeners — `WalletAuditListener` among them — could react to without knowing each other. Subscriptions use `@event_listener(event_types=["WalletOpened", "FundsDeposited", "FundsWithdrawn"])`; handlers receive an `EventEnvelope` whose `event_type` is the domain event class name.

Chapter 9 replaced the mutable aggregate-plus-read-model approach with event sourcing. Every financial movement is an immutable event appended to the ledger. The current balance is computed by replaying the event stream. `EventEnvelope` became the unit of storage, and snapshots kept replay times bounded.

This chapter crossed the network boundary. `MessageBrokerPort` is the single abstraction in front of Kafka, RabbitMQ, or the in-memory broker. Swapping adapters is a configuration change — no business code changes. `@message_listener` gives declarative, zero-boilerplate subscriptions on both standalone functions and `@service` methods. The `retries` and `dead_letter_topic` parameters handle poisoned messages without manual try/except scaffolding. Payloads were encoded as JSON bytes, with Avro and Protobuf available when schema enforcement or binary efficiency matters more than simplicity. `@retry` and `@circuit_breaker` protect the publish path from transient and sustained broker failures.

The domain events flowing through all three chapters are:

Event class	Fields
<code>WalletOpened</code>	<code>wallet_id</code> , <code>owner_id</code> , <code>currency</code>
<code>FundsDeposited</code>	<code>wallet_id</code> , <code>amount</code> , <code>currency</code> , <code>balance</code>
<code>FundsWithdrawn</code>	<code>wallet_id</code> , <code>amount</code> , <code>currency</code> , <code>balance</code>

`amount` and `balance` are always integer minor units (e.g. `5000` for €50.00). `currency` is a string value from the `Currency` StrEnum (`"EUR"` , `"USD"` , `"GBP"`). The `event_type` header value is always the class name — `"FundsDeposited"` — never a dotted path.

Three principles carry forward into Part IV:

- **Depend on the port, not the adapter.** `MessageBrokerPort` is injected; `KafkaAdapter` is a configuration detail.
- **Design consumers for idempotency.** Brokers deliver *at least once*. Guard against duplicate processing with a stable message identifier.
- **Capture events atomically.** The transactional outbox ensures an event is never lost even when the broker is unavailable at write time.

Part IV introduces the `PaymentsService` and `NotificationsService`. Both subscribe to `wallet.events`. The adapter and configuration choices made in this chapter are all they need to start receiving Lumen's facts the moment they connect.

Try it yourself

1. **Swap the adapter in one line.** Start with `provider: "memory"` in `pyfly.yaml` and add the `@message_listener` from Listing 10.4. Write an integration test that publishes a `FundsDeposited` message with `amount=5000` and `currency="EUR"` and asserts the listener receives it. Then switch `provider: "kafka"` in the YAML and confirm the same test (with a Testcontainers-managed Kafka broker) passes without changing the listener or the test assertion.
2. **Add a DLQ monitor.** Create a second `@message_listener` on topic `wallet.events.DLQ` with group `dlq-monitor`. It should log the `x-original-topic` and `x-exception` headers along with the decoded payload. Write a test that simulates a failing consumer by raising `RuntimeError` inside the handler,

configures `retries=2` and `dead_letter_topic="wallet.events.DLQ"`, and confirms the DLQ monitor receives the message with `x-original-topic: "wallet.events"`.

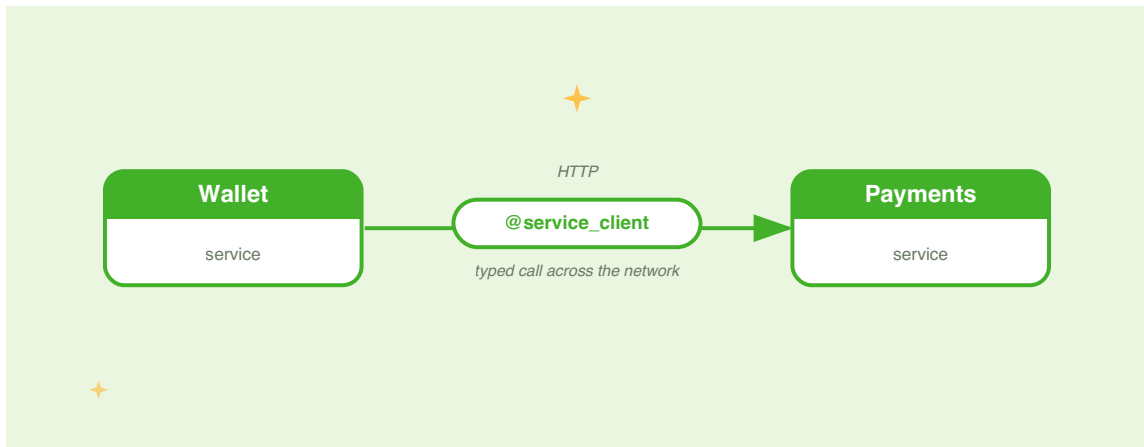
- 3. Evolve the schema with Avro.** Start with the `WALLET_DEPOSITED_SCHEMA` from Listing 10.6. Add an optional `note` field with a default of `None` (Avro union `["null", "string"]`, default `null`). Confirm that a consumer compiled against the original schema can still decode a message encoded with the new schema — this is a *backward-compatible* change. Then try adding a required field without a default and observe the `SchemaParseException` the registry would raise, illustrating why defaults are mandatory for safe evolution.

PART IV

Into Microservices

CHAPTER 11

Splitting the Monolith: HTTP Clients & the BFF



When Lumen was a single service, every capability lived in the same process. The wallet, the balance check, and the payment processing all ran together — straightforward to test, simple to deploy, and perfectly adequate until the team needed to ship features at different cadences. Then came the hard conversation about splitting.

The promise of a microservice split is real: teams own their services independently, scale them separately, and deploy without coordinating a shared release window. But every split introduces a problem the monolith never had — the network. What was a local function call becomes an HTTP request that can time out, fail halfway, or land on an overwhelmed service. That network boundary is not a deployment detail; it is a first-class engineering concern.

This chapter introduces `PaymentsService`, a second service that Lumen's Wallet service calls to settle transfers. Instead of hand-rolling `httpx` sessions and threading circuit-breaker logic through every handler, you will define the Payments client as an ordinary Python class — a typed, declarative interface that PyFly fills in at startup. By the end of the chapter you will also see how a **BFF (Backend for Frontend)** tier sits in front of both services and composes their capabilities into a single, user-journey-focused API.

Why split (and why it hurts)

The monolith comfort zone

A monolith is not an architectural mistake — it is an architectural starting point. Lumen began as one service because one service was right: one team, one deployment pipeline, one set of concerns to reason about. The database transaction that writes a wallet row and publishes a domain event in the same unit of work was not a compromise; it was the optimal choice.

The pressure to split usually arrives from outside the architecture. Payments needs a separate compliance audit trail. Risk scoring needs a specialist team with access to a private data source. Settlement processing demands throughput an order of magnitude higher than balance reads. Any one of these is a good reason to extract a service — and none of them erases the fact that the rest of the system still needs to call the extracted service across a network boundary.

The cost of the network

Network calls fail in ways that local function calls do not. A method on a local object either returns a value or raises an exception. An HTTP call to a remote service can time out (the remote is slow), refuse the connection (the remote is down), return a transient 503 (the remote is overloaded), or succeed only on the third attempt. In a monolith these failure modes are irrelevant; in a distributed system they are your baseline.

The naive fix — use `httpx` directly with `try/except` around every call — works for one call site but does not scale. You end up with circuit- breaker logic duplicated across every service client, retry delays hardcoded in handlers, and timeout values scattered through `pyfly.yaml` fragments that nobody owns. When Payments introduces a new endpoint, every caller must remember to add all the resilience scaffolding again.

PyFly's typed HTTP client eliminates that duplication. You declare what the remote service looks like — its endpoints, paths, and parameter shapes. PyFly generates the implementation at startup, wires in a circuit breaker and retry policy from `pyfly.yaml`, and registers the bean in the container so any handler that needs it can declare it as a constructor argument. Resilience is applied once, consistently, in the right layer.

A typed service client

Declarative over imperative

The core insight behind PyFly's client module is that service-to-service contracts are better expressed as types than as procedural HTTP logic. When you describe `PaymentsClient` as a class with typed method signatures, you get a Python interface that any IDE can navigate, any type checker can verify, and any test can mock — without ever importing `httpx` in the code that uses it.

Two decorators define that contract:

Decorator	Resilience built in	Use for
<code>@service_client</code>	Circuit breaker + retry	Production service-to-service calls
<code>@http_client</code>	None	Lightweight clients, testing, internal tooling

Use `@service_client` whenever the target is another microservice. Reserve `@http_client` for internal utilities and test doubles.

Defining the Payments client

The Payments service exposes two endpoints: one to create a payment instruction and one to retrieve a payment by identifier. Defining the client means writing the class:

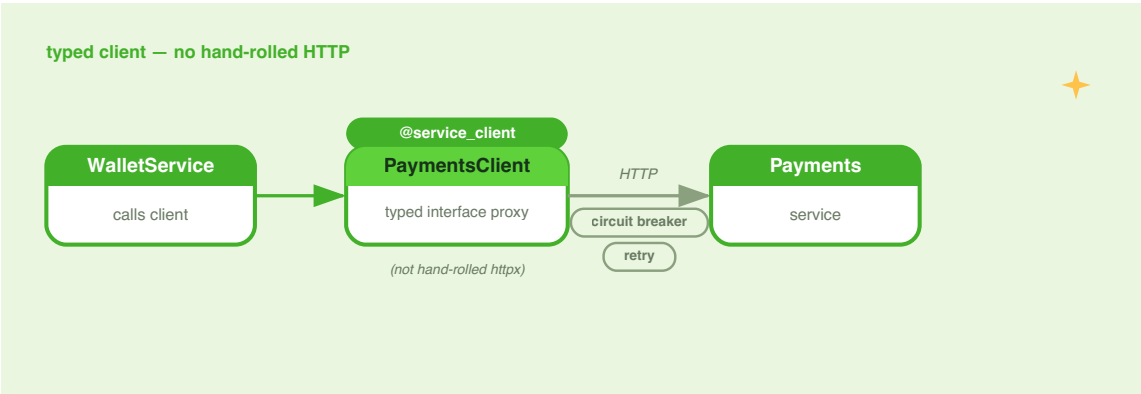


Figure 11.1 — The PyFly declarative client pipeline. You write the interface; `HttpClientBeanPostProcessor` generates the implementation.

```
lumen/sdk/payments_client.py

from __future__ import annotations

from pyfly.client import (
    delete,
```



```

    get,
    patch,
    post,
    service_client,
)

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
    circuit_breaker_failure_threshold=5,
    circuit_breaker_recovery_timeout=60.0,
    retry_base_delay=1.0,
)

class PaymentsClient:
    """Typed HTTP client for the Payments service.

    Method stubs are replaced with real HTTP implementations by
    HttpClientBeanPostProcessor at application startup. Declare
    this class as a constructor argument to have it injected.
    """

    @post("/payments")
    async def create_payment(self, body: dict) -> dict:
        """POST /payments – submit a payment instruction."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(self, payment_id: str) -> dict:
        """GET /payments/:payment_id – fetch a payment by ID."""
        ...

    @patch("/payments/{payment_id}/cancel")
    async def cancel_payment(self, payment_id: str) -> dict:
        """PATCH /payments/:payment_id/cancel – cancel pending."""
        ...

    @delete("/payments/{payment_id}")

```

```

async def delete_payment(self, payment_id: str) -> None:
    """DELETE /payments/:payment_id - remove a completed record."""
    ...

```

Listing 11.1 — Typed Payments client with `@service_client`

How it works — the declaration pipeline:

`@service_client(base_url=...)` stamps metadata attributes on the class and registers it as a singleton bean in the PyFly container — the same `__pyfly_injectable__ = True` mechanism that `@service` uses. The `base_url` is stored as `__pyfly_http_base_url__`; the resilience options land in `__pyfly_resilience__`.

The verb decorators — `@post("/payments")`, `@get("/payments/{payment_id}")`, and the others — each attach two attributes to their method: `__pyfly_http_method__` (the HTTP verb string) and `__pyfly_http_path__` (the path template). The method body itself becomes a stub that raises `NotImplementedError` and should never be called directly.

At startup, `HttpClientBeanPostProcessor.after_init()` inspects every bean. When it finds a class with `__pyfly_http_client__ = True`, it creates an `HttpClientAdapter` for `base_url`, scans every method for `__pyfly_http_method__`, and replaces each stub with a real async implementation. That implementation uses `inspect.signature()` to bind the caller's arguments, interpolates path variables (`{payment_id}` → the actual value), separates remaining parameters into query strings or a JSON body, and calls `client.request()`. Responses with status ≥ 400 raise typed exceptions; successful responses return `response.json()`.

Path variable interpolation is positional: any parameter whose name matches a `{placeholder}` in the path template is substituted. For `get_payment(self, payment_id: str)`, calling `client.get_payment("pay-123")` sends `GET /payments/pay-123`. For `create_payment(self, body: dict)`, calling `client.create_payment({"amount": 5000})` sends `POST /payments` with the dict serialised as the JSON body. Parameters named `body` on POST/PUT/PATCH methods are always treated as the JSON request body; all other non-path parameters on GET/DELETE become query-string parameters.

SPRING PARITY

`@service_client` with `@get / @post / @put / @delete / @patch` is PyFly's counterpart of Spring Cloud OpenFeign's `@FeignClient` with `@GetMapping / @PostMapping` etc. In Feign you annotate an interface; in PyFly you annotate a class with stub methods — the intent is identical. Both frameworks generate the HTTP implementation at startup time, inject the bean through the DI container, and support circuit breakers (Feign via Resilience4j; PyFly via the built-in `CircuitBreaker`). The key difference is that Feign works on Java interfaces while PyFly works on ordinary Python classes, which means you can add helper methods alongside the stub methods — useful for response-shaping logic that belongs inside the client class itself.

Injecting the client into a handler

Because `PaymentsClient` is a singleton bean, any `@service` or `@command_handler` can declare it as a constructor argument. PyFly's container injects it through the same autowiring path used for repositories and domain services.

Lumen's Wallet service already applies the `WalletRepository` and `Money` value object pattern from earlier chapters. When the wallet must call Payments to settle a withdrawal, the handler follows that same pattern: withdraw through the aggregate, then call the external service:

lumen/core/services/wallets/settle_transfer_handler.py

```

from __future__ import annotations

from lumen.core.services.wallets.settle_transfer_command import (
    SettleTransfer,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from lumen.sdk.payments_client import PaymentsClient
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound

@command_handler
@service
class SettleTransferHandler(CommandHandler[SettleTransfer, dict]):
    """Withdraw from the wallet and submit a payment instruction."""

    def __init__(
        self,
        repository: WalletRepository,
        payments: PaymentsClient,
    ) -> None:
        super().__init__()
        self._repository = repository
        self._payments = payments

    async def do_handle(self, command: SettleTransfer) -> dict:
        wallet = await self._repository.find(command.wallet_id)
        if wallet is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet.withdraw(
            Money(amount=command.amount, currency=wallet.currency)
        )
        await self._repository.add(wallet)

```

```

payment = await self._payments.create_payment({
    "wallet_id": command.wallet_id,
    "amount": command.amount,
    "currency": wallet.currency.value,
    "reference": command.reference,
})
return payment

```

Listing 11.2 — *CommandHandler injecting PaymentsClient*

How it works — the injection path:

`payments: PaymentsClient` in the constructor is resolved by the container at startup. `HttpClientBeanPostProcessor` wires `PaymentsClient` before `SettleTransferHandler` is instantiated, so the injected bean is fully operational. The handler calls `await self._payments.create_payment(...)` exactly as if it were a local async method. Connection pooling, header propagation, and error mapping are all invisible to the handler.

`wallet.withdraw(Money(...))` runs before the network call, so the wallet state is committed before Payments is contacted. If Payments is temporarily unavailable, the retry and circuit breaker — described in the next section — handle recovery transparently, without any code in the handler.

Resilience on the wire

Why the client layer is the right place for resilience

Resilience logic inside a handler mixes business concerns with infrastructure plumbing. A handler that catches `httpx.ConnectError` and implements its own backoff loop is doing two things at once: settling a transfer *and* managing HTTP failure modes. Those responsibilities belong in separate layers.

`@service_client` moves the **circuit breaker** and **retry policy** to the client layer, where they belong. You configure them once on the decorator, and every method on the client inherits them uniformly. The handler code stays focused on the business operation.

Circuit breaker

A circuit breaker monitors every call to the remote service. When `failure_threshold` consecutive calls fail, the circuit **opens**: subsequent calls are rejected immediately with `CircuitBreakerException` rather than waiting for a timeout. This prevents a single slow or unavailable service from blocking the event loop and exhausting connection pools across every caller.

After `circuit_breaker_recovery_timeout` seconds, the circuit enters **half-open**: one probe request is admitted. If it succeeds, the circuit closes and normal operation resumes. If it fails, the circuit re-opens and the recovery timer resets.

`@service_client` wires the breaker automatically. If you need it standalone:

```
lumen/sdk/standalone_breaker.py
```

```
from __future__ import annotations

from datetime import timedelta

from pyfly.client import CircuitBreaker
from pyfly.kernel.exceptions import CircuitBreakerException

breaker = CircuitBreaker(
    failure_threshold=3,
    recovery_timeout=timedelta(seconds=30),
)
```

```

async def call_with_breaker(client, payment_id: str) -> dict:
    """Fetch a payment through a standalone circuit breaker."""
    try:
        return await breaker.call(
            client.get_payment,
            payment_id,
        )
    except CircuitBreakerException:
        return {"status": "unavailable", "payment_id": payment_id}

```

Listing 11.3 — Using CircuitBreaker standalone

How it works: `CircuitBreaker.__init__` accepts `failure_threshold` (default 5) and `recovery_timeout` as a `timedelta` (default 30 s). `breaker.call(func, *args)` executes `func(*args)` inside the breaker: on success it resets the failure count; on failure it increments the count and flips the state to `OPEN` once the threshold is reached. The state transitions `CLOSED → HALF_OPEN` are computed lazily with `time.monotonic()` — there is no background timer.

`CircuitBreakerException` is never counted as a failure. It signals that the circuit is already open, so re-raising it without recording another failure prevents the recovery timeout from resetting indefinitely.

Retry policy

Transient failures — a momentary latency spike, a rolling restart, a brief connection reset — do not need a circuit breaker; they need a second attempt. `RetryPolicy` provides exponential backoff with configurable exception filtering:

`lumen/sdk/standalone_retry.py`

```

from __future__ import annotations

from datetime import timedelta

```

```

from pyfly.client import RetryPolicy

policy = RetryPolicy(
    max_attempts=3,
    base_delay=timedelta(milliseconds=500),
    retry_on=(ConnectionError, TimeoutError),
)

async def resilient_fetch(client, payment_id: str) -> dict:
    """Fetch a payment with retry on transient network errors."""
    return await policy.execute(
        client.get_payment,
        payment_id,
    )

```

Listing 11.4 — Using `RetryPolicy` standalone

How it works: `RetryPolicy.__init__` accepts `max_attempts` (default 3, counting the first attempt), `base_delay` (default 1 s), and `retry_on` — a tuple of exception types. The backoff formula is `base_delay * (2 ** attempt)`: for `base_delay=0.5 s`, the delays are 0.5 s, 1 s, 2 s. Only exceptions matching `retry_on` trigger a retry; others propagate immediately. This matters: you do not want to retry a 404 (the resource does not exist) or a 422 (the request is semantically invalid).

When `@service_client` enables both features, the post-processor wraps them in the correct order: circuit breaker *outside*, retry *inside*. A single logical call attempts up to `max_attempts` retries before the circuit breaker records one failure. An open circuit rejects the call immediately, bypassing the retry loop entirely.

Typed error exceptions

When the remote service returns a 4xx or 5xx response, the generated method raises a typed exception instead of returning the error payload as if it were a success. The exception hierarchy lives in `pyfly.client`:

Status	Exception class	retryable
400	<code>ServiceValidationException</code>	False
401 / 403	<code>ServiceAuthenticationException</code>	False
404	<code>ServiceNotFoundException</code>	False
409	<code>ServiceConflictException</code>	False
422	<code>ServiceUnprocessableEntityException</code>	False
429	<code>ServiceRateLimitException</code>	True
5xx	<code>ServiceUnavailableException</code>	True

All exceptions extend `ServiceClientException` (itself an `InfrastructureException`). The `retryable` flag on `ServiceRateLimitException` and `ServiceUnavailableException` tells the post-processor which exceptions to pass to the retry policy. 4xx validation errors and 404s are never retried.

Configuring defaults in `pyfly.yaml`

Per-service overrides on `@service_client` always take precedence. Setting process-wide defaults in `pyfly.yaml` lets new clients inherit sensible values without repeating them on every decorator:

```
pyfly.yaml
```

```

pyfly:
  client:
    timeout: 10
    retry:
      max-attempts: 3
      base-delay: 1.0
    circuit-breaker:
      failure-threshold: 5
      recovery-timeout: 30

```

Listing 11.5 — Client resilience defaults in `pyfly.yaml`

Key	Description	Default
<code>pyfly.client.timeout</code>	Request timeout in seconds	30
<code>pyfly.client.retry.max-attempts</code>	Total attempts including first	3
<code>pyfly.client.retry.base-delay</code>	Base delay in seconds	1.0
<code>pyfly.client.circuit-breaker.failure-threshold</code>	Consecutive failures to open	5
<code>pyfly.client.circuit-breaker.recovery-timeout</code>	Seconds before probing	30

`ClientAutoConfiguration` reads `pyfly.client.timeout` at startup and passes it to `HttpxClientAdapter`. The `retry` and `circuit-breaker` sub-maps are forwarded as `default_retry` and `default_circuit_breaker` to `HttpClientBeanPostProcessor`.

Any value set directly on

`@service_client(circuit_breaker_failure_threshold=...)` overrides the default.

 SET PER-SERVICE TIMEOUTS LOW

The default `timeout: 30` is conservative. In production, each service should carry a `pyfly.yaml` override tuned to its SLA. A payments call that should complete in 500 ms should have `timeout: 2` — not 30 s — so a slow Payments instance fails fast and the circuit breaker can open before threads pile up.

Auth, discovery, and deduplication

Propagating identity downstream

When the Wallet service calls Payments, it often needs to carry the caller's identity — a JWT or an internal service token — so that Payments can enforce its own authorisation rules. The `headers` parameter is treated specially by the post-processor: when a stub method declares `headers: dict`, the value is forwarded as HTTP request headers, not serialised as a query string.

`lumen/sdk/payments_client_auth.py`

```
from __future__ import annotations

from pyfly.client import get, post, service_client

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
)
class AuthenticatedPaymentsClient:
    """Payments client that forwards caller identity on each request."""

    @post("/payments")
    async def create_payment(
```

```

        self,
        body: dict,
        headers: dict | None = None,
    ) -> dict:
        """POST /payments – body is the JSON payload; headers forwarded."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(
        self,
        payment_id: str,
        headers: dict | None = None,
    ) -> dict:
        """GET /payments/:payment_id – headers are forwarded."""
        ...

```

Listing 11.6 — Forwarding auth headers per-call

How it works: The post-processor checks whether a parameter named `headers` is present in the bound arguments and is a `dict`. When both conditions hold, it extracts the value from the query-parameter pool and forwards it as HTTP request headers. The handler passes the incoming `Authorization` header (or a freshly minted service token) as `headers={"Authorization": f"Bearer {token}"}`.

`HttpClientAdapter` also calls `inject_headers(headers)` on every request, propagating the W3C `traceparent` and `tracestate` headers from the current observability context so distributed traces stitch across service boundaries without any application-level work.

📘 SERVICE-TO-SERVICE IDENTITY PATTERNS

For internal services on a trusted network, a shared secret in an `X-Internal-Token` header is the simplest approach. For zero-trust architectures, consider mTLS (mutual TLS at the infrastructure layer) or a service mesh that injects identity certificates. For user-delegated calls, forward the original JWT. Whatever pattern you choose, the `headers` parameter gives you a clean injection point in the declarative client.

Service discovery

When `base_url` is a static string like `http://payments-service:8080`, you rely on DNS-based discovery — a Kubernetes `Service` or a Consul record resolves `payments-service` to the correct cluster IP. This is the recommended starting point and sufficient for most deployments.

For environments that need dynamic URL resolution (multiple environments behind the same client class, feature-flagged routing), supply the URL via configuration instead:

`pyfly.yaml`

```
pyfly:
  client:
    timeout: 10

  services:
    payments:
      base-url: "${PAYMENTS_SERVICE_URL:http://payments-service:8080}"
```

Listing 11.7 — Per-environment base URL in `pyfly.yaml`

A thin factory bean reads the config key and constructs the post-processor with a custom factory that injects the resolved URL. The client class itself does not change — only the factory does.

Request deduplication

Financial operations must be idempotent at the HTTP layer. If `create_payment` is called, times out, and is retried, Payments must not create two payment records. The standard mechanism is an `Idempotency-Key` header: a stable, caller-chosen identifier — typically the command's UUID — that Payments uses to detect and deduplicate repeated requests.

`lumen/core/services/wallets/settle_transfer_idempotent.py`

```
from __future__ import annotations

from lumen.core.services.wallets.settle_transfer_command import (
    SettleTransfer,
)
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from lumen.sdk.payments_client_auth import AuthenticatedPaymentsClient
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.domain import AggregateNotFound

@command_handler
@service
class SettleTransferIdempotentHandler(
    CommandHandler[SettleTransfer, dict]
):
    """Withdraw funds and submit payment with idempotency key."""

    def __init__(
        self,
        repository: WalletRepository,
        payments: AuthenticatedPaymentsClient,
    ) -> None:
        super().__init__()
        self._repository = repository
        self._payments = payments
```

```

async def do_handle(self, command: SettleTransfer) -> dict:
    wallet = await self._repository.find(command.wallet_id)
    if wallet is None:
        raise AggregateNotFound("Wallet", command.wallet_id)

    wallet.withdraw(
        Money(amount=command.amount, currency=wallet.currency)
    )
    await self._repository.add(wallet)

    idempotency_key = str(command.transfer_id)
    return await self._payments.create_payment(
        body={
            "wallet_id": command.wallet_id,
            "amount": command.amount,
            "currency": wallet.currency.value,
        },
        headers={"Idempotency-Key": idempotency_key},
    )

```

Listing 11.8 — Idempotency-Key forwarded via headers parameter

How it works: `command.transfer_id` is the stable identifier for this business operation, determined before the command reaches the handler. If the handler is called again for the same command — from a retry, a re-delivery, or a dead-letter replay — it passes the same `Idempotency-Key`. Payments stores the key alongside the created payment record and returns the existing record when the key has been seen before, rather than creating a second payment. That deduplication is a server-side concern; the client's job is simply to forward the key consistently.

The experience tier: the BFF

Why the frontend cannot talk to both services directly

When a mobile app or web frontend needs to display a wallet summary that includes pending payment instructions, it faces a choice: call Wallet for the balance, call Payments for the pending list, and merge the results in the client — or talk to a single API that does the merging server-side. The first option incurs two round trips, exposes each service's internal shape to the client, and forces the client to implement retry and error handling for two independent failure domains. The second option is the **BFF pattern**.

A **Backend for Frontend** is a lightweight service in the *experience tier* that composes responses from multiple domain services into a shape tailored to a specific frontend's needs. It handles response aggregation, field renaming to match client conventions, and caching of composed results. It never touches the database directly — it depends entirely on domain service clients.

Building the Lumen BFF

The Lumen SDK already ships a `LumenClient` in `lumen/sdk/client.py` that wraps a raw `httpx.AsyncClient`. In the BFF tier you use PyFly's declarative `@service_client` instead — the interface is identical but resilience is built in automatically. Here is the wallet-side client:

```
lumen_bff/sdk/wallet_client.py
```

```
from __future__ import annotations

from pyfly.client import get, service_client

@service_client(
    base_url="http://wallet-service:8080",
```



```

        circuit_breaker=True,
        retry=3,
    )
    class WalletClient:
        """Typed HTTP client for the Lumen Wallet service."""

        @get("/api/v1/wallets/{wallet_id}")
        async def get_wallet(self, wallet_id: str) -> dict:
            """GET /api/v1/wallets/:wallet_id - fetch a wallet."""
            ...

        @get("/api/v1/wallets/{wallet_id}/balance")
        async def get_balance(self, wallet_id: str) -> dict:
            """GET /api/v1/wallets/:wallet_id/balance - current balance."""
            ...

```

Listing 11.9 — WalletClient for the BFF tier

The paths mirror the real Lumen controller — `@request_mapping("/api/v1/wallets")` with `@get_mapping("/{wallet_id}")` — so the BFF client matches exactly what the Wallet service exposes.

The Payments service needs a `list_pending` endpoint that lets the BFF query pending records by wallet:

`lumen_bff/sdk/payments_client_bff.py`

```

from __future__ import annotations

from pyfly.client import get, post, service_client

@service_client(
    base_url="http://payments-service:8080",
    circuit_breaker=True,
    retry=3,
)

```

```

class PaymentsClient:
    """Typed HTTP client for Payments (BFF edition)."""

    @post("/payments")
    async def create_payment(self, body: dict) -> dict:
        """POST /payments – submit a payment instruction."""
        ...

    @get("/payments/{payment_id}")
    async def get_payment(self, payment_id: str) -> dict:
        """GET /payments/:payment_id – fetch a payment by ID."""
        ...

    @get("/payments")
    async def list_pending(self, wallet_id: str) -> list:
        """GET /payments?wallet_id=... – list payments for a wallet."""
        ...

```

Listing 11.10 — Extended PaymentsClient for the BFF

The BFF service then composes the wallet balance with the pending payments list into a single response:

lumen_bff/application/bff_service.py

```

from __future__ import annotations

import asyncio

from lumen_bff.sdk.payments_client_bff import PaymentsClient
from lumen_bff.sdk.wallet_client import WalletClient
from pyfly.container import service

@service
class WalletSummaryService:
    """Composes wallet balance and pending payments into one view.

```

Calls both domain services concurrently using `asyncio.gather` so the total latency is `max(wallet_latency, payments_latency)` rather than their sum.

```

"""

def __init__(
    self,
    wallet: WalletClient,
    payments: PaymentsClient,
) -> None:
    self._wallet = wallet
    self._payments = payments

    async def get_summary(self, wallet_id: str) -> dict:
        """Return a unified summary for the given wallet."""
        wallet_data, pending = await asyncio.gather(
            self._wallet.get_wallet(wallet_id),
            self._payments.list_pending(wallet_id),
            return_exceptions=True,
        )

        balance_minor: int = 0
        if isinstance(wallet_data, dict):
            balance_minor = wallet_data.get("balance_minor", 0)

        pending_list: list = []
        if isinstance(pending, list):
            pending_list = pending

        return {
            "wallet_id": wallet_id,
            "balance_minor": balance_minor,
            "pending_payments": pending_list,
        }

```

Listing 11.11 — BFF service composing Wallet + Payments

How it works — the composition pattern:

`asyncio.gather(...)` fires both upstream calls concurrently. The wallet and payments calls run in parallel, so the composite latency is bounded by the slower of the two rather than their sum — at 50 ms per service, sequential calls cost 100 ms while concurrent calls cost roughly 55 ms.

`return_exceptions=True` is critical for a BFF. Without it, a single upstream failure raises an exception and the caller receives nothing. With it, a failed coroutine returns its exception object as the result instead of propagating it. The service inspects each result with `isinstance(wallet_data, dict)` and degrades gracefully — returning a partial response with a zero balance or an empty payment list rather than an HTTP 500. The BFF should make that decision explicit in its response shape, for example by including an `"errors"` key listing degraded fields.

The field name `balance_minor` follows Lumen's convention: amounts are stored as integer minor units (cents) and the field is named `balance_minor` throughout — in `WalletDto`, in deposit/withdraw responses, and here in the BFF summary.

Each `@service_client` wrapper on `WalletClient` and `PaymentsClient` handles retries and circuit breaking for its upstream call independently. If Payments is circuit-open, the wallet balance still appears; only the pending payments list is empty.

The BFF controller

The BFF exposes its composed response through a standard PyFly web handler. The controller is intentionally thin — its sole job is to delegate to the service:

```
lumen_bff/web/controllers/summary_controller.py
```

```
from __future__ import annotations

from lumen_bff.application.bff_service import WalletSummaryService
from pyfly.container import rest_controller
from pyfly.web import get_mapping, request_mapping
```

```

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletSummaryController:
    """Experience-tier controller for the wallet summary view."""

    def __init__(self, summary: WalletSummaryService) -> None:
        self._summary = summary

    @get_mapping("/{wallet_id}/summary")
    async def get_wallet_summary(self, wallet_id: str) -> dict:
        """GET /api/v1/wallets/:wallet_id/summary"""
        return await self._summary.get_summary(wallet_id)

```

Listing 11.12 — BFF controller

How it works: The BFF controller imports no domain models and touches no repositories — it depends only on `WalletSummaryService`, which in turn depends only on typed client interfaces. The dependency chain is controller → BFF service → declarative clients → remote HTTP. Each layer is independently testable: the controller with a mock service, the service with mock clients, and the clients with a mock `HttpClientPort`.

BFF SCOPE AND TEAM OWNERSHIP

A BFF is scoped to one frontend or one user journey — not one per microservice. Lumen might have a `lumen-mobile-bff` and a `lumen-web-bff`, each composing the same domain services but returning shapes optimised for their respective clients. The BFF is owned by the frontend team, not the domain team. Domain services expose stable contracts; BFFs adapt those contracts to client-specific shapes without coupling the domain services to any particular frontend's conventions.

 SPRING PARITY

The BFF pattern in PyFly mirrors the Spring Boot API Gateway / BFF approach where a thin Spring Boot application aggregates responses from multiple microservices. In the reactive Spring stack, `Mono.zip()` provides the same concurrent aggregation that `asyncio.gather()` does in Python. The `@FeignClient` in the BFF corresponds to `@service_client` in PyFly; the Spring `WebClient` approach of chaining `.flatMap()` calls corresponds to PyFly's `asyncio.gather()` + `isinstance` error handling. The team-ownership model — BFF owned by the frontend team, domain services owned by domain teams — is identical.

 What you built

You started this chapter with a single Lumen service and finished with an architecture that scales in multiple dimensions. Here is what changed and why it matters.

You extracted `PaymentsService`. Payments now runs in its own process with its own deployment pipeline and its own data store. Wallet handlers know nothing about how Payments stores payment records or which database engine it uses — all they see is the typed interface that `PaymentsClient` exposes.

You declared the client, not the implementation. `@service_client` with `@post`, `@get`, `@patch`, and `@delete` stubs gave you a typed interface that IDEs navigate and type checkers verify. `HttpClientBeanPostProcessor` generated the HTTP implementation at startup, read the resilience configuration from `pyfly.yaml`, and registered the bean for injection anywhere.

You made the network resilient. `circuit_breaker=True` and `retry=3` on the decorator wrapped every method with a shared circuit breaker and retry policy — circuit breaker outside, retry inside — so a sustained Payments outage opens the circuit fast

while transient errors recover automatically. Typed exceptions (`ServiceNotFoundException` , `ServiceUnavailableException`) give callers a clean signal without exposing raw HTTP status codes.

You introduced the BFF tier. `WalletSummaryService` composes two upstream calls with `asyncio.gather` , returns a partial response when one service is degraded, and exposes a single contract to the frontend. The BFF absorbs each domain service's independent release cycle and shields the frontend from their internal shapes.

Three principles carry through the rest of Part IV:

- **Depend on the typed client, not on `httpx` directly.** The declaration is your contract; the implementation is a framework detail.
- **Resilience belongs in the client layer.** Configure it once on `@service_client` ; every handler that uses the client inherits it.
- **BFFs compose; domain services provide.** Domain services own stable, fine-grained contracts; BFFs own the coarse-grained compositions that specific frontends need.

Try it yourself

1. **Add a fourth endpoint and verify path interpolation.** Extend `PaymentsClient` with a `@get("/payments")` method that accepts `wallet_id: str` and `status: str = "pending"` as parameters. Call it from a test and assert that the generated HTTP request is `GET /payments?wallet_id=abc&status=pending` . Verify that changing the default to `status="completed"` and calling the method without a `status` argument sends `status=completed` in the query string.

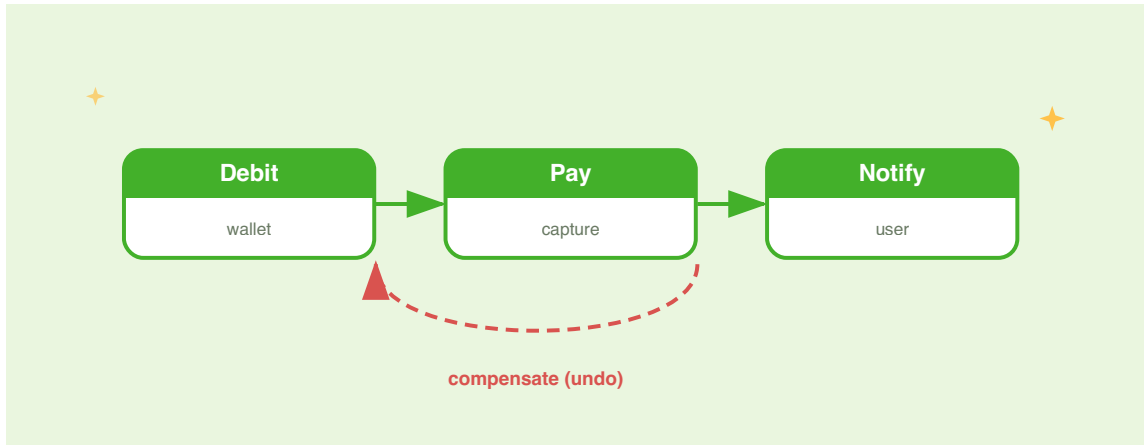
2. Test the BFF with degraded upstream services. Write a unit test for

`WalletSummaryService.get_summary` that mocks `WalletClient.get_wallet` to succeed and `PaymentsClient.list_pending` to raise `ServiceUnavailableException`. Assert that the method returns a dict with the correct `balance_minor` and an empty `pending_payments` list — confirming that the partial-response fallback works and a single upstream failure does not propagate as an exception to the BFF's caller.

3. Tune circuit breaker thresholds for a brittle upstream. Suppose Payments has a known flakiness window during its nightly batch run: it returns 503 on roughly 20 % of requests for about 10 seconds before stabilising. Configure `PaymentsClient` with `circuit_breaker_failure_threshold=2` and `circuit_breaker_recovery_timeout=15.0` and write a test using a mock `HttpClientPort` that simulates two consecutive failures followed by success. Assert that the third call (the probe after the recovery timeout) succeeds and that the circuit transitions back to `CLOSED`.

CHAPTER 12

Distributed Transactions: Sagas, Workflows & TCC



Chapter 10 sent Lumen's wallet events across process boundaries through Kafka. Chapter 11 split the application into co-operating services and showed how to call them over HTTP. Both steps unlocked scale and team autonomy, but they also exposed a new class of danger: multiple aggregates — or multiple services — may all need to change state as part of one business operation, with no distributed ACID transaction to protect you.

Picture a Lumen wallet transfer. You debit the source wallet, then credit the destination. If the source is debited and the credit fails — wrong currency, missing wallet — the source owner loses money with nothing deposited on the other side. You cannot wrap two independent repository calls in a single `BEGIN ... COMMIT` when each aggregate owns its own consistency boundary, and two-phase commit across independent aggregates is operationally fragile.

The answer is **eventual consistency with explicit compensation**. Each step commits to its own store independently, and you design a recovery path — a **compensating transaction** — for every step that could succeed before a later one fails. When the whole sequence succeeds you have your business result; when any step fails, the engine walks back the completed steps in reverse, calling each compensation to restore consistent state. This chapter shows you how to build that with PyFly's `pyfly.transactional` module.

You will model the money transfer as an **orchestrated saga** — a central class that declares each step and its compensation, organised in a DAG (directed acyclic graph) so the engine can run independent steps in parallel. You will then explore compensation in depth, the **Workflow** pattern for long-running or human-in-the-loop flows, and **TCC (Try-Confirm-Cancel)** as a reservation-based alternative. A closing section shows how pluggable persistence lets the engine survive a process crash and automatically resume stale executions.

The problem with distributed writes

Before writing any code, make the failure modes concrete.

Two aggregates, no safety net

Lumen's wallet transfer operates on two `Wallet` aggregates stored in the same PostgreSQL schema but treated as independent domain objects — each is loaded, mutated, and saved in its own round trip. The two steps are:

1. **Debit the source** — withdraw `amount` from the source `Wallet` (enforces `balance >= 0`).

2. Credit the destination — deposit `amount` into the destination `Wallet` (enforces currency match).

In a monolith, both writes could share one database transaction. In the Lumen domain service each step is an independent repository call. A currency mismatch on the destination, or a missing wallet ID, causes step 2 to fail after step 1 has already committed — leaving the source wallet debited and the destination unchanged. The user loses money.

Retrying the whole operation is unsafe: you might debit the source twice. Silently skipping the failed step leaves balances inconsistent. You need a principled pattern that commits each step independently and rolls back every completed step consistently on failure.

Eventual consistency and compensation

A **saga** decomposes the operation into a sequence of local transactions, each committing to its own store independently. When a step fails, the engine runs **compensating transactions** in reverse order for every completed step.

Compensations are not database rollbacks; they are *semantic undos* — new forward operations that reverse the effect. "Re-credit the source wallet" is a new deposit operation that restores the original balance, not a rollback.

SAGAS ARE EVENTUALLY CONSISTENT

A saga does not give you serializability or isolation. Between the moment the source wallet is debited and the moment the destination wallet is credited, another request could read the source wallet and see a balance that is lower than it will ultimately be. This is the trade-off you accept when you choose to operate across independent aggregates without a distributed lock. Sagas give you *consistency in the end* — either all forward steps committed or all are compensated — not *consistency at every point*.

An orchestrated saga

PyFly's `pyfly.transactional` module provides the `@saga` and `@saga_step` decorators. You declare one class per saga, annotate each method as a step with its compensation, and declare the dependency ordering. The engine discovers the class through the DI container, builds a validated DAG at startup, and drives execution asynchronously.

Enabling the engine

The transactional engine is activated by the `@enable_domain_stack` starter decorator on your application class, together with a single YAML property. In Lumen:

`lumen/app.py`

```
from pyfly.core import pyfly_application
from pyfly.starters.domain import enable_domain_stack

@EnableDomainStack
@pyfly_application(
    name="lumen",
    scan_packages=[
        "lumen.models.repositories",
        "lumen.core.services.transfers",
        # ... other packages
    ],
)
class LumenApplication:
    pass
```

Listing 12.1 — Enabling the transactional engine via the domain stack

Add the property in `application.yaml`:

```

pyfly:
  transactional:
    enabled: true

```

How it works: `@enable_domain_stack` imports

`TransactionalEngineAutoConfiguration`, guarded by

`@conditional_on_property("pyfly.transactional.enabled",`

`having_value="true")`. When the property is set, the auto-configuration wires every

engine component — `SagaEngine`, `TccEngine`, `WorkflowEngine`, `SagaRegistry`,

`InMemoryPersistenceAdapter`, and `LoggerEventsAdapter` — into the DI container.

The `OrchestrationBeanPostProcessor` then scans every bean produced at startup:

any bean carrying `__pyfly_saga__` metadata is registered into `SagaRegistry`

automatically. You never call `registry.register_from_bean()` in production code.

Declaring the transfer saga

Lumen's wallet transfer is a two-step saga: debit the source wallet, then credit the

destination. If the credit fails — wrong currency or missing wallet — the engine

compensates by re-crediting the source, returning both balances to their original values.

```
lumen/core/services/transfers/money_transfer_saga.py
```

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Annotated

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.transfers.transfer_request import TransferRequest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.domain import AggregateNotFound
from pyfly.transactional.saga.annotations import (

```

```

        FromStep,
        Input,
        saga,
        saga_step,
    )
    from pyfly.transactional.saga.core.context import SagaContext

MONEY_TRANSFER_SAGA = "money-transfer"

@dataclass(frozen=True)
class DebitResult:
    wallet_id: str
    amount: int
    currency: Currency
    balance: int

@saga(name=MONEY_TRANSFER_SAGA)
@service
class MoneyTransferSaga:
    """Debit source wallet, credit destination; compensate on failure."""

    def __init__(self, repository: WalletRepository) -> None:
        self._repository = repository

    # -- Step 1: debit the source -----

    @saga_step(id="debit-source", compensate="recredit_source")
    async def debit_source(
        self,
        request: Annotated[TransferRequest, Input()],
        ctx: SagaContext,
    ) -> DebitResult:
        entity = await self._repository.find_by_id(
            request.source_wallet_id
        )
        if entity is None:
            raise AggregateNotFound("Wallet", request.source_wallet_id)

```

```

        wallet = to_aggregate(entity)
        wallet.withdraw(
            Money(amount=request.amount, currency=request.currency)
        )
        await self._repository.upsert(to_entity(wallet))
        wallet.clear_events()
        return DebitResult(
            wallet_id=request.source_wallet_id,
            amount=request.amount,
            currency=request.currency,
            balance=wallet.balance.amount,
        )

    async def recredit_source(
        self,
        debit: Annotated[DebitResult, FromStep("debit-source")],
    ) -> int:
        """Compensation: put the money back. Receives the forward step's
        result via FromStep – NOT the saga input."""
        entity = await self._repository.find_by_id(debit.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", debit.wallet_id)
        wallet = to_aggregate(entity)
        wallet.deposit(
            Money(amount=debit.amount, currency=debit.currency)
        )
        await self._repository.upsert(to_entity(wallet))
        wallet.clear_events()
        return wallet.balance.amount

# -- Step 2: credit the destination -----

@saga_step(id="credit-destination", depends_on=["debit-source"])
    async def credit_destination(
        self,
        request: Annotated[TransferRequest, Input()],
        ctx: SagaContext,
    ) -> int:
        entity = await self._repository.find_by_id(

```

```

        request.destination_wallet_id
    )
    if entity is None:
        raise AggregateNotFound(
            "Wallet", request.destination_wallet_id
        )
    wallet = to_aggregate(entity)
    wallet.deposit(
        Money(amount=request.amount, currency=request.currency)
    )
    await self._repository.upsert(to_entity(wallet))
    wallet.clear_events()
    return wallet.balance.amount

```

Listing 12.2 — *MoneyTransferSaga: debit → credit, with compensation*

How it works — step by step:

`@saga(name=MONEY_TRANSFER_SAGA)` stamps `__pyfly_saga__` on the class with the saga name. The decorator only attaches metadata — it does not wrap the class or create a proxy. **The critical requirement** is that `@saga` must be stacked *on top of* `@service`. The `@service` annotation causes the DI container to instantiate and scan the bean at startup; the `OrchestrationBeanPostProcessor.after_init()` hook then sees `__pyfly_saga__` on the bean and calls `SagaRegistry.register_from_bean()`. Without `@service`, the class is never scanned and the saga cannot be executed by name.

`@saga_step` attaches `__pyfly_saga_step__` metadata directly to the async method — no wrapper, no proxy — so `inspect.iscoroutinefunction` keeps returning `True` and the engine correctly `await`s the call. The `compensate="recredit_source"` parameter names the *method on the same class* to invoke when rolling back this step. Omitting `depends_on` (or passing `[]`) means the step can run as soon as the engine starts.

Repository interaction — the load-mutate-save cycle. Each step follows the same three-phase pattern, using the framework

```
WalletRepository(Repository[WalletEntity, str]):
```

1. `find_by_id(id)` — loads the raw `WalletEntity` row from the database.
2. `to_aggregate(entity)` — rehydrates the rich `Wallet` aggregate from that row; the aggregate enforces all invariants (`balance >= 0` , currency match).
3. Mutate — call `wallet.withdraw(...)` or `wallet.deposit(...)` on the aggregate, letting it raise `BusinessRuleViolation` if an invariant is broken before any write occurs.
4. `upsert(to_entity(wallet))` — flattens the mutated aggregate back to a `WalletEntity` and calls `session.merge + flush` , so the write is visible to subsequent steps in the same `AsyncSession` without committing.

Because saga steps share one `AsyncSession` , `upsert` flushes so each step sees the previous step's write; the surrounding application boundary owns the final commit.

Parameter injection uses `typing.Annotated` with **marker instances**, not bare classes:

- `Annotated[TransferRequest, Input()]` — `Input()` is an instance (note the parentheses); bare `Input` without `()` does not resolve.
- `Annotated[DebitResult, FromStep("debit-source")]` — reads the result that step `"debit-source"` stored in `SagaContext` when it completed.
- `ctx: SagaContext` — injected by type; no `Annotated` marker needed.

The resolver inspects type hints at runtime via `typing.get_type_hints(func, include_extras=True)` .

Compensation methods do not receive the saga input. `recredit_source` takes `Annotated[DebitResult, FromStep("debit-source")]` — the value the forward step returned — not the `TransferRequest`. It re-loads the entity via `find_by_id`, rehydrates the aggregate, deposits the original amount back, and upserts — the same load-mutate-save cycle as the forward steps. Compensations always read from `ctx.step_results` via `FromStep`, never from the original input.

The step DAG

The two steps form a linear chain:

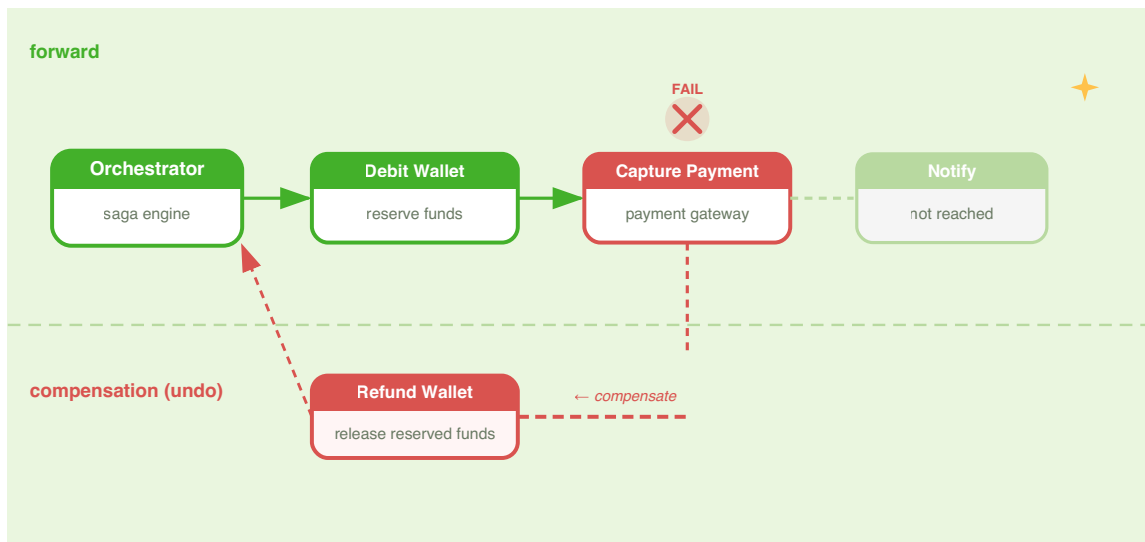


Figure 12.1 — DAG for MoneyTransferSaga: steps run in topological-layer order; independent steps in a layer run with `asyncio.gather`.

```

Layer 0: debit-source
        |
Layer 1: credit-destination

```

Because `credit-destination` depends on `debit-source`, they run sequentially. A more complex saga — a fraud check and a KYC check that are independent of each other but both feed a capture step — would place the two checks in the same layer and run them concurrently with `asyncio.gather`.

Executing the saga

Inject `SagaEngine` from the DI container and call `execute`:

`lumen/core/services/transfers/transfer_service.py`

```
from __future__ import annotations

from typing import Any

from lumen.core.services.transfers.money_transfer_saga import MONEY_TRANSFER_SAGA
from lumen.core.services.transfers.transfer_request import TransferRequest
from pyfly.container import service
from pyfly.transactional.saga.core.result import SagaResult
from pyfly.transactional.saga.engine.saga_engine import SagaEngine

@service
class TransferService:
    """Run the money-transfer saga and report the outcome."""

    def __init__(self, saga_engine: SagaEngine) -> None:
        self._saga_engine = saga_engine

    async def transfer(self, request: TransferRequest) -> dict[str, Any]:
        result: SagaResult = await self._saga_engine.execute(
            saga_name=MONEY_TRANSFER_SAGA,
            input_data=request,
        )

        if result.success:
```

```

        debit = result.result_of("debit-source")
        return {
            "status": "completed",
            "correlation_id": result.correlation_id,
            "source_balance": debit.balance,
            "destination_balance": result.result_of("credit-destination"),
        }

    return {
        "status": "failed",
        "correlation_id": result.correlation_id,
        "failed_steps": list(result.failed_steps().keys()),
        "compensated_steps": list(result.compensated_steps().keys()),
        "error": str(result.error),
    }

```

Listing 12.3 — Executing the money transfer saga

How it works: `saga_engine.execute()` resolves `MoneyTransferSaga` from the registry by name, creates a `SagaContext` with an auto-generated UUID `correlation_id`, and starts executing layers. On success, `SagaResult.success` is `True` and `result_of("debit-source")` returns the `DebitResult` the forward step produced. On failure, `result.failed_steps()` returns a dict of step ID to `StepOutcome` for every step that exhausted its retries; `result.compensated_steps()` returns the steps that were successfully rolled back.

`SagaResult` is an immutable frozen dataclass. Its key members:

- `result.success` — `True` when every forward step completed.
- `result.result_of(step_id)` — the value that step returned, or `None`.
- `result.failed_steps()` — dict of step ID → `StepOutcome` for failed steps.
- `result.compensated_steps()` — dict of step ID → `StepOutcome` for compensated steps.

- `result.correlation_id` — UUID to correlate logs and traces across services.
- `result.error` — the exception that stopped the saga, or `None` on success.

🌿 SPRING PARITY

`@saga / @saga_step` mirror `@Saga / @SagaStep` in the Java `fireflyframework-transactional-engine` library. The decorator-stack rule (`@saga on @service`) mirrors the Java rule that `@Saga` must be on a `@Service`-annotated class so the `WorkflowBeanPostProcessor` can discover it. The parameter-injection markers (`Input()` , `FromStep("id")`) map directly to `@Input` and `@FromStep` in the Java version. The async model differs: Java uses Project Reactor (`Mono<T>`) while PyFly uses native `async/await` with `asyncio.gather` for parallel layers.

Compensation in depth

The happy path is straightforward: every step succeeds and the saga commits. The real design challenge is the unhappy path. Understanding what happens on failure — and why compensation must be designed carefully — is what separates a reliable saga from a brittle one.

What runs on failure

When a step fails after all retries, the engine enters *compensation mode*. It inspects `SagaContext` for every step whose status is `DONE` , then calls their compensation methods in reverse completion order under the default `STRICT_SEQUENTIAL` policy. In `MoneyTransferSaga` , a missing destination wallet causes `credit-destination` to raise `AggregateNotFound` . The engine then compensates the step that already completed:

```
Forward path: debit-source ✓ → credit-destination ✗
Compensation: recredit_source (for debit-source)
```

The net effect: the source wallet is restored to its original balance and the destination wallet was never touched — as if the transfer never happened.

Compensation methods receive their arguments through the same injection system as forward steps. `Annotated[DebitResult, FromStep("debit-source")]` reads the `DebitResult` that `debit_source` stored in the context when it completed — so you always compensate with the actual committed data, never an approximation.

Compensation policies

Five policies govern how the engine runs compensations. Set the global default in YAML or override it per execution:

```
pyfly:
  transactional:
    saga:
      compensation_policy: STRICT_SEQUENTIAL
```

Policy	Behaviour	Use when
<code>STRICT_SEQUENTIAL</code>	Reverse order, one at a time. Stops on first compensation error.	Ordering matters; partial rollback is unacceptable.
<code>GROUPED_PARALLEL</code>	Reverses topology layers; compensates each layer in parallel.	You want speed without violating the dependency structure.
<code>RETRY_WITH_BACKOFF</code>	Reverse order with exponential backoff. Continues if retries succeed.	Transient network failures are likely during compensation.
<code>CIRCUIT_BREAKER</code>	Tracks consecutive failures; opens after 3 and skips the rest.	Avoid cascading failures; manual recovery handles skipped steps.
<code>BEST_EFFORT_PARALLEL</code>	All compensations simultaneously; errors logged, never raised.	Speed critical; separate reconciliation handles partial failures.

⚠ COMPENSATION MUST BE IDEMPOTENT

The engine may call a compensation method more than once. If the engine crashes between calling `void_payment` and persisting the compensation result, it will call `void_payment` again on restart. Your compensation methods must be safe to call multiple times with the same arguments. For payment voids, this means the `PaymentsService` should treat a double-void as a no-op (return success if already voided, do not raise). Design compensation *before* designing the forward step — idempotency is not an afterthought.

Per-step compensation configuration

You can override retry count and timeout for a compensation step without changing forward-step behaviour:

lumen/transfer/transfer_saga_hardened.py

```

from pyfly.container import service
from pyfly.transactional.saga.annotations import saga, saga_step

@saga(name="money-transfer-hardened")
@service
class HardenedTransferSaga:

    @saga_step(
        id="debit-wallet",
        compensate="refund_wallet",
        depends_on=[],
        retry=3,
        backoff_ms=200,
        timeout_ms=5_000,
        compensation_retry=5,
        compensation_backoff_ms=1_000,
        compensation_timeout_ms=8_000,
        compensation_critical=True,
    )
    async def debit_wallet(self, *args: object) -> None: ...

    async def refund_wallet(self, *args: object) -> None: ...

```

Listing 12.4 — Per-step compensation retry and timeout

How it works: `compensation_retry=5` gives the compensation five attempts of its own, independent of the three forward-step retries. `compensation_critical=True` means that if the compensation exhausts all its retries and still fails, the engine raises that exception — surfacing the *compensation failure* as an observable error rather than silently swallowing it.

External compensation steps

When compensation logic is complex enough to warrant its own class, or when it lives in a different module, move it out entirely:

`lumen/transfer/compensation_steps.py`

```
from typing import Annotated

from pyfly.container import service
from pyfly.transactional.saga.annotations import (
    FromStep,
    compensation_step,
)

from lumen.core.services.transfers.money_transfer_saga import DebitResult
from lumen.models.repositories.wallet_repository import WalletRepository

@compensation_step(saga="money-transfer", for_step_id="debit-source")
@service
class SourceRecreditCompensation:

    def __init__(self, repository: WalletRepository) -> None:
        self._repository = repository

    async def execute(
        self,
        debit: Annotated[DebitResult, FromStep("debit-source")],
    ) -> None:
        """External alternative to the inline recredit_source method."""
        ...
```

Listing 12.5 — External compensation step class

The `SagaRegistry` discovers `@compensation_step` classes at startup alongside `@saga` classes and wires them into their step definitions automatically. The `for_step_id` parameter must match the step's `id` string exactly.

Workflows and signals

`@saga` is the right tool when all steps are known upfront and the operation completes within minutes. Some business processes are inherently longer — a loan approval waiting on a compliance officer, a multi-step onboarding blocked on an email click, a payment that needs a cooldown before settling. These fit the **Workflow** pattern.

How workflows differ from sagas

	Saga	Workflow
Duration	Seconds to minutes	Minutes to days
Waiting	Retries only	Signals, timers, child workflows
Human-in-the-loop	No	Yes (<code>@wait_for_signal</code>)
State persistence	Per-saga checkpoint	After every layer
DAG fan-out	Yes (parallel layers)	Yes + gate primitives

Declaring a workflow

`lumen/transfer/approval_workflow.py`

```
from __future__ import annotations

from pyfly.container import service
from pyfly.transactional.core.model import TriggerMode
```

```

from pyfly.transactional.workflow.annotations import (
    compensation_step,
    on_workflow_complete,
    on_workflow_error,
    wait_for_signal,
    workflow,
    workflow_query,
    workflow_step,
)

@workflow(
    id="large-transfer-approval",
    trigger_mode=TriggerMode.SYNC,
    timeout_ms=86_400_000,    # 24 hours
    max_retries=1,
)

@service
class LargeTransferWorkflow:
    """High-value transfers require a compliance officer to approve."""

    @workflow_step(id="enrich-request", depends_on=[])
    async def enrich_request(self, payload: dict) -> dict:
        return {**payload, "risk_score": 0.12}

    @workflow_step(
        id="compliance-review",
        depends_on=["enrich-request"],
        compensatable=True,
        compensation_method="release_review",
        timeout_ms=82_800_000,
    )
    @wait_for_signal("approved", timeout_ms=82_800_000)
    async def compliance_review(self) -> None:
        """Suspends until a compliance officer delivers the signal."""

    @compensation_step(for_step="compliance-review")
    async def release_review(self) -> None:
        """Called if the workflow is cancelled during review."""

```

```

@workflow_step(
    id="settle-transfer",
    depends_on=["compliance-review"],
)
async def settle_transfer(self, payload: dict) -> dict:
    return {"settled": True}

@workflow_query(name="status")
async def get_status(self, ctx: object) -> str:
    return str(getattr(ctx, "status", "UNKNOWN"))

@on_workflow_complete
async def on_done(self, ctx: object) -> None:
    pass    # emit audit event

@on_workflow_error
async def on_error(self, ctx: object, err: Exception) -> None:
    pass    # alert on-call

```

Listing 12.6 — *LargeTransferWorkflow: signal-driven approval for high-value transfers*

How it works: The decorator stack follows the same rule as sagas: `@workflow` on top of `@service`. `@workflow(id=...)` takes keyword-only arguments — `id` is required; all others are optional. `@wait_for_signal("approved", timeout_ms=82_800_000)` stacks on top of `@workflow_step` and tells the engine to suspend at that step until a signal named `"approved"` is delivered. The engine persists the `ExecutionContext` to the configured `ExecutionPersistenceProvider`; if the process restarts, it re-hydrates the context and resumes from the last completed layer.

`@compensation_step(for_step="compliance-review")` uses the keyword argument `for_step` (not positional) and registers `release_review` as the compensation handler for the `compliance-review` step.

`@workflow_query(name="status")` marks a method as a read-side query handler — callable while the workflow is suspended without advancing execution.

Driving the workflow engine

`lumen/transfer/approval_controller.py`

```
from __future__ import annotations

from pyfly.container import service
from pyfly.transactional.workflow.engine import WorkflowEngine
from pyfly.transactional.workflow.result import WorkflowResult

@service
class TransferApprovalService:

    def __init__(self, workflow_engine: WorkflowEngine) -> None:
        self._wf = workflow_engine

    async def request_large_transfer(self, payload: dict) -> str:
        result: WorkflowResult = await self._wf.start(
            "large-transfer-approval",
            input=payload,
        )
        # Returns immediately; workflow is now suspended at compliance-review.
        return result.correlation_id

    async def approve(self, correlation_id: str, reviewer_id: str) -> None:
        await self._wf.deliver_signal(
            correlation_id,
            "approved",
            payload={"by": reviewer_id},
        )

    async def check_status(self, correlation_id: str) -> str:
        return await self._wf.query(correlation_id, "status")
```

Listing 12.7 — Starting a workflow and delivering a signal

How it works: `workflow_engine.start(workflow_id, input=payload)` runs the first layer (`enrich-request`) synchronously, then suspends at `compliance-review` because of `@wait_for_signal`. It returns a `WorkflowResult` immediately with a `correlation_id` — the caller stores this ID and polls later. When `deliver_signal()` is called, the workflow resumes and `settle-transfer` runs to completion.

`WorkflowResult` carries: `workflow_id`, `correlation_id`, `status` (an `ExecutionStatus` enum), `duration_ms`, `step_results` (dict), and `variables`. The boolean `result.successful` is `True` when `status` is `COMPLETED` or `CONFIRMED`.

The programmatic builder

When you need to construct a workflow dynamically — from a database configuration or a rules engine — use `WorkflowBuilder`:

`lumen/transfer/dynamic_workflow.py`

```
from pyfly.transactional.workflow.builder import WorkflowBuilder
from pyfly.transactional.workflow.definition import WorkflowDefinition

async def enrich_fn(payload: dict) -> dict:
    return {**payload, "enriched": True}

async def settle_fn(payload: dict) -> dict:
    return {"settled": True}

definition: WorkflowDefinition = (
    WorkflowBuilder("simple-transfer")
    .step("enrich", enrich_fn, depends_on=[])
    .wait_signal(
        "await-approval",
```

```

        "approved",
        depends_on=["enrich"],
        timeout_ms=3_600_000,
    )
    .step(
        "settle",
        settle_fn,
        depends_on=["await-approval"],
    )
    .build()
)

```

Listing 12.8 — Building a workflow programmatically

`WorkflowBuilder.step(step_id, handler, *, depends_on, timeout_ms, max_retries, ...)` accepts a callable and keyword arguments for dependencies, timeouts, and retries. `wait_signal(step_id, signal, *, depends_on, timeout_ms)` inserts a signal-gate step without a real handler — it creates an internal no-op coroutine that the engine replaces with signal-wait logic. `build()` returns a `WorkflowDefinition` you register directly with `WorkflowEngine`.

TCC: Try-Confirm-Cancel

The saga pattern runs steps forward and compensates backwards. **TCC (Try-Confirm-Cancel)** takes a different approach: all participants first *tentatively reserve* their resources without committing (Try), then either all *commit* those reservations (Confirm) or all *release* them (Cancel). This gives you strong all-or-nothing semantics across participants without a distributed lock.

TCC suits scenarios where each participant can cheaply hold a reservation — for example, pre-authorising a payment card hold rather than immediately charging it.

The three phases

1. **Try** — every participant reserves resources. Reservations are visible internally but not final. If any Try fails, participants that succeeded cancel their reservations.
2. **Confirm** — if all Try phases succeed, the coordinator instructs every participant to commit its reservation.
3. **Cancel** — if any Try phase fails, the coordinator instructs every participant that completed Try to release its reservation.

Declaring a TCC transaction

`lumen/transfer/transfer_tcc.py`

```
from __future__ import annotations

from typing import Annotated

from pyfly.container import service
from pyfly.transactional.tcc.annotations import (
    FromTry,
    cancel_method,
    confirm_method,
    tcc,
    tcc_participant,
    try_method,
)
from pyfly.transactional.tcc.core.context import TccContext

from lumen.wallet.service import WalletService
from lumen.payments.service import PaymentsService

@tcc(
    name="wallet-transfer",
    timeout_ms=30_000,
    retry_enabled=True,
    max_retries=3,
```



```

        backoff_ms=500,
    )
    @service
    class WalletTransferTcc:
        """Reserve funds and payment in lockstep; confirm or cancel together."""

        @tcc_participant(id="wallet-hold", order=1, timeout_ms=5_000)
        class WalletParticipant:

            def __init__(self, wallet_svc: WalletService) -> None:
                self._wallet = wallet_svc

            @try_method(timeout_ms=4_000, retry=2, backoff_ms=200)
            async def try_hold(
                self,
                request: object,
                ctx: TccContext,
            ) -> str:
                """Tentatively hold funds – does not debit yet."""
                return await self._wallet.hold_funds(
                    wallet_id=getattr(request, "sender_id", ""),
                    amount=getattr(request, "amount_cents", 0),
                ) # returns a hold_id

            @confirm_method(timeout_ms=5_000, retry=3)
            async def confirm_hold(
                self,
                hold_id: Annotated[str, FromTry()],
                ctx: TccContext,
            ) -> None:
                await self._wallet.commit_hold(hold_id)

            @cancel_method(timeout_ms=3_000, retry=2)
            async def cancel_hold(
                self,
                hold_id: Annotated[str, FromTry()],
            ) -> None:
                await self._wallet.release_hold(hold_id)

```

```

@tcc_participant(id="payment-auth", order=2, timeout_ms=8_000)
class PaymentParticipant:

    def __init__(self, payments_svc: PaymentsService) -> None:
        self._payments = payments_svc

    @try_method(timeout_ms=6_000, retry=2, backoff_ms=300)
    async def try_auth(
        self,
        request: object,
        ctx: TccContext,
    ) -> str:
        return await self._payments.pre_authorise(
            amount=getattr(request, "amount_cents", 0),
        ) # returns auth_id

    @confirm_method(timeout_ms=8_000, retry=3)
    async def confirm_auth(
        self,
        auth_id: Annotated[str, FromTry()],
        ctx: TccContext,
    ) -> None:
        await self._payments.capture_auth(auth_id)

    @cancel_method(timeout_ms=4_000, retry=2)
    async def cancel_auth(
        self,
        auth_id: Annotated[str, FromTry()],
    ) -> None:
        await self._payments.void_auth(auth_id)

```

Listing 12.9 — *WalletTransferTcc: Try-Confirm-Cancel for payment reservation*

How it works: `@tcc_participant(order=1)` tells the TCC engine to run `WalletParticipant`'s Try phase before `PaymentParticipant`'s — lower `order` means earlier. `FromTry()` is TCC's equivalent of `FromStep`: it injects the value returned by the same participant's `@try_method` into its `@confirm_method` and `@cancel_method`.

The engine runs all participants' Try phases in `order` sequence. If every Try succeeds, it runs all Confirm methods. If any Try fails, it runs Cancel for every participant that completed its Try — again in declared order. An `optional=True` participant that fails its Try does not trigger a global Cancel; its failure is logged and skipped.

Executing a TCC transaction

`lumen/transfer/tcc_service.py`

```
from __future__ import annotations

from typing import Any

from pyfly.container import service
from pyfly.transactional.tcc.core.result import TccResult
from pyfly.transactional.tcc.engine.tcc_engine import TccEngine

from lumen.core.services.transfers.transfer_request import TransferRequest

@service
class TccTransferService:

    def __init__(self, tcc_engine: TccEngine) -> None:
        self._engine = tcc_engine

    async def transfer(self, req: TransferRequest) -> dict[str, Any]:
        result: TccResult = await self._engine.execute(
            tcc_name="wallet-transfer",
            input_data=req,
```

```

)

if result.success:
    hold_id = result.result_of("wallet-hold")
    return {
        "status": "confirmed",
        "hold_id": hold_id,
        "correlation_id": result.correlation_id,
    }

failed = result.failed_participants()
return {
    "status": "cancelled",
    "failed": list(failed.keys()),
    "error": str(result.error),
}

```

Listing 12.10 — Executing a TCC transaction

TCC vs Saga: choosing the right pattern

Use this table to choose between the two approaches:

Question	Saga	TCC
Steps run independently?	Yes — each commits locally	No — all Try phases must succeed first
Needs compensation logic?	Yes, per step	No — Cancel handles rollback
Resource reservation needed?	No	Yes — participants hold resources during Try
Best for	Long sequential operations	Short all-or-nothing locks

Persistence: surviving a crash

The engine stores saga and TCC state through the `TransactionalPersistencePort` protocol. The default adapter keeps state in memory — fast for development, but lost on process restart. Production deployments swap in a durable adapter.

How state flows

Every time a step completes — successfully or otherwise — the engine calls:

1. `persistence_port.update_step_status(correlation_id, step_id, status)` — record the step outcome.
2. `persistence_port.mark_completed(correlation_id, successful)` — record the saga's final result.

On startup, `SagaRecoveryService` queries `persistence_port.get_stale(before)` to find executions that started but never completed. For each stale saga still in `IN_FLIGHT` status, it marks the saga `FAILED` and emits lifecycle events so observability systems can alert on-call.

Configuration

```
pyfly:
  transactional:
    saga:
      persistence_enabled: true
      recovery_enabled: true
      recovery_interval_seconds: 60
      stale_threshold_seconds: 600
      cleanup_older_than_hours: 24
```

With `recovery_enabled: true`, the framework runs `SagaRecoveryService.recover_stale()` on a background task every `recovery_interval_seconds` seconds. Sagas last updated more than `stale_threshold_seconds` seconds ago are considered stuck, marked failed, and surfaced for manual investigation or automatic retry.

Implementing a custom persistence adapter

To persist to a real database, implement `TransactionalPersistencePort` and register your implementation as a `@bean` or `@component`. The auto-configuration detects your bean at startup and uses it in preference to `InMemoryPersistenceAdapter`:

```
lumen/infra/persistence/saga_postgres_adapter.py
```

```
from __future__ import annotations

from datetime import datetime
from typing import Any

from pyfly.container import component
from pyfly.transactional.shared.ports.outbound import (
    TransactionalPersistencePort,
)

@Component
class SagaPostgresAdapter(TransactionalPersistencePort):

    async def persist_state(self, state: dict[str, Any]) -> None:
        # INSERT INTO saga_executions ...
        ...

    async def get_state(
        self, correlation_id: str
    ) -> dict[str, Any] | None:
        # SELECT * FROM saga_executions WHERE ...
```

```

...

async def update_step_status(
    self,
    correlation_id: str,
    step_id: str,
    status: str,
) -> None: ...

async def mark_completed(
    self, correlation_id: str, successful: bool
) -> None: ...

async def get_in_flight(self) -> list[dict[str, Any]]:
    return []

async def get_stale(
    self, before: datetime
) -> list[dict[str, Any]]:
    return []

async def cleanup(self, older_than: datetime) -> int:
    return 0

async def is_healthy(self) -> bool:
    return True

```

Listing 12.11 — Skeleton of a PostgreSQL persistence adapter

💡 USE SAGARECOVERYSERVICE IN INTEGRATION TESTS

In tests that simulate a crash, create a `SagaRecoveryService` with an `InMemoryPersistenceAdapter`, run a saga to a midpoint, manually mark it stale, then call `await recovery.recover_stale(stale_threshold_seconds=0)`. Assert that `SagaResult.success` is `False` and the right steps are marked as failed. This gives you confidence in your recovery logic without spinning up a real database.

The programmatic saga builder

When you need to build a saga from dynamic configuration — loading step definitions from a rules database or a configuration file — `SagaBuilder` gives you the full fluent API without any decorators:

`lumen/transfer/dynamic_saga.py`

```
from __future__ import annotations

from pyfly.transactional.saga.registry.saga_builder import SagaBuilder
from pyfly.transactional.saga.core.result import SagaResult


async def debit_fn(req: object, ctx: object) -> str:
    return "debit-ref-001"


async def capture_fn(req: object, ctx: object) -> str:
    return "txn-001"


async def refund_fn(result: str) -> None:
    pass    # undo debit


saga_def = (
    SagaBuilder("dynamic-transfer")
    .step("debit")
    .handler(debit_fn)
    .compensate(refund_fn)
    .retry(3)
    .backoff_ms(200)
    .timeout_ms(5_000)
    .jitter(enabled=True, factor=0.3)
    .add()
    .step("capture")
)
```



```

        .handler(capture_fn)
        .depends_on("debit")
        .retry(2)
        .backoff_ms(500)
        .add()
    .layer_concurrency(5)
    .build()
)

```

Listing 12.12 — Building a saga programmatically with `SagaBuilder`

How it works: Each `.step(step_id)` call returns a `StepBuilder`. Chain configuration methods —

`.handler()`, `.compensate()`, `.depends_on()`, `.retry()`, `.backoff_ms()`, `.timeout_ms()`, `.j` — then call `.add()` to finalise the step and return the parent `SagaBuilder`. `.build()` runs the same DAG validation as the decorator path: missing handlers, nonexistent `depends_on` references, and cycles all raise `SagaValidationError` immediately at registration time.

✓ What you built

You began with a concrete problem: a wallet transfer across two independent aggregates cannot use a single database transaction. You declared `MoneyTransferSaga` by stacking `@saga` on `@service`, causing the `OrchestrationBeanPostProcessor` to register it into the auto-configured `SagaEngine` at startup. Each step uses `Annotated[T, Input()]` and `Annotated[T, FromStep("step-id")]` marker instances — not bare classes — for parameter injection; `ctx: SagaContext` is injected by type. The compensation method `recredit_source` does not receive the saga input; it pulls the forward step's

`DebitResult` via `FromStep("debit-source")`. When `credit-destination` raises `AggregateNotFound`, the engine automatically runs `recredit_source`, leaving both balances unchanged.

You explored compensation in depth: five policies ranging from strict sequential to best-effort parallel, per-step compensation retries and timeouts, and the non-negotiable requirement that all compensations be idempotent. You saw how `@workflow(id=...)` `@service` and `@wait_for_signal` suspend a long-running workflow until a human delivers a signal, and how `WorkflowResult.successful` reports the final state. You walked through TCC as a reservation-based alternative that locks resources across all participants before committing any. Finally you wired a custom `TransactionalPersistencePort` and configured `SagaRecoveryService` to detect and surface stale executions after a crash.

Key concepts to carry forward:

- **@saga on @service** — the decorator stack that makes a class both a DI bean and a registered saga; `@saga` alone is not enough.
- **Marker instances** — `Input()`, `FromStep("step-id")` must be instances (with parentheses), not bare classes.
- **Compensation via FromStep** — compensation methods receive the forward step's result, never the saga input.
- **SagaEngine.execute(saga_name, input_data)** — the single call that returns `SagaResult` with `.success`, `.result_of()`, `.failed_steps()`, `.compensated_steps()`.
- **@workflow(id=...) @service / @wait_for_signal** — signal-driven, long-running alternative; check `WorkflowResult.successful` for the final state.
- **@tcc on @service / @tcc_participant** — reservation-based coordination; `FromTry()` (instance) injects the try-result into confirm and cancel methods.

- **TransactionalPersistencePort** — implement and register this protocol to give the engine durable state and crash recovery.

Try it yourself

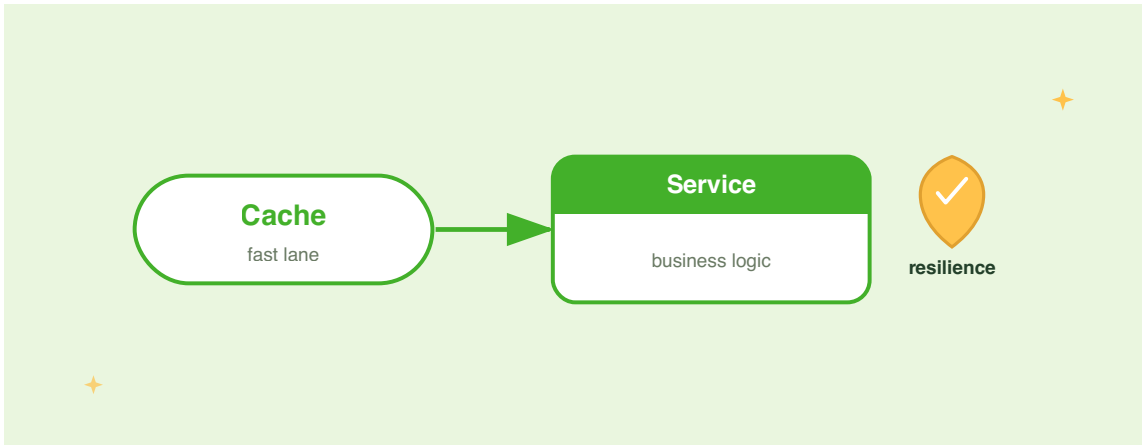
Exercise 1 — Parallel balance validation. Add a `validate-source` step to `MoneyTransferSaga` that checks the source wallet has sufficient funds, without performing the debit. The step should run *in parallel* with nothing (no `depends_on`), and `debit-source` should depend on it. Extend `credit-destination` to depend on `debit-source` as before. Verify the topology via `SagaRegistry.get("money-transfer")` in a test and assert that `definition.steps["debit-source"].depends_on == ["validate-source"]`.

Exercise 2 — Compensation error handler. Change `MoneyTransferSaga`'s global compensation policy to `RETRY_WITH_BACKOFF` in `application.yaml`. Then deliberately make `recredit_source` raise `RuntimeError` on the first call and succeed on the second. Write a pytest test using `AsyncMock` on `WalletRepository` that verifies the saga eventually compensates successfully and `result.compensated_steps()` contains `"debit-source"`.

Exercise 3 — Custom persistence. Implement `TransactionalPersistencePort` backed by a plain Python `dict` that logs every call. Register it as a `@service` and write a test that runs `TransferService`, then calls `get_state(correlation_id)` on your adapter and asserts the recorded `status` is `"COMPLETED"`. Extend the test to simulate a stale saga by manually setting `status = "IN_FLIGHT"` and a past `started_at`, then assert `SagaRecoveryService.recover_stale(stale_threshold_seconds=0)` returns `1`.

CHAPTER 13

Caching & Resilience



In Chapter 11 you split Lumen into separate services and taught its wallet handler to call a downstream `AccountService` over HTTP. In Chapter 12 you added a `DepositSaga` to coordinate multi-step operations across service boundaries, with compensating transactions ready to fire when any step goes wrong.

Those two chapters introduced a new class of problem: latency and failure propagation. Every HTTP hop to `AccountService` is a round trip that could be slow on a busy network, and every call to Lumen's own database competes with concurrent saga participants. In a distributed system, failures are not exceptional events — they are scheduled maintenance. `AccountService` will be upgraded mid-traffic. Redis will hiccup. A payment gateway will spike to three-second response times during peak settlement.

Without protection, Lumen propagates those failures upstream. A slow `AccountService` ties up coroutines, blocking wallet reads for unrelated users. A brief Redis outage wipes cached balances and sends every request straight to the database, multiplying load at exactly the wrong moment.

This chapter makes Lumen **fast** and **fault-tolerant**. The first half covers PyFly's declarative caching layer — `@cacheable`, `@cache_put`, and `@cache_evict` — and shows how to back them with an in-process `InMemoryCache` for development and a shared `RedisCacheAdapter` with automatic failover for production. The second half layers in the resilience toolkit: a token-bucket **rate limiter** that caps inbound traffic, a semaphore **bulkhead** that isolates concurrency, a **time limiter** that cancels hanging coroutines, a **fallback** that degrades gracefully, and **retry** and **circuit-breaker** patterns that protect outbound calls. A closing section shows how to stack all of them in the right order.

By the end of the chapter, every hot path in Lumen will be cached and every outbound dependency wrapped in a resilience fence.

Caching the read path

Why cache wallet reads?

Lumen's most frequent operation is the balance query: "what is wallet `w-001`'s current balance?" Under normal load that query hits the read replica. Under heavy load it competes with deposit commands, saga participants, and snapshot writes. A cached balance costs one Redis lookup — one co-located network round trip — compared with a full SQL query that the read replica must also parse, plan, and execute.

The economics are compelling, but caching introduces a correctness concern: the cached balance may lag the committed balance by up to the TTL. For Lumen, a five-second stale balance is an acceptable trade-off for normal query traffic. When a deposit completes, the handler invalidates the cache entry immediately, so the next balance read reflects the change. Updates that go through the saga use `@cache_put` to refresh the cached value as a side-effect of the write, eliminating any visible staleness window.

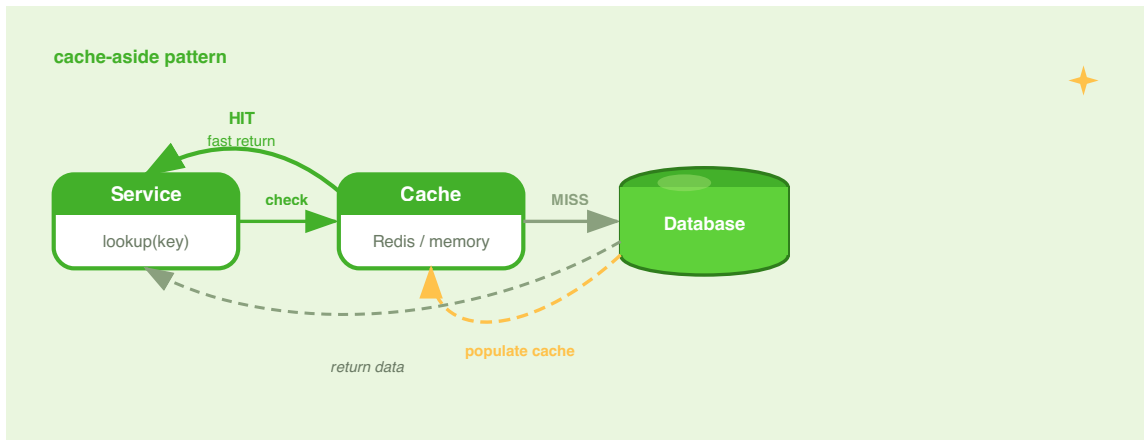


Figure 13.1 — Cache decorators sit in front of the service layer. On a hit the function body never executes; on a miss it runs and the result is stored.

The cache abstraction

PyFly's cache layer follows the hexagonal principle you have seen throughout the book: business logic depends on a `CacheAdapter` protocol, not on any specific backend.

Concrete implementations — `InMemoryCache` for development and `RedisCacheAdapter` for production — are wired in through the DI container. Swapping backends requires no changes to business logic.

The `CacheAdapter` protocol defines the full contract:

Method	Returns	Description
<code>get(key)</code>	<code>Any</code> <code>None</code>	Return the cached value, or <code>None</code> if absent or expired.
<code>put(key, value, ttl=None)</code>	<code>None</code>	Store a value; <code>ttl</code> is a <code>timedelta</code> or <code>None</code> for no expiry.
<code>evict(key)</code>	<code>bool</code>	Remove one key; returns <code>True</code> if it existed.
<code>exists(key)</code>	<code>bool</code>	Check presence without fetching the value.
<code>clear()</code>	<code>None</code>	Flush the entire cache.
<code>start()</code>	<code>None</code>	Called once at application startup.
<code>stop()</code>	<code>None</code>	Called once at application shutdown.

Both `InMemoryCache` and `RedisCacheAdapter` implement this contract.

`InMemoryCache` stores entries in an `OrderedDict` with lazy TTL expiry and optional LRU bounding; it is ideal for single-process development and test suites because it has no external dependencies. `RedisCacheAdapter` wraps a `redis.asyncio.Redis` client, serialises values to JSON before storage, and delegates TTL management to Redis itself — expired keys disappear server-side with zero cleanup overhead on your side.

Setting up a cache backend

For development, a single import is all you need:

```
lumen/cache/config_dev.py
```

```
from pyfly.cache.adapters.memory import InMemoryCache

wallet_cache = InMemoryCache(max_size=1000)
```

Listing 13.1 — `InMemoryCache` for development

`max_size=1000` bounds the LRU eviction window: once the cache holds 1,000 entries, the least-recently-used entry is dropped to make room. Pass `None` (the default) to leave the cache unbounded and rely entirely on TTLs.

For production, point `RedisCacheAdapter` at a `redis.asyncio.Redis` client:

`lumen/cache/config_prod.py`

```
import redis.asyncio as aioredis

from pyfly.cache import CacheAdapter, CacheManager
from pyfly.cache.adapters.memory import InMemoryCache
from pyfly.cache.adapters.redis import RedisCacheAdapter
from pyfly.container import bean, configuration

@configuration
class CacheConfig:

    @bean
    def wallet_cache(self) -> CacheAdapter:
        client = aioredis.from_url("redis://localhost:6379/0")
        primary = RedisCacheAdapter(client)
        fallback = InMemoryCache(max_size=500)
        return CacheManager(primary=primary, fallback=fallback)
```

Listing 13.2 — `RedisCacheAdapter` for production

How it works: `CacheManager` wraps a primary Redis backend and an in-memory fallback. Every write goes to both caches, keeping the fallback warm. On reads, the manager tries Redis first; if Redis raises an exception it logs a `WARNING` and falls back to the in-process store silently. When Redis recovers, new writes immediately repopulate it — no manual intervention required. The `@bean` method tells PyFly's DI container to create a singleton and inject it wherever `CacheAdapter` is declared as a dependency.

 AUTO-CONFIGURATION

Add `pyfly.cache.enabled: true` and `pyfly.cache.provider: redis` to `pyfly.yaml` and PyFly will wire `RedisCacheAdapter` + `InMemoryCache` into a `CacheManager` automatically — no `@configuration` class needed.

@cacheable — skip execution on a hit

`@cacheable` is the most common decorator. On the first call it executes the function body and stores the return value. On every subsequent call with the same key it returns the stored value *without executing the function body at all*.

Lumen's `GetBalanceHandler` is a natural fit: balance reads are frequent, cheap to cache, and tolerate a few seconds of staleness. The handler receives `CacheAdapter` through its constructor — injected by PyFly — and wraps `do_handle` at construction time:

`lumen/core/services/wallets/get_balance_handler.py`

```
from datetime import timedelta

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cacheable
from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):
    """Return a cached :class:`BalanceDto`; bypass the DB on a hit."""

    def __init__(
```

```

        self,
        repository: WalletRepository,
        cache: CacheAdapter,
    ) -> None:
        super().__init__()
        self._repository = repository
        # Wrap do_handle at construction time so `cache` is in scope.
        self.do_handle = cacheable(
            backend=cache,
            key="wallet:balance:{query.wallet_id}",
            ttl=timedelta(seconds=5),
        )(self._fetch)

    async def _fetch(
        self, query: GetBalance
    ) -> BalanceDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listing 13.3 — @cacheable on GetBalanceHandler

KEY TEMPLATE AND SELF

The `key` template `"wallet:balance:{query.wallet_id}"` uses Python's `str.format` syntax. PyFly binds the actual call arguments with `inspect.signature(func).bind(*args, **kwargs)`, then calls `key.format(**bound.arguments)`. Because `_fetch` is wrapped inside `__init__`, the first positional argument is `query` — so `{query.wallet_id}` expands to the wallet id. Calling with `GetBalance(wallet_id="wlt-001")` produces the cache key `"wallet:balance:wlt-001"`. The mapper function `entity_to_balance_dto` goes through `Mapper.project` against the `@projection`-marked `BalanceView` interface, copying only the fields the balance view declares and computing `balance` from `balance_minor`.

`ttl=timedelta(seconds=5)` means the cache entry expires five seconds after it is written. After expiry, the next call re-executes the function body and refreshes the entry. A TTL of `None` (the default) means the entry never expires — appropriate only for truly immutable data.

Null caching: When the function returns `None`, PyFly still stores the entry and records that the key *exists*. A subsequent call finds the key and returns `None` without touching the database. This prevents cache-penetration attacks where an adversary floods requests for non-existent keys, each of which would otherwise fall through to the database.

condition and unless: Both `@cache` and `@cacheable` accept optional predicates. `condition` is a callable with the same signature as the decorated function; if it returns `False`, caching is bypassed for that call. `unless` is a callable that receives the *result*; if it returns `True`, the result is returned but not stored. Both are keyword-only:

```
cacheable(
    backend=cache,
    key="wallet:balance:{query.wallet_id}",
    ttl=timedelta(seconds=5),
    condition=lambda query: not query.wallet_id.startswith("test-"),
    unless=lambda result: result is None,
)(self._fetch)
```

🍃 SPRING PARITY

`@cacheable` mirrors Spring's `@Cacheable`. The `key` template uses Python's `str.format` syntax instead of SpEL, but the semantics — skip-on-hit, store-on-miss, `condition`, `unless` — are identical. `@cache` is a lower-level alias that behaves the same way; use whichever name reads better in your codebase.

@cache_put — always execute, always store

`@cacheable` is for reads: it short-circuits the function when the cache already holds a value. `@cache_put` is for writes: it *always* executes the function and *always* stores the result. Use it when the function is the source of truth — a command handler that modifies the wallet and must keep the cache current.

`DepositFundsHandler` is the canonical example. After a deposit succeeds, the new balance must be visible to the next read without waiting for the TTL to expire. Wrapping `do_handle` with `@cache_put` refreshes the cache entry atomically with the write:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from datetime import timedelta

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cache_put
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance
    in minor units and refreshes the cached balance entry."""

    def __init__(
```

```

        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
        cache: CacheAdapter,
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory
        # Wrap at construction time so `cache` is in scope.
        self.do_handle = cache_put(
            backend=cache,
            key="wallet:balance:{command.wallet_id}",
            ttl=timedelta(seconds=5),
        )(self._deposit)

    @transactional()
    async def _deposit(self, command: DepositFunds) -> int:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)

        wallet = to_aggregate(entity)
        wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))
        await publish_domain_events(self._events, wallet.clear_events())
        return wallet.balance.amount

```

Listing 13.4 — `@cache_put` refreshes the cache on a deposit

How it works: `@cache_put` awaits the wrapped function, then calls `backend.put(resolved_key, result, ttl=ttl)`. Because the function always runs, the cached value after a `DepositFunds` command is the freshly committed balance — not a stale pre-deposit snapshot. `_deposit` runs inside `@transactional()`, so the

`find_by_id` → `to_aggregate` → `mutate` → `upsert` sequence is committed as one unit of work before the cache is refreshed. The next `@cacheable` read in `GetBalanceHandler` picks up this fresh value without touching the database.

❗ CACHE KEY MUST MATCH

The `@cache_put` key `"wallet:balance:{command.wallet_id}"` must match the `@cacheable` key `"wallet:balance:{query.wallet_id}"` when both resolve to the same wallet id. Mismatched keys mean the deposit writes to a different cache slot than the balance read looks up — staleness returns.

Decorator	Function executes?	On hit
<code>@cacheable</code> / <code>@cache</code>	Only on a miss	Returns cached value
<code>@cache_put</code>	Always	Replaces cached value with fresh result

@cache_evict — remove after deletion

When you close a wallet or roll back a transaction, the associated cache entry must be removed. `@cache_evict` runs the function body first, then removes the named key — or clears the entire cache when `all_entries=True`.

`lumen/core/services/wallets/close_wallet_handler.py`

```
from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.cache import CacheAdapter, cache_evict
from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional

@command_handler
@service
class CloseWalletHandler(CommandHandler["CloseWallet", None]):
```

```

"""Close a wallet and evict its cached balance entry."""

def __init__(
    self,
    repository: WalletRepository,
    cache: CacheAdapter,
) -> None:
    super().__init__()
    self._repository = repository
    self.do_handle = cache_evict(
        backend=cache,
        key="wallet:balance:{command.wallet_id}",
    )(self._close)

@transactional()
async def _close(self, command) -> None:
    entity = await self._repository.find_by_id(command.wallet_id)
    if entity is not None:
        await self._repository.delete(entity)

```

Listing 13.5 — @cache_evict after removing a wallet

To flush every cached balance at once — useful for an administrative reset — pass

`all_entries=True`:

```

self.do_handle = cache_evict(
    backend=cache,
    all_entries=True,
)(self._reset_all)

```

How it works: The function body runs first — `repository.delete(entity)` removes the row before eviction, so a failure does not prematurely drop the cache entry. Then either `backend.evict(resolved_key)` removes one key or `backend.clear()` flushes everything. With `CacheManager`, the evict propagates to both primary and fallback caches so no stale entry lingers in either tier.

`all_entries=True` is a blunt instrument reserved for administrative resets. In normal operation, prefer targeted eviction by key.

Invalidation strategy

A coherent strategy matches each operation to the right decorator:

Operation	Decorator	Rationale
<code>GetBalance</code> query	<code>@cacheable</code>	Skip DB on hit; 5 s TTL bounds staleness
<code>DepositFunds</code> command	<code>@cache_put</code>	Refresh the cache entry atomically with the write
<code>WithdrawFunds</code> command	<code>@cache_put</code>	Same — keep the post-withdrawal balance warm
Close wallet	<code>@cache_evict</code>	Remove the entry; the next read rebuilds it from DB
Admin truncate	<code>@cache_evict(all_entries=True)</code>	Bulk reset; full cache flush is correct

⚠️ ASYNC REQUIREMENT

All three decorators require the wrapped function to be declared `async`. Cache adapters are fully async (they `await` backend operations), so a synchronous target will fail with a `TypeError` at decoration time — PyFly raises the error immediately so you catch the mistake at startup rather than at runtime.

Resilience patterns

Why protection matters

Caching makes the happy path fast. Resilience patterns protect Lumen when the happy path is unavailable. Without protection, a slow `AccountService` triggers a cascade:

1. Requests from wallet handlers pile up, each waiting on an HTTP response.
2. Lumen's asyncio event loop — single-threaded by default — processes pending tasks in order; a backlog of slow HTTP calls delays every other operation.
3. Memory and open file-descriptors climb as coroutines stack up.
4. Lumen becomes unavailable to requests that have nothing to do with `AccountService`.

Four complementary patterns break this cascade before it starts:

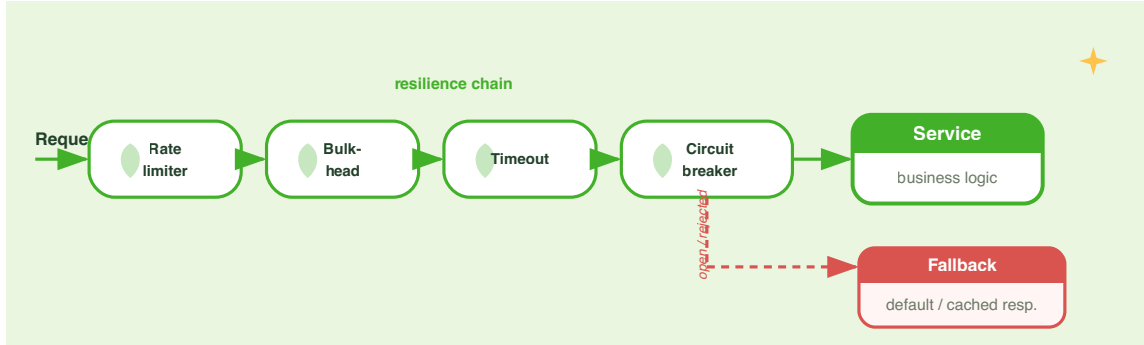


Figure 13.2 — Four resilience layers guard the outbound call. Rate limiter drops excess traffic before it enters the system; bulkhead limits concurrency; timeout cancels slow operations; fallback provides a safe response when all else fails.

Pattern	Protects against	Fail-fast or wait?
Rate limiter	Traffic spikes overwhelming the downstream	Fail-fast (reject excess)
Bulkhead	Too many concurrent calls tying up resources	Fail-fast (reject over limit)
Time limiter	Hanging calls that never return	Cancel after timeout
Fallback	Any failure reaching the caller	Returns degraded value

All four are in `pyfly.resilience`:

```
from pyfly.resilience import (
    RateLimiter, rate_limiter,
    Bulkhead, bulkhead,
    time_limiter,
    fallback,
)
```

Rate limiter — token bucket

`RateLimiter` uses a **token bucket**: the bucket holds up to `max_tokens` tokens and refills at `refill_rate` tokens per second. Each call consumes one token. When the bucket is empty, `RateLimitException` is raised immediately — no queuing, no waiting.

`lumen/resilience/rate_example.py`

```
from pyfly.resilience import RateLimiter, rate_limiter

# Sustained: 20 calls/s; burst: up to 40
account_limiter = RateLimiter(max_tokens=40, refill_rate=20.0)

@rate_limiter(account_limiter)
async def fetch_account(account_id: str) -> dict:
    # This body is reached only when a token is available.
    ...
```

Listing 13.6 — Token-bucket rate limiter on account lookups

How it works: `@rate_limiter(limiter)` calls `await limiter.acquire()` before every invocation. `acquire()` refills the bucket based on elapsed wall-clock time (using `time.monotonic()`), then atomically checks and decrements the token count under a `threading.Lock` — not an asyncio lock — so that both async tasks and sync callers share the same count without races. If fewer than 1.0 tokens remain, `RateLimitException` propagates to the caller.

The token-bucket shape allows controlled bursting: a service that typically sees 10 calls per second can absorb a burst of 40 calls immediately (drawing on saved tokens), then sustains 20 calls per second afterwards. Fixed-window rate limiters cannot express this nuance.

Multiple functions sharing one `RateLimiter` instance enforce a *global* rate across all of them — useful for capping total traffic to a downstream service regardless of which internal method initiates the call.

Bulkhead — concurrency isolation

Bulkhead is a semaphore: it limits the number of calls *in-flight at the same time*. Calls beyond `max_concurrent` are rejected immediately with `BulkheadException`.

`lumen/resilience/bulkhead_example.py`

```
from pyfly.resilience import Bulkhead, bulkhead

# At most 5 concurrent calls to AccountService
account_bulkhead = Bulkhead(max_concurrent=5)

@bulkhead(account_bulkhead)
async def fetch_account(account_id: str) -> dict:
    ...
```

Listing 13.7 — Bulkhead limiting concurrent account service calls

How it works: The decorator acquires a permit (`_acquire_slot`) before entering the function and releases it (`_release_slot`) in a `finally` block, so the slot is always returned even when the function raises. Slots are tracked by a single lock-guarded integer counter shared by async and sync call paths, so one `Bulkhead` instance safely decorates a mix of coroutines and regular functions.

This fail-fast behaviour is intentional: when 5 concurrent calls are in-flight and a 6th arrives, rejecting it immediately lets the caller retry or invoke a fallback — far better than queuing it indefinitely and causing cascading backpressure.

💡 MONITORING BULKHEAD UTILIZATION

`account_bulkhead.available_slots` returns the number of free permits at any moment. Expose this in a health endpoint or feed it to your observability stack to detect persistent saturation before it becomes an outage.

Time limiter — enforcing a deadline

A slow downstream is sometimes worse than a crashed one: indefinitely blocking calls consume resources without bound. `@time_limiter` cancels the coroutine if it does not complete within a `timedelta`:

`lumen/resilience/timeout_example.py`

```
from datetime import timedelta

from pyfly.resilience import time_limiter

@time_limiter(timeout=timedelta(seconds=2))
async def fetch_account(account_id: str) -> dict:
    ...
```

Listing 13.8 — 2-second deadline on account lookup

How it works: Internally, `time_limiter` calls `asyncio.wait_for(func(*args, **kwargs), timeout=timeout_seconds)`. When the deadline passes, `asyncio.wait_for` cancels the underlying task, causing any `await` inside the function to raise `asyncio.CancelledError`. The decorator catches `TimeoutError` and re-raises it as `OperationTimeoutException` with a descriptive message:

```
OperationTimeoutException: fetch_account exceeded timeout of 2.0s
```

Resources acquired inside the timed function should be guarded with `try/finally` so they are released even on cancellation:

```
@time_limiter(timeout=timedelta(seconds=2))
async def fetch_account(account_id: str) -> dict:
    conn = await pool.acquire()
    try:
        return await conn.execute(query, account_id)
    finally:
        await pool.release(conn)
```

Fallback — graceful degradation

`@fallback` is the safety net at the outermost layer: it catches exceptions and returns an alternative response rather than propagating the error to the caller. Lumen's balance summary endpoint can return a degraded response — last known balance, marked as potentially stale — rather than an HTTP 500 when `AccountService` is down.

Two modes are available. The first returns a **static value**:

```
lumen/resilience/fallback_static.py
```

```
from pyfly.resilience import fallback

@fallback(fallback_value={"balance_minor": 0, "source": "fallback"})
```

```

async def fetch_account(account_id: str) -> dict:
    ...

```

Listing 13.9 — Static fallback value

The second invokes a **fallback method** that receives the original arguments plus the exception:

`lumen/resilience/fallback_method.py`

```

from pyfly.cache import CacheAdapter
from pyfly.resilience import fallback

_cache: CacheAdapter # injected elsewhere

async def account_from_cache(
    account_id: str,
    exc: Exception = None,
) -> dict:
    cached = await _cache.get(f"account:{account_id}")
    if cached:
        return {**cached, "source": "cache"}
    return {"account_id": account_id, "balance_minor": 0, "source":
"fallback"}

@fallback(fallback_method=account_from_cache)
async def fetch_account(account_id: str) -> dict:
    ...

```

Listing 13.10 — Fallback method with cached data

How it works: When the primary function raises one of the exception types listed in `on` (default: all `Exception` subclasses), the decorator calls `fallback_method(*args, exc=exc, **kwargs)`. The `exc` keyword argument carries the caught exception so the

fallback can log it, inspect its type, or return different values for different failure modes. If the fallback method returns a coroutine, PyFly awaits it automatically. Narrow the exception filter with `on=(OperationTimeoutException, CircuitBreakerException)` to let programming errors propagate normally.

⚠️ FALLBACK METHOD SIGNATURE

The fallback method must accept `exc` as a keyword argument. PyFly passes the caught exception as `exc=<exception>`. If your fallback method's signature does not include `exc`, you will see a `TypeError` with a clear message at the first failure — not at decoration time.

Retry and circuit breaker

@retry — bounded re-attempts with backoff

Network errors are often transient: a packet is lost, a connection pool is momentarily exhausted, a downstream pod restarts. `@retry` re-invokes the decorated function up to `max_attempts` times with exponential backoff between attempts.

`max_attempts` is the only positional argument; every other parameter is keyword-only:

`lumen/resilience/retry_example.py`

```
from pyfly.resilience import retry

@retry(
    max_attempts=3,
    delay=0.1,
    backoff=2.0,
    max_delay=2.0,
    exceptions=(IOError, TimeoutError),
```

```
)
async def fetch_account(account_id: str) -> dict:
    ...
```

Listing 13.11 — Retry with exponential backoff

How it works: The decorator executes the function, catches exceptions matching `exceptions`, sleeps `delay * backoff ** attempt` seconds (capped at `max_delay`), and tries again. On the final attempt it re-raises the last exception. The sleep uses `await asyncio.sleep(...)` for async functions and `time.sleep(...)` for sync functions — the same implementation handles both. The `jitter` parameter adds randomisation to avoid thundering-herd retries when many instances restart simultaneously.

Parameter	Default	Description
<code>max_attempts</code>	<code>3</code>	Total attempts including the first (≥ 1). Positional.
<code>delay</code>	<code>0.0</code>	Base sleep in seconds before the first retry. Keyword-only.
<code>backoff</code>	<code>1.0</code>	Multiplier applied to <code>delay</code> each attempt. Keyword-only.
<code>max_delay</code>	<code>None</code>	Cap on per-attempt sleep. <code>None</code> means no cap. Keyword-only.
<code>jitter</code>	<code>0.0</code>	Randomisation fraction <code>[0, 1]</code> applied to each wait. Keyword-only.
<code>exceptions</code>	<code>(Exception,)</code>	Exception types that trigger a retry; others propagate immediately. Keyword-only.

⚠️ IDEMPOTENCY IS YOUR RESPONSIBILITY

`@retry` will call the function body multiple times. If the operation is not idempotent — if calling it twice has a different effect than calling it once — you can apply changes more than once. Wallet deposits are not safe to retry naively: retrying a failed deposit could credit the same amount twice. Wrap non-idempotent operations in an idempotency key check (store the operation ID before executing; skip if the ID already exists) or limit `exceptions` to errors that are definitely pre-execution (connection errors, timeouts during the request phase) rather than post-execution ambiguity.

@circuit_breaker — fast failure under sustained outage

Retrying a genuinely unavailable service amplifies load at exactly the moment that service most needs relief. The circuit-breaker pattern solves this: after a threshold of consecutive failures the circuit **opens** and subsequent calls are rejected immediately — without attempting the remote call — until a recovery timeout elapses.

PyFly's circuit breaker has three states:

State	Behaviour
CLOSED	Normal operation. Every call goes through; failures are counted.
OPEN	All calls raise <code>CircuitBreakerException</code> immediately, without network I/O.
HALF_OPEN	After <code>recovery_timeout</code> seconds, a limited probe call is admitted. If it succeeds the circuit closes; if it fails the circuit reopens.

`@circuit_breaker` takes a `CircuitBreaker` **instance** — not keyword arguments.

Construct the `CircuitBreaker` separately and pass it in:

```
lumen/resilience/cb_example.py
```

```
from pyfly.resilience import CircuitBreaker, circuit_breaker
```

```

account_cb = CircuitBreaker(
    failure_threshold=5,
    recovery_timeout=30.0,
    expected=(IOError, TimeoutError),
)

@circuit_breaker(account_cb)
async def fetch_account(account_id: str) -> dict:
    ...

```

Listing 13.12 — Circuit breaker around AccountService

How it works: Before each call, `breaker.before_call()` checks the current state. If OPEN, it raises `CircuitBreakerException` immediately. If HALF_OPEN and the probe budget is exhausted, it also raises. Otherwise the call proceeds. On success, `breaker.on_success()` resets the consecutive-failure counter (or, in HALF_OPEN, closes the circuit once enough probes succeed). On failure, `breaker.on_failure()` increments the counter and opens the circuit when `failure_threshold` is reached.

Only exceptions in `expected` trip the breaker. Business exceptions — `ValueError`, `PermissionError` — propagate normally without affecting the circuit state.

CircuitBreaker constructor parameters (`failure_rate_threshold`, `window_size`, and `half_open_max_calls` are keyword-only):

Parameter	Default	Description
<code>failure_threshold</code>	<code>5</code>	Consecutive failures that trip the circuit.
<code>recovery_timeout</code>	<code>30.0</code>	Seconds in OPEN before moving to HALF_OPEN.
<code>expected</code>	<code>(Exception,)</code>	Exception types that count as failures.
<code>failure_rate_threshold</code>	<code>None</code>	Switch to windowed-rate mode when set (e.g. <code>0.5</code>).
<code>window_size</code>	<code>10</code>	Outcome window size for rate-based tripping.
<code>half_open_max_calls</code>	<code>1</code>	Probe calls required to close from HALF_OPEN.

The `failure_rate_threshold` and `window_size` parameters switch from consecutive-count mode to windowed-rate mode, matching Resilience4j's COUNT_BASED sliding window. Set `failure_rate_threshold=0.5` and `window_size=10` to open the circuit when more than half of the last 10 calls fail.

🌿 SPRING PARITY

`@retry` mirrors Spring Retry's `@Retryable` (with `maxAttempts`, `backoff`, `include`). `CircuitBreaker` mirrors Resilience4j's `CircuitBreaker` (failure threshold, recovery timeout, CLOSED/OPEN/HALF_OPEN state machine, half-open probe calls, expected-exception filter). PyFly does not use the Resilience4j Java library — it is a pure-Python re-implementation with the same semantics.

Composing the layers

Decorator order

PyFly's resilience decorators compose by stacking. Python applies decorators bottom-up at decoration time but executes them top-down at call time. The recommended order, outermost to innermost:

```
@fallback      ← 1. Catch any exception; return degraded response
@rate_limiter  ← 2. Reject excess traffic before it acquires resources
@bulkhead      ← 3. Limit concurrency of rate-limited calls
@time_limiter  ← 4. Cancel if execution takes too long
async def func() ← 5. The actual operation
```

This ordering ensures:

1. **Fallback** catches exceptions from every inner layer — including `RateLimitException`, `BulkheadException`, and `OperationTimeoutException` — so the caller always receives a usable response.
2. **Rate limiter** drops excess requests before they consume a bulkhead slot, preventing a traffic flood from exhausting the concurrency budget.
3. **Bulkhead** limits how many rate-permitted calls run concurrently, protecting the downstream from overload.
4. **Time limiter** applies only to actual execution; when it fires, the bulkhead `finally` block releases the slot correctly.

Add `@retry` and `@circuit_breaker` on the innermost side — wrapping only the actual I/O call — so the fallback absorbs their exceptions and the rate limiter and bulkhead account for retried calls correctly:

```
@fallback
@rate_limiter
@bulkhead
```

```

@time_limiter
@circuit_breaker(account_cb)
@retry(max_attempts=2, delay=0.05, backoff=2.0, exceptions=(IOError,))
async def fetch_account(account_id: str) -> dict: ...

```

With `@retry` below `@time_limiter`, the timeout budget covers the entire retry sequence, not each individual attempt. To bound each attempt independently, move `@time_limiter` below `@retry`.

Putting it all together — Lumen's account gateway

Here is the full pattern assembled into a realistic `AccountGateway` that Lumen's wallet handlers use to look up account information:

`lumen/account/gateway.py`

```

from datetime import timedelta

from pyfly.cache import CacheAdapter, cacheable
from pyfly.container import service
from pyfly.kernel.exceptions import CircuitBreakerException,
OperationTimeoutException
from pyfly.resilience import (
    Bulkhead,
    CircuitBreaker,
    RateLimiter,
    bulkhead,
    circuit_breaker,
    fallback,
    rate_limiter,
    retry,
    time_limiter,
)

_limiter = RateLimiter(max_tokens=50, refill_rate=20.0)
_bh = Bulkhead(max_concurrent=8)
_cb = CircuitBreaker(
    failure_threshold=5,

```

```

        recovery_timeout=30.0,
        expected=(IOError, TimeoutError),
    )

DEGRADED = {"status": "degraded", "balance_minor": None}

@service
class AccountGateway:

    def __init__(self, http_client, cache: CacheAdapter) -> None:
        self._http = http_client
        self._cache = cache

    @cacheable(
        backend=None, # pass self._cache at runtime (see note below)
        key="account:{account_id}",
        ttl=timedelta(seconds=30),
    )
    @fallback(
        fallback_value=DEGRADED,
        on=(OperationTimeoutException, CircuitBreakerException, IOError),
    )
    @rate_limiter(_limiter)
    @bulkhead(_bh)
    @time_limiter(timeout=timedelta(seconds=2))
    @circuit_breaker(_cb)
    @retry(max_attempts=2, delay=0.05, backoff=2.0, exceptions=(IOError))
    async def get_account(self, account_id: str) -> dict:
        resp = await self._http.get(f"/accounts/{account_id}")
        return resp.json()

```

Listing 13.13 — AccountGateway with full resilience stack

❗ WIRING BACKEND IN A CLASS METHOD

Because Python evaluates class-body decorators before `__init__` runs, `self._cache` is not yet available there. The listing above passes `backend=None` as a placeholder to illustrate the stacking order. In practice, wrap `get_account` in `__init__` the same way as the handler examples: `self.get_account = cacheable(backend=cache, key=..., ttl=...)(self._do_get_account)`. Alternatively, use a module-level `InMemoryCache` instance for testing and swap it via the DI container in production.

How a call flows through the layers:

1. `@cacheable` checks the cache. On a hit, every inner layer is skipped entirely.
2. On a miss, `@fallback` becomes the outermost safety net.
3. `@rate_limiter` checks the token bucket; rejects the call if empty.
4. `@bulkhead` checks the permit counter; rejects if at capacity.
5. `@time_limiter` sets a two-second deadline for the layers below.
6. `@circuit_breaker` rejects immediately if the circuit is OPEN.
7. `@retry` attempts the HTTP call up to two times on `IOError`.
8. On success, `@cacheable` stores the response for 30 seconds.
9. If `IOError`, `OperationTimeoutException`, or `CircuitBreakerException` escapes, `@fallback` catches it and returns `DEGRADED`.

Note that `@cacheable` sits *above* `@fallback`. That means:

- A cached `DEGRADED` response from a previous failure cycle is returned as-is for up to 30 seconds without hitting the network.
- If you do not want to cache degraded responses, move `@cacheable` below `@fallback`, or use the `unless` predicate: `unless=lambda r: r.get("status") == "degraded"`.

✓ What you built

This chapter closes Part IV. In Chapter 11 you split Lumen into independent services with typed HTTP clients. In Chapter 12 you added `DepositSaga` to coordinate multi-step operations with compensating transactions. Here you made the whole system fast and fault-tolerant.

Concretely, you learned:

- `@cacheable` short-circuits balance reads on a cache hit; the five-second TTL bounds staleness to an acceptable window. Applied to `GetBalanceHandler` by wrapping `_fetch` at construction time — `_fetch` calls `repository.find_by_id` and projects the resulting `WalletEntity` onto `BalanceDto` via `entity_to_balance_dto` (`Mapper.project` + the `@projection`-marked `BalanceView`).
- `@cache_put` refreshes the cache as a side-effect of each `DepositFunds` command. `_deposit` is decorated with `@transactional()`; it does `find_by_id → to_aggregate → mutate → upsert` as one committed unit of work, then updates the cache with the returned balance. The key template must match the `@cacheable` key to hit the same slot.
- `@cache_evict` removes entries on wallet closure or administrative resets; `all_entries=True` flushes the entire cache in a single call.
- `CacheManager` mirrors writes to both Redis (primary) and `InMemoryCache` (fallback) and fails over transparently; it is the right default for any production deployment.
- `RateLimiter` + `@rate_limiter` cap inbound traffic with a token- bucket algorithm that allows controlled bursting.
- `Bulkhead` + `@bulkhead` isolate concurrency with a fail-fast semaphore that prevents one slow dependency from consuming all available resources.

- `@time_limiter` enforces deadlines using `asyncio.wait_for`, turning indefinitely hanging calls into bounded `OperationTimeoutException` errors.
- `@fallback` provides a degraded but functional response when every other layer has failed; the fallback method receives the original arguments and the caught exception via the `exc` keyword argument.
- `@retry` takes `max_attempts` as its only positional argument; all other parameters (`delay`, `backoff`, `max_delay`, `jitter`, `exceptions`) are keyword-only. It re-invokes operations a bounded number of times with exponential backoff.
- `@circuit_breaker` takes a `CircuitBreaker` **instance** — not keyword arguments — and opens the circuit after a failure threshold, short-circuiting subsequent calls during the recovery window so the downstream has time to recover.
- **Decorator order** matters: fallback outermost, then rate limiter, bulkhead, time limiter, circuit breaker, and retry innermost — with caching above the fallback to cache even degraded responses.

Lumen is now a multi-service, saga-coordinated, cached, and resilient system. Part V adds the final production concerns: observability — metrics, distributed tracing, and health endpoints — so you can see exactly what Lumen is doing in production.

Try it yourself

Exercise 1 — Conditional caching. The `GetBalance` handler is called far more often for active wallets than for test wallets. Add `condition=lambda query: not query.wallet_id.startswith("test-")` to the `cacheable(...)` call inside `GetBalanceHandler.__init__` and verify with a unit test using `InMemoryCache` that queries for test wallet ids always reach the repository.

Exercise 2 — Circuit breaker with rate-based threshold. Replace the consecutive-count circuit breaker in `AccountGateway` with a rate-based one: open the circuit when more than 60% of the last 20 calls fail. Construct

`CircuitBreaker(failure_rate_threshold=0.6, window_size=20, recovery_timeout=60.0, expected=(IOError, TimeoutError))` and write a test that fires 12 failing calls followed by 8 successful ones, asserting that the circuit opens after the 13th failure (crossing 60% of 20).

Exercise 3 — Evict by prefix. Lumen sometimes needs to invalidate all cache entries for a given wallet owner (GDPR deletion). Add a `purge_owner(owner_id: str)` method to a wallet admin service that calls

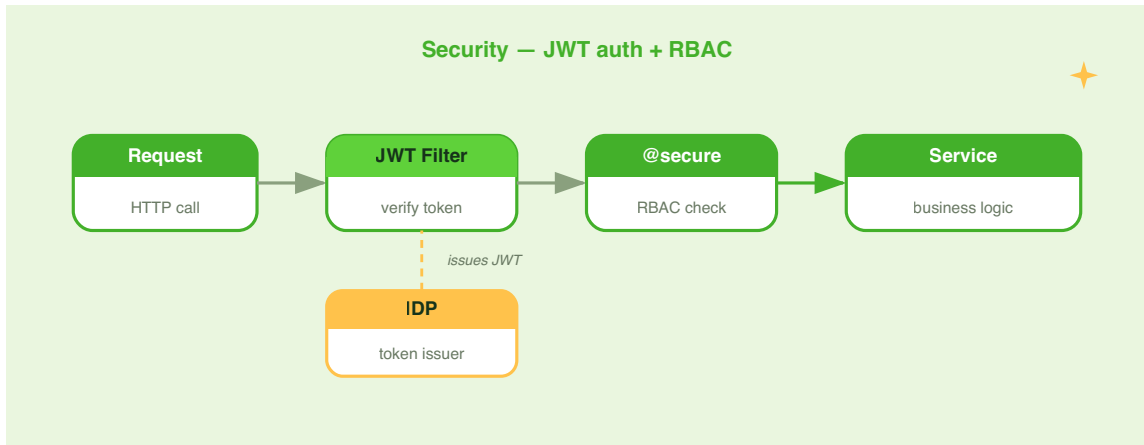
`backend.evict_by_prefix(f"wallet:balance:{owner_id}:")` directly (without a decorator), and write a test that pre-populates three wallet keys for one owner and one for another, calls `purge_owner`, and asserts that only the target owner's entries are gone.

PART V

Secure, Observe & Ship

CHAPTER 14

Security, Sessions & Identity



In Chapter 13 you made Lumen fast and fault-tolerant with caching and resilience decorators. Lumen's API now handles high concurrency without breaking under pressure — but it is wide open. Any caller can create wallets, read balances, or trigger deposits. Before you can ship Part V's remaining production concerns, you need to close that door.

This chapter locks Lumen down. You will:

- **Authenticate** every request with a signed JWT, using `JWTService` to issue and validate tokens and `SecurityMiddleware` to propagate the `SecurityContext` across the request scope.
- **Authorise** individual handlers and commands with the `@secure` decorator, specifying roles, permissions, or full security expressions.

- **Hash passwords** safely with `BcryptPasswordEncoder` so your user store is never a liability.
- **Manage server-side sessions** with `HttpSession`, a pluggable `SessionStore`, and a Redis backend for horizontal scaling.
- **Federate identity** to an external provider — Keycloak, AWS Cognito, or Azure AD — through the `IdpAdapter` port without changing a line of business logic.

If you have worked with Spring Security before, the shape will feel familiar: configure a filter chain, annotate individual methods, and swap the user-detail source for an IDP.

PyFly calls those pieces `SecurityMiddleware + HttpSecurity`, `@secure`, and `IdpAdapter`, but the concepts map one-to-one.

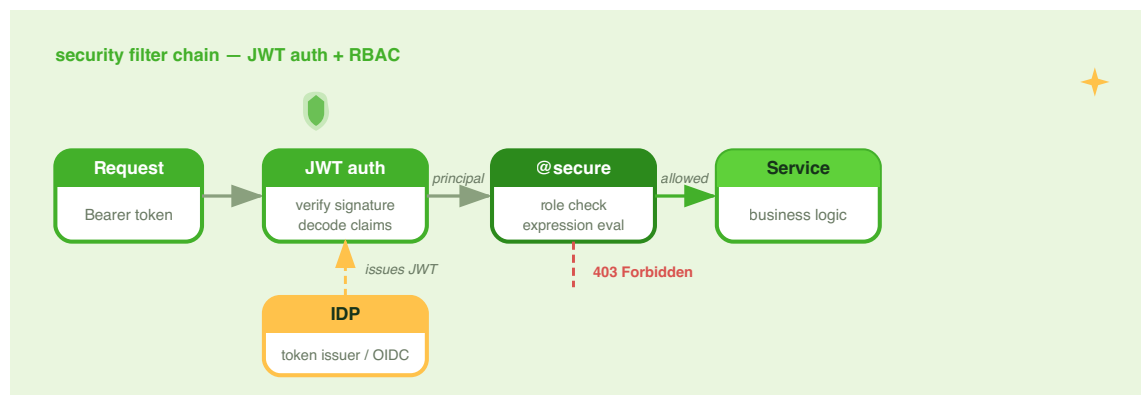


Figure 14.1 — Lumen's security layers. A JWT filter populates the `SecurityContext`; `HttpSecurity` enforces URL-level rules; `@secure` enforces handler-level rules; the IDP port delegates identity to an external provider.

Authentication with JWT

Why JSON Web Tokens?

Lumen is a stateless API. HTTP sessions would require sticky routing or a shared session store on every replica. JWT tokens let each service validate credentials independently — no shared state, no coordination, horizontal scaling by default.

A **JWT** is a signed JSON payload. Lumen's auth service issues a token on login; every subsequent request carries that token in the `Authorization` header;

`SecurityMiddleware` validates the signature and unpacks the token into a `SecurityContext` that the rest of the request can read.

JWTService

`JWTService` wraps PyJWT with three focused operations:

Method	Description
<code>encode(payload)</code>	Sign a payload dict, adding <code>exp</code> if absent
<code>decode(token)</code>	Validate signature + <code>exp</code> ; raises <code>SecurityException</code> on failure
<code>to_security_context(token)</code>	Decode and extract <code>sub</code> , <code>roles</code> , <code>permissions</code> into a <code>SecurityContext</code>

The service always requires an `exp` claim — a token with no expiry is rejected at decode time. That invariant means every token in circulation has a bounded lifetime.

```
lumen/core/services/auth/auth_service.py
```

```
from pyfly.container import service
from pyfly.kernel.exceptions import UnauthorizedException
from pyfly.security import BcryptPasswordEncoder, JWTService, SecurityContext
```

```

@service
class AuthService:

    def __init__(
        self,
        jwt: JWTService,
        encoder: BcryptPasswordEncoder,
        user_repo,
    ) -> None:
        self._jwt = jwt
        self._encoder = encoder
        self._users = user_repo

    async def login(self, username: str, password: str) -> str:
        user = await self._users.find_by_username(username)
        if user is None or not self._encoder.verify(
            password, user.password_hash
        ):
            raise UnauthorizedException(
                "Invalid credentials", code="INVALID_CREDENTIALS"
            )
        # encode() auto-appends exp (default: 3 600 s from now)
        return self._jwt.encode({
            "sub": str(user.id),
            "roles": [user.role],
            "permissions": _permissions_for(user.role),
        })

    async def me(self, ctx: SecurityContext) -> dict:
        user = await self._users.find_by_id(ctx.user_id)
        return {
            "id": str(user.id),
            "username": user.username,
            "role": user.role,
        }

```

```
def _permissions_for(role: str) -> list[str]:
    MAP = {
        "USER": ["wallet:read", "wallet:deposit"],
        "ADMIN": [
            "wallet:read", "wallet:deposit",
            "wallet:create", "wallet:delete",
            "user:read", "user:write",
        ],
    }
    return MAP.get(role, [])
```

Listing 14.1 — Issuing a JWT on successful login

...

How it works. `login` fetches the user record and calls

`BcryptPasswordEncoder.verify` to compare the supplied password against the stored hash. On success it calls `jwt.encode`, which auto-appends an `exp` claim `expiration_seconds` seconds from now (default `3600` — one hour). You never import `datetime`: the service computes the Unix-timestamp expiry as `int(time.time()) + expiration_seconds`. The caller receives a compact, self-contained token string.

The SecurityContext

`SecurityContext` is a frozen dataclass that carries authentication and authorisation data for a single request. The middleware creates it from the validated token; your handlers receive it as an injected parameter.

Field	Type	Description
<code>user_id</code>	<code>str None</code>	Authenticated user's id; <code>None</code> for anonymous
<code>roles</code>	<code>list[str]</code>	Roles granted in the token
<code>permissions</code>	<code>list[str]</code>	Fine-grained permissions
<code>attributes</code>	<code>dict[str, str]</code>	Extra claims (department, tenant, ...)

Key methods:

Method / Property	Returns	Description
<code>is_authenticated</code>	<code>bool</code>	<code>True</code> when <code>user_id</code> is not <code>None</code>
<code>has_role(role)</code>	<code>bool</code>	Exact match against <code>roles</code> list
<code>has_any_role(roles)</code>	<code>bool</code>	Set intersection — any of the listed roles
<code>has_permission(perm)</code>	<code>bool</code>	Exact match against <code>permissions</code> list
<code>SecurityContext.anonymous()</code>	<code>SecurityContext</code>	Create an unauthenticated context

The security filter

`SecurityMiddleware` (canonical location

`pyfly.web.adapters.starlette.security_middleware`, re-exported from `pyfly.security`) sits at the Starlette middleware layer. For every request it:

1. Checks whether the path is in `exclude_paths`; if so, sets an anonymous context and continues.
2. Reads the `Authorization` header and strips the `Bearer` prefix.

3. Calls `jwt_service.to_security_context(token)`.
4. On success, stores the authenticated context in `request.state.security_context`.
5. On any `SecurityException` (expired, tampered, or missing `exp`), logs at DEBUG and sets an anonymous context instead.

The middleware **never rejects requests** — rejection is the job of `@secure` and `HttpSecurity`. An endpoint that requires the user can enforce it; a health-check endpoint can ignore the context entirely.

`lumen/app.py`

```
from pyfly.security import JWTService, SecurityMiddleware
from pyfly.web.adapters.starlette import create_app

def build_app(context):
    app = create_app(title="Lumen", context=context)

    jwt = context.get_bean(JWTService)
    app.add_middleware(
        SecurityMiddleware,
        jwt_service=jwt,
        exclude_paths=[
            "/docs",
            "/openapi.json",
            "/api/auth/login",
            "/api/auth/register",
        ],
    )
    return app
```

Listing 14.2 — Adding the security middleware

...

How it works. `exclude_paths` lists the paths where no token is expected. Login and register cannot require authentication because the token does not exist yet; docs paths are excluded so the API explorer works without credentials. Every other path goes through token validation.

URL-level rules with `HttpSecurity`

`@secure` guards individual handler methods. `HttpSecurity` guards whole URL subtrees at the filter layer — before the route dispatcher runs. The two are complementary: `HttpSecurity` provides fast, broad policy at the edge; `@secure` adds fine-grained, per-handler enforcement behind it.

`lumen/config/security_config.py`

```
from pyfly.container import bean, configuration
from pyfly.security.http_security import HttpSecurity

@Configuration
class SecurityConfig:

    @bean
    def http_security_filter(self):
        hs = HttpSecurity()
        hs.authorize_requests() \
            .request_matchers("/idp/admin/**").has_role("ADMIN") \
            .request_matchers("/api/v1/wallets/**").authenticated() \
            .request_matchers(
                "/health", "/docs", "/openapi.json",
                "/idp/login", "/idp/refresh",
            ).permit_all() \
            .any_request().permit_all()
        return hs.build()
```

Listing 14.3 — `HttpSecurity` DSL

...

How it works. Rules are evaluated in declaration order — first match wins. The `HttpSecurityFilter` runs at `HIGHEST_PRECEDENCE + 350`, after authentication filters have populated `request.state.security_context`, so every role and permission check has a fully hydrated context to inspect. The terminal methods — `has_role`, `has_any_role`, `has_permission`, `authenticated`, `permit_all`, and `deny_all` — cover every common policy; unsatisfied rules return RFC 7807 problem-detail JSON (`application/problem+json`) with the appropriate HTTP status.

TWO-LAYER DEFENSE

`HttpSecurity` provides fast URL-level policy before routes are even dispatched — good for blanket rules like "everything under `/api/v1/wallets` needs authentication." The `@secure` decorators on individual methods are the second, finer-grained layer. Use both together for defense in depth.

SPRING PARITY

`HttpSecurity` mirrors Spring Security's `HttpSecurity.authorizeHttpRequests()` chain. `request_matchers` corresponds to `requestMatchers`, `authenticated()` to `.authenticated()`, `has_role` to `hasRole`, and `build()` triggers registration of the underlying filter just as `build()` finalises the Spring filter chain. The `fnmatch` glob patterns (`/api/admin/**`) behave identically to Spring's Ant-style path matching.

Auto-configuration

You do not need to register `JWTService` or `BcryptPasswordEncoder` manually. Add the relevant properties to `pyfly.yaml` and auto-configuration wires everything:

lumen/resources/pyfly.yaml

```

pyfly:
  security:
    enabled: true
    jwt:
      secret: "${JWT_SECRET}"
      algorithm: HS256
      filter:
        enabled: true
        exclude-patterns: >-
          /docs,/openapi.json,/actuator/health,
          /api/auth/login,/api/auth/register
    password:
      bcrypt-rounds: 12

```

Listing 14.4 — Security auto-configuration

...

Property	Default	Description
<code>pyfly.security.jwt.secret</code>	<code>change-me-in-production</code>	HMAC signing key — must be overridden
<code>pyfly.security.jwt.algorithm</code>	<code>HS256</code>	Signing algorithm
<code>pyfly.security.jwt.filter.enabled</code>	<i>(absent)</i>	Set <code>true</code> to register <code>SecurityFilter</code> bean automatically
<code>pyfly.security.jwt.exclude-patterns</code>	<i>(absent)</i>	Comma-separated paths to skip
<code>pyfly.security.password.bcrypt-rounds</code>	<code>12</code>	Bcrypt cost factor

⚠ PRODUCTION SECRET

Never commit the real JWT secret to source control. Use `${JWT_SECRET}` and inject the value from an environment variable or a secrets manager at deploy time.

Authorization with `@secure`

`@secure` is a function decorator that enforces authentication and authorisation on individual handlers and commands. It reads the `security_context` keyword argument that the middleware has already injected, then evaluates role, permission, and expression checks before the function body runs.

Signature

```
def secure(
    roles: list[str] | None = None,
    permissions: list[str] | None = None,
    expression: str | None = None,
) -> Callable: ...
```

The decorated function must accept `security_context: SecurityContext` as a keyword argument — that is how `@secure` reaches the current user.

Role-based protection

Lumen's wallet endpoints (`/api/v1/wallets`) are the natural place to apply `@secure` . The real `WalletController` injects the command and query buses; add `security_context: SecurityContext` to each method that needs protection and stack `@secure` on top.

The controller exposes seven endpoints under `/api/v1/wallets` :

Method	Path	Handler	Guard
POST	""	open_wallet	USER or ADMIN
POST	/ {wallet_id} / deposit	deposit	USER/ADMIN + wallet:deposit
POST	/ {wallet_id} / withdraw	withdraw	USER/ADMIN + wallet:deposit
GET	""	list_wallets	authenticated
GET	/rich	list_rich_wallets	authenticated
GET	/ {wallet_id} / balance	wallet_balance	USER or ADMIN
GET	/ {wallet_id}	wallet_detail	ADMIN only

The collection handlers (`list_wallets` , `list_rich_wallets`) are named with the `list_` prefix so the framework registers them before the `wallet_detail` / `wallet_balance` handlers — Starlette matches first-registered-wins, so the literal `/rich` segment is found ahead of the `/ {wallet_id}` path variable.

`lumen/web/controllers/wallet_controller.py`

```

from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.list_rich_wallets_query import
ListRichWallets
from lumen.core.services.wallets.list_wallets_query import ListWallets
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.interfaces.dtos.v1.deposit_request import DepositRequest
from lumen.interfaces.dtos.v1.open_wallet_request import OpenWalletRequest

```

```

from lumen.interfaces.dtos.v1.page_dto import PageDto
from lumen.interfaces.dtos.v1.wallet_dto import WalletDto
from pyfly.container import rest_controller
from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
from pyfly.data import Pageable, Sort
from pyfly.kernel import ResourceNotFoundException
from pyfly.security import SecurityContext, secure
from pyfly.web import (
    Body,
    PathVar,
    QueryParam,
    Valid,
    get_mapping,
    post_mapping,
    request_mapping,
)

_NEWEST_FIRST = Sort.by("created_at").descending()

@rest_controller
@request_mapping("/api/v1/wallets")
class WalletController:
    """Digital-wallet REST API: open, deposit, withdraw, list, inspect."""

    def __init__(
        self,
        commands: DefaultCommandBus,
        queries: DefaultQueryBus,
    ) -> None:
        self._commands = commands
        self._queries = queries

    # Any authenticated user may open a wallet.
    @secure(roles=["USER", "ADMIN"])
    @post_mapping("", status_code=201)
    async def open_wallet(
        self,
        request: Valid[Body[OpenWalletRequest]],
    )

```



```

    security_context: SecurityContext,
) -> dict[str, str]:
    wallet_id = await self._commands.send(
        OpenWallet(
            owner_id=request.owner_id,
            currency=request.currency,
        )
    )
    return {"wallet_id": wallet_id}

# Deposit: USER/ADMIN role + wallet:deposit permission.
@secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])
@post_mapping("/{wallet_id}/deposit")
async def deposit(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
    security_context: SecurityContext,
) -> dict[str, int | str]:
    balance = await self._commands.send(
        DepositFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# Withdraw: same guard as deposit.
@secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])
@post_mapping("/{wallet_id}/withdraw")
async def withdraw(
    self,
    wallet_id: PathVar[str],
    request: Valid[Body[DepositRequest]],
    security_context: SecurityContext,
) -> dict[str, int | str]:
    balance = await self._commands.send(
        WithdrawFunds(wallet_id=wallet_id, amount=request.amount)
    )
    return {"wallet_id": wallet_id, "balance_minor": balance}

# Paged list – collection routes registered before {/wallet_id}.

```

```

@secure(roles=["USER", "ADMIN"])
@get_mapping("")
async def list_wallets(
    self,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
    security_context: SecurityContext = None,
) -> PageDto[WalletDto]:
    result = await self._queries.query(
        ListWallets(
            pageable=Pageable.of(page, size, _NEWEST_FIRST)
        )
    )
    return PageDto.from_page(result)

# Rich list: wallets filtered by minimum balance.
@secure(roles=["USER", "ADMIN"])
@get_mapping("/rich")
async def list_rich_wallets(
    self,
    min_minor: QueryParam[int] = 0,
    page: QueryParam[int] = 1,
    size: QueryParam[int] = 20,
    security_context: SecurityContext = None,
) -> PageDto[WalletDto]:
    result = await self._queries.query(
        ListRichWallets(
            min_minor=min_minor,
            pageable=Pageable.of(page, size, _NEWEST_FIRST),
        )
    )
    return PageDto.from_page(result)

# Single-wallet balance: any authenticated user.
@secure(roles=["USER", "ADMIN"])
@get_mapping("/{wallet_id}/balance")
async def wallet_balance(
    self,
    wallet_id: PathVar[str],

```

```

        security_context: SecurityContext,
    ) -> BalanceDto:
        result = await self._queries.query(
            GetBalance(wallet_id=wallet_id)
        )
        if result is None:
            raise ResourceNotFoundException(
                f"Wallet {wallet_id!r} not found",
                code="WALLET_NOT_FOUND",
                context={"wallet_id": wallet_id},
            )
        return result

# Full wallet view: ADMIN only.
@secure(roles=["ADMIN"])
@get_mapping("/{wallet_id}")
async def wallet_detail(
    self,
    wallet_id: PathVar[str],
    security_context: SecurityContext,
) -> WalletDto:
    result = await self._queries.query(
        GetWallet(wallet_id=wallet_id)
    )
    if result is None:
        raise ResourceNotFoundException(
            f"Wallet {wallet_id!r} not found",
            code="WALLET_NOT_FOUND",
            context={"wallet_id": wallet_id},
        )
    return result

```

Listing 14.5 — Role and permission guards on real Lumen endpoints

:::

How it works. Stack `@secure` **above** `@post_mapping` / `@get_mapping` so authorisation runs before route binding. The framework injects `security_context` from `request.state.security_context`, which `SecurityMiddleware` already populated.

When you list multiple roles, the user needs **at least one** (OR semantics). When you list multiple permissions, the user needs **all** of them (AND semantics). When you supply both `roles` and `permissions`, both checks must pass independently.

The collection handlers (`list_wallets` , `list_rich_wallets`) sort alphabetically before `wallet_balance` / `wallet_detail` , so the framework registers the literal `""` and `/rich` routes first — ensuring Starlette resolves `/api/v1/wallets/rich` as a fixed segment rather than as a wallet id.

AMOUNTS IN MINOR UNITS

`DepositRequest.amount` is an `int` in **minor units** (cents). €10.50 is `1050` . This convention avoids floating-point rounding errors throughout the Money domain.

`WalletDto.balance_minor` carries the same integer; `WalletDto.balance` is a `float` rendered for display only.

Expression-based authorization

For policies that cannot be expressed with a flat role list, use the `expression` parameter. PyFly evaluates expressions via safe AST parsing — no `eval()` or `exec()` is involved anywhere in the chain.

`lumen/web/controllers/wallet_controller.py`

```
from pyfly.security import SecurityContext, secure

# ADMIN, or a MANAGER who also holds wallet:write.
@secure(
    expression=(
```

```

        "hasRole('ADMIN')"
```

```

        " or (hasRole('MANAGER') and hasPermission('wallet:write'))"
    )
)
async def approve_large_deposit(
    self,
    deposit_id: str,
    security_context: SecurityContext,
) -> None:
    ...

# Any authenticated, non-guest user can see the wallet dashboard.
@secure(expression="isAuthenticated and not hasRole('GUEST')")
async def dashboard(
    self,
    security_context: SecurityContext,
) -> dict:
    ...

```

Listing 14.6 — Security expressions on wallet endpoints

...

Supported expression vocabulary (full set):

Token	Description
<code>hasRole('X')</code>	User has role <code>X</code>
<code>hasAnyRole('X', 'Y')</code>	User has at least one of the listed roles
<code>hasAuthority('X')</code>	User has role or permission <code>X</code>
<code>hasAnyAuthority('X', 'Y')</code>	At least one of the listed roles/permissions
<code>hasPermission('X')</code>	User has permission <code>X</code>
<code>isAuthenticated</code>	User is authenticated
<code>isAnonymous</code>	User is not authenticated
<code>permitAll</code>	Always allow
<code>denyAll</code>	Always deny
<code>principal / authentication</code>	The current <code>SecurityContext</code> object
<code>and / or / not</code>	Boolean operators
<code>(...)</code>	Grouping

EXPRESSIONS ARE SAFE

PyFly reduces each construct to `True` or `False`, then evaluates only a pure boolean AST. Any node that is not a constant, `BoolOp`, or `UnaryOp(Not)` raises `SecurityException(code="INVALID_EXPRESSION")`. This eliminates injection risks entirely.

Applying @secure to CQRS handlers

`@secure` is not limited to REST controllers. You can protect CQRS command handlers in exactly the same way — it fires before the handler body because the DI container injects `security_context` from `request.state.security_context` when it resolves the handler:

`lumen/core/services/wallets/deposit_funds_handler.py`

```
from pyfly.cQRS import command_handler
from pyfly.security import SecurityContext, secure

from lumen.core.services.wallets.deposit_funds_command import DepositFunds

@command_handler
class DepositFundsHandler:

    @secure(roles=["USER", "ADMIN"], permissions=["wallet:deposit"])
    async def handle(
        self,
        command: DepositFunds,
        security_context: SecurityContext,
    ) -> int:
        # command.amount is in minor units (cents)
        ...
```

Listing 14.7 — @secure on a CQRS command handler

...

The check fires before any business logic runs.

Passwords

Why bcrypt?

MD5 and SHA-256 are engineered to be fast — ideal for data integrity, catastrophic for passwords. An attacker who steals your user table can test billions of SHA-256 guesses per second on commodity hardware. **Bcrypt** is deliberately slow and adjustably expensive: the cost factor (rounds) lets you tune the algorithm so that an attack requires orders of magnitude more time, without meaningfully affecting normal login latency.

BcryptPasswordEncoder

`BcryptPasswordEncoder` implements the `PasswordEncoder` protocol (a `runtime_checkable` Protocol with `hash` and `verify` methods):

`lumen/auth/password_service.py`

```
from pyfly.security import BcryptPasswordEncoder

encoder = BcryptPasswordEncoder(rounds=12)

# During registration – store only the hash, never the raw password
hashed = encoder.hash("correct-horse-battery-staple")

# During login – verify without storing the plaintext
is_match = encoder.verify("correct-horse-battery-staple", hashed)
# True

is_match = encoder.verify("wrong-password", hashed)
# False
```

Listing 14.8 — Hashing and verifying passwords

...

Parameter	Default	Notes
<code>rounds</code>	<code>12</code>	Each increment doubles hashing time. 12 is the recommended production default.

How it works. `hash` calls `bcrypt.gensalt(rounds=self._rounds)` to generate a fresh random salt, then `bcrypt.hashpw` to produce the hash. Both salt and hash are embedded in the returned string — the `$2b$12$...` prefix encodes the algorithm version and cost factor, making every stored hash self-describing. `verify` calls `bcrypt.checkpw`, which re-derives the hash from the raw password and the embedded salt and compares the result with a timing-safe equality check, preventing timing-oracle attacks.

The PasswordEncoder protocol

`PasswordEncoder` is a `runtime_checkable` Protocol. Any class that implements `hash(raw: str) -> str` and `verify(raw: str, hashed: str) -> bool` satisfies it — including `BcryptPasswordEncoder`. You can swap in `argon2` or `scrypt` at any time without touching service code:

```
from pyfly.security import PasswordEncoder

class Argon2PasswordEncoder:
    def hash(self, raw: str) -> str: ...
    def verify(self, raw: str, hashed: str) -> bool: ...

isinstance(Argon2PasswordEncoder(), PasswordEncoder) # True
```

 AUTO-CONFIGURATION

When `pyfly.security.enabled=true` and `bcrypt` is installed, PyFly auto-configures a `BcryptPasswordEncoder` bean with `rounds` read from `pyfly.security.password.bcrypt-rounds`. Declare the bean yourself in a `@configuration` class to override it without touching auto-configuration.

Sessions

Why server-side sessions?

JWT tokens are stateless — once issued, they cannot be revoked before they expire. If a user logs out, their token is still valid until `exp`. For many APIs that trade-off is acceptable. For browser-facing applications — Lumen's admin dashboard, for example — you need the server to be the authoritative source: log out must mean log out.

Server-side sessions give you that control. A `SessionStore` holds session data keyed by a random session ID. The browser receives only the session ID in a cookie. The server revokes a session instantly by deleting its store entry.

HttpSession

`HttpSession` wraps a session's data dictionary with typed accessors and tracks mutation state so the filter knows when to persist:

Property / Method	Description
<code>id</code>	UUID hex string session identifier
<code>is_new</code>	<code>True</code> if created during this request
<code>created_at</code>	Unix timestamp (<code>float</code>) of creation
<code>last_accessed</code>	Unix timestamp (<code>float</code>) of most recent access
<code>modified</code>	<code>True</code> if any attribute was written or session is new
<code>invalidated</code>	<code>True</code> if <code>invalidate()</code> was called
<code>previous_id</code>	Former id after <code>rotate_id()</code> (used by filter to clean up old entry)
<code>get_attribute(name)</code>	Return attribute value or <code>None</code>
<code>set_attribute(name, value)</code>	Write an attribute; marks session modified
<code>remove_attribute(name)</code>	Remove attribute if present
<code>get_attribute_names()</code>	List user-set attribute names (excludes internal <code>_*</code> keys)
<code>rotate_id()</code>	Assign a fresh session id, preserving all data
<code>invalidate()</code>	Mark for deletion on next filter pass
<code>get_data()</code>	Return the raw session dict (includes internal metadata)

`lumen/core/services/auth/session_handler.py`

```
from pyfly.session import HttpSession
```

```
async def post_login(
    username: str,
    password: str,
```

```

    session: HttpSession,
    auth_service,
) -> dict:
    user = await auth_service.authenticate(username, password)

    # Rotate the id before writing auth state (session-fixation prevention)
    session.rotate_id()
    session.set_attribute("user_id", str(user.id))
    session.set_attribute("role", user.role)

    return {"message": "Logged in"}

async def logout(session: HttpSession) -> dict:
    session.invalidate()
    return {"message": "Logged out"}

```

Listing 14.9 — Using the session after login

...

How it works. `rotate_id()` generates a fresh UUID session ID and records the old one in `session.previous_id`. When `SessionFilter` persists the session at the end of the request, it deletes the old store entry (the previous ID can no longer resolve to this session) and saves the new one. An attacker who obtained the pre-auth session ID cannot carry it into the authenticated session — the classic **session-fixation** mitigation.

The SessionStore protocol

All backends implement the `SessionStore` protocol:

```

class SessionStore(Protocol):
    async def get(
        self, session_id: str
    ) -> dict[str, Any] | None: ...

    async def save(

```

```

        self, session_id: str,
        data: dict[str, Any],
        ttl: int,
    ) -> None: ...

    async def delete(self, session_id: str) -> None: ...

    async def exists(self, session_id: str) -> bool: ...

```

Two adapters ship out of the box:

Adapter	Module	Notes
<code>InMemorySessionStore</code>	<code>pyfly.session.adapters.memory</code>	<code>asyncio.Lock</code> -guarded; single-process only; data lost on restart
<code>RedisSessionStore</code>	<code>pyfly.session.adapters.redis</code>	JSON-serialized; keys prefixed <code>pyfly:session:</code> ; TTL managed by Redis

SessionFilter

`SessionFilter` is an `OncePerRequestFilter` ordered at `HIGHEST_PRECEDENCE + 150`. It bookends every request:

1. Reads the session cookie (`PYFLY_SESSION` by default).
2. Loads the session from the store, or creates a new one.
3. Attaches it to `request.state.session`.
4. Calls `call_next(request)` — the rest of the filter chain and the handler run.
5. On the way back, persists a modified or new session, deletes an invalidated one, and re-issues the cookie with a rolling `max_age` (the TTL slides forward on every request).

Cookie attributes set by `SessionFilter`:

Attribute	Value	Reason
<code>httponly</code>	<code>True</code>	Prevents JavaScript access; XSS mitigation
<code>samesite</code>	<code>lax</code>	Blocks most cross-site request forgery flows
<code>secure</code>	configurable	Set <code>True</code> in production (HTTPS only)
<code>max_age</code>	<code>ttl</code>	Rolling expiry

Redis sessions in production

`lumen/config/session_config.py`

```
import redis.asyncio as aioredis

from pyfly.container import bean, configuration
from pyfly.session import SessionFilter
from pyfly.session.adapters.redis import RedisSessionStore

@configuration
class SessionConfig:

    @bean
    def session_store(self) -> RedisSessionStore:
        client = aioredis.from_url(
            "redis://localhost:6379/0"
        )
        return RedisSessionStore(client=client)

    @bean
    def session_filter(
        self,
        store: RedisSessionStore,
    ) -> SessionFilter:
```

```

return SessionFilter(
    store=store,
    cookie_name="LUMEN_SESSION",
    ttl=1800,
    secure=True,
)

```

Listing 14.10 — Redis session store

...

How it works. `RedisSessionStore.save` JSON-serialises the session dictionary (including dataclass attributes such as `SecurityContext`, round-tripped via an allowlisted type-tag mechanism) and calls `client.set(key, raw, ex=ttl)`. The TTL is managed entirely by Redis, so expired sessions disappear server-side with zero cleanup overhead. Keys use the prefix `pyfly:session:` for namespace isolation. On read, only types on the allowlist are deserialised, eliminating arbitrary-object instantiation risks.

💡 AUTO-CONFIGURATION

Add `pyfly.session.enabled: true` and `pyfly.session.store: redis` to `pyfly.yaml`. PyFly auto-configures `RedisSessionStore` (when `redis.asyncio` is installed) and `SessionFilter` for you. The `redis.url` defaults to `redis://localhost:6379/0`; override with `pyfly.session.redis.url`.

🍃 SPRING PARITY

`SessionFilter` mirrors Spring Session's `SessionRepositoryFilter`. `HttpSession` mirrors `javax.servlet.http.HttpSession` (or `jakarta.servlet.http.HttpSession` in Boot 3). `InMemorySessionStore` is equivalent to Spring Session's `MapSessionRepository`; `RedisSessionStore` is equivalent to `RedisSessionRepository`. The cookie attributes (`HttpOnly` , `SameSite=Lax` , rolling `Max-Age`) match Spring Session's defaults exactly.

External identity (IDP)

The problem with managing identity in-house

Lumen currently stores credentials in its own database, which means Lumen must implement password resets, MFA, email verification, account lockout, GDPR deletion, social login, and SSO — undifferentiated work that every service eventually needs. The industry answer is to delegate identity to a dedicated provider: Keycloak for on-premises, AWS Cognito for AWS-native stacks, Azure AD for Microsoft environments.

PyFly's `IdpAdapter` port makes that delegation pluggable behind a single interface. Swap the adapter and the business layer never knows.

IdpAdapter — the port

Every adapter must satisfy the `IdpAdapter` protocol:

```
class IdpAdapter(Protocol):
    name: str

    # User management
    async def create_user(
        self, user: IdpUser, password: str
    ) -> IdpUser: ...
```



```

async def get_user(self, user_id: str) -> IdpUser | None: ...
async def find_by_username(
    self, username: str
) -> IdpUser | None: ...
async def update_user(self, user: IdpUser) -> IdpUser: ...
async def delete_user(self, user_id: str) -> bool: ...
async def list_users(self, *, limit: int = 100) -> list[IdpUser]: ...

# Authentication
async def login(
    self, request: LoginRequest
) -> AuthResult: ...
async def logout(self, access_token: str) -> bool: ...
async def refresh(
    self, refresh_token: str
) -> AuthResult: ...
async def introspect(
    self, access_token: str
) -> SessionIntrospection: ...

# Password / MFA
async def change_password(
    self, request: PasswordChangeRequest
) -> bool: ...
async def reset_password(self, user_id: str) -> str: ...

# Roles
async def assign_role(
    self, user_id: str, role: str
) -> bool: ...
async def revoke_role(
    self, user_id: str, role: str
) -> bool: ...
async def list_roles(self) -> list[IdpRole]: ...

```

Key DTOs:

Class	Purpose
<code>IdpUser</code>	User record: <code>id</code> , <code>username</code> , <code>email</code> , <code>roles</code> , <code>attributes</code> , ...
<code>LoginRequest</code>	<code>username</code> , <code>password</code> , <code>mfa_code</code> (optional)
<code>AuthResult</code>	<code>user</code> , <code>access_token</code> , <code>refresh_token</code> , <code>expires_in</code> , <code>token_type</code>
<code>SessionIntrospection</code>	<code>active</code> , <code>user_id</code> , <code>username</code> , <code>scopes</code> , <code>expires_at</code>
<code>PasswordChangeRequest</code>	<code>user_id</code> , <code>old_password</code> , <code>new_password</code>
<code>IdpRole</code>	<code>name</code> , <code>description</code> , <code>scopes</code>

Keycloak adapter

`lumen/config/idp_config.py`

```

from pyfly.container import bean, configuration
from pyfly.idp import IdpAdapter, KeycloakIdpAdapter

@configuration
class IdpConfig:

    @bean
    def idp_adapter(self) -> IdpAdapter:
        return KeycloakIdpAdapter(
            base_url="https://keycloak.example.com",
            realm="lumen",
            client_id="lumen-backend",
            client_secret="${KEYCLOAK_SECRET}",
            verify_ssl=True,
        )

```

Listing 14.11 — Wiring the Keycloak adapter

...

How it works. `KeycloakIdpAdapter` communicates with Keycloak's Admin REST API (`/admin/realms/{realm}/users`) and token endpoint (`/realms/{realm}/protocol/openid-connect/token`) via `httpx` . It caches the `client_credentials` admin token internally and re-fetches it within a ten-second safety margin of expiry — Keycloak's default client-credentials TTL is 60 s, so without this cache every admin call would require two network round trips.

Using the IDP in a service

`lumen/core/services/auth/idp_auth_service.py`

```
from pyfly.container import service
from pyfly.idp import IdpAdapter, IdpUser, LoginRequest
from pyfly.kernel.exceptions import UnauthorizedException

@service
class IdpAuthService:

    def __init__(self, idp: IdpAdapter) -> None:
        self._idp = idp

    async def register(
        self,
        username: str,
        email: str,
        password: str,
        role: str = "USER",
    ) -> str:
        user = IdpUser(
            username=username,
            email=email,
            roles=[role],
        )
```

```

        created = await self._idp.create_user(user, password)
        result = await self._idp.login(
            LoginRequest(
                username=username, password=password
            )
        )
        return result.access_token

    async def login(
        self, username: str, password: str
    ) -> str:
        try:
            result = await self._idp.login(
                LoginRequest(
                    username=username, password=password
                )
            )
        except PermissionError as exc:
            raise UnauthorizedException(
                "Invalid credentials",
                code="INVALID_CREDENTIALS",
            ) from exc
        return result.access_token

    async def introspect(self, token: str) -> dict:
        info = await self._idp.introspect(token)
        return {
            "active": info.active,
            "user_id": info.user_id,
            "username": info.username,
            "scopes": info.scopes,
        }

```

Listing 14.12 — Using IdpAdapter in the auth service

...

How it works. `IdpAuthService` depends only on `IdpAdapter` — the DI container resolves the concrete `KeycloakIdpAdapter` at startup. The service layer never imports Keycloak, Cognito, or Azure-specific code. Switch provider by changing one line in `IdpConfig`; the service is untouched.

Auto-configuration and the built-in HTTP routes

Enable the IDP subsystem in `pyfly.yaml` and PyFly wires the adapter and a ready-made REST controller automatically:

`lumen/resources/pyfly.yaml`

```
pyfly:
  idp:
    enabled: true
    provider: keycloak
    keycloak:
      base-url: https://keycloak.example.com
      realm: lumen
      client-id: lumen-backend
      client-secret: "${KEYCLOAK_SECRET}"
```

Listing 14.13 — IDP auto-configuration

...

provider value	Adapter
<code>internal-db</code>	<code>InternalDbIdpAdapter</code> (bcrypt in-memory store)
<code>keycloak</code>	<code>KeycloakIdpAdapter</code>
<code>cognito</code> / <code>aws-cognito</code>	<code>AwsCognitoIdpAdapter</code>
<code>azure-ad</code> / <code>azuread</code> / <code>entra</code>	<code>AzureAdIdpAdapter</code>

When Starlette is present, `IdpAutoConfiguration` also registers an `IdpController` bean that exposes the full IDP API under `/idp`:

Route	Method	Description
<code>/idp/login</code>	POST	Authenticate (username + password + optional MFA)
<code>/idp/refresh</code>	POST	Refresh an access token
<code>/idp/logout</code>	POST	Revoke a token
<code>/idp/introspect</code>	POST	Inspect an active session
<code>/idp/admin/users</code>	POST	Create a user
<code>/idp/admin/users</code>	GET	List users
<code>/idp/admin/users/{user_id}</code>	GET / DELETE	Get or delete a user
<code>/idp/admin/users/{user_id}/roles/{role}</code>	POST / DELETE	Assign or revoke a role
<code>/idp/admin/roles</code>	GET	List all roles

💡 CUSTOM ADAPTERS

Any class that satisfies the `IdpAdapter` Protocol can be wired in as the `IdpAdapter` bean. Register it in a `@configuration` class and PyFly's `@conditional_on_missing_bean` skips auto-configuration entirely. This is the standard extension point for on-premises LDAP, in-house SSO, or test-double adapters.

 SPRING PARITY

`IdpAdapter` is PyFly's equivalent of Spring Security's `UserService` + `AuthenticationProvider` combination. `IdpUser` maps to `UserDetails`; `AuthResult` maps to the `Authentication` object returned by `AuthenticationManager.authenticate()`. `KeycloakIdpAdapter` plays the role of the Keycloak Spring Security adapter.

Putting it together — Lumen's auth layer

The listing below shows the complete wiring: IDP adapter, JWT filter, URL-level rules, and a Redis session store for the admin dashboard.

`lumen/config/security_full.py`

```
from pyfly.container import bean, configuration
from pyfly.idp import IdpAdapter, KeycloakIdpAdapter
from pyfly.security.http_security import HttpSecurity

@configuration
class LumenSecurityConfig:

    @bean
    def idp_adapter(self) -> IdpAdapter:
        return KeycloakIdpAdapter(
            base_url="https://keycloak.example.com",
            realm="lumen",
            client_id="lumen-backend",
            client_secret="${KEYCLOAK_SECRET}",
        )

    @bean
    def http_security_filter(self):
```

```

hs = HttpSecurity()
hs.authorize_requests() \
    .request_matchers(
        "/idp/login", "/idp/refresh",
        "/docs", "/openapi.json",
    ).permit_all() \
    .request_matchers(
        "/idp/admin/**"
    ).has_role("ADMIN") \
    .request_matchers(
        "/api/v1/wallets/**"
    ).authenticated() \
    .any_request().permit_all()
return hs.build()

```

Listing 14.14 — Full security configuration

:::

With `pyfly.security.enabled=true`, `pyfly.session.enabled=true`, and `pyfly.session.store=redis` in `pyfly.yaml`, auto-configuration handles `JWTService`, `BcryptPasswordEncoder`, `SessionFilter`, and `RedisSessionStore`. The `@configuration` class above provides only what auto-configuration cannot infer: the Keycloak coordinates and the URL-level policy.

✓ What you built

This chapter opened Part V by closing Lumen's open front door. You:

- Used `JWTService` to issue signed tokens at login and to decode them back into a `SecurityContext` on every subsequent request. The `exp` claim is mandatory — `encode()` auto-adds it using a Unix timestamp so you never need to import `datetime`; tokens without `exp` are rejected at the boundary.

- Added `SecurityMiddleware` to the Starlette application so every request carries a populated `SecurityContext` by the time it reaches a handler.
- Declared URL-level policy with the `HttpSecurity` builder — a fluent DSL that produces an `HttpSecurityFilter` evaluated before the route dispatcher, covering Lumen's real `/api/v1/wallets/**` tree and the IDP admin routes.
- Protected Lumen's real wallet endpoints in `WalletController` with `@secure`, specifying roles, permissions, or full security expressions. The seven routes — `open_wallet`, `deposit`, `withdraw`, `list_wallets` (paged list), `list_rich_wallets` (filtered list), `wallet_balance`, and `wallet_detail` — each carry the minimal guard: `USER+ADMIN` for most, `wallet:deposit` permission for mutations, `ADMIN-only` for the full detail view. Authorization failures raise `SecurityException` (401, code `AUTH_REQUIRED`) for unauthenticated callers and `ForbiddenException` (403, code `FORBIDDEN`) for authenticated callers who lack the required role or permission.
- Hashed passwords with `BcryptPasswordEncoder`, the default adapter for the `PasswordEncoder` protocol. The cost factor is tunable; the stored hash is self-describing; verification is timing-safe.
- Managed server-side sessions with `HttpSession` and the pluggable `SessionStore` protocol. In development, `InMemorySessionStore` requires no dependencies; in production, `RedisSessionStore` serialises to JSON and lets Redis manage TTL. The `SessionFilter` rolls the cookie TTL on every request and deletes it on invalidation.
- Delegated identity to an external provider via the `IdpAdapter` port and the `KeycloakIdpAdapter` implementation. The auto-configured `IdpController` exposes login, refresh, logout, introspect, and admin user management under `/idp` with no extra code.

Try it yourself

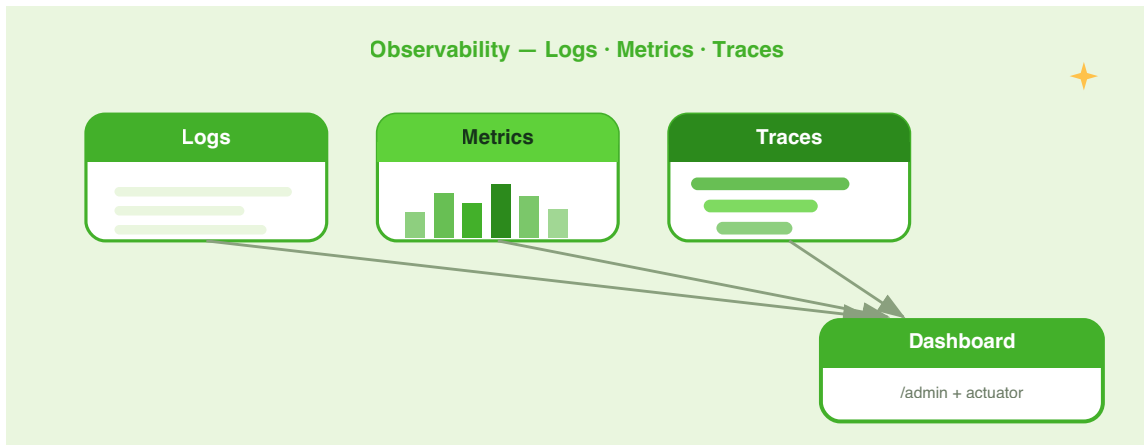
Exercise 1 — Role hierarchy. Lumen currently treats `ADMIN` and `USER` as independent roles. Add a "super-user" role `SUPER` that implicitly holds every `ADMIN` privilege. Implement a `RoleHierarchy` wrapper that pre-expands roles before storing them in `SecurityContext`, and update `_permissions_for` in Listing 14.1 so that a `SUPER`-role token passes every `@secure(roles=["ADMIN"])` check without explicitly carrying the `ADMIN` role. Write a unit test that creates a `SecurityContext(roles=["SUPER"])` after expansion and asserts `has_any_role(["ADMIN"])` returns `True`.

Exercise 2 — Session concurrency control. Lumen's admin users must not be logged in from more than two devices at the same time (a common financial compliance requirement). Enable `pyfly.session.concurrency.enabled: true` with `max-sessions: 2` and `strategy: evict-oldest` in `pyfly.yaml`. Write an integration test using `InMemorySessionStore` that creates three sessions for the same `user_id`, calls the `SessionConcurrencyController`, and asserts that the oldest session has been evicted while the two newest remain valid.

Exercise 3 — Custom IDP adapter. Lumen's staging environment uses a homegrown OAuth2 server. Implement a `StagingIdpAdapter` that satisfies `IdpAdapter`, backed by an in-memory user dictionary. The `login` method should issue a signed JWT using a `JWTService` injected through the constructor. Wire it as the `IdpAdapter` bean in a `@configuration` class tagged `@conditional_on_property("lumen.env", having_value="staging")` and confirm that the production `KeycloakIdpAdapter` bean is not created when that property is active.

CHAPTER 15

Observability & the Admin Dashboard



In Chapter 13 you surrounded Lumen's hot paths with caches and wrapped outbound calls in circuit breakers. In Chapter 14 you secured every endpoint with JWT authentication, role guards, and server-side sessions.

Lumen is now fast and safe — but it is still a black box. When a wallet deposit takes three seconds in production you need to know *where* those seconds went. When a downstream payment service degrades you need a dashboard that lights up red *before* your on-call engineer gets paged. When a compliance auditor asks why a particular transfer was rejected you need structured log records with enough context to reconstruct the decision.

This chapter adds eyes and ears to Lumen. The three pillars of observability answer three complementary questions about a running system:

Pillar	Question	PyFly module
Logging	"What happened, and in what context?"	<code>pyfly.logging</code>
Metrics	"How much? How fast? How often?"	<code>pyfly.observability.metrics</code>
Tracing	"What path did this request take?"	<code>pyfly.observability.tracing</code>

On top of those pillars sits the **Actuator** — production management endpoints that expose health, bean wiring, environment, and live logger levels — and the **Admin Dashboard**, an embedded browser UI that ties everything together in a single pane of glass.

Finally, you will see how **Aspect-Oriented Programming** (AOP) applies logging and metrics to every service method declaratively, without touching the methods themselves.

By the end of the chapter Lumen will produce structured JSON logs with correlation IDs and automatic PII masking, emit Prometheus metrics scraped by any standard collector, propagate OpenTelemetry trace spans across service boundaries, answer Kubernetes liveness and readiness probes, and display all of the above in a zero-configuration dashboard reachable at `/admin`.

Structured logging & PII redaction

Why structured logging?

Traditional log lines look like this:

```
[INFO] Order ord-123 created for customer acme corp (email: sales@acme.com)
```

Searching for `ord-123` in Elasticsearch works — until the format changes. And `sales@acme.com` landing in a log file may violate your data-protection policy without your team even noticing.

Structured logging replaces the interpolated string with an event name and explicit key-value pairs. The pairs render as JSON in production and as readable `key=value` in development. A log aggregation system ingests JSON natively; you query on `wallet_id` or `owner_id` as first-class fields, independent of message format.

get_logger

PyFly exposes a single factory function that returns a structured logger backed by `structlog` (when the `observability` extra is installed) or a zero-dependency stdlib shim otherwise. Both accept the same call signature:

`lumen/logging_demo.py`

```
from pyfly.logging import get_logger

logger = get_logger("lumen.wallet")

logger.info("wallet_opened", wallet_id="wlt-001", owner_id="usr-42")
logger.warning("balance_low", wallet_id="wlt-001", remaining=300)
logger.error(
    "deposit_rejected",
    wallet_id="wlt-001",
    reason="insufficient_funds",
)
```

Listing 15.1 — Structured logger usage

In development with `format: console` the output reads naturally:

```
10:30:00 [info    ] wallet_opened   wallet_id=wlt-001 owner_id=usr-42
10:30:01 [warning  ] balance_low    wallet_id=wlt-001 remaining=300
10:30:02 [error    ] deposit_rejected wallet_id=wlt-001 reason=insufficient_funds
```

In production with `format: json` every line is a self-contained JSON object:

```
{
  "event": "wallet_opened",
  "wallet_id": "wlt-001",
  "owner_id": "usr-42",
  "timestamp": "2026-06-07T10:30:00Z",
  "level": "info",
  "logger": "lumen.wallet"
}
```

Configure logging in `pyfly.yaml`:

```
pyfly:
  logging:
    level:
      root: INFO
      lumen.wallet: DEBUG
      sqlalchemy.engine: WARNING
    format: console          # console | json | logfmt
```

`level.<name>` overrides the root level for any logger whose name begins with that prefix. `sqlalchemy.engine: WARNING` silences query logs without touching your code. An environment variable `PYFLY_LOGGING_LEVEL_ROOT=WARNING` overrides the config key, which is useful for staging builds.

🔍 WHY NOT `STDLIB` LOGGING DIRECTLY?

`logging.getLogger("x").info("event", wallet_id="wlt-001")` raises `TypeError` — the `stdlib` rejects keyword arguments. `get_logger` guarantees the structured signature works regardless of which adapter is active.

Correlation IDs

Correlation IDs link every log line emitted during a single HTTP request. PyFly binds a `transaction_id` to the current async context automatically through `TransactionIdMiddleware`. Your handlers can bind additional fields — such as the authenticated user — so that those fields flow through all subsequent log calls without being passed explicitly:

lumen/wallet/handler.py

```

import structlog

from pyfly.logging import get_logger

logger = get_logger("lumen.wallet")

async def handle_deposit(wallet_id: str, amount: int, owner_id: str) -> dict:
    structlog.contextvars.bind_contextvars(
        wallet_id=wallet_id,
        owner_id=owner_id,
    )

    logger.info("deposit_started", amount=amount)
    # ... business logic ...
    result = {"wallet_id": wallet_id, "new_balance": 1350}
    logger.info("deposit_completed", new_balance=result["new_balance"])

    structlog.contextvars.unbind_contextvars("wallet_id", "owner_id")
    return result

```

Listing 15.2 — Binding correlation context

Every `logger.*` call inside `handle_deposit` — including calls deep in downstream service methods — automatically carries `wallet_id` and `owner_id` without any further plumbing.

PII redaction

PII redaction is enabled by default. Before any log record reaches an output handler, PyFly scans the rendered message for emails, credit-card numbers, IBANs, SSNs, JWTs, bearer tokens, and URL credentials. Detected patterns are replaced with `<EMAIL>`, `<CREDIT_CARD>`, and so on.

The regex engine ships with every install. The Presidio-backed NER engine — which also catches free-text names and addresses — is available via the `[pii]` extra and activates automatically when installed:

```
uv add "pyfly[observability,pii]"
python -m spacy download en_core_web_sm # lighter model; lg for higher recall
```

Configure redaction in `pyfly.yaml`:

```
pyfly:
  logging:
    redaction:
      enabled: true # default; set false to disable entirely
      engine: auto # regex | presidio | auto (presidio if installed)
      mask: placeholder # placeholder (<EMAIL>) | partial (*****@acme.com)
      deny-fields:
        - password
        - token
        - secret
      presidio:
        score-threshold: 0.6
        languages: [en, es]
```

`deny-fields` lists structured log field *names* whose values are unconditionally replaced with `<REDACTED>`. Use it for fields like `password` where you know the value is sensitive without inspecting the content.

🌿 SPRING PARITY

Spring Boot does not ship built-in PII redaction; teams integrate Logback `MaskingMessageConverter` or custom appenders manually. PyFly's redaction applies to *all* loggers — including third-party libraries — through a single `ProcessorFormatter` / `RedactionFilter` installed on the root handler. No per-library configuration is needed.

Rolling file appender

When logs go to a file rather than stdout, configure rotation in `pyfly.yaml`:

```
pyfly:
  logging:
    file:
      name: lumen.log
      path: ./logs
    rolling:
      max-size: 50MB
      max-history: 14
```

PyFly writes to `./logs/lumen.log` and rotates at 50 MB, keeping 14 rotated files before discarding the oldest. The same PII redaction pass applies to file output.

Metrics

The MetricsRegistry

`MetricsRegistry` is a thin wrapper around `prometheus_client` that guarantees each metric name is registered only once. Duplicate calls to `counter()` or `histogram()` with the same name return the existing metric rather than raising a `ValueError`. Inject it from the DI container (auto-configured when `prometheus_client` is installed) or create it manually:

```
lumen/observability/metrics.py
```

```
from pyfly.observability import MetricsRegistry

registry = MetricsRegistry()

deposits_total = registry.counter(
    name="lumen.deposits.total",
```

```

        description="Deposit operations completed",
        labels=["status"],
    )

    deposit_duration = registry.histogram(
        name="lumen.deposits.duration",
        description="Deposit processing time in seconds",
        labels=["status"],
        buckets=(0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0),
    )

```

Listing 15.3 — Creating metrics

`counter()` and `histogram()` return native `prometheus_client.Counter` and `prometheus_client.Histogram` objects, so every Prometheus ecosystem tool — Grafana dashboards, alerting rules, recording rules — works without modification.

Method	Returns	Use for
<code>registry.counter(name, description, labels)</code>	<code>Counter</code>	Monotonically increasing counts
<code>registry.histogram(name, description, labels, buckets)</code>	<code>Histogram</code>	Durations, sizes, latency percentiles
<code>registry.counter(...)</code> called again	Same <code>Counter</code>	Safe deduplication

@timed — automatic duration histogram

`@timed` records how long an async or sync function takes to run, using a labeled histogram. It works on any callable and automatically adds `class`, `method`, and `exception` labels:

```
lumen/core/services/wallets/deposit_funds_handler.py
```

```

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
from pyfly.observability import MetricsRegistry, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

registry = MetricsRegistry()

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """Credit funds to an existing wallet; returns the new balance."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @timed(registry, "lumen.deposit.duration", "Deposit handler latency")
    @transactional()
    async def do_handle(self, command: DepositFunds) -> int:
        entity = await self._repository.find_by_id(command.wallet_id)

```

```

if entity is None:
    raise AggregateNotFound("Wallet", command.wallet_id)
wallet = to_aggregate(entity)
wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
await self._repository.upsert(to_entity(wallet))
await publish_domain_events(self._events, wallet.clear_events())
return wallet.balance.amount

```

Listing 15.4 — `@timed` on `DepositFundsHandler`

The decorator captures `start = time.perf_counter()`, calls the function, and observes the elapsed time in the `finally` block. The `exception` label is `"none"` on success and the exception type name on failure, so you can split latency by outcome in Grafana. The `class` and `method` labels are derived automatically from the function's qualified name.

Histogram names follow Micrometer dot.case convention: `"lumen.deposit.duration"` becomes `lumen_deposit_duration_seconds` in Prometheus, with a `_seconds` suffix added if absent.

@counted — automatic invocation counter

`@counted` increments a counter on every function call. Lumen's `GetBalanceHandler` is a natural fit — every balance read increments the counter, tagged by outcome:

`lumen/core/services/wallets/get_balance_handler.py`

```

from pyfly.container import service
from pyfly.cqrs import QueryHandler, query_handler
from pyfly.observability import MetricsRegistry, counted

from lumen.core.mappers.wallet_mapper import entity_to_balance_dto
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.interfaces.dtos.v1.balance_dto import BalanceDto
from lumen.models.repositories.wallet_repository import WalletRepository

```

```

registry = MetricsRegistry()

@query_handler
@service
class GetBalanceHandler(QueryHandler[GetBalance, BalanceDto | None]):

    def __init__(self, repository: WalletRepository) -> None:
        super().__init__()
        self._repository = repository

    @counted(registry, "lumen.balance.reads", "Balance queries served")
    async def do_handle(self, query: GetBalance) -> BalanceDto | None:
        entity = await self._repository.find_by_id(query.wallet_id)
        return entity_to_balance_dto(entity) if entity is not None else None

```

Listing 15.5 — @counted on GetBalanceHandler

On success the counter is incremented with labels `class="GetBalanceHandler"`, `method="do_handle"`, `result="success"`, and `exception="none"`. On failure it uses `result="failure"` and `exception=<TypeName>`, then re-raises the original exception. The counter name receives a `_total` suffix in Prometheus automatically, per the naming convention.

You can stack both decorators on the same method. The following listing shows Lumen's `WithdrawFundsHandler` timed and counted simultaneously:

`lumen/core/services/wallets/withdraw_funds_handler.py`

```

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher

```

```

from pyfly.observability import MetricsRegistry, counted, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

registry = MetricsRegistry()

@command_handler
@service
class WithdrawFundsHandler(CommandHandler[WithdrawFunds, int]):
    """Debit funds from a wallet; returns the new balance in minor units."""

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @timed(registry, "lumen.withdrawal.duration", "Withdrawal latency")
    @counted(registry, "lumen.withdrawals", "Withdrawal attempts")
    @transactional()
    async def do_handle(self, command: WithdrawFunds) -> int:
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:
            raise AggregateNotFound("Wallet", command.wallet_id)
        wallet = to_aggregate(entity)
        wallet.withdraw(Money(amount=command.amount,
            currency=wallet.currency))
        await self._repository.upsert(to_entity(wallet))

```

```

await publish_domain_events(self._events, wallet.clear_events())
return wallet.balance.amount

```

Listing 15.6 — Stacking `@timed` and `@counted`

`amount` is an `int` in minor units (e.g. 1050 = €10.50) — the `Money` value object enforces the type. Each invocation produces both a histogram observation and a counter increment.

Prometheus scrape endpoint

The actuator (covered in the next section) exposes the metrics registry for scraping. When the actuator is enabled and `prometheus_client` is installed, two endpoints are mounted automatically with no additional code:

- `GET /actuator/metrics` — Micrometer-compatible JSON listing all metric names
- `GET /actuator/prometheus` — standard text-exposition format for scraping

Point your Prometheus `scrape_configs` at `/actuator/prometheus` and all `MetricsRegistry` metrics appear alongside built-in process metrics (CPU, memory, threads, GC).

🍃 SPRING PARITY

`MetricsRegistry` mirrors Spring's `MeterRegistry` from Micrometer. `@timed` corresponds to Spring's `@Timed` and `@counted` to `@Counted`. Dot.case names (`lumen.deposit.duration`) match the Micrometer convention that Spring Boot Actuator's `/actuator/metrics` also exposes.

Distributed tracing

@span — OpenTelemetry span decorator

`@span` wraps an async or sync function in an OpenTelemetry span. Each span is a timed, named unit of work. Spans nest automatically through OpenTelemetry's context propagation, so a `@span`-decorated function called from inside another `@span`-decorated function produces a parent-child relationship in your trace viewer:

`lumen/wallet/service.py`

```
from pyfly.observability import span

class DepositFundsHandler:

    @span("deposit-funds")
    async def do_handle(self, command):
        balance = await self._fetch_wallet(command.wallet_id)
        await self._persist_deposit(command.wallet_id, command.amount)
        return balance + command.amount

    @span("fetch-wallet")
    async def _fetch_wallet(self, wallet_id: str) -> int:
        # ... repository.find(wallet_id) ...
        return 1000

    @span("persist-deposit")
    async def _persist_deposit(self, wallet_id: str, amount: int) -> None:
        # ... repository.add(wallet) ...
        pass
```

Listing 15.7 — @span on CQRS handler methods

In a trace viewer this appears as:


```
deposit-funds [120 ms]
+-- fetch-wallet [15 ms]
+-- persist-deposit [90 ms]
```

`@span` creates a tracer named "pyfly" via `trace.get_tracer("pyfly")`. When the decorated function raises, the span automatically records the error: it sets status to `ERROR`, calls `current_span.record_exception(exc)`, then re-raises so callers see the original exception unmodified. Sync functions are supported identically — no `await` on the decorated side.

OpenTelemetry auto-configuration

PyFly wires up a `TracerProvider` with a `BatchSpanProcessor` automatically when `opentelemetry-api` and `opentelemetry-sdk` are installed:

```
uv add opentelemetry-api opentelemetry-sdk opentelemetry-exporter-otlp
```

Configure the exporter in `pyfly.yaml`:

```
pyfly:
  observability:
    tracing:
      enabled: true
      service-name: "${pyfly.app.name}"
      exporter: otlp
      otlp:
        endpoint: "http://localhost:4318"
```

Exporter selection rules:

- `exporter: otlp` — uses OTLP/HTTP; requires `opentelemetry-exporter-otlp`
- `exporter: console` — prints spans to stdout; useful for local debugging
- `exporter: none` — records spans but discards them (useful in tests)

- Unset — auto-selects OTLP if `pyfly.observability.tracing.otlp.endpoint` or the `OTEL_EXPORTER_OTLP_ENDPOINT` environment variable is configured; otherwise logs a single info line and discards spans silently

💡 CUSTOM TRACERPROVIDER

If you need gRPC export or a non-standard SDK configuration, register your own `TracerProvider` before PyFly starts — `trace.set_tracer_provider(...)`. PyFly detects an existing global provider and skips auto-configuration.

Context propagation — inbound and outbound

Spans stay correlated *within* a process automatically. Keeping the same trace *across* multiple services requires extracting the upstream trace context from inbound HTTP headers and injecting the current context into every outbound call.

PyFly handles both ends without any per-handler code.

Inbound — TracingFilter:

`TracingFilter` is wired into Lumen's filter chain immediately after `CorrelationFilter` by `create_app()`. For each request it reads the W3C `traceparent` header, opens a SERVER span as a child of the upstream context, and keeps that span active for the lifetime of the request. Every `@span` created during the request — and every log line — belongs to the caller's distributed trace:

```
# Simplified view of what TracingFilter does per-request:
parent = extract_context(request.headers) # parse W3C traceparent
with tracer.start_as_current_span(
    f"{request.method} {request.url.path}",
    context=parent,
    kind=trace.SpanKind.SERVER,
) as span:
```

```
response = await call_next(request)
span.set_attribute("http.response.status_code", response.status_code)
```

When OpenTelemetry is not installed, the filter is a transparent pass-through.

Outbound — HttpxClientAdapter:

`HttpxClientAdapter` calls `inject_headers()` on every outbound request so downstream services can continue the same trace:

`lumen/client/inventory_client.py`

```
from pyfly.client.adapters.httpx_adapter import HttpxClientAdapter

class InventoryClient:

    def __init__(self) -> None:
        self._http = HttpxClientAdapter(
            base_url="http://inventory-service:8080"
        )

    async def check_stock(self, sku: str) -> dict:
        # The active traceparent is injected automatically into
        # the outbound request headers – no manual plumbing required.
        resp = await self._http.request("GET", f"/skus/{sku}")
        return resp.json()
```

Listing 15.8 — Trace propagation

Logs carry trace_id and span_id:

`StructlogAdapter` registers a processor that stamps the active span's IDs on every log record. No code change required — any `get_logger(...)` call inside an active span gains `trace_id` and `span_id` fields automatically:

```
{
  "event": "deposit_completed",
  "wallet_id": "wlt-001",
  "new_balance": 1350,
  "trace_id": "1a4b3145ed8f2dd11172ee3584123f4a",
  "span_id": "d2a62aaa81b0ad66",
  "timestamp": "2026-06-07T10:30:00Z",
  "level": "info",
  "logger": "lumen.wallet"
}
```

With `trace_id` in every log record you can jump from a Grafana Loki search for `wallet_id=wlt-001` directly to the correlated Tempo trace view, and from there to the Prometheus latency charts for that time window — all three pillars joined on a single identifier.

The low-level propagation helpers are available if you ever need them directly:

```
from pyfly.observability.propagation import (
    extract_context,    # parse traceparent from inbound headers
    inject_headers,     # write traceparent into outbound headers
    current_trace_ids,  # -> (trace_id, span_id) hex, or None
    has_otel,          # True if opentelemetry is importable
)
```

Health checks & the Actuator

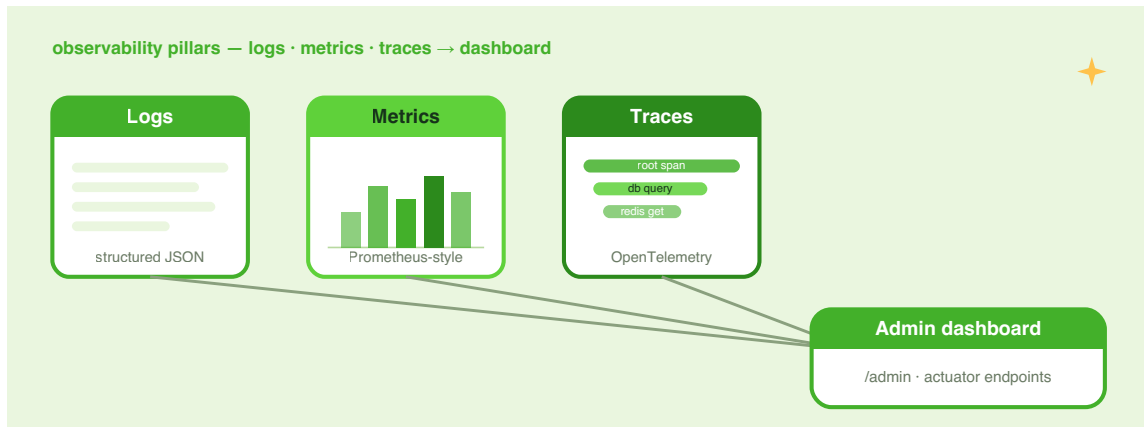


Figure 15.1 — The Actuator exposes health, beans, loggers, and Prometheus metrics over HTTP. Kubernetes liveness and readiness probes hit the dedicated sub-paths.

The **Actuator** gives Kubernetes and your ops tooling a stable contract: a set of management endpoints that expose health, bean wiring, environment state, and metric scrapers. You configure it once and every tool from `kubectl` to Grafana can consume it without custom code.

Enabling the Actuator

Pass `actuator_enabled=True` to `create_app()`, or set the flag in `pyfly.yaml`:

```
pyfly:
  web:
    actuator:
      enabled: true
  app:
    name: lumen
    version: 1.0.0
    description: Lumen wallet service
```

When enabled, `create_app()` automatically scans the DI container for `HealthIndicator` beans, creates a `HealthAggregator`, instantiates all built-in endpoints, and mounts them at `/actuator/*`.

Built-in endpoints

Endpoint	Method	Description
<code>/actuator</code>	GET	HAL-style index of all enabled endpoints
<code>/actuator/health</code>	GET	Aggregated status: <code>UP</code> (200) or <code>DOWN</code> (503)
<code>/actuator/health/liveness</code>	GET	Kubernetes liveness probe sub-path
<code>/actuator/health/readiness</code>	GET	Kubernetes readiness probe sub-path
<code>/actuator/beans</code>	GET	All registered DI beans with stereotype and scope
<code>/actuator/env</code>	GET	Active configuration profiles
<code>/actuator/info</code>	GET	Application name, version, description
<code>/actuator/loggers</code>	GET, POST	List loggers; change levels at runtime
<code>/actuator/metrics</code>	GET	Micrometer-compatible JSON metric names
<code>/actuator/prometheus</code>	GET	Prometheus text-exposition scrape target
<code>/actuator/heapdump</code>	GET	Snapshot of all live threads and stack traces
<code>/actuator/refresh</code>	POST	Evict refresh-scoped beans; re-bind config

Custom HealthIndicator

Any `@component` bean with an `async def health(self) -> HealthStatus` method is automatically discovered and registered as a health indicator. Lumen's `WalletRepository` is a good candidate — `count()` issues a lightweight `SELECT COUNT(*)` against the live database session without mutating any data:

`lumen/health/indicators.py`

```
from pyfly.actuator import HealthStatus
from pyfly.container import component

from lumen.models.repositories.wallet_repository import WalletRepository

@component
class WalletRepositoryHealthIndicator:
    """Checks the wallet store is reachable with a lightweight probe."""

    def __init__(self, repository: WalletRepository) -> None:
        self._repository = repository

    async def health(self) -> HealthStatus:
        try:
            # count() issues SELECT COUNT(*) – fast, read-only probe.
            total = await self._repository.count()
            return HealthStatus(
                status="UP",
                details={"store": "wallet-repository", "rows": total},
            )
        except Exception as exc:
            return HealthStatus(
                status="DOWN",
                details={"error": str(exc)},
            )

@component
```

```

class DatabaseHealthIndicator:
    """Checks database connectivity via a lightweight SELECT 1."""

    def __init__(self, session_factory) -> None:
        self._factory = session_factory

    async def health(self) -> HealthStatus:
        try:
            async with self._factory() as session:
                await session.execute("SELECT 1")
            return HealthStatus(
                status="UP",
                details={"type": "postgresql", "pool_active": 3},
            )
        except Exception as exc:
            return HealthStatus(
                status="DOWN",
                details={"error": str(exc)},
            )

```

Listing 15.9 — HealthIndicator beans

`HealthStatus.status` accepts four values: `"UP"`, `"DOWN"`, `"OUT_OF_SERVICE"`, or `"UNKNOWN"`. The aggregator applies a severity ordering (`DOWN > OUT_OF_SERVICE > UP > UNKNOWN`) and returns the worst-case status across all indicators. If any indicator's `health()` method raises, that indicator is treated as `"DOWN"` with `details={"error": "check failed"}`; the exception is logged but does not crash the health endpoint.

A healthy response looks like:

```

{
  "status": "UP",
  "components": {
    "WalletRepositoryHealthIndicator": {
      "status": "UP",
      "details": {"store": "wallet-repository", "rows": 42}
    },

```



```

"DatabaseHealthIndicator": {
  "status": "UP",
  "details": {"type": "postgresql", "pool_active": 3}
}
}
}

```

Changing log levels at runtime

The loggers endpoint lets you inspect and change log levels without restarting Lumen — invaluable when a production incident needs DEBUG output for exactly one package:

```

# List all loggers with configured and effective levels
curl http://localhost:8080/actuator/loggers

# Enable DEBUG for the wallet module – takes effect immediately
curl -X POST http://localhost:8080/actuator/loggers/lumen.wallet \
-H "Content-Type: application/json" \
-d '{"configuredLevel": "DEBUG"}'

# Reset to inherit from parent
curl -X POST http://localhost:8080/actuator/loggers/lumen.wallet \
-H "Content-Type: application/json" \
-d '{"configuredLevel": null}'

```

The endpoint uses Spring Boot's level vocabulary (`OFF` , `ERROR` , `WARN` , `INFO` , `DEBUG` , `TRACE`) and is drop-in compatible with Spring Boot Actuator tooling.

Custom actuator endpoint

To expose a custom endpoint, implement the `ActuatorEndpoint` protocol and annotate the class with `@component` . PyFly discovers it during context startup and mounts it at `/actuator/{endpoint_id}` automatically:

```
lumen/actuator/git_info.py
```

```

from pyfly.container import component

@component
class GitInfoEndpoint:
    """Exposes build metadata at /actuator/git."""

    @property
    def endpoint_id(self) -> str:
        return "git"

    @property
    def enabled(self) -> bool:
        return True

    async def handle(self, context=None) -> dict:
        return {
            "branch": "main",
            "commit": {
                "id": "5c6f83b",
                "time": "2026-06-07T08:30:00Z",
            },
            "build": {
                "version": "1.0.0",
            },
        }

```

Listing 15.10 — Custom actuator endpoint

Kubernetes probe configuration

Point your pod spec at the dedicated liveness and readiness sub-paths so Kubernetes can make independent restart and traffic decisions:

```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080

```

```

initialDelaySeconds: 10
periodSeconds: 30
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10

```

The separate sub-paths let you group indicators independently — an in-flight migration that degrades readiness need not trigger a liveness restart and container recreation.

🌿 SPRING PARITY

PyFly's Actuator mirrors Spring Boot Actuator. `HealthIndicator`, `HealthStatus`, `HealthAggregator`, `ActuatorEndpoint`, and `ActuatorRegistry` correspond directly to their Spring counterparts. The loggers endpoint uses the same Spring Boot level vocabulary and `configuredLevel` / `effectiveLevel` response shape, making it compatible with Spring Boot Admin and Actuator-aware tooling out of the box. `MetricsAutoConfiguration` and `MetricsActuatorAutoConfiguration` mirror Spring Boot's Micrometer auto-configuration: when `prometheus_client` is installed, `/actuator/prometheus` appears without any manual wiring.

The Admin Dashboard

The **Admin Dashboard** is a zero-build, zero-dependency browser UI served directly from the `pyfly.admin` package. One configuration line enables it; navigate to `/admin` — no separate server, no `npm` build step.

Enabling the dashboard

```

pyfly:
  admin:

```

```

enabled: true
title: "Lumen Admin"
theme: auto # auto | light | dark
refresh_interval: 5000

```

The dashboard auto-discovers beans, health indicators, loggers, scheduled tasks, HTTP mappings, caches, CQRS handlers, sagas, and metrics from the running `ApplicationContext`. It presents them in **15 built-in views** with real-time Server-Sent Event (SSE) updates — no WebSocket, no polling loop in your code.

Built-in views

Dashboard section:

View	Description
Overview	App info, uptime, health badge, bean counts by stereotype
Health	Component status with color-coded UP / DOWN / UNKNOWN badges; live SSE

Application section:

View	Description
Beans	All DI beans with stereotype, scope, and dependency graph
Environment	Active profiles and masked environment variables
Configuration	Resolved config tree for all namespaces with source tracking
Loggers	Logger levels with runtime level-change UI; TRACE and OFF supported

Monitoring section:

View	Description
Metrics	CPU, memory, threads, GC, uptime; optional Prometheus metrics; live trend chart
Scheduled Tasks	All <code>@scheduled</code> tasks with cron expressions and status
HTTP Traces	Request/response traces with p50/p90/p95/p99 latency, error-rate bar
Log Viewer	Live-tail with level filters, search, and pause/resume

Infrastructure section:

View	Description
Mappings	All HTTP routes with handler, parameters, return type, and docstring
Caches	Adapter type, entry count, per-key eviction, bulk evict-all
CQRS	Command and query handlers with bus pipeline introspection
Transactions	Saga step DAGs and TCC participant coverage; in-flight count

Fleet section (server mode):

View	Description
Instances	All registered remote application instances with health status

Real-time SSE streams

The dashboard never polls the backend with `setInterval`. It opens a single `EventSource` connection and the server pushes events as they occur:

SSE endpoint	Event name	What it streams
<code>/admin/api/sse/health</code>	<code>health</code>	Status change whenever aggregate health changes
<code>/admin/api/sse/metrics</code>	<code>metrics</code>	Full metric-name list at each refresh interval
<code>/admin/api/sse/traces</code>	<code>trace</code>	Individual HTTP traces as they arrive
<code>/admin/api/sse/logfile</code>	<code>log</code>	New log records from the in-memory ring buffer
<code>/admin/api/sse/beans</code>	<code>beans</code>	Bean registry snapshot at each refresh interval

The log viewer ring buffer holds 2,000 records; the HTTP traces ring buffer holds 500. Admin and actuator paths (`/admin/*` , `/actuator/*`) are excluded from trace capture automatically so they do not pollute the trace panel.

Runtime logger management

The Loggers view uses the same `/admin/api/loggers/{name}` endpoint as the actuator. Click a logger row to open an inline level selector and submit — the new level takes effect immediately, and the UI re-fetches to confirm the change. The Reset button sends `null` to return the logger to `NOTSET` (inherit from parent).

Custom view extension

To add your own sidebar view, implement `AdminViewExtension` and annotate with `@component`. The dashboard discovers it at startup:

```
lumen/admin/deployment_view.py
```

```
from pyfly.container import component
```

```

@component
class DeploymentView:
    """Shows deployment metadata in the admin sidebar."""

    @property
    def view_id(self) -> str:
        return "deployments"

    @property
    def display_name(self) -> str:
        return "Deployments"

    @property
    def icon(self) -> str:
        return "upload-cloud"

    async def get_data(self, context=None) -> dict:
        return {
            "last_deploy": "2026-06-07T08:00:00Z",
            "version": "1.0.0",
            "environment": "production",
        }

```

Listing 15.11 — Custom admin view

`view_id` sets the sidebar URL fragment (`#deployments`), `display_name` appears in the sidebar menu, and `icon` maps to a Feather icon. `get_data()` is called by `GET /admin/api/views` and can query the DI container, a database, or any external source.

Security

Restrict dashboard access to operators in production:

```

pyfly:
  admin:
    enabled: true
    require_auth: true

```

allowed_roles:

- ADMIN
- OPS

When `require_auth: true`, every `/admin/api/*` route — data, mutation, SSE, and instance-registry endpoints — requires an authenticated principal whose roles overlap with `allowed_roles`. Unauthenticated requests receive `401`; authenticated users who lack every listed role receive `403`. The static SPA shell remains public so the dashboard can boot and display the error message.

Fleet monitoring — server mode

For a fleet of Lumen instances, run one dedicated admin-server and point every application instance at it:

```
# Admin server instance
pyfly:
  admin:
    enabled: true
  server:
    enabled: true
    poll_interval: 10000
  instances:
    - name: lumen-1
      url: http://lumen-1:8080
    - name: lumen-2
      url: http://lumen-2:8080
```

```
# Each application instance
pyfly:
  admin:
    enabled: true
  client:
    url: http://admin-server:8080
    auto_register: true
```


`StaticDiscovery` seeds the registry from the YAML list. `AdminClientRegistration` registers the instance on startup and deregisters on shutdown. HTTP calls use `httpx` when available and fall back to `urllib.request`; registration errors are silently swallowed so an unreachable admin server never aborts application startup.

🌿 SPRING PARITY

PyFly Admin maps directly to Spring Boot Admin. `server.enabled: true` replaces `@EnableAdminServer`. `client.url` replaces `spring.boot.admin.client.url`. The Vaadin/React frontend is replaced with a vanilla-JS SPA that requires no build tooling. SSE streams replace Spring Boot Admin's WebSocket notifications. The built-in Log Viewer replaces the Spring Boot Admin logfile viewer backed by `/actuator/logfile`; PyFly's ring buffer approach avoids the file-path configuration that Spring Boot Admin requires.

AOP for cross-cutting concerns

What is AOP?

Aspect-Oriented Programming separates cross-cutting concerns — logging, metrics, security, auditing — from business logic. Without AOP, every service method begins with `logger.info(...)` and ends with `metrics.increment(...)`. With AOP, you write that logic once in an `@aspect` class and apply it to every matching method via a pointcut expression — the methods themselves stay clean.

PyFly's AOP module ships five advice types:

Advice	Decorator	Runs
Before	<code>@before</code>	Before the target method
After returning	<code>@after_returning</code>	After the method succeeds
After throwing	<code>@after_throwing</code>	After the method raises
After (finally)	<code>@after</code>	Always, success or failure
Around	<code>@around</code>	Wraps the entire call; must call <code>jp.proceed()</code>

@aspect — declaring an aspect

`@aspect` marks a class as a PyFly aspect. The class is automatically registered in the DI container as a singleton and receives injected dependencies via `__init__`. No explicit base class is required:

`lumen/aspects/logging_aspect.py`

```

from pyfly.aop import aspect, before, after_returning, after_throwing,
JoinPoint
from pyfly.container.ordering import order
from pyfly.logging import get_logger

logger = get_logger("lumen.audit")

@aspect
@order(-50)
class AuditLoggingAspect:
    """Logs entry, exit, and failure for every service method."""

    @before("service.*.*")
    def log_entry(self, jp: JoinPoint) -> None:
        logger.info(
            "method_called",
            cls=type(jp.target).__name__,

```

```

        method=jp.method_name,
    )

    @after_returning("service.*.*")
    def log_return(self, jp: JoinPoint) -> None:
        logger.info(
            "method_returned",
            cls=type(jp.target).__name__,
            method=jp.method_name,
        )

    @after_throwing("service.*.*")
    def log_error(self, jp: JoinPoint) -> None:
        logger.error(
            "method_raised",
            cls=type(jp.target).__name__,
            method=jp.method_name,
            exc=type(jp.exception).__name__,
        )

```

Listing 15.12 — A logging aspect

The pointcut `"service.*.*"` matches every public method on every `@service`-stereotype bean. `*` matches exactly one dot-separated segment; `**` matches one or more. Partial globs are supported within a segment: `"service.*.do_handle"` matches all `do_handle` methods on all service-stereotype handlers.

Qualified names follow the pattern `"{stereotype}.{ClassName}.{method_name}"`, so `service.DepositFundsHandler.do_handle` uniquely identifies the `do_handle` method on `DepositFundsHandler`.

 @BEFORE HANDLERS MUST BE SYNCHRONOUS

`@before`, `@after_returning`, `@after_throwing`, and `@after` handlers are always called synchronously by the weaver. Only `@around` handlers can be async (and must `await jp.proceed()` when advising an async method).

@around — metrics without decorators

`@around` is the most powerful advice type. It wraps the entire method execution; call `await jp.proceed()` to invoke the original method (or the next advice in the chain) and add behaviour on either side:

lumen/aspects/metrics_aspect.py

```
import time

from pyfly.aop import JoinPoint, around, aspect
from pyfly.container.ordering import order
from pyfly.observability import MetricsRegistry

registry = MetricsRegistry()

@aspect
@order(50)
class MetricsAspect:
    """Records duration and call counts for every service method."""

    @around("service.*.*")
    async def record_metrics(self, jp: JoinPoint):
        start = time.perf_counter()
        exc_name = "none"
        try:
            result = await jp.proceed()
            return result
        except Exception as exc:
            exc_name = type(exc).__name__
```

```

        raise
    finally:
        elapsed = time.perf_counter() - start
        histogram = registry.histogram(
            f"service.{jp.method_name}.duration",
            f"Duration of {jp.method_name}",
            labels=["exception"],
        )
        histogram.labels(exception=exc_name).observe(elapsed)

```

Listing 15.13 — `@around metrics aspect`

`@order(-50)` on `AuditLoggingAspect` and `@order(50)` on `MetricsAspect` ensure the logging aspect fires first in the advice chain. `HIGHEST_PRECEDENCE = -(2^31)` runs earliest; `LOWEST_PRECEDENCE = 2^31 - 1` runs last.

Automatic weaving — `AspectBeanPostProcessor`

In production you never call `weave_bean()` manually. `AopAutoConfiguration` registers `AspectBeanPostProcessor` unconditionally. During context startup the post-processor:

1. Collects every bean whose class has `__pyfly__aspect__ = True` into an `AspectRegistry`.
2. For every non-aspect bean, checks whether any registered pointcut matches any public method.
3. Wraps matching methods in-place with the full advice chain via `weave_bean()`.

The result is zero-configuration AOP: define aspects, define services, start the application — the weaver wires them together.

JoinPoint reference

Every advice handler receives a `JoinPoint` dataclass:

Attribute	Available in	Description
<code>target</code>	All	The bean instance being intercepted
<code>method_name</code>	All	Name of the intercepted method
<code>args</code>	All	Positional arguments passed to the method
<code>kwargs</code>	All	Keyword arguments passed to the method
<code>return_value</code>	<code>@after_returning</code> , <code>@after</code>	Return value (after success)
<code>exception</code>	<code>@after_throwing</code> , <code>@after</code>	The raised exception (or <code>None</code>)
<code>proceed</code>	<code>@around</code> only	Callable; <code>await</code> -able for async methods

Putting it together — full observability on DepositFundsHandler

Lumen's deposit handler with all three observability pillars applied — zero observability code inside the business logic:

`lumen/core/services/wallets/deposit_funds_handler.py`

```

from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

from pyfly.container import service
from pyfly.cqrs import CommandHandler, command_handler
from pyfly.data.relational.sqlalchemy import transactional
from pyfly.domain import AggregateNotFound
from pyfly.eda import EventPublisher
from pyfly.logging import get_logger
from pyfly.observability import MetricsRegistry, counted, span, timed

from lumen.core.mappers.wallet_mapper import to_aggregate, to_entity
from lumen.core.services.wallets.deposit_funds_command import DepositFunds

```

```

from lumen.core.services.wallets.event_publishing import publish_domain_events
from lumen.models.entities.v1.money import Money
from lumen.models.repositories.wallet_repository import WalletRepository

logger = get_logger("lumen.wallet")
registry = MetricsRegistry()

@command_handler
@service
class DepositFundsHandler(CommandHandler[DepositFunds, int]):
    """
    Credits funds to an existing wallet and returns the new balance
    (in minor units, e.g. 1350 = €13.50).

    Logging, metrics, and tracing are applied by decorators and aspects;
    the business logic here stays free of cross-cutting concerns.
    """

    def __init__(
        self,
        repository: WalletRepository,
        events: EventPublisher,
        session_factory: async_sessionmaker[AsyncSession],
    ) -> None:
        super().__init__()
        self._repository = repository
        self._events = events
        self._session_factory = session_factory

    @timed(registry, "lumen.wallet.deposit.duration", "Deposit latency")
    @counted(registry, "lumen.wallet.deposits", "Total deposit attempts")
    @span("wallet-deposit")
    @transactional()
    async def do_handle(self, command: DepositFunds) -> int:
        logger.info("deposit_started", wallet_id=command.wallet_id,
                    amount=command.amount)
        entity = await self._repository.find_by_id(command.wallet_id)
        if entity is None:

```

```

    raise AggregateNotFound("Wallet", command.wallet_id)
    wallet = to_aggregate(entity)
    wallet.deposit(Money(amount=command.amount, currency=wallet.currency))
    await self._repository.upsert(to_entity(wallet))
    await publish_domain_events(self._events, wallet.clear_events())
    logger.info("deposit_completed", wallet_id=command.wallet_id,
               new_balance=wallet.balance.amount)
    return wallet.balance.amount

```

Listing 15.14 — *DepositFundsHandler* with full observability

How it works. `@span` opens an OpenTelemetry span. `@timed` records the `do_handle` duration. `@counted` increments the call counter. `AuditLoggingAspect` fires `@before` and `@after_returning` on every service method. `MetricsAspect` adds its own `@around` histogram. PII redaction strips sensitive values from log output automatically. The actuator exposes `/actuator/health`, `/actuator/prometheus`, and `/actuator/loggers`. The Admin Dashboard shows live traces and log records.

`command.amount` is an `int` in minor units — enforced by the `DepositFunds` command's validator (`amount > 0`). The `Money` value object wraps it with the wallet's `Currency`, preventing cross-currency arithmetic at the domain boundary.

Seven lines of decorators and one `get_logger` call — and Lumen's deposit path is fully observable.

✓ What you built

You started with a production-ready but opaque service. By the end of this chapter Lumen:

- Emits **structured JSON logs** with correlation IDs, async-context-bound fields, and automatic PII redaction via regex and optional Presidio NER.

- Exports **Prometheus metrics** from `MetricsRegistry`, tagged with `@timed` duration histograms and `@counted` invocation counters, scraped at `/actuator/prometheus`.
- Propagates **OpenTelemetry traces** end-to-end: `TracingFilter` opens a SERVER span from the inbound `traceparent` header, `@span` creates child spans for handler calls, `HttpxClientAdapter` injects context into outbound requests, and `StructlogAdapter` stamps every log record with `trace_id` and `span_id`.
- Answers **Kubernetes health probes** at `/actuator/health/liveness` and `/actuator/health/readiness` via auto-discovered `HealthIndicator` beans, including a `WalletRepositoryHealthIndicator` that probes the wallet store.
- Surfaces all of the above in the **embedded Admin Dashboard** at `/admin`, with 15 built-in views, real-time SSE streams, a live log tail, an HTTP trace analytics panel, and runtime logger-level management.
- Applies logging and metrics **cross-cuttingly** via `@aspect`, `@before`, `@after_returning`, `@after_throwing`, and `@around`, woven automatically by `AspectBeanPostProcessor` without touching handler code.

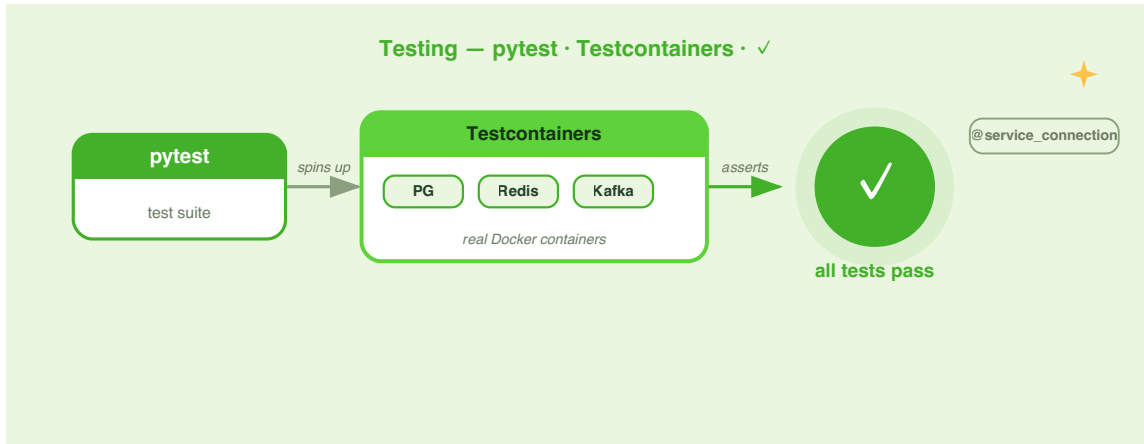
Try it yourself

1. **PII redaction audit.** Add a log statement to `DepositFundsHandler.do_handle` that includes a fake email address as a field value (`customer_email="alice@example.com"`) and a `token` field with an arbitrary string. Run Lumen locally with `format: console`, observe that the email is replaced with `<EMAIL>` and the token field value is replaced with `<REDACTED>`. Then switch `engine: presidio` (after installing `pyfly[pil]` and `en_core_web_sm`) and compare the output.

2. **Custom HealthIndicator.** Write a `StripeHealthIndicator` that calls `https://status.stripe.com/api/v2/status.json` with `httpx`, parses `indicator.status`, and returns `UP` if the value is `"none"` or `DOWN` otherwise. Register it as a `@component` and verify that `/actuator/health` includes a `StripeHealthIndicator` component. Test with a mocked `httpx` call that raises `httpx.ConnectError` and verify the aggregated status becomes `DOWN`.
3. **Metrics aspect with per-method thresholds.** Extend `MetricsAspect` from Listing 15.13 with a configurable `slow_threshold` (default 0.5 s) injected from `pyfly.yaml` via `@config_properties`. When a service method exceeds the threshold, emit a `logger.warning("slow_method", ...)` with `method_name` and `elapsed` fields. Write a pytest test that uses a `FakeClock` or `unittest.mock.patch("time.perf_counter")` to simulate a slow call and asserts the warning is logged.

CHAPTER 16

Testing PyFly Applications



The wallet works. Deposits land, balances update, events propagate through the bus, and the saga coordinator rolls back cleanly on failure. What you do not yet have is **confidence** that it will keep working after the next refactor. That confidence comes from tests — tests that run in milliseconds, prove the domain model enforces its invariants, verify the CQRS pipeline dispatches correctly, exercise the repository's derived queries and Specification predicates against a real SQLite database, and boot the full application context in an integration test that proves the entire DI + persistence composition.

PyFly treats testing as a first-class concern. The `pyfly.testing` module ships higher-level helpers — `PyFlyTestCase`, `create_test_container`, `assert_event_published`, Testcontainers wiring — that you can reach for when you

need them. Lumen's own test suite does not use them: it wires real components directly from `conftest.py`, uses standard pytest fixtures, and covers every layer of the pyramid with no boilerplate. That is the approach this chapter teaches.

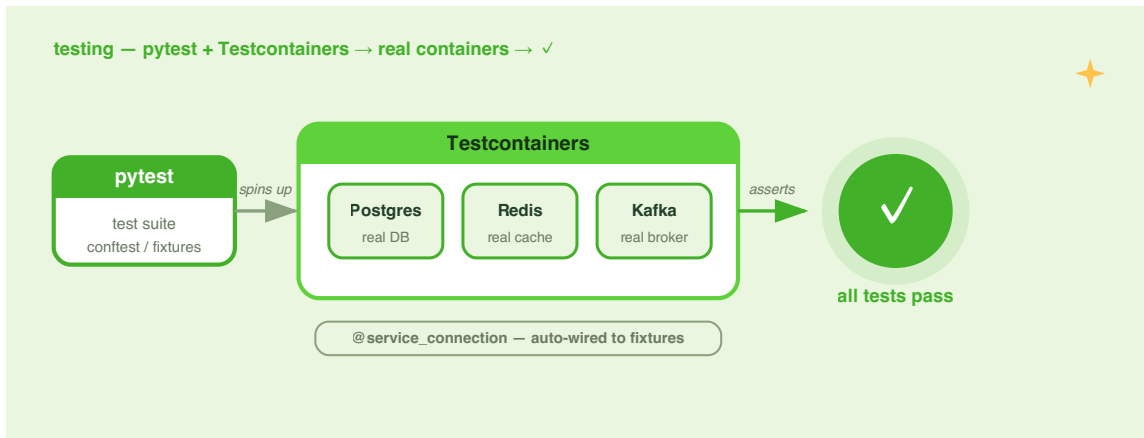


Figure 16.1 — PyFly's testing pyramid. Fast unit tests form the wide base; integration tests sit in the middle; real-DB adapter tests and a booted-context integration test crown the top.

The pyramid has four levels. **Unit tests** sit at the base — many of them, running in milliseconds, exercising the domain model with no dependencies. **CQRS flow tests** occupy the next tier — the full open/deposit/withdraw/query cycle routed through the real bus and the real repository, all wired in `conftest.py`. **Repository tests** exercise derived queries, pagination, and Specification predicates against SQLite. At the peak, a **booted-context integration test** starts the real `ApplicationContext` — DI scan, CQRS auto-config, `RepositoryBeanPostProcessor`, `@transactional` seam, EDA — and drives the full lifecycle.

Level	Dependencies	Speed	Lumen approach
Unit	None	Fast	plain pytest, no fixtures
CQRS flow	Real bus + repository over SQLite	Fast	conftest.py fixtures
Repository	SQLite + aiosqlite	Fast	<code>tmp_path</code> + SQLAlchemy
Booted-context	Full ApplicationContext + SQLite	Fast	<code>monkeypatch</code> env override

The project uses pytest with `pytest-asyncio` in **auto mode**. Enable it once in `pyproject.toml` and every `async def test_*` function is collected automatically:

```
[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
pythonpath = ["src"]
```

Install dev dependencies and run the suite:

```
uv run --extra dev pytest -q
```

The bare `uv sync` (without `--extra dev`) omits the dev group, so pytest is not installed. Always include `--extra dev` when running tests.

Unit-testing the domain

The domain model — `Money` and `Wallet` — has no framework dependencies. It never touches a database, a message bus, or an HTTP client. That purity makes it the ideal target for fast unit tests: construct objects, call methods, assert outcomes. No mocks, no fixtures, no `async`.

Testing Money

`Money` is a frozen dataclass. Every operation either succeeds and returns a new `Money` instance, or raises `BusinessRuleViolation`. Each violation carries a `.rule` string that names the violated invariant — useful for asserting the exact rule in tests.

tests/test_money.py

```
from __future__ import annotations

import pytest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money

from pyfly.domain import BusinessRuleViolation


def test_value_equality_is_structural() -> None:
    assert Money(1050, Currency.EUR) == Money(1050, Currency.EUR)
    assert Money(1050, Currency.EUR) != Money(1050, Currency.USD)
    assert Money(1050, Currency.EUR) != Money(999, Currency.EUR)


def test_money_is_immutable() -> None:
    money = Money(1050, Currency.EUR)
    with pytest.raises(Exception): # frozen dataclass -> FrozenInstanceError
        money.amount = 0 # type: ignore[misc]


def test_add_and_subtract_same_currency() -> None:
    a = Money(1050, Currency.EUR)
    b = Money(450, Currency.EUR)
    assert a.add(b) == Money(1500, Currency.EUR)
    assert a.subtract(b) == Money(600, Currency.EUR)


def test_zero_factory_and_major_units() -> None:
    assert Money.zero(Currency.USD) == Money(0, Currency.USD)
```

```

assert Money(1050, Currency.EUR).major_units == 10.5
assert str(Money(1050, Currency.EUR)) == "10.50 EUR"

def test_currency_mismatch_is_rejected() -> None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Money(100, Currency.EUR).add(Money(100, Currency.USD))
    assert exc.value.rule == "money-currency-mismatch"

def test_non_integer_amount_is_rejected() -> None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Money(10.5, Currency.EUR) # type: ignore[arg-type]
    assert exc.value.rule == "money-amount-integer"

```

Listing 16.1 — Pure unit tests for the Money value object

Every test is synchronous — no `async`, no `await`, no fixtures. Pytest collects the module-level functions automatically. `Currency.EUR` is an enum value, not a plain string, matching the domain model's type contract exactly.

💡 MINOR-UNIT ARITHMETIC

Money stores amounts in **minor units** (integer cents). `Money(1050, Currency.EUR)` represents €10.50 — verified by `major_units == 10.5` and `str(...) == "10.50 EUR"`. The `Money.zero(currency)` factory returns a `Money(0, currency)`, useful for initialising wallet balances.

Testing the Wallet aggregate

`Wallet` enforces several invariants: the owner must be a non-blank string, deposits must be positive, withdrawals must not overdraw, and amounts must match the wallet's currency. Each violation carries a `.rule` attribute for precise assertion.

`tests/test_wallet_aggregate.py`

```

from __future__ import annotations

import pytest
from lumen.interfaces.enums.v1.currency import Currency
from lumen.models.entities.v1.money import Money
from lumen.models.entities.v1.wallet_entity import (
    FundsDeposited,
    FundsWithdrawn,
    Wallet,
    WalletOpened,
)

from pyfly.domain import BusinessRuleViolation

def test_open_creates_empty_wallet() -> None:
    wallet = Wallet.open("wlt-1", "owner-1", Currency.EUR)
    assert wallet.owner_id == "owner-1"
    assert wallet.currency is Currency.EUR
    assert wallet.balance == Money.zero(Currency.EUR)
    [event] = wallet.pending_events()
    assert isinstance(event, WalletOpened)
    assert event.wallet_id == "wlt-1"
    assert event.currency == "EUR"

def test_open_requires_owner() -> None:
    with pytest.raises(BusinessRuleViolation) as exc:
        Wallet.open("wlt-x", " ", Currency.EUR)
    assert exc.value.rule == "wallet-owner-required"

def test_deposit_then_withdraw_happy_path() -> None:
    wallet = Wallet.open("wlt-2", "owner-2", Currency.EUR)
    wallet.clear_events()

    wallet.deposit(Money(1000, Currency.EUR))
    assert wallet.balance == Money(1000, Currency.EUR)

```



```

[event] = wallet.clear_events()
assert isinstance(event, FundsDeposited)
assert event.amount == 1000
assert event.balance == 1000

wallet.withdraw(Money(400, Currency.EUR))
assert wallet.balance == Money(600, Currency.EUR)
[event] = wallet.clear_events()
assert isinstance(event, FundsWithdrawn)
assert event.amount == 400
assert event.balance == 600

def test_withdraw_cannot_overdraw() -> None:
    wallet = Wallet.open("wlt-3", "owner-3", Currency.EUR)
    wallet.deposit(Money(500, Currency.EUR))
    wallet.clear_events()
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.withdraw(Money(501, Currency.EUR))
    assert exc.value.rule == "wallet-insufficient-funds"
    # invariant held: balance unchanged, no event raised
    assert wallet.balance == Money(500, Currency.EUR)
    assert wallet.pending_events() == []

def test_deposit_must_be_positive() -> None:
    wallet = Wallet.open("wlt-4", "owner-4", Currency.EUR)
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.deposit(Money(0, Currency.EUR))
    assert exc.value.rule == "wallet-deposit-positive"

def test_currency_mismatch_is_rejected() -> None:
    wallet = Wallet.open("wlt-5", "owner-5", Currency.EUR)
    with pytest.raises(BusinessRuleViolation) as exc:
        wallet.deposit(Money(100, Currency.USD))
    assert exc.value.rule == "wallet-currency-mismatch"

```

Listing 16.2 — Unit tests for the Wallet aggregate root

Three details deserve attention. First, `Wallet.open` takes three positional arguments: a pre-generated `wallet_id`, an `owner_id`, and a `Currency` enum value — the aggregate does not generate its own ID. Second, `pending_events()` returns buffered events without draining; `clear_events()` returns and drains. The `test_deposit_then_withdraw_happy_path` test calls `clear_events()` after opening so each assertion sees exactly one event. Third, `FundsDeposited` and `FundsWithdrawn` carry `amount` (the operation amount in minor units) and `balance` (the post-operation running balance) — not `new_balance`. Always verify real event dataclass fields before asserting them.

SPRING PARITY

Testing a DDD aggregate in isolation is the same discipline in any stack. In Spring / jMolecules you would call the aggregate's methods directly and check `aggregate.domainEvents()` (provided by `AbstractAggregateRoot`) before calling `afterDomainEventPublication()` to drain the buffer. PyFly's `clear_events()` plays the same role — drain, assert, move on.

Wiring the test stack with `conftest.py`

The CQRS and event-listener tests need real infrastructure: a SQLite-backed `WalletRepository`, an event bus, command and query handlers, and a running bus. Rather than recreating this in every test module, Lumen declares the wiring once in `tests/conftest.py`. Pytest discovers the file automatically and makes the fixtures available to every test in the package.

The key difference from a hand-rolled adapter test is that the `repository` fixture uses the **real framework** `WalletRepository` — the same Spring-Data-style class the application boots — and runs it through the **real** `RepositoryBeanPostProcessor`, which compiles derived-query stubs from method names at startup.

`tests/conftest.py`

```
from __future__ import annotations

import sys
from collections.abc import AsyncIterator
from pathlib import Path

import pytest_asyncio
from sqlalchemy.ext.asyncio import (
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

# Make the sample's `src/` importable
_HERE = Path(__file__).resolve().parent
_SRC = _HERE.parent / "src"
sys.path.insert(0, str(_SRC))

from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets import (
    DepositFundsHandler,
    GetBalanceHandler,
    GetWalletHandler,
    ListRichWalletsHandler,
    ListWalletsHandler,
    OpenWalletHandler,
    WithdrawFundsHandler,
)

from lumen.models.entities.v1.wallet_orm import WalletEntity
from lumen.models.repositories import WalletRepository
```

```

from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus, HandlerRegistry
from pyfly.data.relational.sqlalchemy import Base
from pyfly.data.relational.sqlalchemy.post_processor import (
    RepositoryBeanPostProcessor,
)
from pyfly.eda.adapters.memory import InMemoryEventBus

```

```

@pytest_asyncio.fixture
async def session_factory() ->
    AsyncIterator[async_sessionmaker[AsyncSession]]:
        """An in-memory SQLite engine + session factory, schema created.

        Mirrors the framework's relational auto-configuration: build the
        async engine and run Base.metadata.create_all.
        """

        engine = create_async_engine("sqlite+aiosqlite:///memory:")
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)
        factory = async_sessionmaker(engine, expire_on_commit=False)
        try:
            yield factory
        finally:
            await engine.dispose()

```

```

@pytest_asyncio.fixture
async def repository(
    session_factory: async_sessionmaker[AsyncSession],
) -> AsyncIterator[WalletRepository]:
    """The framework WalletRepository, post-processed.

    RepositoryBeanPostProcessor compiles derived-query stubs
    (e.g. find_by_owner_id) onto the bean – the same step the
    ApplicationContext runs at startup.
    """

    session = session_factory()
    repo = WalletRepository(WalletEntity, session)

```

```

RepositoryBeanPostProcessor().after_init(repo, "walletRepository")
try:
    yield repo
finally:
    await session.close()

@pytest_asyncio.fixture
async def event_bus() -> AsyncIterator[InMemoryEventBus]:
    """A real in-memory EDA bus – the same EventPublisher used in
    production."""
    yield InMemoryEventBus()

@pytest_asyncio.fixture
async def audit_listener(
    event_bus: InMemoryEventBus,
) -> AsyncIterator[WalletAuditListener]:
    """The wallet audit projection, subscribed to the bus exactly as the
    ApplicationContext auto-wires it at startup."""
    listener = WalletAuditListener()
    method = listener.on_wallet_event
    for pattern in method.__pyfly_event_patterns__:
        event_bus.subscribe(pattern, method)
    yield listener

@pytest_asyncio.fixture
async def command_bus(
    repository: WalletRepository,
    event_bus: InMemoryEventBus,
    session_factory: async_sessionmaker[AsyncSession],
) -> AsyncIterator[DefaultCommandBus]:
    registry = HandlerRegistry()
    registry.register_command_handler(
        OpenWalletHandler(
            repository=repository,
            events=event_bus,
            session_factory=session_factory,

```

```

    )
)
registry.register_command_handler(
    DepositFundsHandler(
        repository=repository,
        events=event_bus,
        session_factory=session_factory,
    )
)
registry.register_command_handler(
    WithdrawFundsHandler(
        repository=repository,
        events=event_bus,
        session_factory=session_factory,
    )
)
yield DefaultCommandBus(registry=registry)

@pytest_asyncio.fixture
async def query_bus(
    repository: WalletRepository,
) -> AsyncIterator[DefaultQueryBus]:
    registry = HandlerRegistry()
    registry.register_query_handler(GetWalletHandler(repository=repository))
    registry.register_query_handler(GetBalanceHandler(repository=repository))
    registry.register_query_handler(
        ListWalletsHandler(repository=repository)
    )
    registry.register_query_handler(
        ListRichWalletsHandler(repository=repository)
    )
    yield DefaultQueryBus(registry=registry)

```

Listing 16.3 — *conftest.py*: real framework components wired with no mocks

Each fixture is declared with `@pytest_asyncio.fixture` (not the bare `@pytest.fixture`) so `pytest-asyncio` manages the async iterator lifecycle. `asyncio_mode = "auto"` in `pyproject.toml` makes async fixtures and tests work without per-function decorators — but the fixture decorator itself must still be `pytest_asyncio.fixture`.

The `session_factory` fixture is shared. `repository` and `command_bus` both receive it, so the same in-memory SQLite engine backs reads, writes, and the `@transactional` boundary the handlers open. The `audit_listener` and `command_bus` fixtures both receive `event_bus`; `pytest` instantiates that once per test and shares it between them, so events published by the command handlers are visible to the listener.

NO MOCKS ANYWHERE

Every component in `conftest.py` is the real production implementation.

`WalletRepository` is the same class the application boots.

`RepositoryBeanPostProcessor` is the same post-processor the `ApplicationContext` runs at startup to compile derived-query stubs. `InMemoryEventBus` is the same bus used in non-Kafka deployments. The goal is to test the real code paths, not the wiring.

Testing the CQRS flow end to end

With the fixtures from `conftest.py`, exercising the full command/query cycle is a matter of calling `command_bus.send(...)` and `query_bus.query(...)`. No handler is instantiated in the test body — the bus dispatches to the handler already registered in the fixture.

`tests/test_cqrs_flow.py`

```

from __future__ import annotations

import pytest
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
from lumen.core.services.wallets.get_balance_query import GetBalance
from lumen.core.services.wallets.get_wallet_query import GetWallet
from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus

@pytest.mark.asyncio
async def test_full_wallet_lifecycle(
    command_bus: DefaultCommandBus,
    query_bus: DefaultQueryBus,
) -> None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    assert isinstance(wallet_id, str) and wallet_id.startswith("wlt-")

    balance = await command_bus.send(
        DepositFunds(wallet_id=wallet_id, amount=1500)
    )
    assert balance == 1500

    balance = await command_bus.send(
        WithdrawFunds(wallet_id=wallet_id, amount=500)
    )
    assert balance == 1000

    wallet = await query_bus.query(GetWallet(wallet_id=wallet_id))
    assert wallet is not None
    assert wallet.id == wallet_id
    assert wallet.owner_id == "u-1"
    assert wallet.currency is Currency.EUR

```



```

assert wallet.balance_minor == 1000
assert wallet.balance == 10.0

balance_dto = await query_bus.query(GetBalance(wallet_id=wallet_id))
assert balance_dto is not None
assert balance_dto.balance_minor == 1000
assert balance_dto.balance == 10.0

@pytest.mark.asyncio
async def test_get_wallet_returns_none_for_unknown_id(
    query_bus: DefaultQueryBus,
) -> None:
    assert await query_bus.query(
        GetWallet(wallet_id="wlt-does-not-exist")
    ) is None

@pytest.mark.asyncio
async def test_overdraw_is_rejected_through_the_bus(
    command_bus: DefaultCommandBus,
) -> None:
    from pyfly.cqrs.exceptions import CommandProcessingException

    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-2", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=100))

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            WithdrawFunds(wallet_id=wallet_id, amount=999)
        )

@pytest.mark.asyncio
async def test_validation_rejects_non_positive_deposit(
    command_bus: DefaultCommandBus,
) -> None:

```

```

from pyfly.cqrs.exceptions import CommandProcessingException

wallet_id = await command_bus.send(
    OpenWallet(owner_id="u-3", currency=Currency.EUR)
)
with pytest.raises(CommandProcessingException):
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=0))

@pytest.mark.asyncio
async def test_deposit_to_unknown_wallet_is_rejected(
    command_bus: DefaultCommandBus,
) -> None:
    from pyfly.cqrs.exceptions import CommandProcessingException

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            DepositFunds(wallet_id="wlt-nope", amount=100)
        )

```

Listing 16.4 — End-to-end CQRS tests through the real bus

`test_full_wallet_lifecycle` is the primary smoke test: it sends every command in the natural order and then queries both the full wallet DTO and the balance DTO. The wallet DTO exposes `balance_minor` (integer minor units) and `balance` (major units as a float); both derive from the same `WalletEntity` row stored through the repository.

The error-path tests verify that the bus surfaces domain violations correctly.

`CommandProcessingException` is the bus's wrapper for any exception raised inside a handler — including `BusinessRuleViolation` from the aggregate. Calling code never sees the raw domain exception; it always sees the bus wrapper.

❗ ASYNCIO_MODE = \"AUTO\" AND @PYTEST.MARK.ASYNCIO

With `asyncio_mode = \"auto\"` every async test is collected and run automatically. The `@pytest.mark.asyncio` decorator is **not required** but is harmless and makes the async intent explicit at a glance. Lumen keeps it for clarity.

Testing the repository adapter

The CQRS flow tests prove the full open/deposit/withdraw/query pipeline. The repository adapter tests go one level deeper: they directly exercise the `WalletRepository` API — CRUD, **derived queries** compiled from method names, **Pageable / Page** pagination, and **Specification** predicates — against a temporary SQLite file database. No Docker, no external process, no network. pytest's built-in `tmp_path` fixture provides a temporary directory cleaned up automatically after each test.

The local fixture `_make_repo` mirrors what the `ApplicationContext` does at startup: construct the repository and run `RepositoryBeanPostProcessor.after_init` to compile the derived-query stubs. Without that call, methods like `find_by_owner_id` would raise `NotImplementedError`.

tests/test_sql_wallet_repository.py

```
from __future__ import annotations

from collections.abc import AsyncIterator
from datetime import UTC, datetime, timedelta
from pathlib import Path

import pytest
import pytest_asyncio
from sqlalchemy.ext.asyncio import (
```

```

    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)

from lumen.models.entities.v1.wallet_orm import WalletEntity
from lumen.models.repositories.wallet_repository import (
    WalletRepository,
    balance_at_least,
)

from pyfly.data import Pageable, Sort
from pyfly.data.relational.sqlalchemy import Base
from pyfly.data.relational.sqlalchemy.post_processor import (
    RepositoryBeanPostProcessor,
)

def _entity(
    wid: str,
    owner: str,
    minor: int,
    *,
    currency: str = "EUR",
    age_days: int = 0,
) -> WalletEntity:
    created = datetime.now(UTC) - timedelta(days=age_days)
    return WalletEntity(
        id=wid,
        owner_id=owner,
        currency=currency,
        balance_minor=minor,
        created_at=created,
    )

def _make_repo(session: AsyncSession) -> WalletRepository:
    repo = WalletRepository(WalletEntity, session)
    # Mirror the ApplicationContext: compile derived-query stubs.
    RepositoryBeanPostProcessor().after_init(repo, "walletRepository")

```

```

    return repo

@pytest_asyncio.fixture
async def sqlite_factory(
    tmp_path: Path,
) -> AsyncIterator[tuple[async_sessionmaker[AsyncSession], str]]:
    """A temp-file SQLite engine + session factory, schema created.

    Yields the session factory and the database URL so the test can
    reconnect with a fresh engine to verify true persistence.
    """
    db_url = f"sqlite+aiosqlite:/// {tmp_path / 'wallets.db'}"
    engine = create_async_engine(db_url)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    factory = async_sessionmaker(engine, expire_on_commit=False)
    try:
        yield factory, db_url
    finally:
        await engine.dispose()

@pytest.mark.asyncio
async def test_upsert_inserts_then_updates_and_persists(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) -> None:
    factory, db_url = sqlite_factory

    async with factory() as session:
        repo = _make_repo(session)
        await repo.upsert(_entity("wlt-1", "owner-42", 0, currency="USD"))
        # update: same PK, new balance
        await repo.upsert(_entity("wlt-1", "owner-42", 2500, currency="USD"))
        await session.commit()

    got = await repo.find_by_id("wlt-1")
    assert got is not None
    assert got.owner_id == "owner-42"

```

```

    assert got.currency == "USD"
    assert got.balance_minor == 2500
    assert await repo.count() == 1

# prove persistence: reconnect with a brand-new engine/session
fresh_engine = create_async_engine(db_url)
fresh_factory = async_sessionmaker(fresh_engine, expire_on_commit=False)
try:
    async with fresh_factory() as fresh_session:
        fresh_repo = _make_repo(fresh_session)
        persisted = await fresh_repo.find_by_id("wlt-1")
        assert persisted is not None, "wallet should survive a reconnect"
        assert persisted.balance_minor == 2500
finally:
    await fresh_engine.dispose()

@pytest.mark.asyncio
async def test_find_by_id_unknown_returns_none(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) -> None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        assert await repo.find_by_id("wlt-nope") is None

@pytest.mark.asyncio
async def test_derived_find_by_owner_id(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) -> None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        await repo.upsert(_entity("wlt-1", "alice", 100))
        await repo.upsert(_entity("wlt-2", "alice", 200))
        await repo.upsert(_entity("wlt-3", "bob", 300))
        await session.commit()

```

```

owned = await repo.find_by_owner_id("alice")
assert sorted(w.id for w in owned) == ["wlt-1", "wlt-2"]
assert await repo.find_by_owner_id("nobody") == []

```

```
@pytest.mark.asyncio
```

```

async def test_specification_find_rich_paged_and_sorted(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) -> None:
    factory, _ = sqlite_factory
    async with factory() as session:
        repo = _make_repo(session)
        # age_days drives created_at so we can assert newest-first ordering.
        await repo.upsert(_entity("wlt-poor", "a", 50, age_days=3))
        await repo.upsert(_entity("wlt-mid", "b", 1000, age_days=2))
        await repo.upsert(_entity("wlt-rich", "c", 5000, age_days=1))
        await session.commit()

        # Specification: balance_minor >= 1000, newest first, page size 1.
        newest_first = Sort.by("created_at").descending()
        page = await repo.find_rich(1000, Pageable.of(1, 1, newest_first))
        assert page.total == 2 # mid + rich
        assert page.total_pages == 2
        assert page.has_next is True
        assert [w.id for w in page.items] == ["wlt-rich"]

        page2 = await repo.find_rich(1000, Pageable.of(2, 1, newest_first))
        assert [w.id for w in page2.items] == ["wlt-mid"]

        # The bare predicate also works through find_all_by_spec.
        rich = await repo.find_all_by_spec(balance_at_least(5000))
        assert [w.id for w in rich] == ["wlt-rich"]

```

```
@pytest.mark.asyncio
```

```

async def test_find_all_pageable_counts_and_pages(
    sqlite_factory: tuple[async_sessionmaker[AsyncSession], str],
) -> None:
    factory, _ = sqlite_factory

```

```

async with factory() as session:
    repo = _make_repo(session)
    for i in range(5):
        await repo.upsert(
            _entity(f"wlt-{i}", "owner", i * 100, age_days=5 - i)
        )
    await session.commit()

    page = await repo.find_all(
        Pageable.of(1, 2, Sort.by("created_at").descending())
    )
    assert page.total == 5
    assert page.total_pages == 3
    assert len(page.items) == 2
    # newest first -> wlt-4 (age 1 day), then wlt-3
    assert [w.id for w in page.items] == ["wlt-4", "wlt-3"]

```

Listing 16.5 — Repository tests: CRUD, derived query, pagination, Specification

Four things to note. First, `_make_repo` calls

`RepositoryBeanPostProcessor().after_init(repo, ...)` — without this, `find_by_owner_id` is still a stub and raises `NotImplementedError`. The post-processor compiles the method name into a SQLAlchemy `WHERE owner_id = :owner_id` clause. Second, `upsert` is the repository's insert-or-update; after each batch of upserts, `await session.commit()` flushes to SQLite. Third, `find_rich` takes a minimum balance and a `Pageable`; it delegates to `find_all_by_spec_paged(balance_at_least(min), pageable)`. Fourth, the two-engine pattern in `test_upsert_inserts_then_updates_and_persists` proves true durability: data committed through one engine is readable by a completely fresh engine and session.

🍃 SPRING PARITY

This test layer is the Python equivalent of `@DataJpaTest` with an embedded H2 database in Spring Boot. `@DataJpaTest` loads only the JPA layer (entities, repositories, Flyway) and wires a fresh in-memory H2 for every test class. The `sqlite_factory` fixture does the same: create the schema, run the tests, dispose the engine. No Docker, no external process.

💡 DERIVED QUERIES ARE METHOD-NAME CONVENTIONS

`WalletRepository.find_by_owner_id` is declared as a stub (`raise NotImplementedError`). `RepositoryBeanPostProcessor` inspects the method name at startup — `find_by_owner_id` → `WHERE owner_id = :value` — and replaces the stub with a real coroutine. Testing this method therefore also tests that the post-processor convention is working correctly.

Testing the event listener

`WalletAuditListener` listens for domain events published by the command handlers. Testing it end to end — command runs on the bus, handler publishes events, listener receives them — requires all three components to share the same `InMemoryEventBus`. The `conftest.py` fixtures already arrange this: both `command_bus` and `audit_listener` accept an `event_bus` argument, and pytest injects the same instance into both.

tests/test_event_listener.py

```
from __future__ import annotations

import pytest
from lumen.core.services.listeners import WalletAuditListener
from lumen.core.services.wallets.deposit_funds_command import DepositFunds
```

```

from lumen.core.services.wallets.open_wallet_command import OpenWallet
from lumen.core.services.wallets.withdraw_funds_command import WithdrawFunds
from lumen.interfaces.enums.v1.currency import Currency

from pyfly.cqrs import DefaultCommandBus

@pytest.mark.asyncio
async def test_listener_observes_wallet_events(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) -> None:
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-1", currency=Currency.EUR)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=1500))
    await command_bus.send(WithdrawFunds(wallet_id=wallet_id, amount=400))

    entries = audit_listener.entries_for(wallet_id)
    assert [e.event_type for e in entries] == [
        "WalletOpened",
        "FundsDeposited",
        "FundsWithdrawn",
    ]

    # The payload carried the real domain-event fields.
    deposited = entries[1]
    assert deposited.payload["amount"] == 1500
    assert deposited.payload["currency"] == "EUR"
    assert deposited.payload["balance"] == 1500
    assert deposited.event_id # the aggregate's DomainEvent.event_id

    # The running-total projection reflects deposit - withdrawal.
    assert audit_listener.running_total(wallet_id) == 1100

@pytest.mark.asyncio
async def test_listener_records_nothing_before_any_command(
    audit_listener: WalletAuditListener,

```

```

) -> None:
    assert audit_listener.entries == []
    assert audit_listener.running_total("anything") == 0

@pytest.mark.asyncio
async def test_event_type_matches_domain_event_class_names(
    command_bus: DefaultCommandBus,
    audit_listener: WalletAuditListener,
) -> None:
    # A rejected withdrawal raises no event – it must not appear in the log.
    wallet_id = await command_bus.send(
        OpenWallet(owner_id="u-2", currency=Currency.USD)
    )
    await command_bus.send(DepositFunds(wallet_id=wallet_id, amount=100))

    from pyfly.cqrs.exceptions import CommandProcessingException

    with pytest.raises(CommandProcessingException):
        await command_bus.send(
            WithdrawFunds(wallet_id=wallet_id, amount=9999)
        )

    types = [e.event_type for e in audit_listener.entries_for(wallet_id)]
    assert types == ["WalletOpened", "FundsDeposited"]
    assert audit_listener.running_total(wallet_id) == 100

```

Listing 16.6 — Event listener tests: command publishes, listener observes

`test_listener_observes_wallet_events` is the core integration proof: three commands produce three events, the listener records all three in order, the payload fields match the aggregate's event dataclass fields, and the `running_total` projection equals the arithmetic result. No bus mock, no event capture list — the production listener runs on the production bus.

`test_event_type_matches_domain_event_class_names` proves a domain invariant: a rejected command (overdraw) raises no event. The audit log must never record a side effect from a failed operation.

💡 EVENT_TYPE IS THE CLASS NAME

PyFly's event publisher sets `event_type` to the domain event class name:

"WalletOpened", "FundsDeposited", "FundsWithdrawn". The `@event_listener(pattern)` decorator on `WalletAuditListener.on_wallet_event` uses a glob pattern ("Wallet*", "Funds*") to subscribe to all three. The test asserts the string class names directly.

Booted-context integration test

The unit tests, CQRS flow tests, and repository tests each wire one layer of the stack. The booted-context integration test wires everything at once: it starts the real `ApplicationContext` — DI component scan, CQRS auto-configuration, relational auto-configuration, `RepositoryBeanPostProcessor`, `@transactional` seam, EDA event bus — then resolves the `DefaultCommandBus` and `DefaultQueryBus` from the context and drives the full wallet lifecycle.

The database URL is overridden via an environment variable so the test never touches the developer's `lumen.db`.

`tests/test_app_context_integration.py`

```
from __future__ import annotations

import logging
import sys
from collections.abc import AsyncIterator
```

```

from pathlib import Path

import pytest
import pytest_asyncio
from sqlalchemy.ext.asyncio import AsyncSession

_HERE = Path(__file__).resolve().parent
_SRC = _HERE.parent / "src"
sys.path.insert(0, str(_SRC))

@pytest_asyncio.fixture
async def booted_context(
    tmp_path: Path,
    monkeypatch: pytest.MonkeyPatch,
) -> AsyncIterator[object]:
    """Boot the full LumenApplication against an isolated SQLite file."""
    db_path = tmp_path / "lumen-it.db"
    monkeypatch.setenv(
        "PYFLY_DATA_RELATIONAL_URL",
        f"sqlite+aiosqlite:/// {db_path}",
    )
    # Silence the pool-GC warning that aiosqlite emits at teardown.
    logging.getLogger(
        "sqlalchemy.pool.impl.AsyncAdaptedQueuePool"
    ).setLevel(logging.CRITICAL)

    from lumen.app import LumenApplication
    from pyfly.core import PyFlyApplication

    app = PyFlyApplication(
        LumenApplication,
        config_path=str(_HERE.parent / "pyfly.yaml"),
    )
    await app.startup()
    try:
        yield app.context
    finally:
        await app.context.get_bean(AsyncSession).close()

```

```

    await app.shutdown()

@pytest.mark.asyncio
async def test_full_lifecycle_through_booted_context(
    booted_context: object,
) -> None:
    from lumen.core.services.wallets.deposit_funds_command import DepositFunds
    from lumen.core.services.wallets.get_balance_query import GetBalance
    from lumen.core.services.wallets.get_wallet_query import GetWallet
    from lumen.core.services.wallets.list_rich_wallets_query import (
        ListRichWallets,
    )
    from lumen.core.services.wallets.list_wallets_query import ListWallets
    from lumen.core.services.wallets.open_wallet_command import OpenWallet
    from lumen.core.services.wallets.withdraw_funds_command import
WithdrawFunds
    from lumen.interfaces.enums.v1.currency import Currency
    from pyfly.cqrs import DefaultCommandBus, DefaultQueryBus
    from pyfly.data import Pageable

    ctx = booted_context
    commands = ctx.get_bean(DefaultCommandBus)
    queries = ctx.get_bean(DefaultQueryBus)

    # --- open -> deposit -> withdraw, each a committed unit of work -----
    w1 = await commands.send(OpenWallet(owner_id="u-1",
currency=Currency.EUR))
    w2 = await commands.send(OpenWallet(owner_id="u-2",
currency=Currency.EUR))
    assert w1.startswith("wlt-") and w2.startswith("wlt-")

    assert await commands.send(DepositFunds(wallet_id=w1, amount=5000)) ==
5000
    assert await commands.send(WithdrawFunds(wallet_id=w1, amount=1500)) ==
3500
    assert await commands.send(DepositFunds(wallet_id=w2, amount=100)) == 100

    # --- persistence survived: reload the aggregate via the query side ---

```

```

reloaded = await queries.query(GetWallet(wallet_id=w1))
assert reloaded is not None
assert reloaded.owner_id == "u-1"
assert reloaded.balance_minor == 3500

# --- paged list (find_all(pageable) + Page.map) -----
page = await queries.query(ListWallets(pageable=Pageable.of(1, 10)))
assert page.total == 2
assert {w.id for w in page.items} == {w1, w2}

# --- Specification: only wallets with balance >= 1000 -----
rich = await queries.query(
    ListRichWallets(min_minor=1000, pageable=Pageable.of(1, 10))
)
assert rich.total == 1
assert [w.id for w in rich.items] == [w1]

everyone = await queries.query(
    ListRichWallets(min_minor=0, pageable=Pageable.of(1, 10))
)
assert everyone.total == 2

# --- projection-backed balance -----
balance = await queries.query(GetBalance(wallet_id=w1))
assert balance is not None
assert balance.balance_minor == 3500
assert balance.balance == 35.0

```

Listing 16.7 — Booted-context integration: full DI + persistence composition

`booted_context` uses pytest's built-in `monkeypatch` fixture to set `PYFLY_DATA_RELATIONAL_URL` before the application boots. The framework reads this environment variable during relational auto-configuration, so the context uses the isolated temp-file SQLite database for the life of the test, then disposes of it when the fixture tears down.

`test_full_lifecycle_through_booted_context` exercises every query type the application exposes: `GetWallet` (aggregate reload), `ListWallets` (paginated list using `find_all(pageable)`), `ListRichWallets` (Specification predicate using `find_all_by_spec_paged`), and `GetBalance` (projection-backed balance). It proves that the `RepositoryBeanPostProcessor`, the `@transactional` boundary around each command handler, and the DI wiring all compose correctly in a single boot.

SPRING PARITY

This test is the Python equivalent of `@SpringBootTest` with an embedded H2 database. `@SpringBootTest` loads the full application context, including all auto-configurations and the JPA layer; you set `spring.datasource.url` in `application-test.properties` to redirect to H2. PyFly's environment variable override (`monkeypatch.setenv`) plays the same role. Both approaches prove that the composed application works end to end without any external infrastructure.

Framework testing helpers

The tests above cover Lumen's full pyramid with standard pytest primitives and PyFly's real production components. For larger applications or teams that prefer more structure, `pyfly.testing` ships higher-level helpers that mirror Spring Boot's testing annotations.

`PyFlyTestCase` + `mock_bean(T)` work like `@MockBean` in `@SpringBootTest`. Declare `repo = mock_bean(WalletDomainRepository)` on the class body; `setup()` installs an `AsyncMock(spec=T)` into the application context and wires it into any collaborator that depends on it.

`create_test_container(overrides={Interface: Implementation})` builds a DI container with fakes registered for specific interfaces. Resolve the class under test from it and its dependencies are already injected.

`assert_event_published(events, event_type, payload_contains=...)` scans a captured `EventEnvelope` list for the first envelope with the given type, optionally checks payload keys, and returns the envelope for further assertions.

`assert_no_events_published(events)` fails if the list is non-empty.

Testcontainers integration (`postgres_container()`, `redis_container()`, `pyfly_config(container, base={...})`) is PyFly's equivalent of Spring Boot's `@Testcontainers` + `@ServiceConnection`. Start a real Postgres container; `pyfly_config` rewrites the sync `psycpg2://` URL to `postgresql+asyncpg://` and merges it into a `Config` ready to boot an `ApplicationContext`. Install support with:

```
pip install 'pyfly[testcontainers]'
```

Guard every Testcontainers test with `@requires_docker` so it skips cleanly on machines without Docker and runs automatically on CI runners that have it:

```
from pyfly.testing import postgres_container, pyfly_config, requires_docker

@requires_docker
async def test_wallet_round_trip_against_real_postgres():
    with postgres_container() as pg:
        config = pyfly_config(pg, base={"pyfly.data.enabled": True})
        assert config.get("pyfly.data.relational.url").startswith(
            "postgresql+asyncpg://"
        )
    ...
```

Lumen does not use these helpers — SQLite covers the persistence layer without Docker, and the in-memory bus covers event routing. Reach for them when your project has infrastructure that cannot be reproduced without a real daemon.

✓ What you built

Lumen now has 41 passing tests across six files, exercising every layer of the pyramid.

At the base, `test_money.py` and `test_wallet_aggregate.py` prove the domain model's arithmetic, immutability, and invariant rules. All tests are synchronous, pure Python functions — no fixtures, no DI, no `async`. The `BusinessRuleViolation.rule` attribute makes each assertion specific to the exact violated invariant.

In the middle, `conftest.py` wires the real components — the framework `WalletRepository` over an in-memory SQLite engine, `InMemoryEventBus`, all five command and query handlers (including `ListWalletsHandler` and `ListRichWalletsHandler`), and `WalletAuditListener` — into reusable async fixtures that pytest shares automatically across modules. The `RepositoryBeanPostProcessor` is applied to the repository fixture exactly as the `ApplicationContext` applies it at startup. `test_cqrs_flow.py` dispatches commands and queries through the real bus and checks every field of the query DTOs. `test_event_listener.py` proves that the audit listener observes exactly the events produced by successful commands and nothing from rejected ones.

`test_sql_wallet_repository.py` exercises the `WalletRepository` directly against a temporary SQLite file, covering the full CRUD surface, the derived query `find_by_owner_id` (compiled from the method name by `RepositoryBeanPostProcessor`), the `find_all(pageable)` API that returns a `Page` with total count and page metadata, and the `Specification` predicate path via `find_rich` / `find_all_by_spec`. The two-engine reconnect pattern proves true durability.

At the peak, `test_app_context_integration.py` boots the real `LumenApplication` with the database URL overridden to an isolated SQLite file, then drives the full open → deposit → withdraw → list → rich → balance lifecycle through the context-resolved buses. This single test proves that the DI scan, CQRS auto-configuration, `RepositoryBeanPostProcessor`, and `@transactional` boundary all compose correctly.

Concretely, you learned:

- `asyncio_mode = "auto" + pythonpath = ["src"]` in `pyproject.toml` — all async tests run without decorators; the `src/` layout is importable.
- `uv run --extra dev pytest -q` — the bare `uv sync` drops the dev group; always include `--extra dev` to get pytest.
- `@pytest_asyncio.fixture` — async fixture lifecycle managed by pytest-asyncio; plain `@pytest.fixture` does not handle async generators.
- **Shared fixture instances** — when two fixtures request the same fixture name (e.g., `event_bus`, `session_factory`), pytest resolves it once per test and shares the instance.
- `RepositoryBeanPostProcessor().after_init(repo, name)` — must be called in tests that exercise derived queries; without it, the method stubs raise `NotImplementedError`.
- **Derived queries** (`find_by_owner_id`) — declared as stubs; the post-processor compiles them to `WHERE owner_id = :value` at startup.
- `Pageable.of(page, size, sort) + Page` — the `find_all(pageable)` API returns a `Page` with `total`, `total_pages`, `has_next`, and `items`; assert each field for pagination correctness.

- **Specification predicates** — `balance_at_least(n)` is passed to `find_rich` / `find_all_by_spec` to filter by an arbitrary predicate without adding a new derived-query method.
- **`pending_events()` vs `clear_events()`** — `pending_events()` reads without draining; `clear_events()` drains. Always call `clear_events()` in arrange steps so assertions only see events from the act step.
- **`BusinessRuleViolation.rule`** — assert the exact rule string, not just the exception class, to prove that the right invariant fired.
- **`monkeypatch.setenv`** — override configuration before booting the context in integration tests; the framework reads environment variables during auto-configuration.
- **Framework helpers** (`PyFlyTestCase`, `mock_bean`, `create_test_container`, `Testcontainers`) — available in `pyfly.testing` for projects that need them; Lumen keeps things simple with real components.

Try it yourself

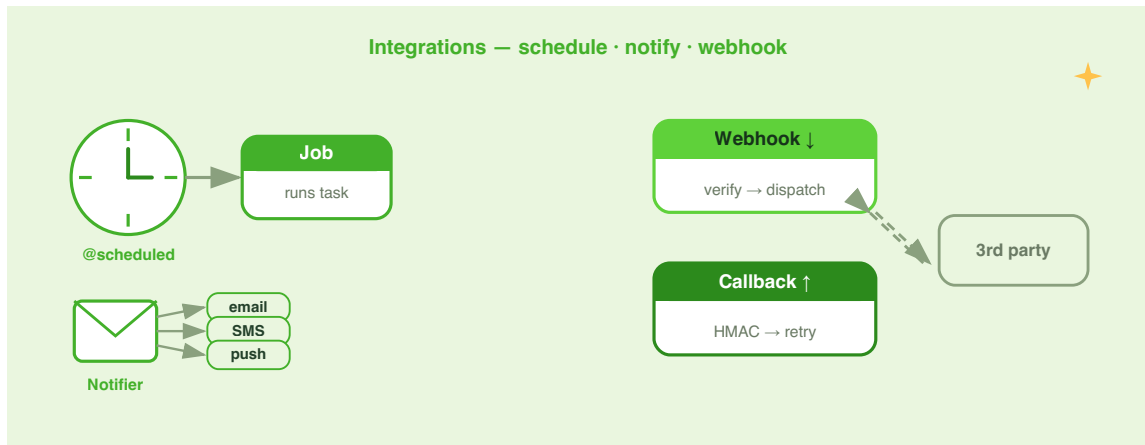
1. **Add a test for zero-amount withdrawal.** In `test_wallet_aggregate.py`, add `test_withdraw_zero_is_rejected`. Open a wallet, deposit 500 EUR, then attempt `wallet.withdraw(Money(0, Currency.EUR))`. Assert that a `BusinessRuleViolation` is raised and check its `.rule` attribute. Compare the rule name with the deposit equivalent — are the rules symmetric?
2. **Test an unknown wallet via the CQRS bus.** In `test_cqrs_flow.py`, add `test_withdraw_from_unknown_wallet_is_rejected`. Send only a `WithdrawFunds(wallet_id="wlt-ghost", amount=50)` command without opening

a wallet first. Assert that `CommandProcessingException` is raised. Confirm that the repository still holds no wallets by querying `GetWallet(wallet_id="wlt-ghost")` and asserting `None`.

3. **Extend the listener test with a second wallet.** In `test_event_listener.py`, add a test that opens two wallets (`u-A` and `u-B`), deposits different amounts into each, and then calls `audit_listener.entries_for(wallet_id_A)` and `audit_listener.entries_for(wallet_id_B)` separately. Assert that each returns exactly two entries (`WalletOpened` + `FundsDeposited`) and that the payload `amount` values differ. This proves that `entries_for` filters by wallet ID, not by event type.
4. **Add a multi-owner pagination test.** In `test_sql_wallet_repository.py`, add a test that inserts ten wallets with two different owners, calls `find_by_owner_id` for each owner, and then calls `find_all` with `Pageable.of(1, 3, Sort.by("balance_minor").descending())`. Assert that `page.total == 10`, `page.total_pages == 4`, and that the first item in `page.items` has the highest `balance_minor`. This proves that pagination is independent of the derived-query filter.

CHAPTER 17

Scheduling, Notifications, Webhooks & Callbacks



In Chapter 16 you gave Lumen a full test harness — unit tests for the domain, CQRS flow tests through the real bus, and a SQLite adapter test that proves true persistence. Lumen is now well-tested and resilient. But it still lives entirely inside its own process: it reacts to requests, but it never reaches out unprompted.

Real financial platforms are different. They send nightly account statements, fire an SMS the moment funds land, receive payment-status webhooks from a payment provider at midnight, and call back partner systems to confirm that a disbursement was booked. That is four distinct integration patterns — scheduling, notifications, inbound webhooks, and outbound callbacks — and this chapter covers all of them.

By the end of the chapter Lumen will:

- run a **nightly scheduled job** that tallies daily wallet balances using `@scheduled` with a cron expression;
- send a **"funds received" email and push notification** via PyFly's pluggable `EmailService` and `PushService` ports, triggered by the real `FundsDeposited` domain event;
- accept **inbound webhooks** from an illustrative payment provider, verifying the HMAC-SHA256 signature, deduplicating replays, and dispatching to a typed listener;
- dispatch **outbound callbacks** to partner systems, signing each payload, retrying on transient failures, and recording every delivery attempt.

Install the two optional extras before you start:

```
uv add "pyfly[scheduling,notifications]"
```

Scheduled tasks

Why schedule instead of trigger?

Many operations in a financial platform cannot wait for an HTTP request to arrive. Nightly reconciliation must run at 02:00 regardless of whether any user is active. A cache-warming pass should fire 30 seconds after startup — before real traffic arrives — not when the first slow request triggers a miss. A heartbeat metric should emit every 10 seconds so the ops dashboard shows a live signal, not a stale reading.

PyFly's scheduling module provides a declarative, decorator-driven way to define all three patterns without manually managing threads, event loops, or timer wheels.

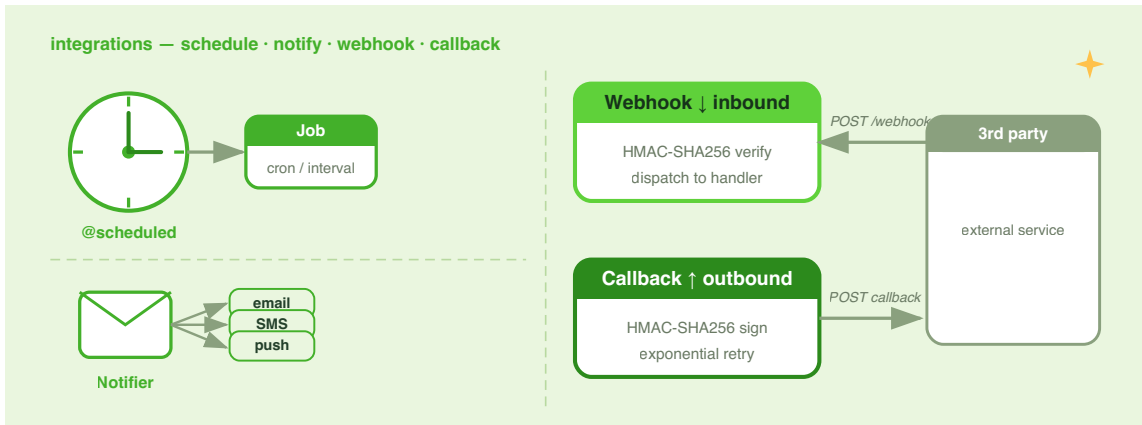


Figure 17.1 — The four\

integration layers added in this chapter. Scheduled tasks fire\ internally; notifications flow outward to users; inbound webhooks\ arrive from partners; outbound callbacks close the feedback loop.

The @scheduled decorator

`@scheduled` marks any `async` method on a `@service` bean for periodic execution. It accepts exactly one *trigger*: `fixed_rate`, `fixed_delay`, or `cron`. Providing zero or more than one trigger raises a `ValueError` at decoration time, so mistakes surface at startup rather than silently at three in the morning.

`lumen/ledger/daily_rollup.py`

```
from datetime import timedelta

from lumen.models.repositories.wallet_repository import WalletRepository
from pyfly.container import service
from pyfly.scheduling import scheduled

@service
class DailyRollupService:
```



```

"""Tallies all wallet balances once per night at 02:00 UTC."""

def __init__(self, wallet_repo: WalletRepository) -> None:
    self._wallets = wallet_repo

@scheduled(cron="0 2 * * *")
async def run(self) -> None:
    wallets = await self._wallets.find_all()
    # find_all() returns WalletEntity rows; balance_minor is the
    # persisted integer (cents).
    total_minor_units = sum(w.balance_minor for w in wallets)
    # Persist or ship the nightly snapshot; here we log it.
    print(
        f"[rollup] {len(wallets)} wallets, "
        f"total {total_minor_units / 100:.2f} "
        f"(minor units: {total_minor_units})"
    )

```

Listing 17.1 — Nightly wallet balance rollup with `@scheduled`

How it works. `@scheduled(cron="0 2 * * *")` fires every day at 02:00 UTC. The scheduler calculates `seconds_until_next()` via `CronExpression`, sleeps exactly that long, then submits the method to the executor.

That is all the wiring Lumen needs. With `pyfly[scheduling]` installed, `SchedulingAutoConfiguration` automatically:

1. registers a `TaskScheduler` bean;
2. scans every `@service` bean for `@scheduled` methods;
3. starts the scheduler during `ApplicationContext` startup;
4. stops it gracefully on shutdown.

No `SchedulerManager` required.

fixed_rate vs. fixed_delay

fixed_rate measures from the **start** of one execution to the start of the next.

fixed_delay measures from the **end** of one execution to the start of the next. Use

fixed_rate for heartbeats and metrics where you need a steady cadence regardless of execution time. Use **fixed_delay** when you need a guaranteed breathing gap — for example, when polling an upstream API that rate-limits on request frequency.

lumen/health/monitor.py

```

from datetime import timedelta

from pyfly.container import service
from pyfly.scheduling import scheduled

@service
class HealthMonitor:

    @scheduled(
        fixed_rate=timedelta(seconds=10),
        initial_delay=timedelta(seconds=5),
    )
    async def heartbeat(self) -> None:
        """Emit a liveness metric every 10 s, starting 5 s after startup."""
        # metrics.gauge("lumen.up", 1)
        pass

@service
class ExchangeRatePoller:

    def __init__(self, fx_repo) -> None:
        self._repo = fx_repo

    @scheduled(fixed_delay=timedelta(minutes=5))
    async def poll(self) -> None:
        """Fetch the latest exchange rates, then wait 5 min before

```

```
repeating."""
    rates = await self._repo.fetch_latest()
    await self._repo.store(rates)
```

Listing 17.2 — *fixed_rate* heartbeat and *fixed_delay* poll

`initial_delay` postpones the first run; it is available for both `fixed_rate` and `fixed_delay` (ignored for `cron` triggers, which always wait for the next matching calendar instant).

CronExpression

`CronExpression` is also usable directly — convenient when you need to display upcoming schedule times in a UI or validate a user-supplied expression before storing it.

`lumen/ledger/schedule_preview.py`

```
from pyfly.scheduling import CronExpression

def preview_rollup_schedule(expression: str, n: int = 5) -> list[str]:
    """Return the next N fire times for a given cron expression."""
    cron = CronExpression(expression)
    return [str(t) for t in cron.next_n_fire_times(n)]
```

Listing 17.3 — Using `CronExpression` standalone

`CronExpression` accepts both the standard 5-field format (`min hour dom month dow`) and the Spring-style 6-field format with seconds first (`sec min hour dom month dow`). The Spring `?` wildcard is normalised to `*` transparently.

Expression	Fires
* * * * *	Every minute
0 * * * *	Every hour, on the hour
0 2 * * *	Every day at 02:00
0 9 * * 1-5	Weekdays at 09:00
30 2 1 * *	1st of each month at 02:30
*/15 * * * *	Every 15 minutes
0 0 12 * * *	Noon every day (6-field, seconds-first)

Time-zone-aware cron

Cron expressions are evaluated in **UTC** by default. Pass `zone` with an IANA time-zone name to evaluate fire times in a specific zone instead:

```
@scheduled(cron="0 2 * * *", zone="America/New_York")
async def close_books(self) -> None:
    """02:00 New York time – follows DST automatically."""
    ...
```

The same `zone` parameter is available on `CronExpression`:

```
cron = CronExpression("0 9 * * *", zone="Europe/Madrid")
next_run = cron.next_fire_time() # zone-aware datetime
```

DST transitions are handled by the `zoneinfo` database; no manual offset adjustment is required.

Distributed locking

When multiple Lumen instances run behind a load balancer, every instance schedules the same `@scheduled` methods. Without coordination, the nightly rollup fires once per instance and writes duplicate records. The `lock` parameter solves this the same way Spring's `@SchedulerLock` (ShedLock) does: before each tick the scheduler tries to acquire a named lock and **skips the run** if the lock is already held elsewhere.

```
@scheduled(cron="0 2 * * *", lock=True, lock_ttl=timedelta(minutes=5))
async def run(self) -> None:
    """lock=True auto-names the lock 'DailyRollupService.run'."""
    ...
```

- `lock=True` — derives the lock name from `"ClassName.method_name"`.
- `lock="shared-name"` — explicit name; useful when two methods must be mutually exclusive.
- `lock_ttl` — safety-valve TTL; set it comfortably longer than the job's worst-case runtime.

By default `TaskScheduler` uses `LocalLock`, which always acquires — single-instance behaviour is unchanged. For cross-process coordination, implement `DistributedLock` and register it as a bean:

`lumen/infra/redis_lock.py`

```
from pyfly.container import bean, configuration
from pyfly.scheduling import DistributedLock

class RedisLock:
    """Best-effort named lock backed by Redis SET NX PX."""

    def __init__(self, redis) -> None:
        self._redis = redis
```

```

async def try_acquire(self, name: str, ttl: float) -> bool:
    ok = await self._redis.set(
        f"pyfly:lock:{name}", "1",
        nx=True, px=int(ttl * 1000),
    )
    return ok is True

async def release(self, name: str) -> None:
    await self._redis.delete(f"pyfly:lock:{name}")

@configuration
class LockConfig:

    @bean
    def distributed_lock(self) -> DistributedLock:
        import redis.asyncio as aioredis
        client = aioredis.from_url("redis://localhost:6379/1")
        return RedisLock(client)

```

Listing 17.4 — Redis-backed DistributedLock

`SchedulingAutoConfiguration` detects the `DistributedLock` bean in the container automatically and passes it to the `TaskScheduler`. Any object with conforming `try_acquire` and `release` coroutines satisfies the protocol.

@async_method

`@async_method` marks a method for fire-and-forget execution via the `TaskExecutorPort`. The caller returns immediately; the framework routes the coroutine through the configured executor in the background:

```

from pyfly.scheduling import async_method

@service
class AlertService:

```

```

@async_method
async def send_alert(self, msg: str) -> None:
    """Caller does not await – AlertService dispatches asynchronously."""
    ...

```

Under the hood `@async_method` sets `__pyfly_async__ = True` on the function; the framework detects this flag and submits the coroutine to the `TaskExecutorPort`.

🍃 SPRING PARITY

```

@scheduled(fixed_rate=...) mirrors Spring's @Scheduled(fixedRate=...) .
@scheduled(fixed_delay=...) mirrors @Scheduled(fixedDelay=...) .
@scheduled(cron=...) mirrors @Scheduled(cron=...) . zone= mirrors Spring's zone
attribute. lock=True mirrors ShedLock's @SchedulerLock . @async_method mirrors
Spring's @Async .

```

Configuration reference

```

pyfly:
  scheduling:
    enabled: true          # set false to disable all loops
  thread-pool:
    max-workers: 4        # threads for ThreadPoolTaskExecutor

```

When `enabled` is `false`, `TaskScheduler` starts no loops and all `@scheduled` methods are silently skipped.

Notifications

Lumen needs to tell customers that their money has arrived — email for the balance confirmation, and optionally an SMS or mobile push for the real-time alert.

PyFly's notifications module defines three **port protocols** and three **default services**. Your business logic depends on the protocols; the concrete provider adapters — SMTP, SendGrid, Twilio, Firebase — live behind the port boundary and can be swapped without touching a single line of domain code.

The port hierarchy

Protocol	Service class	Method
EmailProvider	DefaultEmailService	send(EmailMessage) -> NotificationResult
SmsProvider	DefaultSmsService	send(SmsMessage) -> NotificationResult
PushProvider	DefaultPushService	send(PushMessage) -> NotificationResult

DefaultEmailService, DefaultSmsService, and DefaultPushService are thin wrappers: each delegates to a provider, catches any provider exception, and returns a structured NotificationResult with status=FAILED and the error string rather than propagating the exception. A transient SendGrid outage does not take down the deposit handler.

Messages and results

FundsDeposited carries amount and balance as integer minor units (cents). The Money.major_units property converts them for display — Money(25000, Currency.EUR).major_units is 250.0. Keep that in mind when formatting notification bodies.

```
lumen/notifications/models_overview.py
```

```
from pyfly.notifications import (
    EmailMessage,
    NotificationResult,
```



```

        PushMessage,
        SmsMessage,
    )

    # Email – full field set
    email = EmailMessage(
        to=["alice@example.com"],
        sender="no-reply@lumenbank.com",
        subject="Funds received",
        body_text="EUR 250.00 has been credited to your wallet.",
        body_html=(
            "<p><strong>EUR 250.00</strong> has been credited "
            "to your wallet.</p>"
        ),
    )

    # SMS – compact
    sms = SmsMessage(
        to="+34600000001",
        body="Lumen: EUR 250.00 received. New balance: EUR 750.00.",
        sender="LUMEN",
    )

    # Push – structured payload
    push = PushMessage(
        device_tokens=["FCM_TOKEN_GOES_HERE"],
        title="Funds received",
        body="EUR 250.00 credited",
        data={"wallet_id": "w-001", "amount_minor": 25000},
    )

```

Listing 17.5 — The core DTOs

`NotificationResult` carries `id`, `provider`, `status` (`EmailStatus.SENT` | `DELIVERED` | `FAILED` | ...), an optional `provider_id` (e.g. the SendGrid message ID), and an optional `error`.

Wiring the SMTP provider

For development and self-hosted deployments, `SmtplibEmailProvider` runs `smtplib` from a thread pool so the async event loop is never blocked:

`lumen/notifications/config.py`

```
from pyfly.container import bean, configuration
from pyfly.notifications import DefaultEmailService, EmailService
from pyfly.notifications.providers.smtp import SmtplibEmailProvider

@configuration
class NotificationConfig:

    @bean
    def email_provider(self) -> SmtplibEmailProvider:
        return SmtplibEmailProvider(
            "smtp.lumenbank.internal",
            port=587,
            username="notifications",
            password="s3cr3t",
            use_tls=True,
        )

    @bean
    def email_service(
        self, provider: SmtplibEmailProvider,
    ) -> EmailService:
        return DefaultEmailService(provider=provider)
```

Listing 17.6 — SMTP provider wired as a `@bean`

`SmtplibEmailProvider` accepts `host`, `port` (default `587`), `username`, `password`, and `use_tls` (default `True`). Swap it for `SendGridEmailProvider` or `ResendEmailProvider` by changing one `@bean` method — `DefaultEmailService` is indifferent to the provider behind it.

 AVAILABLE PROVIDERS

pyfly.notifications ships eight built-in adapters: `DummyEmailProvider` / `DummySmsProvider` / `DummyPushProvider` (log-only, for dev/tests), `SmtplibEmailProvider`, `SendGridEmailProvider`, `ResendEmailProvider` (email), `TwilioSmsProvider` (SMS), and `FirebasePushProvider` (push). All satisfy their respective `*Provider` protocol.

Sending a "funds received" notification

Lumen publishes a `FundsDeposited` domain event every time the `deposit()` command succeeds (see Chapter 8). The right place to trigger the notification is an EDA listener subscribed to that event — not the command handler itself, which keeps the deposit path free of notification concerns.

`FundsDeposited` carries `wallet_id: str`, `amount: int` (minor units), `currency: str`, and `balance: int` (new balance, minor units). The listener converts `amount` to a display string via `amount / 100`:

`lumen/wallet/deposit_notification_listener.py`

```
from pyfly.container import service
from pyfly.eda import EventEnvelope, event_listener
from pyfly.notifications import (
    EmailMessage,
    EmailService,
    PushMessage,
    PushService,
)

@service
class DepositNotificationListener:
    """Sends email + push when a FundsDeposited event is observed."""

    def __init__(
        self,
```

```

        email_service: EmailService,
        push_service: PushService,
    ) -> None:
        self._email = email_service
        self._push = push_service

    @event_listener(event_types=["FundsDeposited"])
    async def on_funds_deposited(
        self, envelope: EventEnvelope,
    ) -> None:
        payload = envelope.payload
        wallet_id = str(payload.get("wallet_id", ""))
        amount_minor = int(payload.get("amount", 0))
        currency = str(payload.get("currency", "EUR"))
        balance_minor = int(payload.get("balance", 0))
        amount_str = f"{amount_minor / 100:.2f} {currency}"
        balance_str = f"{balance_minor / 100:.2f} {currency}"

        # Fetch contact details from a wallet profile service in prod;
        # hardcoded here for illustration.
        email = "customer@example.com"
        device_token = "FCM_TOKEN_GOES_HERE"

        await self._email.send(EmailMessage(
            to=[email],
            sender="no-reply@lumenbank.com",
            subject=f"Funds received: {amount_str}",
            body_text=(
                f"{amount_str} has been credited to wallet "
                f"{wallet_id}. New balance: {balance_str}."
            ),
        ))

        await self._push.send(PushMessage(
            device_tokens=[device_token],
            title="Funds received",
            body=f"{amount_str} credited",
            data={
                "wallet_id": wallet_id,
                "amount_minor": amount_minor,
            }
        ))

```

```

        "currency": currency,
    },
))

```

Listing 17.7 — Notifying on FundsDeposited

Both calls return a `NotificationResult` ; inspect the `status` field to log failures or schedule retries.

SPRING PARITY

`EmailService` / `SmsService` / `PushService` are the Python equivalents of Spring's `JavaMailSender` (email) and third-party Spring integrations for SMS and push. The hexagonal port/adaptor split is identical: your domain code depends on the protocol; concrete provider adapters live in the infrastructure layer.

Inbound webhooks

An illustrative payment provider POSTs a `payment_intent.succeeded` event to Lumen whenever a customer tops up their wallet by card. Lumen must:

1. **verify the HMAC-SHA256 signature** to reject forged payloads;
2. **deduplicate** replays using the idempotency key so a retry does not credit a wallet twice;
3. **dispatch** the verified event to a typed listener.

PyFly's `pyfly.webhooks` module handles all three steps.

WebhookEvent and AbstractWebhookEventListener

Every inbound event is modelled as a `WebhookEvent` dataclass:

```

@dataclass
class WebhookEvent:
    id: str          # auto-generated UUID
    source: str       # e.g. "payment-provider"
    event_type: str   # from body["type"]
    headers: dict[str, str]
    body: dict[str, Any]
    raw_body: bytes
    received_at: datetime
    idempotency_key: str | None

```

Subclass `AbstractWebhookEventListener` and set `source` to the name you will pass to `WebhookProcessor.process()`:

`lumen/webhooks/payment_listener.py`

```

from pyfly.container import service
from pyfly.webhooks import AbstractWebhookEventListener, WebhookEvent

@service
class PaymentWebhookListener(AbstractWebhookEventListener):
    source = "payment-provider"

    def __init__(self, deposit_handler) -> None:
        self._handler = deposit_handler

    async def handle(self, event: WebhookEvent) -> None:
        if event.event_type == "payment_intent.succeeded":
            pi = event.body.get("data", {}).get("object", {})
            wallet_id = pi.get("metadata", {}).get("wallet_id")
            amount_minor = int(pi.get("amount_received", 0))
            currency_code = pi.get("currency", "EUR").upper()
            if wallet_id and amount_minor > 0:
                # Delegate to the CQRS command handler so the aggregate
                # enforces the balance invariant and raises FundsDeposited.
                from lumen.core.services.wallets.deposit_funds_command import
(

```

```

        DepositFunds,
    )
    await self._handler.handle(
        DepositFunds(
            wallet_id=wallet_id,
            amount=amount_minor,
        )
    )

    async def on_error(
        self, event: WebhookEvent, error: BaseException,
    ) -> None:
        # Override to DLQ or page on-call.
        pass

```

Listing 17.8 — Payment-provider webhook listener

`on_error` is called when `handle` raises; the default is a no-op. Override it to publish to a dead-letter queue or emit a metric.

WebhookProcessor — verify, dedupe, dispatch

`WebhookProcessor` wires together a signature validator, an idempotency store, and a list of listeners:

`lumen/webhooks/processor_config.py`

```

from pyfly.container import bean, configuration
from pyfly.webhooks import (
    HmacSignatureValidator,
    WebhookProcessor,
)
from lumen.webhooks.payment_listener import PaymentWebhookListener

@configuration
class WebhookConfig:

```

```

@Bean
def webhook_processor(
    self, payment_listener: PaymentWebhookListener,
) -> WebhookProcessor:
    return WebhookProcessor(
        listeners=[payment_listener],
        signature_validators={
            "payment-provider": HmacSignatureValidator(
                secret="whsec_REPLACE_ME",
            ),
        },
    )

```

Listing 17.9 — Assembling WebhookProcessor

`HmacSignatureValidator` expects the `sha256=<hex>` header format and uses `hmac.compare_digest` for a constant-time comparison. Change the `header_prefix` parameter if your provider uses a different scheme.

Handling a webhook in an HTTP handler

Call `processor.process()` from your inbound HTTP handler. Pass the raw request body (unmodified bytes) — the validator computes the HMAC over the exact bytes received:

`lumen/webhooks/payment_handler.py`

```

from pyfly.container import service
from pyfly.web import Request, Response, router
from pyfly.webhooks import WebhookProcessor

@Service
class PaymentWebhookHandler:

    def __init__(self, processor: WebhookProcessor) -> None:
        self._processor = processor

```



```

@router.post("/webhooks/payment")
async def receive(self, request: Request) -> Response:
    raw_body = await request.body()
    headers = {
        "X-Signature": request.headers.get(
            "X-Webhook-Signature", ""
        ),
        "X-Idempotency-Key": request.headers.get(
            "X-Idempotency-Key", ""
        ),
    }
    try:
        await self._processor.process(
            source="payment-provider",
            raw_body=raw_body,
            headers=headers,
        )
    except ValueError:
        return Response(status=400, body=b"invalid signature")
    return Response(status=200, body=b"ok")

```

Listing 17.10 — Inbound payment-provider webhook endpoint

The `process()` signature accepts `signature_header` and `idempotency_header` keyword arguments to override the default header names (`X-Signature` and `X-Idempotency-Key`).

What happens inside `process()` :

1. The validator is looked up by `source` ; if none is registered, a `NoOpSignatureValidator` is used — always passes, safe for development but not production.
2. If signature validation fails, `ValueError` is raised immediately and no listeners are called.

3. The raw body is decoded as JSON; on failure the raw bytes are stored under `body["_raw"]`.
4. If `idempotency_key` is present and already seen, the event is returned but listeners are **not** called.
5. Each listener for the source is called in registration order; if one raises, the error is logged and `on_error` is called before continuing to the next listener.

IN-MEMORY IDEMPOTENCY STORE

The default `InMemoryWebhookEventStore` holds seen keys in a Python `set`. For production clusters, implement the `WebhookEventStore` protocol backed by Redis or a database: `python class RedisEventStore: async def already_processed( self, key: str, ) -> bool: ... async def remember(self, key: str) -> None: ...` Pass it as `event_store=RedisEventStore(...)` to `WebhookProcessor`.

Outbound callbacks

When Lumen books a disbursement to a partner bank, that partner expects a `DisbursementSettled` POST to their webhook URL — signed, retried on failure, and auditable. PyFly's `pyfly.callbacks` module handles the outbound side.

Subscriptions and config

Each partner is modelled as a `CallbackConfig` — a tenant-scoped record that holds the webhook secret, event subscriptions, and retry policy:

```
lumen/callbacks/register_partner.py
```

```
from pyfly.callbacks import (
    CallbackConfig,
    CallbackSubscription,
```

```

    InMemoryCallbackConfigRepository,
    InMemoryCallbackExecutionRepository,
)

async def register_clearance_bank(configs) -> None:
    await configs.save(CallbackConfig(
        tenant_id="lumen",
        name="clearance-bank",
        secret="cb-secret-xyz",
        max_attempts=5,
        backoff_ms=2_000,
        subscriptions=[
            CallbackSubscription(
                event_type="DisbursementSettled",
                target_url=(
                    "https://api.clearancebank.example.com"
                    "/hooks/lumen"
                ),
            ),
            CallbackSubscription(
                event_type="*",
                target_url=(
                    "https://audit.clearancebank.example.com"
                    "/all-events"
                ),
            ),
        ],
    ))

```

Listing 17.11 — Registering a partner callback

`event_type="*"` is a catch-all: every event dispatched for the tenant matches. Named types match only the exact event type string.

Dispatching an event

`CallbackDispatcher.dispatch()` fans the event out to every matching subscription:

lumen/callbacks/dispatcher_config.py

```

from pyfly.callbacks import (
    CallbackDispatcher,
    InMemoryCallbackConfigRepository,
    InMemoryCallbackExecutionRepository,
)
from pyfly.container import bean, configuration

@configuration
class CallbackConfig_:

    @bean
    def callback_configs(self) -> InMemoryCallbackConfigRepository:
        return InMemoryCallbackConfigRepository()

    @bean
    def callback_executions(
        self,
    ) -> InMemoryCallbackExecutionRepository:
        return InMemoryCallbackExecutionRepository()

    @bean
    def callback_dispatcher(
        self,
        configs: InMemoryCallbackConfigRepository,
        executions: InMemoryCallbackExecutionRepository,
    ) -> CallbackDispatcher:
        return CallbackDispatcher(
            configs=configs,
            executions=executions,
        )

```

Listing 17.12 — Wiring and calling CallbackDispatcher

Then in the domain service — note the payload uses `amount` in minor units (cents), so `50_000` is EUR 500.00:

```
results = await dispatcher.dispatch(
    "lumen",          # tenant_id
    "DisbursementSettled",
    {"id": "txn-009", "amount": 50_000, "currency": "EUR"},
)
```

`dispatch()` returns one `CallbackExecution` record per matching subscription, each with `status`, `attempts`, `response_status`, and `delivered_at`.

HMAC signing and retry logic

When `CallbackConfig.secret` is set, `CallbackDispatcher` signs the canonical JSON payload before every POST using HMAC-SHA256:

```
# canonical body – compact, keys sorted
canonical = json.dumps(payload, separators=(",", ":"), sort_keys=True)
sig = hmac.new(secret, canonical.encode(), hashlib.sha256).hexdigest()
headers["X-Pyfly-Signature"] = f"sha256={sig}"
```

The recipient can verify the signature using PyFly's own `HmacSignatureValidator` — the same class used for inbound webhooks.

Retry policy:

- The dispatcher retries up to `max_attempts` times (default `5`).
- Between retries it applies exponential backoff: `delay = min(backoff_ms * 2(attempt-1), 300_000) ms`.
- Only *transient* HTTP status codes trigger a retry: `408`, `429`, `500`, `502`, `503`, `504`, or any `>= 500`.
- Permanent client errors (`4xx` except `408 / 429`) mark the execution as `FAILED` immediately without retrying.
- On success (`2xx`) the execution is marked `DELIVERED` and `delivered_at` is stamped.

lumen/callbacks/models_overview.py

```

from pyfly.callbacks import CallbackStatus

# After a successful delivery:
assert execution.status == CallbackStatus.DELIVERED
assert execution.delivered_at is not None
assert execution.response_status == 200

# After all retries are exhausted:
assert execution.status == CallbackStatus.FAILED
assert execution.attempts == 5
assert execution.last_error is not None

```

Listing 17.13 — CallbackExecution status lifecycle

SSRF protection — authorized domains

The `authorized_domains` field on `CallbackConfig` acts as an allowlist. When set, `CallbackDispatcher` checks that the target URL's hostname matches one of the allowed domains before making any outbound request. A URL that fails the check is immediately marked `FAILED` with `last_error="Domain not authorized"` — no HTTP request is made.

```

from pyfly.callbacks import AuthorizedDomain, CallbackConfig

config = CallbackConfig(
    tenant_id="lumen",
    name="safe-config",
    secret="s3cr3t",
    authorized_domains=[
        AuthorizedDomain(domain="clearancebank.example.com"),
    ],
    subscriptions=[...],
)

```

Subdomains of allowed domains are also accepted (e.g.

`api.clearancebank.example.com`).

🌿 SPRING PARITY

PyFly's `@scheduled` / `CronExpression` / `TaskScheduler` trio mirrors Spring's `@Scheduled` / `CronExpression` / `ThreadPoolTaskScheduler`. Notification ports map to Spring's `MailSender` / `JavaMailSender`. `WebhookProcessor` corresponds to a Spring `@RestController` + Spring Security's `HMAC HmacRequestMatcher`. `CallbackDispatcher` with HMAC signing and retry mirrors Spring's `WebhookPublisher` pattern from Spring Modulith.

✓ What you built

You extended Lumen into a connected system that operates independently of incoming requests:

- **Scheduled tasks** — `@scheduled` with `cron`, `fixed_rate`, and `fixed_delay` triggers runs work on a calendar or timer. `CronExpression` drives fire-time calculations, including time-zone-aware scheduling with DST handled automatically. `lock=True` serialises cluster-wide execution via the `DistributedLock` port. `@async_method` offloads fire-and-forget work to the executor.
- **Notifications** — `EmailService`, `SmsService`, and `PushService` port protocols decouple business logic from provider adapters (SMTP, SendGrid, Resend, Twilio, Firebase). `DefaultEmailService` and its siblings catch provider errors and return structured `NotificationResult` values rather than propagating exceptions. `DepositNotificationListener` subscribes to the real `FundsDeposited` EDA event and converts minor-unit amounts to display strings before sending.

- **Inbound webhooks** — `AbstractWebhookEventListener` defines typed consumers. `WebhookProcessor` gates every event with `HmacSignatureValidator` and `WebhookEventStore` before dispatching to listeners. `on_error()` hooks allow DLQ integration without breaking the dispatch loop.
- **Outbound callbacks** — `CallbackDispatcher` fans events out to `CallbackSubscription` targets, signs payloads with HMAC-SHA256 under the `X-Pyfly-Signature` header, and retries on transient failures with exponential backoff. `CallbackExecution` records provide a full delivery audit trail. `authorized_domains` prevents SSRF.

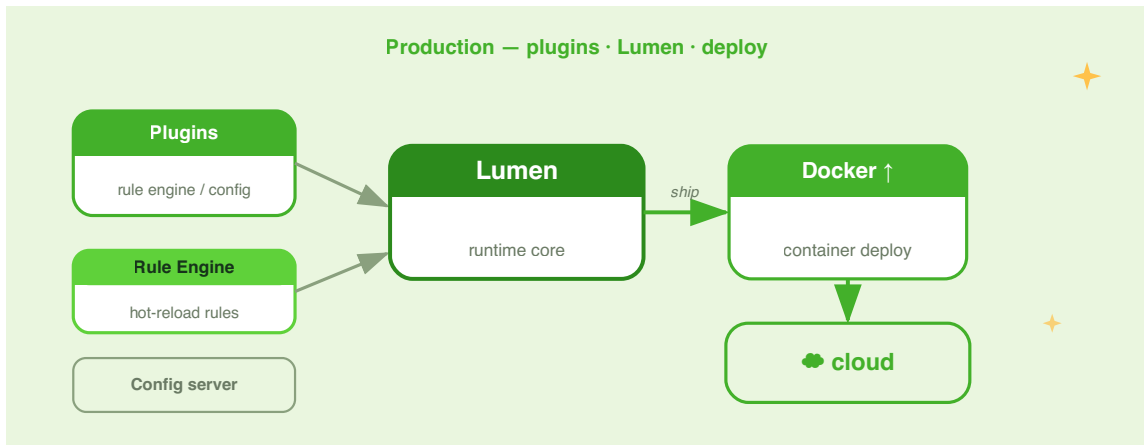
Try it yourself

1. **Cluster lock.** Add a second instance of `DailyRollupService` to a test that spins up two `ApplicationContext` instances sharing a `FakeDistributedLock` (one that only returns `True` for the first acquirer). Assert that `run()` is called exactly once across both instances for a single cron tick.
2. **Provider swap.** Replace `SmtplibEmailProvider` with a `DummyEmailProvider` in the test suite. Write a test that deposits EUR 100 (10 000 minor units) into wallet `w-001` via the deposit handler, triggering a `FundsDeposited` event. Assert that `DummyEmailProvider.last_message` contains the wallet ID and the formatted amount (`100.00 EUR`) in its `body_text`.
3. **Signature replay attack.** Write a test that calls `PaymentWebhookHandler.receive()` twice with the same raw body, headers, and idempotency key. Assert that the first call returns 200 and credits the wallet

(triggering `FundsDeposited`), and the second call also returns 200 (the duplicate is silently ignored by `InMemoryWebhookEventStore`) but does **not** credit the wallet a second time.

CHAPTER 18

Extending PyFly & Going to Production



Lumen is no longer a toy. Over the past seventeen chapters you built a wallet service from a single annotated class to a full event-driven microservice — CQRS, sagas, EDA, HTTP clients, caching, resilience, observability, security, and a test suite backed by real containers. You know how to run it, debug it, and deploy it.

This final chapter is about the distance between "it works on my laptop" and "it runs reliably for real users." That distance has three dimensions. First: **extensibility** — the ability to add behaviour without forking the framework. Second: the features your domain may need right now — a rules engine, centralised config, multiple languages, real-time push, a CLI. Third: the operational habits that separate a weekend project from a production service.

You will move quickly. Every section picks one topic, shows the minimal working API, and connects it to Lumen. By the end you will have a complete picture of the PyFly ecosystem and a production checklist worth keeping close.

Plugins and extension points

Why a plugin system?

The core framework is intentionally small. Optional features — formatters, audit sinks, notification channels — should be pluggable so teams can compose only what they need and ship additions without touching framework internals.

PyFly's `pyfly.plugins` module mirrors Spring's plugin registry: you declare an **extension point** (a named slot), **extensions** (concrete contributions), and bundle them into a **plugin** with a lifecycle.

`lumen/plugins/audit.py`

```
from pyfly.plugins import (
    PluginManager,
    extension,
    extension_point,
    plugin,
)

@extension_point(id="audit-sinks")
class AuditSink:
    """Interface that all audit sinks must implement."""

    def record(self, event: dict) -> None: ...

@plugin(id="console-audit", version="1.0.0")
```

```

class ConsoleAuditPlugin:

    @extension(point="audit-sinks", priority=10)
    class ConsoleSink(AuditSink):
        name = "console"

        def record(self, event: dict) -> None:
            print(f"[AUDIT] {event}")

    async def start(self) -> None:
        print("ConsoleAuditPlugin started")

    async def stop(self) -> None:
        print("ConsoleAuditPlugin stopped")

```

Listing 18.1 — Declaring an audit-sink plugin

How it works. `@extension_point(id="audit-sinks")` registers a named slot and declares the interface every contribution must implement. `@plugin(id="console-audit", version="1.0.0")` declares a plugin class with a mandatory `id` and `version`. `@extension(point="audit-sinks", priority=10)` marks an inner class as a contribution to that slot; the inner class must inherit the extension-point interface so the registry can validate it. Higher priority wins first position when iterating results.

Loading and running the plugin:

`lumen/plugins/runner.py`

```

import asyncio

from pyfly.plugins import PluginManager

from lumen.plugins.audit import ConsoleAuditPlugin

async def main() -> None:

```

```

manager = PluginManager()
await manager.add(ConsoleAuditPlugin)
await manager.start_all()

sinks = await manager.registry.get("audit-sinks")
for sink in sinks:
    sink.record({"action": "deposit", "amount_minor": 100})

await manager.stop_all()

asyncio.run(main())

```

Listing 18.2 — Driving the plugin lifecycle

`PluginManager.add()` inspects the class for nested `@extension_point` declarations, then registers each `@extension` contribution. `start_all()` invokes each plugin's `init` then `start` hooks in dependency order; `stop_all()` reverses the sequence, calling `stop` then `unload`. Circular dependencies raise `PluginResolutionError` before any code runs.

Method	Description
<code>await manager.add(cls)</code>	Scan and register a plugin class
<code>await manager.start_all()</code>	<code>init</code> → <code>start</code> in dependency order
<code>await manager.stop_all()</code>	<code>stop</code> → <code>unload</code> in reverse order
<code>await manager.remove(plugin_id)</code>	Unload one plugin; returns <code>False</code> if unknown
<code>await manager.registry.get(point_id)</code>	Extensions for a slot, priority-sorted

🍃 SPRING PARITY

`@plugin / @extension_point / @extension mirror @Plugin / ExtensionPoint / @Extension` from Spring's plugin API. `PluginManager.start_all()` plays the role of the Spring plugin container's lifecycle management. The dependency-order boot and reverse-order shutdown are identical in semantics.

Business rules with the Rule Engine

Most real-world services carry logic that belongs to the business, not the code: "flag orders over 500,000 cents", "block shipments to sanctioned regions", "apply a surcharge after hours." Hard-coding those thresholds in Python means a rebuild every time the business changes its mind.

PyFly's `pyfly.rule_engine` gives product owners a YAML dial they can turn without touching source code.

Defining rules in YAML

Rules live in a separate YAML file that any team member can review. The evaluator parses this file once at startup — or on each fetch if you hot-reload from a Config Server (see the next section).

`lumen/rules/transaction_rules.yaml`

```
id: transaction-rules
name: Lumen transaction rules

rules:
  - id: daily-limit
    priority: 20
    when:
      op: ge
```

```

    field: transaction.daily_total
    value: 500000
  then:
    - type: set
      target: flags.limit_exceeded
      value: true
    - type: log
      value: "daily limit exceeded"

- id: fraud-country
  priority: 10
  when:
    op: in
    field: transaction.country
    value: ["XX", "YY", "ZZ"]
  then:
    - type: set
      target: flags.fraud_risk
      value: true
    - type: log
      value: "high-risk country detected"

- id: high-value
  priority: 5
  when:
    op: ge
    field: transaction.amount
    value: 100000
  then:
    - type: set
      target: flags.high_value
      value: true

```

Listing 18.3 — Fraud and daily-limit rules (amounts in minor units)

Each rule has a `when` condition and a list of `then` actions. Amounts are always **integer minor units** (cents), so `100000` is €1,000.00. Conditions use these operators:

Comparison	Logical
<code>eq</code> , <code>ne</code> , <code>gt</code> , <code>ge</code> , <code>lt</code> , <code>le</code>	<code>and</code> , <code>or</code> , <code>not</code>
<code>in</code> , <code>not_in</code> , <code>regex</code>	(with <code>conditions: [...]</code>)

Actions are `set` (write to a context path), `increment`, or `log`. Subclass `RuleEvaluator` and override `_execute_action` to add `call`, `calculate`, or any custom verb.

Evaluating rules in a service

`lumen/rules/risk_service.py`

```

from pathlib import Path

from pyfly.container import service
from pyfly.rule_engine import RuleSetEvaluator, RuleSetLoader

@service
class RiskService:
    """Evaluate transaction-level rules and return risk flags."""

    def __init__(self) -> None:
        yaml_text = (
            Path(__file__).parent / "transaction_rules.yaml"
        ).read_text()
        self._ruleset = RuleSetLoader.from_yaml(yaml_text)
        self._evaluator = RuleSetEvaluator()

    def assess(
        self,
        amount: int,
        daily_total: int,
        country: str,
    ) -> dict:

```



```

ctx = {
    "transaction": {
        "amount": amount,
        "daily_total": daily_total,
        "country": country,
    },
    "flags": {},
}

self._evaluator.evaluate(self._ruleset, ctx)
return ctx["flags"]

```

Listing 18.4 — Evaluating rules against a transaction

How it works. `RuleSetLoader.from_yaml(text)` parses the YAML into an AST. `RuleSetEvaluator.evaluate(ruleset, ctx)` walks every rule in priority order, evaluates the `when` clause, and applies matching `then` actions — mutating `ctx` in place and returning a `list[EvaluationResult]`. The `flags` dict is the authoritative output: a downstream handler rejects, queues, or flags the transaction based on whatever keys are set. `amount` and `daily_total` are in minor units (cents) to match the Lumen domain.

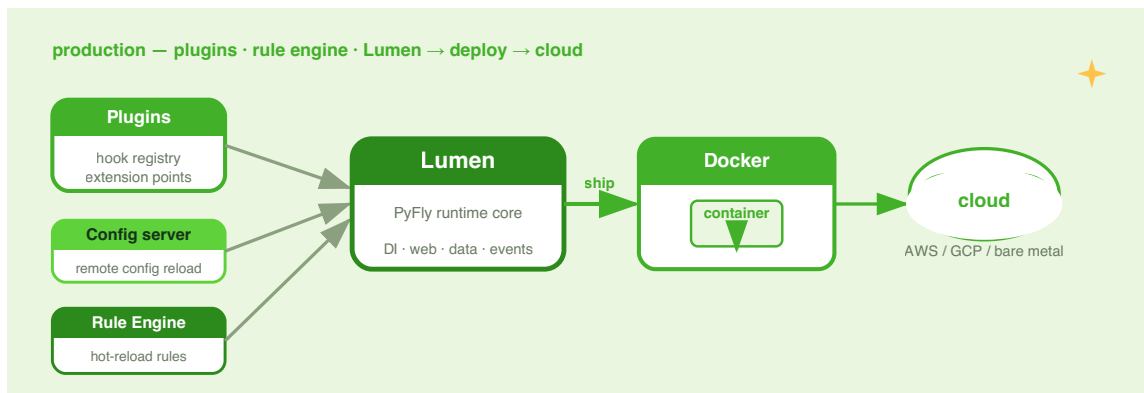


Figure 18.1 — Rule evaluation at the service boundary. YAML rules are parsed once at startup; each transaction passes through the evaluator as a mutable context dict.

💡 HOT-RELOAD RULES WITHOUT REDEPLOYMENT

Store `transaction_rules.yaml` in the Config Server (see the next section) and re-parse on every fetch. Your rules engine becomes a live dial the business controls.

Centralised config (Config Server)

As Lumen grows to multiple services, each carries its own copy of database URLs, timeouts, and feature flags. The Config Server module removes that duplication: one service owns the truth; every other service fetches on startup.

Running the server

Enable the server in `pyfly.yaml`:

```
pyfly:
  config-server:
    enabled: true
    backend:
      root: /etc/lumen/config
```

That is all. PyFly auto-configures a `ConfigServer` backed by a `FilesystemConfigBackend` and mounts the HTTP routes automatically:

Method	Path	Purpose
GET	<code>/app/{profile}</code>	Fetch merged config bundle
GET	<code>/app/{profile}/{label}</code>	Fetch for a specific label
POST	<code>/app/{profile}</code>	Save a config bundle
GET	<code>/_list</code>	List all stored bundles

The response shape is Spring Cloud Config-compatible, so an existing Spring service can consume the same endpoint without changes.

Saving and fetching config programmatically

`lumen/config/seed.py`

```
import asyncio

from pyfly.config_server import (
    ConfigClient,
    ConfigServer,
    FilesystemConfigBackend,
)

async def seed() -> None:
    server = ConfigServer(FilesystemConfigBackend("/etc/lumen/config"))
    await server.save(
        "wallet",
        "prod",
        {
            "db.url": "postgres://db:5432/lumen",
            "cache.ttl": 30,
        },
    )
    bundle = await server.fetch("wallet", "prod")
    print(bundle)

asyncio.run(seed())
```

Listing 18.5 — Seeding and reading a config bundle

Client services fetch on startup with:

`lumen/config/bootstrap.py`

```

from pyfly.config_server import ConfigClient

async def load_remote() -> dict:
    client = ConfigClient(
        url="http://config:8888",
        application="wallet",
        profile="prod",
        label="main",
    )
    return await client.fetch()

```

Listing 18.6 — Fetching remote config at startup

`ConfigClient.fetch()` GETs `{url}/{application}/{profile}/{label}`, merges the `propertySources` array (highest priority first), and returns a flat `{dotted_key: value}` dict. In normal operation you never call `ConfigClient` directly — set `pyfly.cloud.config.uri` in `pyfly.yaml` and `PyFlyApplication` calls it automatically during bootstrap, merging the result into the application `Config` as a high-precedence source.

FALLBACK PRIORITY

The server assembles up to four overlay layers: `{app}/{profile}`, `{app}/default`, `application/{profile}`, `application/default`. A client merges them with the first source winning, so environment-specific overrides always beat shared defaults.

Internationalisation (i18n)

Lumen's error messages and notifications currently live as Python string literals. When users speak different languages, that approach does not scale.

Enable the i18n subsystem with a single flag:

```
pyfly:
  i18n:
    enabled: true
    base-path: i18n/
    default-locale: en
```

Writing resource bundles

```
# i18n/messages_en.yaml
wallet:
  deposit_ok: "Deposited {0} minor units to wallet {1}."
  limit_exceeded: "Daily limit exceeded. Maximum is {0} minor units."

# i18n/messages_es.yaml
wallet:
  deposit_ok: "Se depositaron {0} unidades menores en la billetera {1}."
  limit_exceeded: "Se superó el límite diario. El máximo es {0} unidades menores."
```

`ResourceBundleMessageSource` resolves keys with dot notation and substitutes `{n}` placeholders (zero-based) following `MessageFormat` semantics. Missing codes fall back to `default-locale`; if they are absent there too, `get_message` raises `KeyError`.

Using MessageSource in a service

`lumen/i18n/notification_service.py`

```
from pyfly.container import service
from pyfly.i18n import AcceptHeaderLocaleResolver, MessageSource

@service
class NotificationService:
    """Renders user-facing messages in the caller's preferred locale."""

    def __init__(
```

```

        self,
        messages: MessageSource,
        locale_resolver: AcceptHeaderLocaleResolver,
    ) -> None:
        self._messages = messages
        self._resolver = locale_resolver

    def deposit_confirmation(
        self,
        request,
        amount_minor: int,
        wallet_id: str,
    ) -> str:
        locale = self._resolver.resolve_locale(request)
        return self._messages.get_message_or_default(
            "wallet.deposit_ok",
            default="Deposit successful.",
            args=(amount_minor, wallet_id),
            locale=locale,
        )

```

Listing 18.7 — Locale-aware notification service

`AcceptHeaderLocaleResolver` parses the `Accept-Language` header and returns the highest-`q` primary subtag. Use `FixedLocaleResolver` for single-language deployments or tests. Auto-configuration registers both when `pyfly.i18n.enabled: true`; inject either `MessageSource` (the port protocol) or the concrete `ResourceBundleMessageSource` — both work.

🍃 SPRING PARITY

`MessageSource`, `ResourceBundleMessageSource`, `AcceptHeaderLocaleResolver`, and `FixedLocaleResolver` are direct name equivalents of the Spring MVC i18n stack. The API differs only in the use of positional `{n}` placeholders instead of SpEL inside message strings.

Real-time updates with WebSocket

The Lumen admin dashboard currently polls for balance changes. A WebSocket endpoint eliminates the poll — the server pushes an update the instant a deposit commits.

`lumen/web/balance_ws_controller.py`

```
import asyncio

from pyfly.container import rest_controller
from pyfly.web import request_mapping
from pyfly.websocket import WebSocketSession, websocket_mapping

@rest_controller
@request_mapping("/ws")
class BalanceFeedController:
    """Streams balance updates to connected clients."""

    def __init__(self, wallet_service) -> None:
        self._wallet = wallet_service
        self._clients: set[WebSocketSession] = set()

    @websocket_mapping("/balance/{wallet_id}")
    async def balance_feed(self, session: WebSocketSession) -> None:
        wallet_id = session.path_params["wallet_id"]
        await session.accept()
        self._clients.add(session)
        try:
            while True:
                balance = await self._wallet.get_balance(wallet_id)
                await session.send_json(
                    {"wallet_id": wallet_id, "balance_minor": balance}
                )
                await asyncio.sleep(1)
```

```

    finally:
        self._clients.discard(session)

    async def on_disconnect(self, session: WebSocketSession) -> None:
        self._clients.discard(session)

```

Listing 18.8 — Live balance feed via WebSocket

How it works. `@websocket_mapping("/balance/{wallet_id}")` mounts the endpoint at `ws://<host>/ws/balance/{wallet_id}`. The full path is the controller's `@request_mapping` base (`/ws`) concatenated with the decorator's path.

`WebSocketSession` exposes the connection lifecycle:

Method	Description
<code>await accept(subprotocol=None)</code>	Complete the handshake
<code>await send_json(data)</code>	Serialise and send a JSON message
<code>await send_text(data)</code>	Send a plain string
<code>await receive_text()</code>	Block until a text message arrives
<code>await receive_json()</code>	Block until a JSON message arrives
<code>await close(code=1000, reason=None)</code>	Close the connection cleanly

`session.path_params`, `session.query_params`, and `session.headers` expose connection metadata. WebSocket routes are auto-discovered alongside HTTP routes — no extra configuration required.

The optional `on_disconnect` method is invoked automatically by the registrar after the `@websocket_mapping` handler returns or the client closes the connection (only if the connection was accepted first), giving controllers a safe place to release resources.

 BROADCASTING

Keep a `set[WebSocketSession]` on the controller and fan out with `for client in list(self._clients): await client.send_json(payload)`. Because controller beans are singletons the set lives for the lifetime of the application.

Shell commands and startup runners

Not every feature lives behind an HTTP endpoint. Database seed scripts, one-time data migrations, and scheduled batch jobs are better expressed as CLI commands that run inside the same DI container — sharing services, configuration, and repositories with the main application.

@shell_component and @shell_method

`lumen/cli/wallet_commands.py`

```
from pyfly.shell import (
    shell_argument,
    shell_component,
    shell_method,
    shell_option,
)

@shell_component
class WalletCommands:
    """Operational commands for the Lumen wallet service."""

    def __init__(self, wallet_service) -> None:
        self._wallet = wallet_service

    @shell_method(group="wallet", help="Deposit funds into a wallet")
    @shell_argument("wallet_id", help="Target wallet identifier")
```

```

@shell_option("--amount", help="Amount in minor units (integer cents)")
async def deposit(
    self, wallet_id: str, amount: int = 100
) -> str:
    result = await self._wallet.deposit(wallet_id, amount)
    return f"New balance: {result['balance_minor']} minor units"

@shell_method(group="wallet", help="Show current balance")
@shell_argument("wallet_id", help="Wallet to inspect")
async def balance(self, wallet_id: str) -> str:
    data = await self._wallet.get_balance(wallet_id)
    return f"{wallet_id}: {data['balance_minor']} minor units"

```

Listing 18.9 — DI-powered shell commands

Enable the shell in `pyfly.yaml`:

```

pyfly:
  shell:
    enabled: true

```

PyFly auto-configures a `ClickShellAdapter` and wires every `@shell_method` at startup. The group name becomes a sub-command:

```

python -m lumen wallet deposit w-001 --amount 500
python -m lumen wallet balance w-001
python -m lumen          # no args → drops into REPL mode

```

CommandLineRunner — one-shot post-startup tasks

For tasks that run once at startup — seeding, warm-up, connection checks — implement `CommandLineRunner`:

```
lumen/runners/seed_runner.py
```

```

from pyfly.container import service
from pyfly.shell import CommandLineRunner

```

```

@service
class SeedRunner(CommandLineRunner):
    """Seed the database with a default admin wallet on first boot."""

    def __init__(self, wallet_service) -> None:
        self._wallet = wallet_service

    async def run(self, args: list[str]) -> None:
        if "--seed" in args:
            await self._wallet.ensure_default_wallet()
            print("Default wallet ensured.")

```

Listing 18.10 — Post-startup database seeder

Any bean whose class implements `async def run(self, args: list[str]) -> None` structurally satisfies the `CommandLineRunner` protocol. The framework detects it via `isinstance()` (the protocol is `@runtime_checkable`) after `ApplicationReadyEvent` fires, then invokes it with the raw CLI arguments. Use `@order(n)` to control execution order when multiple runners coexist.

🍃 SPRING PARITY

`@shell_component`, `@shell_method`, `@shell_option`, `@shell_argument`, and `CommandLineRunner` are direct equivalents of Spring Shell's `@ShellComponent`, `@ShellMethod`, `@ShellOption`, `@ShellArgument`, and Spring Boot's `CommandLineRunner` interface. Click replaces JLine as the terminal library, but the programming model is identical.

Generating an SDK from the OpenAPI spec

When Lumen exposes an HTTP API, downstream services should call it via a generated client — not hand-written `httpx` calls that drift out of sync. PyFly builds and serves an OpenAPI 3.1 spec automatically at `/openapi.json`.

`OpenAPIGenerator` assembles the spec from route metadata collected by `ControllerRegistrar`:

- **Info** — populated from `title`, `version`, and `description` passed to `create_app()`.
- **Paths** — one operation per `@get_mapping` / `@post_mapping` etc., with parameters inferred from `PathVar[T]`, `QueryParam[T]`, `Header[T]`, and `Body[BaseModel]` type hints.
- **Schemas** — Pydantic models registered in `components.schemas` via `model_json_schema()` and referenced with `$ref`.

With the spec available, generate a Python client in one command:

```
# Download the spec from a running instance
curl http://localhost:8080/openapi.json -o lumen-spec.json

# Generate a Python client package
openapi-generator-cli generate \
  -i lumen-spec.json \
  -g python \
  -o lumen-client \
  --package-name lumen_client
```

The generated `lumen_client` package contains typed models and a `DefaultApi` with one method per operation. Consumer services add it as a dependency and call it without knowing anything about HTTP:

payment/services/wallet_client.py

```

from lumen_client import ApiClient, Configuration, DefaultApi

class WalletGateway:
    """Typed façade over the generated Lumen client SDK."""

    def __init__(self, base_url: str) -> None:
        cfg = Configuration(host=base_url)
        self._api = DefaultApi(ApiClient(cfg))

    def get_balance(self, wallet_id: str) -> int:
        result = self._api.get_wallet_balance(wallet_id)
        return result.balance_minor

```

Listing 18.11 — Consuming the generated Lumen SDK

💡 KEEP THE SPEC VERSIONED

Check `lumen-spec.json` into the Lumen repository and regenerate client packages in CI whenever the spec changes. Downstream teams pin to a specific spec version via their dependency manager — the same discipline Java teams use with Maven artifact versions.

Going to production

Packaging with Docker

`pyfly new` generates a `Dockerfile` for every archetype. For a web service it looks like this after production hardening:

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```

COPY pyproject.toml uv.lock ./
RUN pip install uv && \
    uv sync --no-dev --extra web --extra data-relational \
        --extra security --extra observability

COPY src/ src/
COPY pyfly.yaml .

EXPOSE 8080
CMD ["pyfly", "run", "--host", "0.0.0.0", \
    "--port", "8080", "--server", "granian", "--workers", "2"]

```

Install only the extras your service actually uses — the `full` meta-extra pulls in Kafka, RabbitMQ, and MongoDB drivers even when you need none of them.

Environment variables and secrets

Never bake secrets into `pyfly.yaml`. PyFly resolves `${ENV_VAR}` placeholders anywhere in configuration:

```

pyfly:
  data:
    relational:
      url: ${DATABASE_URL}

  security:
    jwt:
      secret: ${JWT_SECRET}

```

PyFly reads the actual values from the container environment at startup. In Kubernetes, back those variables with a `Secret`; in Docker Compose, use an `.env` file that is never committed. The `pyfly doctor` command checks that required tools are present but does not validate secrets — that responsibility remains yours.

Graceful shutdown

PyFly honours graceful shutdown by default. Set `pyfly.server.graceful-timeout` (seconds) to control how long the server waits for in-flight requests to complete before forcing exit:

```
pyfly:
  server:
    graceful-timeout: 30
```

SIGTERM triggers the shutdown sequence: the server stops accepting new connections, `ApplicationContext.stop()` runs `@pre_destroy` hooks and `stop_all()` for plugins, and the process exits cleanly. In Kubernetes set `terminationGracePeriodSeconds` to at least five seconds more than `graceful-timeout`.

Server selection

The ASGI server is selected by priority at runtime:

Priority	Server	Install extra
1	Granian (Rust/tokio)	<code>granian</code>
2	Uvicorn	<code>web</code> (default)
3	Hypercorn	<code>hypercorn</code>

For production, prefer Granian — it delivers roughly 3× the throughput of Uvicorn with native HTTP/2. Pair it with uvloop on Linux for an additional 2–4× event-loop speedup:

```
uv add "pyfly[web-fast]" # granian + uvloop in one shot
```

Pin the choice in YAML to avoid surprises on machines where multiple servers happen to be installed:

```

pyfly:
  server:
    type: granian
    event-loop: uvloop
    workers: 4
    graceful-timeout: 30

```

Health endpoints

PyFly exposes Spring Boot-style actuator endpoints out of the box:

Endpoint	Purpose
GET /actuator/health	Aggregate health (UP / DOWN)
GET /actuator/health/liveness	Kubernetes liveness probe
GET /actuator/health/readiness	Kubernetes readiness probe
GET /actuator/metrics	Prometheus-compatible metrics

Configure them in `pyfly.yaml`:

```

pyfly:
  management:
    endpoints:
      web:
        exposure:
          include: health,metrics
        endpoint:
          health:
            show-details: when-authorized

```

Then wire your Kubernetes deployment:

```

livenessProbe:
  httpGet:
    path: /actuator/health/liveness

```



```

    port: 8080
    initialDelaySeconds: 10
    periodSeconds: 15
  readinessProbe:
    httpGet:
      path: /actuator/health/readiness
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 10

```

The production checklist

- [] All secrets are environment variables — none are in `pyfly.yaml` or source control.
- [] Docker image installs only the extras the service uses.
- [] Server is pinned to Granian + uvloop; `workers` is set explicitly (not left at `1` for multi-core machines).
- [] `graceful-timeout` is at least 15 s; Kubernetes `terminationGracePeriodSeconds` is at least 5 s more.
- [] Liveness and readiness probes are configured and tested.
- [] `/actuator/health` returns UP before traffic is sent.
- [] Prometheus metrics endpoint is scraped by the monitoring stack.
- [] Structured logging is enabled (`pyfly[observability]`) and log level is `INFO` in production, not `DEBUG` .
- [] OpenTelemetry exporter is pointed at the production collector.
- [] Database migrations (`pyfly db upgrade`) run in a pre-deploy step, not at application startup.
- [] The generated OpenAPI spec is versioned in CI and downstream SDK packages pin to a specific spec revision.
- [] `pyfly doctor` passes on every developer machine and CI runner.

✓ What you built

Seventeen chapters ago you typed `@pyfly_application` and watched the DI container wire your first `@service`. Today that same container powers a production wallet platform. Here is what you constructed along the way.

Chapters 1–3 gave you the foundation: a first-class DI container, flexible configuration (YAML, env, profiles, SpEL expressions), and a complete HTTP layer with request mapping, filters, content negotiation, and a JSON serialisation layer you can replace without touching a single controller.

Chapters 4–6 introduced persistence. You mapped entities with SQLAlchemy, managed schema evolution with Alembic, and structured the domain with DDD tactical patterns — aggregates, value objects, and repositories that keep business logic out of infrastructure.

Chapters 7–9 brought the architecture alive. CQRS split reads from writes at the handler level. EDA let services react to events without polling. Event sourcing made every state change a first-class fact — replayable, auditable, and the foundation for projections.

Chapters 10–12 taught Lumen to leave its own process. Resilient HTTP clients called downstream services without cascading failures. Sagas orchestrated multi-step transactions across service boundaries with automatic compensation when any step went wrong.

Chapters 13–15 hardened the platform. Caching cut database pressure. Rate limiters, bulkheads, timeouts, and circuit breakers turned every dependency into a controlled blast radius. Distributed tracing, structured logging, and a live admin dashboard gave you eyes inside the system at every layer.

Chapters 16–17 closed the feedback loop. A structured test suite — unit, integration, and Testcontainers-backed persistence tests — made the platform safe to change. Scheduled tasks, push notifications, webhooks, and callbacks let Lumen reach out to the world on its own schedule.

Chapter 18 showed you what lies beyond the core: a plugin system for open extension, a YAML rule engine for business-owned logic, a Config Server for fleet-wide configuration, i18n for global audiences, WebSocket for real-time UX, a Shell module for operational tooling, an OpenAPI spec that generates typed client SDKs automatically, and the production habits that keep all of it running.

PyFly is not magic. Every abstraction in this book has a cost, and you now understand what that cost is: a DI container that starts in milliseconds but requires you to think about bean scopes; an async HTTP server that handles thousands of concurrent connections but requires you to avoid blocking calls; a saga engine that survives partial failures but requires you to write compensating transactions.

Understanding the cost is what separates a practitioner from a reader who merely copies patterns. You are now a practitioner.

Try it yourself

1. **Add a custom plugin.** Implement an `AuditPlugin` that contributes an extension to an `"audit-sinks"` extension point. The extension class must implement an `AuditSink` interface and write each event to a file. In a test, call `PluginManager.registry.get("audit-sinks")` and assert that your extension is returned with the expected `name`.

2. **Ship a rules change without redeployment.** Store `transaction_rules.yaml` in the Config Server (`pyfly.config-server.enabled: true`). Write a `RiskService` that fetches the YAML via `ConfigClient` on each call to `assess()` (or caches it with a short TTL). Update the `value` threshold through the `POST /{app}/{profile}` route and verify that `assess()` picks up the new value without restarting the service.
 3. **Localise a rejection message.** Add `wallet.limit_exceeded` to `i18n/messages_en.yaml` and `i18n/messages_es.yaml` . Wire a `NotificationService` that reads the locale from the `Accept-Language` header and returns the correct string. Write two tests — one with `Accept-Language: en` , one with `Accept-Language: es` — and assert each returns the right message.
-

Lumen is ready for production. For what comes next — new modules, community plugins, and release notes — visit the framework documentation at github.com/fireflyframework/fireflyframework-pyfly. Every concept in this book lives in that repository; the source is the ultimate reference.

Appendices

APPENDIX A

Spring Boot → PyFly Cheat-Sheet

If you have shipped production services with Spring Boot, the concepts in PyFly will feel immediately familiar: stereotypes, constructor injection, `@configuration` + `@bean` factories, typed config binding, derived queries, saga orchestration — they are all here. What changes is the syntax (Python decorators instead of Java annotations), the runtime model (native `async/await` instead of Project Reactor or servlet threads), and a handful of deliberate design choices made to fit idiomatic Python. This cheat-sheet maps every Spring Boot concept you already know to its PyFly equivalent, so you can start reading and writing PyFly code without relearning the architecture from scratch.

Application Bootstrap & Stereotypes

Spring Boot	PyFly	Notes
<code>@SpringBootApplication</code>	<code>@pyfly_application</code>	Combines <code>@EnableAutoConfiguration</code> + <code>@ComponentScan</code> . <code>scan_packages</code> replaces classpath scanning — list packages explicitly.
<code>SpringApplication.run(...)</code>	<code>PyFlyApplication(App); await app.startup()</code>	Entry point is async.
<code>@Component</code>	<code>@component</code>	Generic singleton managed bean.
<code>@Service</code>	<code>@service</code>	Business-logic layer.
<code>@Repository</code>	<code>@repository</code>	Data-access layer.
<code>@RestController</code>	<code>@rest_controller</code>	API endpoint class. No <code>@Controller</code> (no view rendering).
<code>@Configuration</code>	<code>@configuration</code>	Bean factory class.
<code>@Bean</code>	<code>@bean</code>	Factory method inside a <code>@configuration</code> class. Return type hint is the bean's registered type.
<code>@Primary</code>	<code>@primary</code> or <code>@bean(primary=True)</code>	Default when multiple candidates exist.
<code>@Order(N)</code>	<code>@order(N)</code>	Lifecycle and injection ordering.
<code>@Lazy</code>	<code>@lazy</code>	Bean is not created until first resolution.

Dependency Injection

Spring Boot	PyFly	Notes
Constructor <code>@Autowired</code> (implicit in modern Spring)	Plain constructor with type-hinted params	Container reads <code>__init__</code> type hints automatically.
Field <code>@Autowired</code>	<code>field: T = Autowired()</code>	<code>Autowired(required=False)</code> for optional.
<code>@Qualifier("name")</code>	<code>Annotated[T, Qualifier("name")]</code>	Python's <code>Annotated</code> carries the qualifier without losing the base type.
<code>Optional<T></code> injection	<code>Optional[T]</code> param	Resolves to <code>None</code> when no bean is registered.
<code>List<T></code> injection	<code>list[T]</code> param	Collects all registered implementations of <code>T</code> .
<code>Map<String, T></code> injection	<code>dict[str, T]</code> param	<code>{bean-name: bean}</code> for every named bean of type <code>T</code> .
<code>Repository<User></code> generic injection	<code>Repository[User, int]</code> param	Container matches on generic type arguments, honours <code>@primary</code> for ties.
<code>ObjectFactory<T></code> / <code>Provider<T></code>	<code>Provider[T]</code>	Deferred resolution; each <code>.get()</code> re-resolves — safe for <code>TRANSIENT</code> beans.

 PREFER CONSTRUCTOR INJECTION

Constructor injection keeps dependencies visible in the class signature, prevents missing-dependency bugs at startup rather than at runtime, and lets you write plain-Python unit

tests without a container: `svc = WalletService(repo=MockRepo(),
events=MockEvents())` .

Conditions & Auto-Configuration

Spring Boot	PyFly	Notes
<code>@ConditionalOnProperty</code>	<code>@conditional_on_property</code>	Register when a config key equals a specific value.
<code>@ConditionalOnClass</code>	<code>@conditional_on_class("module")</code>	Register when a Python module is importable.
<code>@ConditionalOnMissingBean</code>	<code>@conditional_on_missing_bean(T)</code>	Register when no bean of type <code>T</code> exists yet.
<code>@ConditionalOnBean</code>	<code>@conditional_on_bean(T)</code>	Register only if a bean of type <code>T</code> is present.
<code>@ConditionalOnSingleCandidate</code>	<code>@conditional_on_single_candidate(T)</code>	Exactly one candidate, or one marked <code>@primary</code> .
<code>@ConditionalOnWebApplication</code>	<code>@conditional_on_web_application()</code>	Web stack (Starlette/FastAPI) present.
<code>@ConditionalOnResource</code>	<code>@conditional_on_resource(path)</code>	Filesystem path exists.

Spring Boot	PyFly	Notes
@ConditionalOnExpression	@conditional_on_expression("#{...}")	SpEL-lite expression — supports \$ {key:default} + arithmetic, comparison, boolean. AST-parsed, no eval .

Lifecycle Hooks & Scopes

Spring Boot	PyFly	Notes
<code>@PostConstruct</code>	<code>@post_construct</code>	Called after DI; can be <code>async def</code> .
<code>@PreDestroy</code>	<code>@pre_destroy</code>	Called on graceful shutdown; can be <code>async def</code> .
Default (singleton) scope	Default (singleton)	One instance per application.
<code>@SessionScope</code>	<code>@component(scope=Scope.SESSION)</code>	One instance per <code>HttpSession</code> .
Custom <code>Scope</code> SPI	<code>Container.register_scope(name, handler)</code>	Implement the <code>ScopeHandler</code> protocol.
<code>@RefreshScope</code> (Spring Cloud)	<code>@refresh_scope</code>	Evicted and rebuilt on <code>POST /actuator/refresh</code> .
<code>ContextRefresher.refresh()</code>	<code>ContextRefresher.refresh()</code> (injectable)	Evicts refresh-scoped beans, resets <code>@config_properties</code> , returns the changed keys.
<code>ApplicationEventPublisher</code>	<code>ApplicationEventPublisher</code> (injectable)	<code>await publisher.publish(event)</code> .
<code>@EventListener</code>	<code>@app_event_listener</code>	Dispatch by <code>isinstance</code> ; sync listeners are allowed.

Configuration & Profiles

Spring Boot	PyFly	Notes
<code>application.yml</code>	<code>pyfly.yaml</code>	Same hierarchical structure.
<code>application-{profile}.yml</code>	<code>pyfly-{profile}.yaml</code>	Profile overlays.
<code>spring.profiles.active=dev</code>	<code>PYFLY_PROFILES_ACTIVE=dev</code> (env var) or <code>pyfly.profiles.active: dev</code> in <code>pyfly.yaml</code>	Activation is identical in priority order.
<code>@ConfigurationProperties(prefix=...)</code>	<code>@config_properties(prefix=...)</code> on a <code>@dataclass</code> (from <code>pyfly.core import config_properties</code>)	Pydantic-backed: validated and frozen at startup.
<code>@Value("\${key}")</code>	<code>field: str = Value("\${key}")</code> (from <code>pyfly.core import Value</code>)	Raises on missing key.
<code>@Value("\${key:default}")</code>	<code>field: str = Value("\${key:default}")</code>	Default after the colon.
<code>@Value("#{expr}")</code> SpEL	<code>Value("#{...}")</code> SpEL-lite	Arithmetic, comparison, boolean, <code>\${...}</code> substitution, <code>env</code> mapping. Constructor injection: <code>Annotated[bool, Value("#{...}")]</code> .
<code>@Bean @Profile("dev")</code>	<code>@bean(profile="dev")</code>	Profile expression (<code>& \ ! ()</code>) on any <code>@bean</code> .

Spring Boot	PyFly	Notes
Boolean <code>@Profile("prod & cloud")</code>	<code>profile="prod & cloud"</code>	Spring Boot 2.4+ operators; legacy comma-OR form still works.
<code>spring.application.name</code>	<code>pyfly.app.name</code>	Application name key in <code>pyfly.yaml</code> .

Property-source priority (lowest → highest):

1. `pyfly-defaults.yaml` (framework built-ins)
2. `pyfly.yaml` (application defaults)
3. `pyfly-{profile}.yaml` (profile overlays)
4. Environment variables (runtime, highest priority)

This is identical to Spring Boot's property-source ordering.

Web Layer

Imports: `from pyfly.web import (Body, PathVar, QueryParam, Valid, get_mapping, post_mapping, put_mapping, delete_mapping, patch_mapping, request_mapping)`. Stereotypes: `from pyfly.container import rest_controller, service, repository, component, configuration`. 404: `from pyfly.kernel import ResourceNotFoundException`.

Spring Boot	PyFly	Notes
<code>@RestController</code>	<code>@rest_controller</code>	
<code>@RequestMapping("/path")</code>	<code>@request_mapping("/path")</code>	Class-level prefix.
<code>@GetMapping</code>	<code>@get_mapping</code>	All handler methods are <code>async def</code> .
<code>@PostMapping</code>	<code>@post_mapping</code>	
<code>@PutMapping</code>	<code>@put_mapping</code>	
<code>@DeleteMapping</code>	<code>@delete_mapping</code>	
<code>@PatchMapping</code>	<code>@patch_mapping</code>	
<code>@PathVariable Long id</code>	<code>id: PathVar[int]</code> <code>param</code>	Type annotation required; matched by name.
<code>@RequestParam(defaultValue="0") int page</code>	<code>page:</code> <code>QueryParam[int] = 0</code>	Python default replaces <code>defaultValue</code> .
<code>@RequestBody @Valid T body</code>	<code>body:</code> <code>Valid[Body[T]]</code>	Pydantic deserialization + validation combined.
<code>@RequestHeader("X-Token") String t</code>	<code>t: Header[str]</code>	
<code>@ResponseStatus(HttpStatus.CREATED)</code>	<code>@post_mapping("/", status_code=201)</code>	Status code on the mapping decorator.
<code>@ControllerAdvice + @ExceptionHandler</code>	<code>@exception_handler</code> or built-in exception hierarchy	<code>ResourceNotFoundException</code> → 404, <code>ValidationException</code> → 422, etc.

Spring Boot	PyFly	Notes
<code>spring.mvc.problemdetails.enabled</code>	<code>pyfly.web.problem-details.enabled: true</code>	RFC 7807 <code>application/problem+json</code> responses.

JSON & Content Negotiation

Spring Boot	PyFly	Notes
<code>spring.jackson.property-naming-strategy</code>	<code>pyfly.web.json.property-naming-strategy</code>	<code>as-is</code> (default) or <code>camelCase</code> .
<code>spring.jackson.default-property-inclusion: non_null</code>	<code>pyfly.web.json.exclude-none: true</code>	
Jackson <code>ObjectMapper</code>	<code>PyFlyJsonSerializer</code>	Central serialization boundary.
Jackson <code>Module</code> / custom serializer	<code>JsonSerializers.register(Type, encode=fn)</code>	Non-Pydantic type encoders.
<code>@JsonNaming(CamelCase...)</code>	<code>CamelModel</code> base class	Opt-in camelCase model — <code>order_id</code> serializes as <code>orderId</code> .
<code>HttpMessageConverter</code>	<code>MessageConverter</code> / <code>MessageConverterRegistry</code>	Built-in: JSON (first) then XML; add custom converters via <code>registry.add(...)</code> .

Data Access

Imports: `from pyfly.data.relational.sqlalchemy import (Repository, BaseEntity, Base, Specification, transactional, Propagation, Isolation)`
`from pyfly.data import Page, Pageable, Sort` `from pyfly.data.query import query` `from pyfly.container import repository`

Repositories

Spring Boot	PyFly	Notes
<code>JpaRepository<E, ID></code> / <code>CrudRepository</code>	<code>Repository[E, ID]</code>	from <code>pyfly.data.relational.sqla</code> import <code>Repository</code> ; decorate <code>@repository</code> . Subclass with type params; <code>AsyncSession</code> injected automatically at startup
<code>@Repository</code> interface <code>WalletRepo</code> extends <code>JpaRepository<...></code>	<code>@repository class</code> <code>WalletRepository(Repository[WalletEntity,</code> <code>str])</code>	Class, not interface; body holds derived-query stubs and custom methods only.
<code>findByOwnerId(String</code> <code>id)</code>	<code>async def find_by_owner_id(self,</code> <code>owner_id: str) -></code> <code>list[WalletEntity]: ...</code>	Stub body <code>...</code> triggers comp by <code>RepositoryBeanPostProcess</code> startup. Prefixes: <code>find_by_</code> , <code>count_by_</code> , <code>exists_by_</code> , <code>delete_by_</code> .
<code>@Query("SELECT u</code> <code>FROM User u WHERE</code> <code>...")</code>	<code>@query("SELECT u FROM User u WHERE ...")</code> (JPQL-like) or <code>@query("SELECT ...",</code> <code>native=True)</code> (raw SQL)	from <code>pyfly.data.query</code> import <code>query</code> . Params use <code>:name</code> syntax
<code>Specification<T></code>	<code>Specification(lambda root, q:</code> <code>q.where(root.status == "ACTIVE"))</code>	Compose with <code>&</code> / <code>\ </code> / <code>~</code> ; r <code>repo.find_all_by_spec(spec)</code> <code>repo.find_all_by_spec_pageable)</code> .

Entities

Spring Boot	PyFly	Notes
<code>@Entity</code> <code>class Order</code>	<code>class Order(Base)</code>	from pyfly.data.relational.sqlalchemy import Base . Registers the table in Base.metadata .
<code>@Entity</code> + surrogate UUID PK + audit columns	<code>class Order(BaseEntity)</code>	from pyfly.data.relational.sqlalchemy import BaseEntity . Inherits id: UUID , created_at , updated_at , created_by , updated_by — all mapped as SQLAlchemy 2.0 Mapped / mapped_column .
<code>@Column</code> / <code>@Id</code>	<code>id: Mapped[str] = mapped_column(String(64), primary_key=True)</code>	SQLAlchemy 2.0 typed columns. Base (no audit) lets the entity own its own PK type, as Lumen's WalletEntity does with a str id.
<code>@SoftDelete</code> (Hibernate 6)	<code>SoftDeleteMixin + SoftDeleteRepository</code>	from pyfly.data.relational.sqlalchemy import SoftDeleteMixin . Adds deleted_at ; repository filters it automatically.
Optimistic locking <code>@Version</code>	<code>VersionedMixin</code>	Adds a version: int column; SQLAlchemy raises StaleDataError on conflict.

Pagination & Sorting

Spring Boot	PyFly	Notes
<pre>Pageable / PageRequest.of(page, size, Sort.by(...).descending())</pre>	<pre>Pageable.of(page, size, Sort.by("created_at").descending())</pre>	<pre>from pyfly.data import Pageable, Sort . Sort.by(*fields) returns ascending; .descending() flips all.</pre>
<pre>Page<T></pre>	<pre>Page[T]</pre>	<pre>from pyfly.data import Page . Attributes: .items, .total, .p transforms items, preserving metadata.</pre>
<pre>page.getContent() / page.getTotalElements()</pre>	<pre>page.items / page.total</pre>	<pre>Python naming; .total_pages derived as ceil(total / size) .</pre>
<pre>repo.findAll(pageable)</pre>	<pre>await repo.find_all(pageable)</pre>	<pre>Returns Page[T] .</pre>
<pre>repo.findAll(spec, pageable)</pre>	<pre>await repo.find_all_by_spec_paged(spec, pageable)</pre>	<pre>Applies WHERE, ORDER BY, and LIMIT/OFFSET in one call.</pre>

Transactions & **save**

Spring Boot	PyFly	Notes
<code>@Transactional</code>	<code>@transactional()</code>	from pyfly.d import transactional _session = session injected = session success, ...
<code>@Transactional(propagation = REQUIRES_NEW)</code>	<code>@transactional(propagation=Propagation.REQUIRES_NEW)</code>	Full Propagation REQUIRES_NEW, ...
<code>@Transactional(isolation = READ_COMMITTED)</code>	<code>@transactional(isolation=Isolation.READ_COMMITTED)</code>	Full Isolation
<code>@Transactional(readOnly = true)</code>	<code>@transactional(read_only=True)</code>	Routes to read-only Routing read_only
<code>repo.save(entity)</code>	<code>await repo.save(entity)</code>	Calls save flushes batch @transactional
<code>session.merge(entity)</code> (upsert)	<code>await repo.upsert(entity)</code> (<i>extend</i>) or <code>session.merge(entity)</code> + flush	No built-in your repository WalletRepository handles the PK.
<code>AbstractRoutingDataSource</code>	<code>RoutingSessionFactory</code>	factory force a session
Multiple <code>DataSource</code> beans	<code>NamedDataSources</code>	Config: pyfly.datasource inject NamedDataSources .get("<

Projections & Mapper

Spring Boot	PyFly	Notes
Interface projection <pre>interface OrderSummary { ... }</pre>	<pre>@projection class OrderSummary: id: str; status: str</pre>	<pre>from pyfly.data.projection import projection</pre> A concrete dataclass (not a <code>Protocol</code>), not a JDK proxy; registered with <pre>mapper.register_projection(src, proj)</pre>
<pre>repo.findAll(cls, Projection.class)</pre>	<pre>mapper.project(entity, OrderSummary)</pre>	<pre>from pyfly.data.mapper import Mapper</pre>
MapStruct <code>@Mapper</code>	<pre>Mapper + @mapping</pre> decorator	Runtime reflection; no codegen. <pre>mapper.map(obj, TargetDTO), mapper.map_list(...)</pre>

Migrations

Spring Boot	PyFly	Notes
Flyway migrations	<pre>pyfly.data.migrations.*</pre>	Same concept: versioned SQL scripts run at startup.

Caching

Spring Boot	PyFly	Notes
<code>@Cacheable(value="cache", key="#id")</code>	<code>@cacheable(backend=cache, key="item:{id}")</code>	<code>{param}</code> template syntax in the key.
<code>@Cacheable(condition=..., unless=...)</code>	<code>@cacheable(condition=..., unless=...)</code>	<code>condition</code> bypasses on args; <code>unless</code> skips storing based on result.
<code>@CacheEvict</code>	<code>@cache_evict(backend=cache, key="item:{id}")</code>	
<code>@CachePut</code>	<code>@cache_put(backend=cache, key="item:{id}")</code>	Update cache after mutation.
<code>CacheManager</code> (auto-configured)	<code>InMemoryCache</code> / <code>RedisCacheAdapter</code> (auto-configured)	Redis selected when the <code>redis</code> extra is installed; falls back to in-memory.

Messaging & Events

Spring Boot	PyFly	Notes
<code>@KafkaListener(topics=..., groupId=...)</code>	<code>@message_listener(topic=..., group_id=...)</code>	Handler is <code>async def</code>
<code>KafkaTemplate.send(topic, event)</code>	<code>await publisher.publish(dest, event_type, payload)</code>	<code>EventPublisher</code> port (<code>from pyfly.eda import EventPublisher</code>); switch adapters via config.
<code>@RetryableTopic</code> / DLT	<code>@message_listener(retries=3, retry_delay=1.0, dead_letter_topic="...")</code>	Linear-backoff retry; exhausted messages to DLQ with <code>x-origin-topic</code> / <code>x-exception</code> headers.
<code>ApplicationEvent</code>	<code>EventEnvelope</code>	Domain event container
<code>@EventListener</code>	<code>@event_listener(event_types=["TypeName"])</code>	In-process EDA handler. <code>event_type</code> is the class name string. No <code>@domain_event_listener</code>
<code>ApplicationEventPublisher</code>	<code>ApplicationEventPublisher</code> (injectable)	<code>await publisher.publish(event)</code> for Spring-style app events

💡 MESSAGING VS EDA

PyFly separates **broker messaging** (`pyfly.messaging` — Kafka/RabbitMQ transport) from **domain events** (`pyfly.eda` — `EventEnvelope` + `EventBus`). Start with `InMemoryEventBus` inside a monolith; switch to a Kafka adapter later by changing one configuration key, not your handlers.

Security

Spring Boot	PyFly	Notes
<code>SecurityAutoConfiguration</code>	<code>JwtAutoConfiguration</code> + <code>PasswordEncoderAutoConfiguration</code>	Split by optional dep
JWT filter chain	<code>JwtAutoConfiguration</code> auto-wired	Enable with <code>pyfly.security.jw</code>
<code>@PreAuthorize("hasRole('ADMIN')")</code>	<code>@pre_authorize("hasRole('ADMIN')")</code>	Same SpEL subset: <code>hasAnyRole</code> , <code>hasA</code> , <code>isAuthenticated</code> , <code>denyAll</code> , <code>#param</code> , AST-walked, no ev
<code>@PostAuthorize("returnObject.owner == principal")</code>	<code>@post_authorize("returnObject.owner == principal")</code>	<code>returnObject</code> is be method's return valu
<code>RoleHierarchy</code> bean	<code>RoleHierarchy.from_string("ADMIN > USER")</code> + <code>set_role_hierarchy(...)</code>	<code>expand(roles)</code> cor <code>hasRole</code> / <code>hasAnyRo</code>
<code>ClientRegistration</code> (OAuth2 auth-code)	<code>ClientRegistration(...)</code>	PKCE: add <code>use_pk</code> generates <code>code_ve</code> <code>code_challenge</code> (S
<code>maximumSessions</code> / <code>SessionRegistry</code>	<code>SessionConcurrencyController</code> + <code>SessionRegistry</code>	Cap per-principal co (<code>evict-oldest</code> or Enable: <code>pyfly.session.cor</code> <code>true</code> .

Scheduling

Spring Boot	PyFly	Notes
<code>@Scheduled(fixedRate = 5000)</code>	<code>@scheduled(fixed_rate=5.0)</code>	Spring uses milliseconds; PyFly uses seconds .
<code>@Scheduled(fixedDelay = 1000)</code>	<code>@scheduled(fixed_delay=1.0)</code>	
<code>@Scheduled(cron = "0 0 2 * * ?")</code>	<code>@scheduled(cron="0 2 * * *")</code>	Spring: 6-field (seconds-first). PyFly: standard 5-field; also accepts the 6-field Spring form and <code>?</code> .
<code>@Scheduled(cron=..., zone=...)</code>	<code>@scheduled(cron=..., zone="America/New_York")</code>	IANA time zone; ignored for <code>fixed_rate</code> / <code>fixed_delay</code> .
ShedLock / <code>@SchedulerLock</code>	<code>@scheduled(lock=True, lock_ttl=30)</code> + <code>DistributedLock</code> bean	Skips a tick when the lock is held elsewhere. Defaults to in-process <code>LocalLock</code> ; register a Redis <code>DistributedLock</code> for cross-process single-firing.
<code>@EnableScheduling</code>	<code>SchedulingAutoConfiguration</code>	Auto-enabled when <code>croniter</code> is installed. No explicit <code>@Enable...</code> needed.

Resilience

```
from pyfly.resilience import retry, CircuitBreaker, circuit_breaker,
RateLimiter, rate_limiter, Bulkhead, bulkhead, time_limiter, fallback
```

Spring (Resilience4j)	PyFly	Notes
<code>@Retry</code>	<code>@retry(max_attempts=3, *, delay=0.1, backoff=2.0, jitter=0.1, exceptions=(IOError,))</code>	<code>delay / backoff / jitter</code> are keyword-only. <code>jitter</code> is a float fraction in <code>[0,1]</code> .
<code>@CircuitBreaker</code>	<code>breaker = CircuitBreaker(...); @circuit_breaker(breaker)</code>	Pass a <code>CircuitBreaker</code> instance. Count-based (<code>failure_threshold</code>) or rate-based (<code>failure_rate_threshold</code> + <code>window_size</code>).
<code>@RateLimiter</code>	<code>limiter = RateLimiter(max_tokens=100, refill_rate=100/60); @rate_limiter(limiter)</code>	Token-bucket; pass an instance.
<code>@Bulkhead</code>	<code>bh = Bulkhead(max_concurrent=10); @bulkhead(bh)</code>	Concurrency cap; pass an instance.
<code>@TimeLimiter</code>	<code>@time_limiter(timeout=timedelta(seconds=2))</code>	Raises <code>asyncio.TimeoutError</code> on breach.
<code>fallbackMethod</code>	<code>@fallback(fallback_method=fn)</code> or <code>@fallback(fallback_value=v)</code>	Static or callable fallback.

AOP

Spring Boot	PyFly	Notes
<code>@Aspect + @Component</code>	<code>@aspect + @component</code>	
<code>@Before("execution(...)")</code>	<code>@before("execution(...)")</code>	
<code>@After</code>	<code>@after</code>	Always runs (like <code>finally</code>).
<code>@Around</code>	<code>@around</code>	Call <code>await</code> <code>join_point.proceed()</code> to continue.
<code>@AfterReturning</code>	<code>@after_returning</code>	
<code>@AfterThrowing</code>	<code>@after_throwing</code>	
<code>@EnableAspectJAutoProxy</code>	<code>AopAutoConfiguration</code>	Always active; no opt-in needed.

Pointcut DSL: `execution(* pkg.services.*.*(..))` for method patterns;
`annotation(timed)` for decorator-targeted matching.

Observability & Actuator

Spring Boot	PyFly
<code>management.endpoints.web.exposure.include</code>	<code>pyfly.management.endpoints.web.exposure.include</code>
<code>/actuator/health</code>	<code>/actuator/health</code>
<code>/actuator/info</code>	<code>/actuator/info</code>
<code>/actuator/beans</code>	<code>/actuator/beans</code>
<code>/actuator/env</code>	<code>/actuator/env</code>
<code>POST /actuator/refresh</code> (Spring Cloud)	<code>POST /actuator/refresh</code>
Micrometer <code>@Timed</code>	<code>@timed("metric_name")</code>
Micrometer <code>@Counted</code>	<code>@counted("metric_name")</code>
Prometheus <code>MetricsRegistry</code>	<code>MetricsRegistry</code> (Prometheus backend)
Sleuth / Micrometer Tracing (W3C)	<code>TracingFilter</code> (inbound) + <code>HttpClientAdapter</code> (outbound)

CQRS & Sagas

```
from pyfly.cqrs import (Command, CommandHandler, DefaultCommandBus, Query,
QueryHandler, DefaultQueryBus, command_handler, query_handler) from
pyfly.transactional.saga.annotations import (saga, saga_step, Input,
FromStep)
```

Spring Boot / Axon	PyFly	Notes
<code>@CommandHandler</code>	<code>@command_handler</code> + <code>@service</code> stacked	Both decorators required; <code>@service</code> registers the bean. Override <code>do_handle(self, cmd)</code> .
<code>@QueryHandler</code>	<code>@query_handler</code> + <code>@service</code> stacked	Same rule. Override <code>do_handle(self, qry)</code> . Bus method: <code>.query(...)</code> .
Controller bus injection	<code>commands: DefaultCommandBus,</code> <code>queries: DefaultQueryBus</code>	Inject the concrete classes, not the protocol. <code>commands.send(cmd)</code> , <code>queries.query(qry)</code> .
<code>@Repository</code> + port	<code>@repository</code> on a class that inherits the <code>@runtime_checkable</code> <code>Protocol</code> port	Port is a <code>Protocol</code> ; adapter class inherits it.
<code>@EventHandler</code> (event-sourcing)	<code>@event_handler</code>	Sourced from <code>EventStore</code> .
<code>@Saga</code>	<code>@saga(name="...",</code> <code>layer_concurrency=N)</code> + <code>@service</code>	Both decorators required for DI + engine registration.
<code>@SagaStep(id=...,</code> <code>compensate=...)</code>	<code>@saga_step(id="...",</code> <code>compensate="method_name")</code>	
<code>@Input</code>	<code>Annotated[T, Input()]</code>	Marker is an instance : <code>Input()</code> . Injects the saga's initial payload.
<code>@FromStep("id")</code>	<code>Annotated[T, FromStep("id")]</code>	Marker is an instance : <code>FromStep("step-id")</code> . Injects a prior step's result.

Spring Boot / Axon	PyFly	Notes
<code>@Tcc</code>	<code>@tcc(name="...")</code>	TCC (Try-Confirm-Cancel) transaction class.
<code>@TccParticipant</code>	<code>@tcc_participant(id="...", order=N)</code>	
<code>@TryMethod</code> / <code>@ConfirmMethod</code> / <code>@CancelMethod</code>	<code>@try_method</code> / <code>@confirm_method</code> / <code>@cancel_method</code>	TCC three-phase methods.
<code>@FromTry</code>	<code>Annotated[T, FromTry]</code>	Inject the try-phase result into confirm/cancel.

Event sourcing: `from pyfly.eventsourcing import AggregateRoot, EventSourcedRepository`. Aggregate uses `self.when(EventType, handler_fn)` to register apply-handlers. Data: `from pyfly.data.relational.sqlalchemy import Base` (requires `pyfly[data-relational]`).

Integration Testing

Spring Boot	PyFly	Notes
<code>@SpringBootTest</code>	<code>service_slice(*beans) / slice_context(...)</code>	Minimal started context; <code>overrides</code> accept a class or pre-built instance.
<code>@WebMvcTest</code>	<code>web_slice(*controllers, overrides=...) → (context, client)</code>	Starts minimal context + <code>PyFlyTestClient</code> .
<code>@DataJpaTest</code>	<code>data_slice(*beans) → context</code>	Data-layer slice.
<code>@Testcontainers + @Container</code>	<code>with postgres_container() as pg:</code>	Python context manager handles lifecycle.
<code>@ServiceConnection</code>	<code>pyfly_config(pg) / pyfly_config_for(pg)</code>	Maps container connection details into PyFly config keys.
<code>@DynamicPropertySource</code>	<code>pyfly_config(*containers, base=...)</code>	One-call <code>Config</code> for several containers.
<code>PostgreSQLContainer</code>	<code>postgres_container()</code>	URL auto-rewritten to <code>asyncpg</code> .
<code>MySQLContainer</code>	<code>mysql_container()</code>	URL rewritten to <code>aiomysql</code> .
<code>GenericContainer (Redis)</code>	<code>redis_container()</code>	Cache + session URLs wired.
<code>KafkaContainer</code>	<code>kafka_container()</code>	
<code>@requires_docker</code>	<code>@requires_docker</code>	Skips test cleanly when Docker daemon is absent.

Install with: `pip install 'pyfly[testcontainers]'`

Embedded Server

Spring Boot	PyFly	Notes
<code>server.port</code>	<code>pyfly.web.port</code>	HTTP listen port.
Tomcat (default)	Granian (default)	Rust/tokio HTTP runtime; highest priority.
Jetty (fallback)	Uvicorn (fallback)	Ecosystem-standard ASGI fallback.
Undertow (alternative)	Hypercorn (alternative)	Advanced protocol support (HTTP/3).
<code>server.tomcat.*</code>	<code>pyfly.server.granian.*</code>	Server-specific tuning.
<code>WebServer</code> interface	<code>ApplicationServerPort</code> protocol	Contract for the embedded ASGI server.
<code>EventLoopGroup</code> (Netty)	<code>EventLoopPort</code> protocol	Contract for the I/O runtime.
<code>server.type: auto</code>	<code>pyfly.server.type: auto</code> (default)	Granian → Uvicorn → Hypercorn cascade.

💡 AUTO-CONFIGURATION CASCADE

PyFly's auto-configuration uses the same conditional-bean pattern as Spring Boot: if Granian is installed, `GranianServerAdapter` wins; if not, Uvicorn is tried next; then Hypercorn. Override at any point by providing your own `ApplicationServerPort` bean.

APPENDIX B

MongoDB & Document Data

PyFly's document data layer wraps MongoDB through **Beanie ODM** and **Motor** (the async MongoDB driver). The API mirrors the relational adapter deliberately — the same `MongoRepository[T, ID]` base class, the same derived query naming convention, the same `Page / Pageable / Sort` vocabulary — so switching between relational and document storage touches only the document class definition and the repository base class, not the service layer.

All concrete types live in `pyfly.data.document.mongodb`. Shared types (`Page` , `Pageable` , `Sort`) come from `pyfly.data`.

Installation and configuration

Install the extra:

terminal

```
uv add "pyfly[data-document]"
```

Listing B.1 — Install the data-document extra

Enable the adapter in `pyfly.yaml`:

pyfly.yaml

```
pyfly:
  data:
    document:
      enabled: true
```

```

uri: "mongodb://localhost:27017"
database: "myapp"
min_pool_size: 5
max_pool_size: 50

```

Listing B.2 — Minimal MongoDB configuration

Configuration reference

pyfly.yaml key	Type	Default	Description
<code>pyfly.data.document.enabled</code>	bool	<code>false</code>	Enable the MongoDB adapter
<code>pyfly.data.document.uri</code>	str	<code>mongodb://localhost:27017</code>	Connection URI
<code>pyfly.data.document.database</code>	str	<code>pyfly</code>	Database name
<code>pyfly.data.document.min_pool_size</code>	int	<code>0</code>	Motor pool minimum
<code>pyfly.data.document.max_pool_size</code>	int	<code>100</code>	Motor pool maximum

Every key has a matching environment variable: replace dots with underscores and uppercase — e.g. `PYFLY_DATA_DOCUMENT_URI`. For MongoDB Atlas or a replica set:

pyfly.yaml

```

# Atlas
pyfly:
  data:
    document:
      enabled: true
      uri: >-

```

```

    mongodb+srv://user:secret@cluster.mongodb.net/
    ?retryWrites=true&w=majority
    database: production_db

# Replica set (required for transactions)
# pyfly:
#   data:
#     document:
#       uri: "mongodb://m1:27017,m2:27017,m3:27017/?replicaSet=rs0"

```

Listing B.3 — Atlas and replica-set URIs

BaseDocument

`BaseDocument` extends `beanie.Document` with an audit trail. Every document class in a PyFly application should inherit from it.

Field	Type	Default	Description
<code>id</code>	<code>PydanticObjectId</code>	Auto-generated	Document primary key (ObjectId)
<code>created_at</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	Insert timestamp
<code>updated_at</code>	<code>datetime</code>	<code>datetime.now(UTC)</code>	Last-update timestamp
<code>created_by</code>	<code>str \ None</code>	<code>None</code>	Creator identifier
<code>updated_by</code>	<code>str \ None</code>	<code>None</code>	Last-updater identifier

`use_state_management = True` is set on the base `Settings` class, enabling Beanie's change-tracking so `save_changes()` produces efficient partial updates.

A typical document class:

catalog/product_document.py

```

from pydantic import BaseModel, Field
from beanie import Indexed, PydanticObjectId
from pyfly.data.document.mongodb import BaseDocument

class Dimensions(BaseModel):
    width_cm: float
    height_cm: float
    depth_cm: float

class ProductDocument(BaseDocument):
    name: str
    sku: Indexed(str, unique=True)
    description: str = ""
    price: float = Field(gt=0)
    category: Indexed(str)
    tags: list[str] = Field(default_factory=list)
    dimensions: Dimensions | None = None
    active: bool = True

class Settings:
    name = "products"

```

Listing B.4 — ProductDocument with index and nested model

The `Settings.name` attribute sets the MongoDB collection name. Omitting it causes Beanie to derive the name from the class name, which is rarely what you want.

For compound or descending indexes use `Settings.indexes`:

catalog/product_document.py

```

from pymongo import IndexModel, ASCENDING, DESCENDING
from pyfly.data.document.mongodb import BaseDocument

```

```

class OrderDocument(BaseDocument):
    customer_id: str
    status: str
    total: float
    region: str

class Settings:
    name = "orders"
    indexes = [
        IndexModel(
            [("customer_id", ASCENDING), ("status", ASCENDING)],
            name="idx_customer_status",
        ),
        IndexModel(
            [("region", ASCENDING), ("total", DESCENDING)],
            name="idx_region_total",
        ),
    ]

```

Listing B.5 — Compound index via Settings.indexes

Spring Data ↔ MongoRepository mapping

The table below shows how Spring Data MongoDB concepts map to PyFly's document layer. The surface is intentionally identical to `Repository[T, ID]` in the relational adapter, so the same service-layer patterns apply to both.

Spring Data MongoDB	PyFly	Notes
<code>MongoRepository<E, ID></code>	<code>MongoRepository[E, ID]</code>	<code>from pyfly.data.document.mongodb import MongoRepository ; decorate with @repository .</code>
<code>@Document class Product + @Id</code>	<code>class ProductDocument(BaseDocument)</code>	<code>from pyfly.data.document.mongodb import BaseDocument . Inherits id (Beanie PydanticObjectId), created_at , updated_at , created_by , updated_by . Collection name set in class Settings: name = "products" .</code>
<code>findByCategory(String c)</code>	<code>async def find_by_category(self, category: str) -> list[ProductDocument]: ...</code>	Stub body <code>...</code> compiled by <code>MongoRepositoryBeanPostProcessor</code> at startup. Same prefixes as relational: <code>find_by_</code> , <code>count_by_</code> , <code>exists_by_</code> , <code>delete_by_</code> .
<code>@Query("{ 'status': ? 0 }")</code>	<code>@query('{"status": ":status"}')</code>	<code>from pyfly.data.query import query .</code> JSON filter or aggregation pipeline; <code>:param</code> substitution.
<code>MongoSpecification</code>	<code>MongoSpecification(lambda root, q: {"active": True})</code>	<code>from pyfly.data.document.mongodb import MongoSpecification .</code> Compose with <code>& / \ / ~</code> ; run via <code>find_all_by_spec(spec)</code> or <code>find_all_by_spec_paged(spec, pageable)</code> .
<code>PageRequest.of(page, size, Sort.by(...))</code>	<code>Pageable.of(page, size, Sort.by("name").descending())</code>	<code>from pyfly.data import Pageable, Sort</code> — identical to the relational adapter.

Spring Data MongoDB	PyFly	Notes
<code>Page<T></code>	<code>Page[T]</code>	<code>.items</code> , <code>.total</code> , <code>.page</code> , <code>.size</code> , <code>.total</code> — same as relational.

MongoRepository[T, ID]

Subclass `MongoRepository[T, ID]` and annotate with `@repository`. The framework extracts the document type and ID type from the generic parameters at class-definition time via `__init_subclass__`. No `__init__` is required.

catalog/product_repository.py

```

from beanie import PydanticObjectId
from pyfly.container import repository
from pyfly.data.document.mongodb import MongoRepository

from catalog.product_document import ProductDocument

@repository
class ProductRepository(MongoRepository[ProductDocument, PydanticObjectId]):

    # --- derived query method stubs (compiled at startup) ---

    async def find_by_category(
        self, category: str
    ) -> list[ProductDocument]: ...

    async def find_by_active_and_category(
        self, active: bool, category: str
    ) -> list[ProductDocument]: ...

    async def find_by_price_greater_than_order_by_price_desc(

```

```
        self, min_price: float
    ) -> list[ProductDocument]: ...

    async def find_by_name_containing(
        self, fragment: str
    ) -> list[ProductDocument]: ...

    async def count_by_category(self, category: str) -> int: ...

    async def exists_by_sku(self, sku: str) -> bool: ...

    async def delete_by_active(self, active: bool) -> int: ...
```

Listing B.6 — ProductRepository: CRUD + derived queries

Built-in CRUD methods

Method	Return type	Description
<code>save(entity)</code>	<code>T</code>	Insert or update via Beanie <code>entity.save()</code>
<code>find_by_id(id)</code>	<code>T \ None</code>	Find by primary key
<code>find_all(**filters)</code>	<code>list[T]</code>	Find all; keyword args become equality filters
<code>find_all(sort)</code>	<code>list[T]</code>	Fetch all documents, ordered by a <code>Sort</code>
<code>find_all(pageable)</code>	<code>Page[T]</code>	Paged query: counts the total, applies the Pageable's sort, slices with skip/limit, returns <code>Page[T]</code>
<code>stream_all(sort)</code>	<code>AsyncIterator[T]</code>	Stream all documents (the Flux analogue); optional <code>Sort</code> and equality filters
<code>delete(entity)</code>	<code>None</code>	Delete a loaded document instance
<code>delete_by_id(id)</code>	<code>None</code>	Delete by primary key; no-op if not found
<code>count()</code>	<code>int</code>	Count all documents in the collection
<code>exists_by_id(id)</code>	<code>bool</code>	True if a document with this ID exists
<code>save_all(entities)</code>	<code>list[T]</code>	Bulk insert via <code>insert_many</code>
<code>find_all_by_id(ids)</code>	<code>list[T]</code>	Find all with IDs in a list

Method	Return type	Description
<code>delete_all_by_id(ids)</code>	<code>None</code>	Delete all with IDs in a list
<code>delete_all(entities=None)</code>	<code>None</code>	Delete the given documents; with no args, truncate the entire collection
<code>find_all_by_spec(spec)</code>	<code>list[T]</code>	Find matching a <code>MongoSpecification</code>
<code>find_all_by_spec_paged(spec, pageable)</code>	<code>Page[T]</code>	Find matching a <code>MongoSpecification</code> with pagination and sort

`find_all(**filters)` translates keyword arguments into MongoDB equality filters:

```
# {"status": "PENDING", "customer_id": "abc"}
orders = await repo.find_all(status="PENDING", customer_id="abc")
```

Derived query methods

PyFly compiles stub methods on `MongoRepository` subclasses into real MongoDB queries at startup. The naming convention is identical to the relational adapter and to Spring Data: `{prefix}_by_{predicates}[_order_by_{fields}]`.

Prefixes: `find_by`, `count_by`, `exists_by`, `delete_by`

Connectors: `_and_`, `_or_`

Operator mapping

Method suffix	MongoDB filter	Args consumed
<i>(none, default)</i>	<code>{field: value}</code>	1
<code>_not</code>	<code>{field: {"\$ne": value}}</code>	1
<code>_greater_than</code>	<code>{field: {"\$gt": value}}</code>	1
<code>_greater_than_equal</code>	<code>{field: {"\$gte": value}}</code>	1
<code>_less_than</code>	<code>{field: {"\$lt": value}}</code>	1
<code>_less_than_equal</code>	<code>{field: {"\$lte": value}}</code>	1
<code>_between</code>	<code>{field: {"\$gte": low, "\$lte": high}}</code>	2
<code>_like</code>	<code>{field: {"\$regex": pattern}}</code> (SQL % → .*)	1
<code>_containing</code>	<code>{field: {"\$regex": ".*val.*", "\$options": "i"}}</code>	1
<code>_in</code>	<code>{field: {"\$in": values}}</code>	1 (list)
<code>_is_null</code>	<code>{field: None}</code>	0
<code>_is_not_null</code>	<code>{field: {"\$ne": None}}</code>	0

Ordering: append `_order_by_{field}_{asc|desc}`. Multiple sort fields are chained:

```
# sort=[("name", ASC), ("created_at", DESC)]
async def find_by_active_order_by_name_asc_created_at_desc(
    self, active: bool
) -> list[ProductDocument]: ...
```

The `MongoRepositoryBeanPostProcessor` detects stubs (bodies containing only `...` or `pass`) and replaces them with compiled callables. Source: `src/pyfly/data/document/mongodb/post_processor.py` and `src/pyfly/data/document/mongodb/query_compiler.py`.

Custom queries with `@query`

For queries that cannot be expressed through naming conventions, `@query` accepts a MongoDB filter document (`{...}`) or aggregation pipeline (`[...]`) as a JSON string. Named parameters use `:param_name` syntax.

`catalog/order_repository.py`

```
from pyfly.container import repository
from pyfly.data.document.mongodb import MongoRepository
from pyfly.data.query import query

from catalog.order_document import OrderDocument

@repository
class OrderRepository(MongoRepository[OrderDocument, str]):

    @query('{"status": ":status", "total": {"$gte": ":min_total"}}')
    async def find_by_status_min_total(
        self, status: str, min_total: float
    ) -> list[OrderDocument]: ...

    @query(
        ['{"$match": {"customer_id": ":cid"}},',
         '{"$group": {"_id": "$category",',
         '              "total": {"$sum": "$amount"}}}]'
    )
    async def totals_by_category(
```

```

        self, cid: str
    ) -> list[dict]: ...

```

Listing B.7 — @query filter and aggregation examples

Substitution rules:

- A JSON string value that is *exactly* `:param_name` is replaced by the Python value, preserving its type (`int` , `bool` , `list` , etc.).
- `:param_name` embedded inside a larger string is replaced via `str(value)` .
- Dicts and lists are recursed. Non-string JSON values pass through unchanged.

`MongoQueryExecutor` parses the query template once at startup, detects whether it is a filter or pipeline, and substitutes parameters at call time.

Pagination

catalog/product_service.py

```

from pyfly.data import Page, Pageable, Sort
from pyfly.data.document.mongodb import MongoRepository

from catalog.product_document import ProductDocument

async def list_products(
    repo: MongoRepository[ProductDocument, str],
    page: int = 1,
    size: int = 20,
) -> Page[ProductDocument]:
    pageable = Pageable.of(
        page=page,
        size=size,

```

```

        sort=Sort.by("name"),
    )
    return await repo.find_all(pageable)

```

Listing B.8 — Paginated product listing

`find_all(pageable)` counts the total, applies the Pageable's sort, slices with `.skip((page-1)*size)` and `.limit(size)` on the Beanie query, and returns `Page[T]`. Pageable is 1-based, so page `1` is the first page.

Transaction management

Multi-document transactions require a **replica set** deployment. Standalone MongoDB does not support them.

`pyfly.yaml`

```

# Start MongoDB: mongod --replSet rs0 --bind_ip localhost
# Init (once, in mongosh): rs.initiate()
pyfly:
  data:
    document:
      enabled: true
      uri: "mongodb://localhost:27017/?replicaSet=rs0"
      database: myapp

```

Listing B.9 — Single-node replica set for local development

The `@mongo_transactional` decorator wraps an async function in a Motor session and transaction. The function's Beanie operations participate automatically:

`billing/transfer.py`

```

from motor.motor_asyncio import AsyncIOMotorClient
from pyfly.data.document.mongodb import mongo_transactional

from billing.account_document import AccountDocument

def make_transfer_fn(client: AsyncIOMotorClient):
    @mongo_transactional(client)
    async def transfer(
        from_id: str, to_id: str, amount: float
    ) -> None:
        src = await AccountDocument.get(from_id)
        dst = await AccountDocument.get(to_id)
        if src is None or dst is None or src.balance < amount:
            raise ValueError("Invalid transfer")
        src.balance -= amount
        dst.balance += amount
        await src.save()
        await dst.save()
    return transfer

```

Listing B.10 — Atomic fund transfer with `@mongo_transactional`

On success the transaction commits; on any exception it aborts and re-raises. Unlike the relational `@reactive_transactional`, the Motor session is not injected as an argument — Beanie picks it up through the Motor context.

The `motor_client` bean is registered automatically by `DocumentAutoConfiguration` when the adapter is enabled. Inject it into your service via the DI container.

⚠ REPLICASET REQUIRED

`@mongo_transactional` will raise an error against a standalone MongoDB instance. Use the `?replicaSet=rs0` URI fragment (see Listing B.9) even for local dev.

Auto-configuration

`DocumentAutoConfiguration` activates when:

1. `beanie` is importable (`@conditional_on_class("beanie")`), and
2. `pyfly.data.document.enabled` is `"true"` in config.

It registers three beans automatically:

Bean	Type	Role
<code>motor_client</code>	<code>AsyncIOMotorClient</code>	Async MongoDB connection pool
<code>mongo_post_processor</code>	<code>MongoRepositoryBeanPostProcessor</code>	Compiles derived query stubs
<code>odm_initializer</code>	<code>BeanieInitializer</code>	Calls <code>init_beanie()</code> at startup

`BeanieInitializer.start()` discovers `BaseDocument` subclasses in two passes: first from every registered `MongoRepository.entity_type` (set by `__init_subclass__`), then directly registered `BaseDocument` subclasses. This means defining a repository is sufficient — you do not need to register document models separately.

Source files: `src/pyfly/data/document/auto_configuration.py`, `src/pyfly/data/document/mongodb/initializer.py`.

Testing

For unit tests, use [mongomock-motor](#) or point at a dedicated test database:

pyfly-test.yaml

```
pyfly:
  data:
    document:
      enabled: true
      database: "myapp_test"
```

Listing B.11 — Test database configuration

For integration tests, PyFly's Testcontainers support spins up a real MongoDB container automatically — see the testing chapter and

```
@ServiceConnection(MongoDBContainer) .
```

APPENDIX C

ECM: Content & E-Signature

`pyfly.ecm` provides hexagonal abstractions for enterprise document management: blob storage, document metadata, folder hierarchies, and e-signature workflows. The module ships fully wired adapters for local storage, AWS S3, and Azure Blob Storage, plus e-signature adapters for DocuSign, Adobe Sign, and Logalty — all selectable without changing a line of application code, purely via YAML.

i NO EXTRA DEPENDENCY FOR THE MODULE ITSELF

The core ECM module (`pyfly.ecm`) has no third-party dependencies beyond the standard library. Cloud storage adapters (`boto3` , `azure-storage-blob`) and e-signature SDK calls are made through lightweight async wrappers in each adapter class; install only what you need.

Domain model

All ECM types are plain Python dataclasses defined in `src/pyfly/ecm/models.py` .

Class	Key fields	Description
<code>Document</code>	<code>id</code> , <code>name</code> , <code>folder_id</code> , <code>content_type</code> , <code>size_bytes</code> , <code>metadata</code> , <code>versions</code>	Logical document record
<code>DocumentVersion</code>	<code>version</code> , <code>content_hash</code> , <code>size_bytes</code> , <code>storage_uri</code>	Immutable version snapshot
<code>Folder</code>	<code>id</code> , <code>name</code> , <code>parent_id</code> , <code>path</code>	Folder node in the hierarchy
<code>Recipient</code>	<code>name</code> , <code>email</code> , <code>role</code>	E-signature recipient
<code>SignatureRequest</code>	<code>document_id</code> , <code>recipients</code> , <code>subject</code> , <code>message</code>	Signature request payload
<code>ESignatureEnvelope</code>	<code>id</code> , <code>provider</code> , <code>status</code> , <code>provider_envelope_id</code>	Envelope tracking record

`ESignatureStatus` is a `StrEnum` with values `DRAFT`, `SENT`, `SIGNED`, `DECLINED`, and `EXPIRED`.

Ports (contracts)

Four `Protocol` classes defined in `src/pyfly/ecm/ports.py` form the hexagonal boundary. Your application services depend on these contracts; adapters implement them.

DocumentStoragePort

```
class DocumentStoragePort(Protocol):
    name: str
    async def upload(
```

```

    self, document: Document, content: bytes
) -> DocumentVersion: ...
async def download(
    self, document: Document, version: int | None = None
) -> bytes: ...
async def delete(
    self, document: Document, version: int | None = None
) -> bool: ...

```

MetadataStoragePort

```

class MetadataStoragePort(Protocol):
    async def save(self, document: Document) -> Document: ...
    async def get(self, document_id: str) -> Document | None: ...
    async def list(
        self, folder_id: str | None = None, *, limit: int = 100
    ) -> list[Document]: ...
    async def delete(self, document_id: str) -> bool: ...

```

FolderRepositoryPort

```

class FolderRepositoryPort(Protocol):
    async def save(self, folder: Folder) -> Folder: ...
    async def get(self, folder_id: str) -> Folder | None: ...
    async def list(self, parent_id: str | None = None) -> list[Folder]: ...
    async def delete(self, folder_id: str) -> bool: ...

```

ESignatureAdapter

```

class ESignatureAdapter(Protocol):
    name: str
    async def send(
        self, request: SignatureRequest
    ) -> ESignatureEnvelope: ...
    async def get(self, envelope_id: str) -> ESignatureEnvelope | None: ...
    async def cancel(self, envelope_id: str) -> bool: ...

```

DocumentService

`DocumentService` orchestrates the storage and metadata ports. It is the primary entry point for document upload, download, listing, and deletion.

`contracts/ecm_usage.py`

```
from pyfly.ecm import (
    DocumentService,
    LocalFilesystemStorageAdapter,
)
from pyfly.ecm.in_memory import (
    InMemoryFolderRepository,
    InMemoryMetadataStorage,
)

service = DocumentService(
    storage=LocalFilesystemStorageAdapter("/var/ecm/docs"),
    metadata=InMemoryMetadataStorage(),
    folders=InMemoryFolderRepository(),
)

async def demo() -> None:
    # Upload
    doc = await service.upload(
        name="contract.pdf",
        content=b"%PDF-1.4 ...",
        content_type="application/pdf",
        metadata={"department": "legal", "year": "2026"},
    )

    # Download (latest version)
    content = await service.download(doc.id)

    # Download a specific version
    v1_content = await service.download(doc.id, version=1)
```

```
# List documents in a folder
docs = await service.list(folder_id=doc.folder_id)

# Delete (returns False if blob delete partially fails)
ok = await service.delete(doc.id)
```

Listing C.1 — DocumentService: upload, download, delete

delete semantics

`DocumentService.delete(document_id)` removes both the stored blob and the metadata record and returns the **logical AND** of the two delete results. If the document ID is unknown it returns `False` immediately. If the blob delete fails the metadata record is still removed (so the logical document is gone), but the method returns `False` to signal a partial delete. This contract is verified in the source at `src/pyfly/ecm/services.py`.

Folder management

`contracts/ecm_folders.py`

```
from pyfly.ecm import DocumentService, Folder

async def setup_folders(service: DocumentService) -> None:
    root = await service.create_folder(
        Folder(name="contracts", path="/contracts")
    )
    legal = await service.create_folder(
        Folder(name="legal", parent_id=root.id, path="/contracts/legal")
    )
    doc = await service.upload(
        name="nda.pdf",
        content=b"...",
        folder_id=legal.id,
```

```
        content_type="application/pdf",
    )
```

Listing C.2 — Creating folder hierarchies

FOLDER PORT IS OPTIONAL

Pass `folders=None` to `DocumentService` if you do not need folder support. Calling `create_folder` on such an instance raises `RuntimeError`.

ESignatureService

`ESignatureService` wraps an `ESignatureAdapter` and exposes three operations: `request`, `get`, and `cancel`.

`contracts/esignature_usage.py`

```
from pyfly.ecm import (
    ESignatureService,
    NoOpESignatureAdapter,
    Recipient,
    SignatureRequest,
)

esig = ESignatureService(adapter=NoOpESignatureAdapter())

async def send_for_signature(document_id: str) -> str:
    envelope = await esig.request(
        SignatureRequest(
            document_id=document_id,
            recipients=[
                Recipient(
                    name="Alice Johnson",
```



```

        email="alice@example.com",
        role="signer",
    ),
    ],
    subject="Please sign your employment contract",
    message="Review and sign at your earliest convenience.",
)
)
return envelope.id

```

Listing C.3 — Requesting an e-signature

Storage adapters

Adapter class	pyfly.ecm.storage.provider value	Notes
<code>LocalFilesystemStorageAdapter</code>	<code>local</code> (default)	Stores blobs as <code><base_dir>/<id>/v<n></code>
<code>AwsS3StorageAdapter</code>	<code>s3</code> or <code>aws</code>	Requires <code>boto3</code> ; async via <code>run_in_executor</code>
<code>AzureBlobStorageAdapter</code>	<code>azure</code>	Requires <code>azure-storage-blob</code>

All three implement `DocumentStoragePort` . `LocalFilesystemStorageAdapter` creates the base directory on construction and stores each version as a separate file, making it suitable for development and testing without external services.

E-signature adapters

Adapter class	<code>pyfly.ecm.esignature.provider</code> value	Notes
<code>NoOpESignatureAdapter</code>	<code>noop</code> (default)	Returns stub envelopes; safe for dev/test
<code>DocuSignESignatureAdapter</code>	<code>docusign</code>	REST API calls via <code>httpx</code>
<code>AdobeSignESignatureAdapter</code>	<code>adobe</code>	REST API calls via <code>httpx</code>
<code>LogaltyESignatureAdapter</code>	<code>logalty</code>	REST API calls via <code>httpx</code>

Implement `ESignatureAdapter` to add a custom provider (e.g. an in-house signing service). Register your implementation as a bean and inject it into `ESignatureService`.

Auto-configuration

`EcmAutoConfiguration` activates when `pyfly.ecm.enabled=true` is set in config. It registers five beans:

Bean	Default implementation
<code>metadata_storage</code>	<code>InMemoryMetadataStorage</code>
<code>folder_repository</code>	<code>InMemoryFolderRepository</code>
<code>document_storage</code>	<code>LocalFilesystemStorageAdapter</code> (temp dir)
<code>document_service</code>	<code>DocumentService</code> wiring all three above
<code>esignature_adapter</code>	<code>NoOpESignatureAdapter</code>
<code>esignature_service</code>	<code>ESignatureService</code> wrapping the adapter

The storage and e-signature adapters are selected from config so you switch providers purely via YAML — no code change required:

`pyfly.yaml`

```
pyfly:
  ecm:
    enabled: true
    storage:
      provider: s3
      local:
        base-dir: /var/ecm
      s3:
        bucket: my-docs-bucket
        region: eu-west-1
        key-prefix: documents/
      azure:
        container: my-container
        connection-string: "DefaultEndpointsProtocol=https;..."
        account-url: "https://acct.blob.core.windows.net"
        key-prefix: documents/
    esignature:
      provider: docusign
      docusign:
```

```

    base-url: "https://demo.docusign.net/restapi"
    account-id: "your-account-id"
    access-token: "your-access-token"
  adobe:
    api-base: "https://api.na4.adobesign.com"
    access-token: "your-token"
  logalty:
    api-base: "https://api.logalty.com"
    api-key: "your-api-key"

```

Listing C.4 — Full ECM YAML with S3 storage and DocuSign

💡 START WITH LOCAL + NOOP

In development, omit the `provider` keys and let the defaults take effect: `local` storage uses a fresh temp directory and `noop` e-signature returns deterministic stub data. No external services needed.

Implementing a custom storage adapter

Implement `DocumentStoragePort` and register it as a `@bean` :

infra/gcs_storage.py

```

from pyfly.ecm.models import Document, DocumentVersion
from pyfly.ecm.ports import DocumentStoragePort

class GcsStorageAdapter:
    """Google Cloud Storage adapter (skeleton)."""

    name = "gcs"

    def __init__(self, bucket: str, key_prefix: str = "") -> None:
        self._bucket = bucket

```

```

self._prefix = key_prefix

async def upload(
    self, document: Document, content: bytes
) -> DocumentVersion:
    version_no = (
        (document.versions[-1].version + 1) if document.versions else 1
    )
    key = f"{self._prefix}{document.id}/v{version_no}"
    # ... upload content to GCS ...
    return DocumentVersion(
        version=version_no,
        content_hash="sha256-placeholder",
        size_bytes=len(content),
        storage_uri=f"gs://{self._bucket}/{key}",
    )

async def download(
    self, document: Document, version: int | None = None
) -> bytes:
    target = version or document.versions[-1].version
    key = f"{self._prefix}{document.id}/v{target}"
    # ... download from GCS ...
    return b""

async def delete(
    self, document: Document, version: int | None = None
) -> bool:
    # ... delete object(s) from GCS ...
    return True

```

Listing C.5 — Custom storage adapter skeleton

Source files: - `src/pyfly/ecm/models.py` — domain dataclasses - `src/pyfly/ecm/ports.py` — Protocol contracts - `src/pyfly/ecm/services.py` — `DocumentService`, `ESignatureService` - `src/pyfly/ecm/adapters/` — bundled storage and e-signature

```
adapters - src/pyfly/ecm/in_memory.py — InMemoryMetadataStorage ,  
InMemoryFolderRepository - src/pyfly/ecm/auto_configuration.py —  
EcmAutoConfiguration
```

APPENDIX D

CLI & Troubleshooting

The `pyfly` CLI provides project scaffolding, application startup, database migrations, and environment diagnostics. It is built with **Click** for command parsing and **Rich** for coloured terminal output.

Install: `uv add "pyfly[cli]"` (or `uv sync --extra cli`). Requires Click, Rich, Jinja2, and questionnaire.

Entry point: `pyfly` — registered as a console script in `pyproject.toml`.

Command reference

Command	Description
<code>pyfly new [NAME]</code>	Scaffold a new project (interactive or direct)
<code>pyfly run</code>	Start the ASGI application server
<code>pyfly info</code>	Show Python version and installed extras
<code>pyfly doctor</code>	Diagnose environment (Python, tools, PyFly)
<code>pyfly db init</code>	Initialise an Alembic migration environment
<code>pyfly db migrate [-m MSG]</code>	Auto-generate a migration revision
<code>pyfly db upgrade [REVISION]</code>	Apply pending migrations (default: <code>head</code>)
<code>pyfly db downgrade REVISION</code>	Revert to a previous revision
<code>pyfly license</code>	Display the Apache 2.0 license text
<code>pyfly sbom [--json]</code>	Software Bill of Materials table
<code>pyfly --version</code>	Print the installed PyFly version
<code>pyfly --help</code>	Print the banner and all commands

pyfly new

Creates a complete project directory from one of seven archetypes.

```
pyfly new <name> [--archetype ARCHETYPE] [--features FEAT,...] [--directory DIR]
pyfly new                                # interactive mode (questionary TUI)
```


Archetypes

Archetype	Default features	What is generated
<code>core</code>	<i>(none)</i>	DI container, config, Dockerfile
<code>web-api</code>	<code>web</code>	REST controllers, services, repositories (Todo CRUD)
<code>fastapi-api</code>	<code>fastapi</code>	FastAPI stack, native OpenAPI
<code>web</code>	<code>web</code>	Server-rendered HTML with Jinja2 templates
<code>hexagonal</code>	<code>web</code>	Ports & adapters, domain/application/infra/api layers
<code>library</code>	<i>(none)</i>	PEP 561 <code>py.typed</code> package, no PyFly runtime
<code>cli</code>	<code>shell</code>	Shell commands with DI, no ASGI entry point

Available `--features` values

Feature	What it adds
<code>web</code>	Starlette HTTP server, REST controllers, OpenAPI docs
<code>fastapi</code>	FastAPI HTTP server, native OpenAPI
<code>granian</code>	Granian ASGI server (Rust/tokio)
<code>hypercorn</code>	Hypercorn ASGI server (HTTP/2, HTTP/3)
<code>data-relational</code>	SQLAlchemy ORM (async), Alembic migrations
<code>data-document</code>	Beanie ODM, Motor (MongoDB)
<code>eda</code>	In-memory event bus, Kafka + RabbitMQ support
<code>cache</code>	Caching layer (in-memory default; Redis if installed)
<code>client</code>	Resilient HTTP client (httpx + retry/circuit-breaker)
<code>security</code>	JWT authentication, bcrypt password hashing
<code>scheduling</code>	Cron-based task scheduling
<code>observability</code>	Prometheus metrics, OpenTelemetry tracing
<code>cqrs</code>	Command/Query Responsibility Segregation
<code>shell</code>	DI-powered interactive shell commands

Examples**terminal**

```
# Minimal core service
pyfly new wallet-service
```

```

# REST API with SQL data layer
pyfly new lumen --archetype web-api --features web,data-relational

# Hexagonal service with cache and security
pyfly new payment-svc --archetype hexagonal \
    --features web,data-relational,cache,security

# MongoDB-backed microservice
pyfly new catalog-svc --archetype web-api \
    --features web,data-document

# CLI application with interactive shell
pyfly new admin-tool --archetype cli

# Interactive mode (arrow keys + checkboxes)
pyfly new

```

Listing D.1 — *pyfly new examples*

In interactive mode the wizard prompts for name, package name (pre-filled from the project name), archetype (single-select with arrows), and features (multi-select with space bar). A confirmation summary is shown before creation. Press Ctrl+C at any prompt to exit cleanly.

Project names containing hyphens are automatically converted to valid Python package names (`my-service` → `my_service`). Use the **Package name** prompt in interactive mode to override this derived value.

pyfly run

Starts the ASGI application using the auto-selected server.

```

pyfly run [--host HOST] [--port PORT] [--server SERVER]
          [--workers N] [--reload] [--app MODULE:VAR]

```

Option	Default	Description
<code>--host</code>	<code>0.0.0.0</code>	Bind address
<code>--port</code>	Config or <code>8080</code>	Port (CLI → <code>pyfly.web.port</code> → 8080)
<code>--server</code>	Auto-detect	<code>granian</code> , <code>uvicorn</code> , or <code>hypercorn</code>
<code>--workers</code>	Config or <code>0</code>	Worker processes (<code>0</code> = cpu count)
<code>--reload</code>	off	Auto-reload on code changes
<code>--app</code>	Auto-discovered	Import path, e.g. <code>myapp.main:app</code>

Server priority (when `--server` is omitted): Granian › Uvicorn › Hypercorn. The first importable server wins.

Application discovery (when `--app` is omitted):

1. `pyfly.app.module` in `pyfly.yaml`
2. `app.module` in `pyfly.yaml` (flat layout)
3. `src/<package>/main.py` auto-scan

`pyfly run` adds `src/` to `sys.path` automatically for src-layout projects.

terminal

```
# Development with auto-reload
pyfly run --reload

# Production: Granian, bind all interfaces, all CPU cores
pyfly run --host 0.0.0.0 --port 80 --server granian --workers 0

# Explicit application path
pyfly run --app my_service.main:app
```

Listing D.2 — pyfly run examples

pyfly info

Displays two Rich tables: Python version, platform, and architecture; plus installation status of every PyFly optional module. Useful after a selective install to confirm which adapters are available.

```
pyfly info
```

pyfly doctor

Runs a comprehensive environment health check.

```
pyfly doctor
```

Check	Pass condition	Failure impact
Python version	<code>>= 3.12</code>	Fatal — doctor reports fail
Virtual environment	<code>sys.prefix != sys.base_prefix</code>	Warning only
<code>git</code> on PATH	<code>shutil.which("git")</code> returns a path	Fatal
<code>uv</code> on PATH	<code>shutil.which("uv")</code> returns a path	Fatal
<code>uvicorn</code> on PATH	present	Advisory (-)
<code>alembic</code> on PATH	present	Advisory (-)
<code>ruff</code> on PATH	present	Advisory (-)
<code>mypy</code> on PATH	present	Advisory (-)
PyFly importable	<code>import pyfly</code> succeeds	Fatal

Exits with **"All checks passed!"** or **"Some issues found."**.

pyfly db

Database migration commands backed by [Alembic](#).

REQUIRES DATA-RELATIONAL EXTRA

All `pyfly db` subcommands require Alembic (`pyfly[data-relational]`).

pyfly db init

```
pyfly db init
```

Creates `alembic/` and `alembic.ini` in the current directory. Overwrites `alembic/env.py` with a PyFly template that wires `async_engine_from_config` and `Base.metadata` from `pyfly.data.relational.sqlalchemy`. Exits with an error if `alembic/` already exists.

pyfly db migrate

```
pyfly db migrate [-m "description"]
```

Runs `alembic revision --autogenerate`. Compares current `Base.metadata` against the live database and writes a new version file to `alembic/versions/`. Requires `alembic.ini` (run `pyfly db init` first).

pyfly db upgrade / downgrade

```
pyfly db upgrade [REVISION]      # default: head
pyfly db downgrade REVISION      # e.g. -1, base, abc123
```

Typical migration workflow

terminal

```
# 1. Initialise Alembic (once per project)
pyfly db init

# 2. Generate initial migration from your entity models
pyfly db migrate -m "initial schema"

# 3. Apply migrations
pyfly db upgrade

# 4. After modifying entities, generate a new revision
pyfly db migrate -m "add order status column"
pyfly db upgrade

# 5. Roll back one step
```

```
pyfly db downgrade -1  
  
# 6. Revert all migrations  
pyfly db downgrade base
```

Listing D.3 — Database migration lifecycle

pyfly sbom

Prints a Rich table of every PyFly dependency with required and installed versions. The `--json` flag outputs machine-readable JSON for compliance pipelines.

```
pyfly sbom  
pyfly sbom --json
```

Typical development workflow

terminal

```
# Verify environment  
pyfly doctor  
  
# Scaffold the project  
pyfly new lumen --archetype web-api \  
    --features web,data-relational,security  
  
cd lumen  
  
# Check what was installed  
pyfly info  
  
# Initialise migrations  
pyfly db init
```



```
pyfly db migrate -m "initial schema"
pyfly db upgrade

# Develop with auto-reload
pyfly run --reload

# Add a new column, migrate, upgrade
pyfly db migrate -m "add shipment_id to orders"
pyfly db upgrade
```

Listing D.4 — End-to-end workflow from setup to development

Troubleshooting

Common problems and fixes

Symptom	Likely cause	Fix
<code>command not found: pyfly</code>	PATH not updated after install	<code>source ~/.zshrc</code> (or <code>~/.bashrc</code>); or <code>export PATH="\$HOME/.pyfly/bin:\$PATH"</code>
Python 3.11 found (3.12 required)	Old Python on PATH	<code>brew install python@3.12</code> (macOS) or <code>sudo apt install python3.12</code> (Ubuntu)
<code>ModuleNotFoundError: No module named 'starlette'</code>	<code>web</code> extra not installed	<code>uv add "pyfly[web]"</code> or <code>uv sync --extra web</code>
<code>ModuleNotFoundError: No module named 'beanie'</code>	<code>data-document</code> extra missing	<code>uv add "pyfly[data-document]"</code>
<code>ModuleNotFoundError: No module named 'sqlalchemy'</code>	<code>data-relational</code> extra missing	<code>uv add "pyfly[data-relational]"</code>
<code>alembic is not installed</code>	Missing Alembic	<code>uv add "pyfly[data-relational]"</code>
No application found	No <code>pyfly.yaml</code> or <code>--app</code> flag	Add <code>pyfly.app.module: myapp.main:app</code> to <code>pyfly.yaml</code> , or pass <code>--app myapp.main:app</code>
No ASGI server found	No server extra installed	<code>uv add "pyfly[web]"</code> (adds Uvicorn)
<code>venv module not found</code>	Split-package Python (Debian/Ubuntu)	<code>sudo apt install python3.12-venv</code>
Directory 'alembic' already exists	<code>db init</code> run twice	<code>rm -rf alembic alembic.ini</code> then re-run <code>pyfly db init</code>
<code>uv cache clean</code> fixes slow installs	Corrupt uv cache	Run <code>uv cache clean</code> then retry

Symptom	Likely cause	Fix
Ctrl+C in <code>pyfly new</code> leaves no files	Interactive mode cancelled cleanly	Re-run <code>pyfly new</code> ; no cleanup needed

Uninstalling

For an installer-based installation (`bash install.sh`):

terminal

```
# Remove the installation directory
rm -rf ~/.pyfly

# Remove the PATH line from your shell profile
# Delete the line: export PATH="$HOME/.pyfly/bin:$PATH" # PyFly Framework
```

Listing D.5 — Uninstall PyFly

For a manual `uv` / `pip` installation: `uv remove pyfly` or `pip uninstall pyfly`.

Extending the CLI

The CLI is built on Click. Add custom commands by importing the `cli` group and calling `cli.add_command`:

myapp/generate.py

```
import click
from pyfly.cli.main import cli

@click.command()
@click.argument("entity_name")
```

```
def generate(entity_name: str) -> None:
    """Generate boilerplate for a new entity."""
    click.echo(f"Generating {entity_name} entity ...")

cli.add_command(generate, name="generate")
```

Listing D.6 — Adding a custom CLI command

Register the command module in your `pyproject.toml` entry points or import it in your application's `__init__.py` before the CLI is invoked.

APPENDIX

Glossary

Adapter — A concrete class that implements a port by delegating to a specific library or infrastructure technology (PostgreSQL, Redis, Kafka, etc.). Adapters live at the edge of the hexagonal architecture and can be swapped without touching the domain or application layers. In PyFly, the async SQLAlchemy session factory,

`RedisCacheAdapter`, and `KafkaMessageBroker` are all adapters (Chapters 5, 10, 13).

Aggregate root — The single entry point to a cluster of domain objects that must stay consistent together; all state changes are routed through its methods, which enforce invariants and emit domain events. External code loads and saves only the root, never its inner objects. In PyFly, `AggregateRoot[ID]` is the state-stored base class; `Wallet` is Lumen's state-stored aggregate root and `LedgerAccount` is its event-sourced counterpart (Chapters 6, 9).

AOP (Aspect-Oriented Programming) — A technique for adding cross-cutting concerns — logging, metrics, security checks — declaratively to methods without modifying their source code. PyFly's `@aspect` and `@before` / `@after` / `@around` advice decorators implement AOP; Chapter 15 uses it to attach observability to every service method.

ApplicationContext — The central DI container that discovers beans, resolves dependencies, and manages their lifecycle from startup to shutdown. It fires `ContextRefreshedEvent` once all beans are wired and `ContextClosedEvent` on graceful shutdown. `ApplicationContext` is the PyFly equivalent of Spring's `ApplicationContext` (Chapter 2).

Async/await — Python's native coroutine syntax for non-blocking I/O. PyFly is built async-native: every HTTP handler, repository method, bus call, and service client is declared `async def` and scheduled on the event loop. Blocking calls inside an `async def` function freeze the loop and must be avoided (Chapter 1).

Autowiring — The mechanism by which the DI container resolves a bean's constructor arguments from type hints alone, with no factory code required. PyFly inspects `__init__` signatures at startup and injects matching beans automatically; the `@primary` annotation selects the preferred implementation when multiple candidates exist (Chapter 2).

Bean — Any Python object that the DI container creates, wires, and manages. You declare a class as a bean by applying a stereotype decorator (`@service` , `@repository` , `@component` , `@configuration` , or `@rest_controller`), or by annotating a factory method with `@bean` inside a `@configuration` class (Chapter 2).

BFF (Backend for Frontend) — An API gateway layer that sits in front of multiple microservices and composes their capabilities into a user-journey-focused API, tailored to a specific frontend client. In Chapter 11, Lumen's BFF tier aggregates wallet and payment data so the mobile app makes one request instead of two.

Bounded context — A DDD-level scope within which a domain model has a single, unambiguous meaning. Services in a microservices architecture often map one-to-one to bounded contexts: `WalletService` owns the wallet model; `PaymentsService` owns the payment model. Shared kernel or anti-corruption layers are needed when two contexts must exchange data (Chapter 6).

Bulkhead — A resilience pattern that limits the number of concurrent calls to a particular resource or downstream service, preventing a slow dependency from exhausting the thread or coroutine pool and degrading unrelated paths. PyFly's `@bulkhead(max_concurrent=N)` decorator implements the semaphore bulkhead (Chapter 13).

Circuit breaker — A resilience pattern that counts consecutive failures to a remote dependency; once the failure threshold is reached, the breaker trips to the open state and short-circuits further calls with a fast error, giving the dependency time to recover before calls resume. PyFly's `@circuit_breaker` decorator and `@service_client` inject this behaviour automatically (Chapters 11, 13).

Command (CQRS) — An immutable, frozen dataclass that expresses a single write intent — "open a wallet", "deposit funds" — and carries the exact data needed to fulfil it. Commands inherit from `Command[R]` where `R` is the handler's return type, and flow through the `CommandBus`. They may optionally implement `validate()` and `authorize()` (Chapter 7).

CommandBus — The pipeline that receives a `Command`, runs validation and authorization, dispatches it to the matching `CommandHandler`, and then publishes any domain events buffered by the aggregate. One handler is registered per command type; the bus enforces this constraint at startup (Chapter 7).

Compensation — A forward operation that semantically reverses the effect of a previously completed saga step when a later step fails. Unlike a database rollback, compensation is a new write that explicitly undoes the earlier change (for example, "refund payment" compensates "capture payment"). Each `@saga_step` names its compensation method (Chapter 12).

Component — A generic stereotype for a managed bean that does not fit the more specific roles of `@service`, `@repository`, or `@rest_controller`. `@component` registers the class with the container and enables injection, but carries no additional semantic meaning (Chapter 2).

Configuration class — A class decorated with `@configuration` that groups `@bean` factory methods. The container treats it as a source of bean definitions, calling each factory method and registering the return value as a named, typed bean. Profile guards and conditional annotations can be applied to the class or to individual factory methods (Chapter 3).

Container (DI) — See *ApplicationContext*.

Convention over configuration — The principle that sensible defaults eliminate boilerplate: a class annotated `@service` is automatically singleton-scoped, discovered by component scan, and injected by type without any XML or explicit registration. PyFly adopts this as a core design value, requiring explicit overrides only when the default is wrong (Chapter 1).

CQRS (Command Query Responsibility Segregation) — An architectural pattern that separates write operations (commands) from read operations (queries) into distinct code paths, buses, and potentially distinct data models. Reads can be cached independently of writes; handlers can be tested in isolation; cross-cutting concerns are applied uniformly by the respective bus (Chapter 7).

Dead-letter queue (DLQ) — A dedicated message queue or topic that receives messages which could not be processed after the configured number of retries. PyFly's `@message_listener` routes poison messages to the DLQ automatically, preventing them from blocking healthy messages (Chapter 10).

Dependency injection (DI) — A design pattern in which an object declares its collaborators as constructor parameters and an external container provides the concrete instances. DI decouples construction from use, making it straightforward to swap implementations and to test classes with fakes or mocks (Chapter 2).

Domain event — An immutable record of a business fact that has already occurred — "wallet opened", "funds deposited". Domain events are produced by aggregate roots via `raise_event()`, published through the `EventPublisher` port, and consumed by independent listeners. In event sourcing, they are also the source of truth for aggregate state (Chapters 6, 8, 9).

DTO (Data Transfer Object) — A plain, serializable object used to carry data across a layer boundary — between the HTTP controller and the service, or between a service and its API client — without exposing internal domain types. In PyFly, Pydantic `BaseModel` subclasses serve as request/response DTOs (Chapter 4).

EDA (Event-Driven Architecture) — An architectural style in which services communicate by publishing and subscribing to events rather than by direct synchronous calls. Producers and consumers are decoupled: a producer does not know which consumers exist. PyFly's `EventPublisher` and `@event_listener` are the intra-process EDA primitives; `MessageBrokerPort` extends the pattern across process boundaries (Chapters 8, 10).

Entity — A domain object with a stable identity that persists over time and through state changes. Two entities are equal if and only if they share the same non-null `id`. In PyFly, `Entity[TID]` tracks identity; `BaseEntity` adds audit columns (`created_at` , `updated_at`) for the persistence layer (Chapters 5, 6).

Event sourcing — A persistence strategy in which every state change is stored as an immutable domain event in an append-only stream. The current state of an aggregate is computed by replaying all events from the stream. PyFly's `pyfly.eventsourcing` module provides `AggregateRoot`, `EventStore`, `EventSourcedRepository`, snapshot support, and a `ProjectionRunner` (Chapter 9).

EventEnvelope — The metadata wrapper that packages a domain event payload for delivery: event ID, event type, aggregate stream ID, sequence number, timestamp, correlation ID, and causation ID. Every event reaching a listener arrives in an `EventEnvelope`; you never construct one manually (Chapters 8, 9).

EventStore — The append-only persistence layer for an event-sourced system. It records domain events indexed by stream ID and sequence number, enforces optimistic concurrency via the `version` token, and supports range queries for replay. PyFly ships `SQLAlchemyEventStore` and `InMemoryEventStore` (Chapter 9).

Hexagonal architecture — An architectural style that places the domain and application logic at the centre, surrounded by ports (interfaces), with adapters at the edges. Business code depends only on ports; adapters implement those ports using specific technologies. This is the organising principle of every PyFly module (Chapters 1, 2, 5).

Idempotency — The property that performing the same operation multiple times produces the same result as performing it once. Idempotent handlers are essential in messaging and saga compensation: network retries or at-least-once delivery guarantees mean a message may arrive more than once. Deduplication keys or idempotency tokens are used to detect and discard duplicate executions (Chapters 10, 12).

Migration — A versioned, ordered script that evolves a relational database schema without destroying data. PyFly integrates Alembic for migration management; `pyfly db migrate` auto-generates a migration from entity changes and `pyfly db upgrade` applies pending migrations (Chapter 5).

Outbox pattern — A technique for publishing domain events reliably alongside a database write: both the state change and the event records are written in the same local transaction; a background relay process reads unsent events from the outbox table and forwards them to the broker. This eliminates the two-phase commit between the database and the message broker (Chapters 9, 12).

Port — A Python `Protocol` class that defines the interface a piece of business logic depends on, without specifying any implementation. The DI container wires the concrete adapter that satisfies the protocol at startup. Ports enable the hexagonal architecture and make adapters swappable with zero business-logic changes (Chapters 1, 2, 5).

Primary bean — When multiple beans satisfy the same type, the one annotated `@primary` is preferred for injection. Without a `@primary` annotation the container raises an ambiguity error. `@primary` is how you designate the production adapter among several alternatives (Chapter 2).

Profile — A named activation tag that selects which beans and configuration values are active at runtime. PyFly loads `pyfly-{profile}.yaml` over the base `pyfly.yaml` and activates beans annotated `@profile("prod")` only when `prod` is in the active profile list. The active profile is set via `PYFLY_PROFILES_ACTIVE` or `pyfly.yaml` (Chapter 3).

Projection — A read model derived by consuming a stream of events. A projection subscribes to specific event types and incrementally builds a queryable view — a balance cache, an audit table, a dashboard aggregate — without touching the write model. In event sourcing, `ProjectionRunner` replays the `EventStore` to rebuild projections from scratch (Chapters 8, 9).

Query (CQRS) — An immutable dataclass that expresses a read intent — "get wallet balance", "list transactions". Queries inherit from `Query[R]` and flow through the `QueryBus`, which can cache results transparently. Queries never mutate state (Chapter 7).

QueryBus — The pipeline that receives a `Query`, optionally returns a cached result, dispatches to the matching `QueryHandler`, and optionally stores the result in cache. Separating the query bus from the command bus allows different cross-cutting behaviour — caching, read-replica routing — for read paths (Chapter 7).

Rate limiter — A resilience component that caps the number of requests an endpoint or service client can accept within a time window, preventing overload from bursty traffic. PyFly's `RateLimiter(max_tokens=N, refill_rate=r) + @rate_limiter(limiter)` implements a token-bucket limiter (Chapter 13).

Repository — A collection-like abstraction over the persistence layer that allows the application to load and save aggregates or entities without any SQL in the business code. PyFly's `CrudRepository[E, ID]` provides typed `find_by_id`, `save`, `delete`, and derived-query helpers; custom implementations annotated `@repository` replace the in-memory defaults (Chapters 2, 5).

Retry — A resilience pattern that re-executes a failed operation after a delay, up to a configured maximum attempt count. Retries handle transient failures — brief network glitches, momentary 503s — without operator intervention. PyFly's

`@retry(max_attempts=N, backoff=...)` implements exponential back-off with jitter; `@service_client` includes retry by default (Chapters 11, 13).

Saga — A sequence of local transactions coordinated by a central orchestrator. Each step commits to its own service's database; if a later step fails, the orchestrator calls the compensating transaction for each already-committed step in reverse order. PyFly's `@saga` and `@saga_step` decorators implement the orchestrated saga pattern with a parallel-execution DAG (Chapter 12).

Serialization — The process of converting an in-memory object to a wire format (JSON, Protobuf, Avro) for HTTP responses or message publishing, and deserializing the reverse. PyFly's central `PyFlyJsonSerializer` bean (`from pyfly.web import PyFlyJsonSerializer`) configures Pydantic-backed JSON serialization once and applies it to every HTTP response and message envelope (Chapters 4, 10).

Service — A managed bean decorated with `@service` that houses business logic and orchestrates calls to repositories, event publishers, and other services. Services are the application layer in hexagonal architecture: they translate incoming commands into domain operations and persist results (Chapter 2).

Snapshot — A point-in-time serialized copy of an event-sourced aggregate's state, stored alongside the event stream to accelerate replay. On load, the repository restores the snapshot and replays only the events that occurred after the snapshot version, capping replay time regardless of stream length (Chapter 9).

Stereotype — A decorator that registers a class as a bean and signals its architectural role: `@service`, `@repository`, `@component`, `@configuration`, or `@rest_controller`. All stereotypes are technically equivalent in the container; the semantic difference is for human readers and tooling (Chapter 2).

TCC (Try-Confirm-Cancel) — A distributed-transaction pattern in which each participant first *reserves* a resource (Try), then the coordinator either *confirms* all reservations or *cancels* them based on whether all Tries succeeded. TCC is useful when exact, immediate reservation semantics are required — for example, holding funds before capturing a payment. PyFly's `@tcc(name="...")` + `@tcc_participant(id="...", order=N)` decorators implement the TCC protocol alongside the saga engine (Chapter 12).

Testcontainers — A library that starts real Docker containers (PostgreSQL, Redis, Kafka) for integration tests and tears them down when the suite finishes. PyFly's `pyfly.testing` module provides `postgres_container` and `redis_container` fixtures that wire Testcontainers into the DI container via `@ServiceConnection`-style configuration (Chapter 16).

Value object — An immutable domain object identified by its value rather than by an identity field. Two value objects are equal if all their fields are equal. In PyFly, `ValueObject` is the base class; apply `@dataclass(frozen=True)` to enforce immutability. `Money` — an integer amount in **minor units** (cents) and a `Currency` `StrEnum` — is Lumen's canonical value object; `Wallet` is the aggregate root that owns it (Chapter 6).

Webhook — An inbound HTTP callback that an external provider (Stripe, Twilio, etc.) calls to notify your service of an asynchronous event, such as a payment-status change. PyFly's `@webhook_listener` decorator verifies the HMAC-SHA256 signature, deduplicates replays via a nonce cache, and routes the payload to a typed handler (Chapter 17).

Workflow — A long-running variant of the saga pattern where steps may pause for minutes, hours, or human approval before resuming. Workflows persist their state between steps so they survive process restarts; PyFly's `@workflow` and `@workflow_step` decorators provide this capability on top of the same saga engine (Chapter 12).